JavaScript Mastery

শূন্য থেকে প্রফেশনাল

৩০টি অধ্যায় • ৩০টি Practical Project • Browser থেকে Backend, Testing, Security ও System Design

প্রথম পূর্ণাঙ্গ খসড়া সংস্করণ (Draft Edition) • অক্টোবর ২০২৬

Moynul Hasan

IT Officer, M&N Holidays

JavaScript Mastery — শূন্য থেকে প্রফেশনাল

লেখক: **Moynul Hasan**

প্রথম পূর্ণাঙ্গ খসড়া সংস্করণ (Draft Edition), অক্টোবর ২০২৬

যোগাযোগ: Moynul Hasan, Bogura, Bangladesh • ফোন: 01745334044

© ২০২৬ Moynul Hasan। সর্বস্বত্ব সংরক্ষিত। লেখকের লিখিত অনুমতি ছাড়া এই বইয়ের কোনো অংশ কোনো মাধ্যমে পুনরুৎপাদন, সংরক্ষণ বা বিতরণ করা যাবে না।

খসড়া সংস্করণ সম্পর্কে: এটি বইটির প্রথম পূর্ণাঙ্গ খসড়া। বিষয়বস্তু, Code, ভাষা ও বিন্যাস এখনো সম্পাদনা এবং কারিগরি পর্যালোচনার অধীনে রয়েছে। চূড়ান্ত সংস্করণে অধ্যায় সম্প্রসারণ, সংশোধন ও পুনর্বিন্যাস হতে পারে।

দায়মুক্তি: এই বইয়ের সব Code, Project ও উদাহরণ শিক্ষামূলক উদ্দেশ্যে তৈরি। এগুলো Production System হিসেবে ব্যবহারের উপযোগী নয়; বাস্তব প্রয়োগের আগে নিজস্ব Security, Testing ও Code Review আবশ্যক। বইয়ের তথ্য ব্যবহারের ফলে সৃষ্ট কোনো ক্ষতির জন্য লেখক দায়ী নন। উদাহরণে ব্যবহৃত ব্যক্তি, Booking, Package, মূল্য ও অন্যান্য Data কাল্পনিক।

Trademark: JavaScript যুক্তরাষ্ট্রে Oracle Corporation-এর Trademark। Node.js OpenJS Foundation-এর Trademark। TypeScript ও Windows Microsoft Corporation-এর এবং Chrome Google LLC-এর Trademark। বইয়ে উল্লিখিত অন্যান্য Product ও Service-এর নাম তাদের নিজ নিজ মালিকের Trademark; এখানে শুধু বর্ণনামূলক উদ্দেশ্যে ব্যবহৃত।

Software: বইয়ের Code উদাহরণ ও Project-গুলো Node.js 20 বা পরবর্তী সংস্করণ এবং আধুনিক Browser (Chrome, Edge, Firefox) লক্ষ্য করে লেখা।

ভূমিকা

JavaScript আজ শুধু Web Page-কে Interactive করার ভাষা নয়; Browser, Server, Mobile, Desktop এবং Cloud—সব জায়গায় এর ব্যবহার। এই বইয়ের লক্ষ্য তোমাকে একেবারে প্রথম `console.log()` থেকে শুরু করে পরীক্ষিত, নিরাপদ ও রক্ষণাবেক্ষণযোগ্য JavaScript Application তৈরির পেশাদার মানসিকতা পর্যন্ত নিয়ে যাওয়া।

এই বই কার জন্য

- যারা Programming একেবারে নতুন শুরু করছে এবং বাংলায় ধাপে ধাপে শিখতে চায়।
- যারা JavaScript-এর মৌলিক বিষয় জানে, কিন্তু Async, Testing, Security, Architecture বা Backend-এ আত্মবিশ্বাসী হতে চায়।
- যারা Job Interview বা Portfolio Project-এর জন্য বাস্তবসম্মত প্রস্তুতি নিতে চায়।

বইটির গঠন

বইটি ৩০টি অধ্যায়ে সাজানো, এবং প্রতিটি অধ্যায় আগের অধ্যায়ের ওপর ভিত্তি করে এগোয়:

- **অধ্যায় ১–৯ — মৌলিক ভিত্তি:** Variables, Data Types, Operators, Loops, Functions, Arrays, Objects, Strings এবং DOM ও Events।
- **অধ্যায় ১০–১৮ — আধুনিক JavaScript:** Asynchronous Programming, Error Handling, Modules, OOP, Closures, Browser APIs, Testing, Regular Expressions এবং Functional Programming।
- **অধ্যায় ১৯–২৪ — পেশাদার Engineering:** Performance, Security, Design Patterns, Testing Mastery, Profiling এবং TypeScript।
- **অধ্যায় ২৫–৩০ — Backend থেকে System Design:** Node.js Backend, Database, Real-Time Application, Deployment, Distributed System এবং Full-Stack Capstone ও Career Roadmap।

প্রতিটি অধ্যায় কীভাবে সাজানো

প্রায় প্রতিটি অধ্যায়ে পাবে: শেখার লক্ষ্য, উদাহরণসহ ধারণার ব্যাখ্যা, একটি Practical Project, সাধারণ ভুল ও Debugging নির্দেশনা, অনুশীলনী ও তার সমাধান, এবং উত্তর ও ব্যাখ্যাসহ MCQ। প্রথমে নিজে চেষ্টা করো, তারপর সমাধান মিলিয়ে দেখো।

Code ও Project চালানোর নির্দেশিকা

- প্রতিটি অধ্যায়ের Project-এর সম্পূর্ণ Source Code বইয়ের Companion Source Code-এর `source-code/chapter-NN/` Folder-এ রয়েছে (যেমন অধ্যায় ৫-এর জন্য `source-code/chapter-05/`)। বইয়ে মূল অংশগুলো ব্যাখ্যা করা হয়েছে।

- Browser Project-এর জন্য Project Folder-এ `python -m http.server 8000`-এর মতো একটি Local HTTP Server চালিয়ে `http://localhost:8000` খোলো; ES Modules সাধারণত `file://` থেকে ঠিকমতো চলে না।
- Automated Test চালাতে Node.js 20 বা পরবর্তী সংস্করণ Install করে Project Folder-এ `node --test` বা অধ্যায়ে দেওয়া Command চালাও।
- Code Block-এর ওপরের ছোট Label (JavaScript, Terminal, HTML ইত্যাদি) Code-এর ধরন নির্দেশ করে। দীর্ঘ Line পাতার প্রস্থে ভেঙে দেখানো হয়েছে; আসল File-এ সেগুলো এক Line।

কিছু গুরুত্বপূর্ণ নীতি

- টাকার হিসাব সাধারণত পূর্ণসংখ্যা BDT বা Paisa-এর মতো Minor Unit-এ করা হয়েছে, যাতে Floating-Point ত্রুটি এড়ানো যায়।
- User Input দেখাতে `innerHTML`-এর বদলে `textContent` ব্যবহার করা হয়েছে—নিরাপত্তা এই বইয়ের একটি ধারাবাহিক বিষয়।
- বইয়ের Project-গুলো শেখার জন্য ইচ্ছাকৃতভাবে ছোট রাখা হয়েছে। প্রতিটি অধ্যায়ে স্পষ্টভাবে বলা আছে কোন অংশ Production-এর জন্য যথেষ্ট নয় এবং বাস্তবে আর কী প্রয়োজন।

খসড়া সংস্করণ সম্পর্কে

এটি বইটির প্রথম পূর্ণাঙ্গ খসড়া সংস্করণ। পরবর্তী সংস্করণে অধ্যায়গুলো আরও বিস্তৃত করা, Diagram যোগ করা এবং ভাষা ও পরিভাষা আরও সামঞ্জস্যপূর্ণ করার পরিকল্পনা রয়েছে। ভুল বা অস্পষ্টতা চোখে পড়লে তা চিহ্নিত করে রাখো—তোমার প্রতিক্রিয়া পরবর্তী সংস্করণকে আরও ভালো করবে।

শুভ কোডিং!

লেখক পরিচিতি

নাম	Moynul Hasan
পেশা	IT Officer, M&N Holidays
শিক্ষা	Computer Science and Engineering (CSE), Daffodil International University
ঠিকানা	Bogura, Bangladesh
ফোন	01745334044

সূচিপত্র

ভূমিকা 3
লেখক পরিচিতি 5
অধ্যায় ১ — JavaScript-এর জগতে প্রথম পদক্ষেপ 8
অধ্যায় ২ — Variables, Data Types এবং Memory 12
অধ্যায় ৩ — Operators, Expressions এবং Decision Making 19
অধ্যায় ৪ — Loops এবং Iteration: Repeated Work থেকে Cart Analyzer 28
অধ্যায় ৫ — Functions: Reusable Code, Parameters, Return Values এবং Scope 34
অধ্যায় ৬ — Arrays: Data Collection, Array Methods এবং Practical Data Processing 43
অধ্যায় ৭ — Objects: Properties, Methods, Destructuring এবং Real-World Data Modeling 53
অধ্যায় ৮ — Strings এবং Modern Text Processing 61
অধ্যায় ৯ — DOM Manipulation, Events এবং Interactive Web Applications 69
অধ্যায় ১০ — Asynchronous JavaScript: Callbacks, Promises, async/await এবং Event Loop 76
অধ্যায় ১১ — Error Handling, Debugging এবং Defensive Programming 85
অধ্যায় ১২ — Modern JavaScript (ES6+), Modules এবং Clean Code Practices 93
অধ্যায় ১৩ — Object-Oriented Programming (OOP): Classes, Prototypes, Inheritance এবং Encapsulation 100
অধ্যায় ১৪ — Advanced Functions, Closures, Higher-Order Patterns এবং Functional Programming 108
অধ্যায় ১৫ — JavaScript Browser APIs, Web Storage, Fetch, URL এবং Real-World Browser Applications 114
অধ্যায় ১৬ — JavaScript Error Handling, Debugging, Testing এবং Software Reliability 123
অধ্যায় ১৭ — JavaScript Regular Expressions, Advanced Validation এবং Secure Input Processing 131
অধ্যায় ১৮ — Functional Programming, Immutability এবং Advanced Data Transformation 138
অধ্যায় ১৯ — JavaScript Performance Optimization, Memory Management এবং Web Application Efficiency 146
অধ্যায় ২০ — JavaScript Security Fundamentals, Web Application Security এবং Secure Coding Practices 153
অধ্যায় ২১ — JavaScript Design Patterns, Software Architecture এবং Scalable Application Structure 161
অধ্যায় ২২ — JavaScript Testing Mastery — Unit Testing, Integration Testing, Mocking, TDD এবং E2E 170
অধ্যায় ২৩ — JavaScript Performance Optimization, Memory Management, Profiling এবং Web Application Performance 178
অধ্যায় ২৪ — JavaScript থেকে TypeScript — Static Typing, Type Safety এবং Professional Development Workflow 185
অধ্যায় ২৫ — JavaScript Backend Development with Node.js 191
অধ্যায় ২৬ — Advanced Database Integration, ORM, Query Optimization এবং Scalable Data Architecture 198

অধ্যায় ২৭ — Real-Time JavaScript Applications 204

অধ্যায় ২৮ — Professional JavaScript Deployment, Docker, CI/CD, Cloud Infrastructure এবং Production Monitoring 210

অধ্যায় ২৯ — Advanced JavaScript System Design, Microservices, Distributed Systems এবং Enterprise Architecture 217

অধ্যায় ৩০ — JavaScript Professional Mastery, Full-Stack Capstone এবং Career Roadmap 225

অধ্যায় ১ — JavaScript-এর জগতে প্রথম পদক্ষেপ

শেখার লক্ষ্য

এই অধ্যায় শেষে পাঠক JavaScript-এর কাজ, HTML/CSS/JavaScript-এর ভূমিকা, Browser Console, Variable, Basic Function, DOM এবং একটি ছোট Interactive Application বুঝতে পারবে।

১.১ JavaScript কী এবং কেন প্রয়োজন?

JavaScript হলো একটি উচ্চস্তরের Programming Language, যা Browser-এ Interactive User Interface তৈরি করতে এবং Node.js-এর মাধ্যমে Server-Side Application চালাতে ব্যবহৃত হয়। HTML একটি Web Page-এর Structure নির্ধারণ করে; CSS এর Presentation নির্ধারণ করে; JavaScript Page-এর Behavior পরিচালনা করে। উদাহরণস্বরূপ, একটি Travel Package-এর Booking Button Click করলে Form Validation, Price Calculation এবং API Request পরিচালনায় JavaScript কাজ করতে পারে।

JavaScript এবং Java আলাদা ভাষা। JavaScript-এর Syntax-এর কিছু অংশ Java-র মতো দেখালেও Language Model, Runtime এবং Ecosystem ভিন্ন।

১.২ প্রথম JavaScript Program

Browser-এর Developer Tools খুলে Console Tab-এ লিখো:

```
console.log('Hello, JavaScript!');
const destination = 'Bhutan';
console.log(`Next destination: ${destination}`);
```

`console.log()` Debugging ও Learning-এর জন্য সুবিধাজনক। Production Application-এ Sensitive User Data Console-এ Log করা উচিত নয়।

১.৩ HTML, CSS এবং JavaScript-এর সম্পর্ক

```
HTML  → Page Structure
CSS   → Page Appearance
JS    → Page Behavior
```

Browser HTML Parse করে DOM Tree তৈরি করে। JavaScript DOM-এর Elements Select, Read এবং Update করতে পারে।

১.৪ Variables এবং Basic Values

```
const packageName = 'Mountain Escape';
let availableSeats = 12;
const priceBdt = 65800;
const isAvailable = availableSeats > 0;
```

```
console.log({ packageName, availableSeats, priceBdt, isAvailable });
```

`const` Binding Reassignment বন্ধ করে; `let` পুনরায় Value Assign করতে দেয়। Object বা Array-এর Internal Values `const` থাকলেও পরিবর্তনযোগ্য হতে পারে—পরবর্তী অধ্যায়ে বিস্তারিত থাকবে।

১.৫ Functions: Reusable Behavior

```
function makeGreeting(name) {
  const cleanName = name.trim();
  return cleanName ? `স্বাগতম, ${cleanName}!` : 'স্বাগতম!';
}

console.log(makeGreeting('Moynul'));
```

Function একই Logic বারবার না লিখে ব্যবহার করতে সাহায্য করে। Input Validation প্রয়োজন হলে Function-এর শুরুতেই যাচাই করা যায়।

১.৬ DOM এবং Events

`document.querySelector()` দিয়ে Element Select করা যায়। `addEventListener()` দিয়ে Click বা Input Event-এর Response লেখা যায়। User Input সরাসরি `innerHTML`-এ বসানো ঝুঁকিপূর্ণ; সাধারণ Text-এর জন্য `textContent` নিরাপদ Default।

১.৭ Practical Project — Greeting App

Project Structure

```
greeting-app/
├── index.html
├── style.css
└── app.js
```

index.html

```
<!doctype html>
<html lang="bn">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Greeting App</title>
  <link rel="stylesheet" href="style.css">
  <script src="app.js" defer></script>
</head>
<body>
  <main class="card">
    <h1>JavaScript Greeting App</h1>
    <label for="name">তোমার নাম</label>
    <input id="name" type="text" maxlength="80" autocomplete="name">
    <button id="greet" type="button">Say Hello</button>
    <p id="result" role="status" aria-live="polite"></p>
  </main>
</body>
</html>
```

style.css

```
* { box-sizing: border-box; }
body { margin: 0; min-height: 100vh; display: grid; place-items: center; font-family: system-ui, sans-serif; background: #f0f5ff; color: #183153; }
.card { width: min(92vw, 460px); padding: 28px; background: white; border: 1px solid #d9e4f4; border-radius: 18px; }
label, input, button { display: block; width: 100%; }
input, button { padding: 12px; margin-top: 10px; border-radius: 8px; }
input { border: 1px solid #b8c8dc; }
button { border: 0; background: #175cd3; color: white; cursor: pointer; }
#result { min-height: 28px; font-weight: 600; }
```

app.js

```
'use strict';
const nameInput = document.querySelector('#name');
const greetButton = document.querySelector('#greet');
const result = document.querySelector('#result');

function greeting(name) {
  const cleanName = name.trim();
  return cleanName ? `স্বাগতম, ${cleanName}! JavaScript শেখা শুরু হোক।` : 'অনুগ্রহ করে তোমার নাম লিখো।';
}

greetButton.addEventListener('click', () => {
  result.textContent = greeting(nameInput.value);
});
nameInput.addEventListener('keydown', event => {
  if (event.key === 'Enter') greetButton.click();
});
```

Run ও Test

`index.html` Browser-এ Open করো। নাম লিখে Button Click করো; ফাঁকা Input দিয়ে Validation পরীক্ষা করো; Enter চাপলেও Greeting দেখাবে। কোনো External Dependency প্রয়োজন নেই।

১.৮ Exercises

১. Greeting-এর সঙ্গে বর্তমান সময় অনুযায়ী Morning/Afternoon/Evening Message যোগ করো।

২. Input ফাঁকা থাকলে Error Message দেখাও।

৩. Button Click Count দেখাও।

৪. Greeting Text-এর Color পরিবর্তন করার Button যোগ করো।

৫. `console.log()` এবং `textContent`-এর ব্যবহারিক পার্থক্য ব্যাখ্যা করো।

Suggested Solutions

১: `new Date().getHours()` থেকে Hour নিয়ে Conditional Greeting তৈরি করো। **২:** `name.trim().length === 0` যাচাই করো। **৩:** `let clickCount = 0` দিয়ে Event Handler-এ Increment করো। **৪:** `classList.toggle('highlight')` ব্যবহার করো। **৫:** Console Developer Output; `textContent` Page-এর Visible DOM Text Update করে।

১.৯ Knowledge Check

Q1. HTML-এর প্রধান কাজ কী? A) Data Query B) Structure C) Deployment D) Encryption। **উত্তর: B**।

Q2. `const` **কী বন্ধ করে?** A) Reassignment B) All Mutation C) All Functions D) DOM Events। **উত্তর: A**।

Q3. Safe Plain Text DOM Update কোনটি? A) `eval` B) `innerHTML` C) `textContent` D) `document.write`। **উত্তর: C**।

Q4. Button Click শোনার জন্য কোন API? A) `addEventListener` B) `fetch` C) `JSON.parse` D) `setInterval`। **উত্তর: A**।

Q5. JavaScript Server-Side-এ চালাতে কোন Runtime ব্যবহৃত হতে পারে? A) CSS B) Node.js C) HTML D) PNG। **উত্তর: B**।

১.১০ অধ্যায়ের সারসংক্ষেপ

আমরা JavaScript-এর উদ্দেশ্য, Browser Console, Variables, Functions, DOM এবং Event-Driven UI-এর প্রথম Practical Project শিখেছি। পরবর্তী অধ্যায়ে Data Types, Variable Scope এবং Memory Model গভীরভাবে আলোচনা করব।

অধ্যায় ২ — Variables, Data Types এবং Memory

অধ্যায়ের লক্ষ্য

এই অধ্যায় শেষে তুমি `let`, `const`, `var`, JavaScript-এর Primitive ও Reference Values, `typeof`, Type Conversion, Truthy/Falsy, Scope, `null`/`undefined`, Memory Reference এবং সাধারণ Mutation Bugs বুঝতে পারবে। শেষে সম্পূর্ণ **Student Result Calculator** তৈরি করবে।

২.১ Variable আসলে কী?

Variable হলো একটি নামযুক্ত Binding, যার মাধ্যমে Program কোনো Value ব্যবহার করে। JavaScript-এর Variable-কে শুধু "Memory Box" ভাবলে কিছু ক্ষেত্রে ভুল ধারণা তৈরি হয়; Object-এর ক্ষেত্রে Variable Object-এর Reference ধরে রাখতে পারে।

```
let seats = 12;
seats = 10; // reassignment allowed
const destination = "Dhaka";
// destination = "Sylhet"; // TypeError
```

Naming: `camelCase` ব্যবহার করো; `totalPrice`, `studentName` অর্থবহ। নামের শুরুতে সংখ্যা রাখা যায় না, তবে `_` ও `$` অনুমোদিত। Reserved Keyword (`const`, `return`) নাম হতে পারে না।

২.২ `let`, `const`, `var` — পার্থক্য

বৈশিষ্ট্য	`let`	`const`	`var`
Scope	Block	Block	Function / Global
Reassignment	হ্যাঁ	না	হ্যাঁ
Same scope-এ redeclare	না	না	হ্যাঁ
Initialization বাধ্যতামূলক	না	হ্যাঁ	না
Hoisting behavior	TDZ	TDZ	`undefined` initialization

```
let count = 1;
if (true) {
  let count = 2; // different block-scoped binding
  console.log(count); // 2
}
console.log(count); // 1
```

Recommended Practice: Default হিসেবে `const`; Value পুনরায় Assign করতে হলে `let`; নতুন Code-এ `var` এড়িয়ে চলো। `const` মানে Object Deeply Immutable নয়।

```
const profile = {name: "Rafi"};
profile.name = "Rafi Ahmed"; // allowed
// profile = {}; // TypeError
```

২.৩ JavaScript Data Types

JavaScript-এর সাতটি Primitive Type: `string`, `number`, `bigint`, `boolean`, `undefined`, `symbol`, `null`; আর Objects-এর মধ্যে Array, Function, Date ইত্যাদি পড়ে। `null`-এর `typeof` ফল `"object"` একটি ঐতিহাসিক Language Quirk।

```
const city = "Dhaka";                // string
const fare = 65000;                  // number
const largeId = 90071992547409 93n; // bigint
const available = true;              // boolean
let coupon;                          // undefined
const missing = null;                // null
const key = Symbol("key");          // symbol
const student = {name: "Mina"};     // object
console.log(typeof missing);         // "object" (historical quirk)
console.log(Array.isArray([1, 2])); // true
```

`number` এবং Precision

JavaScript-এর সাধারণ `number` IEEE-754 Double-Precision Floating-Point ব্যবহার করে। তাই:

```
console.log(0.1 + 0.2); // 0.30000000000000004
console.log(Number.isNaN(Number("abc"))); // true
console.log(Number.isFinite(Infinity)); // false
```

Money Calculation-এ Minor Unit (যেমন পয়সা) বা উপযুক্ত Decimal Library বিবেচনা করা ভালো। `BigInt` বড় Integer-এর জন্য, কিন্তু `number`-এর সঙ্গে সরাসরি Arithmetic করা যায় না।

২.৪ `undefined`, `null`, `NaN`

- `undefined`: Value Assign করা হয়নি বা Property নেই।
- `null`: ইচ্ছাকৃতভাবে "কোনো Value নেই" বোঝাতে ব্যবহৃত।
- `NaN`: Invalid Numeric Result; `typeof NaN` হলো `"number"`।

```
let email;
const middleName = null;
const badNumber = Number("hello");
console.log(email === undefined); // true
console.log(middleName === null); // true
console.log(Number.isNaN(badNumber)); // true
```

২.৫ Type Conversion বনাম Coercion

Explicit Conversion-এ Programmer ইচ্ছাকৃতভাবে Type বদলায়; **Implicit Coercion**-এ Operator বা Context পরিবর্তন ঘটায়।

```
Number("42");       // 42
String(42);         // "42"
Boolean(0);         // false
"5" + 2;            // "52" (concatenation)
"5" - 2;            // 3 (numeric coercion)
Number("");         // 0 — input validation-এ বিপজ্জনক!
Number("abc");      // NaN
```

Form Input থেকে সব Value সাধারণত String হিসেবে আসে। `Number("")` হলো `0`, তাই Empty Input আগে আলাদাভাবে Reject করতে হবে।

২.৬ Truthy এবং Falsy

`false`, `0`, `-0`, `0n`, `""`, `null`, `undefined`, `NaN` হলো Falsy। অন্য সব Object (খালি Array বা Object-সহ) Truthy।

```
console.log(Boolean([]));  // true
console.log(Boolean({}));  // true
console.log(Boolean("0")); // true
console.log(Boolean(0));   // false
```

Common Mistake: `if (marks)` ব্যবহার করলে `0` নম্বরকে Missing ধরে নিতে পারো। বরং `marks === ""` দিয়ে Empty Field পরীক্ষা করো।

২.৭ Scope, Hoisting এবং Temporal Dead Zone

`let` ও `const` Block-Scoped। Declaration-এর আগে একই Scope-এ Access করলে `ReferenceError` হয়—এই অবস্থাকে Temporal Dead Zone (TDZ) বলা হয়।

```
function demo() {
  // console.log(score); // ReferenceError
  const score = 95;
  if (score > 80) {
    const message = "Excellent";
    console.log(message);
  }
  // console.log(message); // ReferenceError
}
```

`var` Function-Scoped এবং Declaration-এর আগে Access করলে অনেক ক্ষেত্রে `undefined` পাওয়া যায়; তাই পুরোনো Code-এ সূক্ষ্ম Bug তৈরি হতে পারে।

২.৮ Primitive Value এবং Object Reference

Primitive Assignment সাধারণত Value Copy-এর মতো আচরণ করে। Object Assignment-এ দুই Binding একই Object-কে Refer করতে পারে।

```
let a = 5;
let b = a;
b = 9;
console.log(a); // 5

const first = {name: "Nila"};
const second = first;
second.name = "Nila Islam";
console.log(first.name); // "Nila Islam"
```

Shallow Copy তৈরি করতে Spread Syntax ব্যবহার করা যায়, কিন্তু Nested Object Shared থাকতে পারে:

```
const original = {name: "A", address: {city: "Dhaka"}};
const copy = {...original};
copy.address.city = "Khulna";
console.log(original.address.city); // "Khulna"
```

```
const deepCopy = structuredClone(original); // supported modern runtimes
```

Memory Note: JavaScript Runtime সাধারণত Garbage Collection দিয়ে Unreachable Objects-এর Memory পুনরুদ্ধার করে। `const` Variable-কে Permanent Memory Allocation ভাবা ভুল।

২.৯ `===`, `??` এবং Optional Chaining-এর পরিচয়

```
console.log(0 === "0"); // false
console.log(0 == "0");  // true (coercion)
const preferredSeats = 0;
console.log(preferredSeats ?? 2); // 0
console.log(preferredSeats || 2); // 2
const user = {profile: null};
console.log(user.profile?.email ?? "Not provided");
```

সাধারণত `===` ও `!==` ব্যবহার করো। `??` শুধু `null` বা `undefined` হলে Fallback নেয়; `||` সব Falsy Value-তে Fallback নেয়।

২.১০ Practical Project — Student Result Calculator

Requirements: Student Name, বাংলা, ইংরেজি ও গণিতের নম্বর নিতে হবে। প্রতিটি নম্বর ০–১০০-এর মধ্যে Integer হতে হবে। মোট, গড়, পাস/ফেল ও Grade দেখাতে হবে। কোনো একটি বিষয়ে ৩৩-এর নিচে হলে Overall Fail। Grade Policy এখানে শেখার জন্য উদাহরণ; এটি কোনো নির্দিষ্ট Education Board-এর Official Grading Standard নয়।

Project Files:

```
source-code/chapter-02/
├── index.html
├── style.css
├── calculator.js
├── app.js
└── calculator.test.js
```

Logic Breakdown: `calculateResult(name, marks)` একটি Pure Function; এটি DOM-এর ওপর নির্ভর করে না। `app.js` Form-এর Data নিয়ে Function Call করে। `textContent` দিয়ে Result দেখানো হয় যাতে User Input HTML হিসেবে Execute না হয়।

calculator.js — Core Logic

```
"use strict";
function calculateResult(name, marks) {
  if (typeof name !== "string" || name.trim().length === 0 || name.trim().length > 80) {
    throw new TypeError("শিক্ষার্থীর নাম ১–৮০ অক্ষরের হতে হবে।");
  }
  if (!Array.isArray(marks) || marks.length !== 3 || marks.some(
    (mark) => typeof mark !== "number" || !Number.isInteger(mark) || mark < 0 || mark > 100
  )) {
    throw new RangeError("প্রতিটি বিষয়ে ০–১০০-এর মধ্যে পূর্ণসংখ্যা দাও।");
  }
  const total = marks.reduce((sum, mark) => sum + mark, 0);
  const average = total / marks.length;
  const passed = marks.every((mark) => mark >= 33);
  let grade;
```

```
  if (!passed) grade = "F";
  else if (average >= 80) grade = "A+";
  else if (average >= 70) grade = "A";
  else if (average >= 60) grade = "A-";
  else if (average >= 50) grade = "B";
  else if (average >= 40) grade = "C";
  else if (average >= 33) grade = "D";
  else grade = "F";
  return {name: name.trim(), marks: [...marks], total, average, passed, grade};
}
```

Complete `index.html`, `style.css`, `app.js` ও Test Code একই Package-এর `source-code/chapter-02/` Folder-এ রয়েছে। `index.html` Browser-এ খুলে Application চালাও।

Input → Processing → Output

```
Form Input → Empty Check → Number Conversion → Validation
     → Total / Average / Pass → Grade → Safe DOM Output
```

Example: `Sara`, `[80, 90, 70]` → Total `240`, Average `80.00`, Passed `true`, Grade `A+`। `Sara`, `[100, 100, 20]` → Total `220`, কিন্তু Math-এ Fail হওয়ায় Grade `F`।

২.১১ Debugging Checklist

1. Input Empty থাকলে `Number("")` দিয়ে `0` হয়ে যাচ্ছে কি?
2. Grade-এর আগে সব বিষয়ে Pass Condition যাচাই হচ্ছে কি?
3. `NaN`, Negative বা `101` Reject হচ্ছে কি?
4. User Name Output-এ `textContent` ব্যবহার করা হচ্ছে কি?
5. `const` Object-এর Nested Property পরিবর্তন হলে কেন হচ্ছে বুঝতে পারছ কি?

২.১২ Practice Exercises

Exercise 1: `const` ও `let`-এর মধ্যে Reassignment-এর পার্থক্য Code দিয়ে দেখাও।

Exercise 2: `"25"`, `25`, `null`, `undefined`, `[]` এবং `NaN`-এর `typeof` বের করো।

Exercise 3: কেন `Number("")` Form Validation-এ Problem তৈরি করে? একটি Guard লেখো।

Exercise 4: `const settings = {theme: {mode: "light"}}`-এর Shallow Copy বানিয়ে Nested Mode বদলাও। Original কেন বদলায় ব্যাখ্যা করো।

Exercise 5: Calculator-এ একটি Subject-এ ৩২ ও অন্য দুটিতে ৯০ থাকলে Final Grade কী হবে? কেন?

Exercise 6 (Challenge): Calculator-এ ৪টি Subject Support করতে Function-টিকে Generalize করো; তবে Empty List Reject করবে।

২.১৩ Exercise Solutions

Solution 1:

```
let points = 10;
points = 20; // works
const maxPoints = 100;
// maxPoints = 200; // TypeError
```

Solution 2: যথাক্রমে `"string"`, `"number"`, `"object"`, `"undefined"`, `"object"`, `"number"`। Array-এর জন্য `Array.isArray()` দরকার।

Solution 3:

```
function parseMark(raw) {
  if (raw.trim() === "") throw new Error("Required");
  const value = Number(raw);
  if (!Number.isInteger(value) || value < 0 || value > 100) throw new Error("Invalid mark");
  return value;
}
```

Solution 4: Spread Copy শুধু Top Level Copy করে; Nested `theme` Object Shared থাকে।

```
const settings = {theme: {mode: "light"}};
const shallow = {...settings};
shallow.theme.mode = "dark";
console.log(settings.theme.mode); // "dark"
```

Solution 5: `F`; কারণ যেকোনো Subject-এ ৩৩-এর কম হলে Fail Policy প্রয়োগ করা হয়েছে।

Solution 6 (One Approach):

```
function summarizeMarks(marks) {
  if (!Array.isArray(marks) || marks.length === 0 ||
      marks.some((n) => !Number.isInteger(n) || n < 0 || n > 100)) {
    throw new RangeError("Invalid marks");
  }
  const total = marks.reduce((sum, n) => sum + n, 0);
  return {total, average: total / marks.length, passed: marks.every((n) => n >= 33)};
}
```

২.১৪ Quick Quiz — ১০টি প্রশ্ন

1. `const`-এর Object Property পরিবর্তন করা যায়? **(A) কখনো না (B) হ্যাঁ (C) শুধু Strict Mode-এ**
2. `typeof null` কী? **(A) null (B) undefined (C) object**
3. `Number("")` কত? **(A) 0 (B) NaN (C) undefined**
4. `Boolean([])` কী? **(A) false (B) true (C) error**
5. `"5" + 2` কী? **(A) 7 (B) "52" (C) NaN**
6. `0 ?? 10` কী? **(A) 0 (B) 10 (C) null**
7. `0 || 10` কী? **(A) 0 (B) 10 (C) false**
8. `Number.isNaN(NaN)` কী? **(A) true (B) false (C) error**
9. `let`-এর Scope সাধারণত কী? **(A) Function only (B) Block (C) Always global**
10. `const x = [1]`-এর পর `x.push(2)` কি সম্ভব? **(A) না (B) হ্যাঁ (C) শুধু Node.js-এ**

Quiz Answer Key ও Explanation

প্রশ্ন	উত্তর	ব্যাখ্যা
1	B	`const` Binding স্থির, Object Contents নয়।
2	C	Historical Quirk।
3	A	Empty String Numeric Conversion-এ Zero।
4	B	Empty Array-ও Truthy Object।
5	B	`+` String Concatenation করে।
6	A	`??` Zero-কে Missing ধরে না।
7	B	`||` Zero-কে Falsy ধরে।
8	A	`Number.isNaN` সঠিক NaN Detection।
9	B	`let` Block Scoped।
10	B	Array Contents Mutable।

২.১৫ অধ্যায়ের সারসংক্ষেপ

Variable Binding, Primitive বনাম Object Reference, Scope, Type Conversion ও Input Validation বুঝলে JavaScript-এর অনেক Common Bug আগেই এড়ানো যায়। এই অধ্যায়ের Calculator Project পরের অধ্যায়ের Operators ও Decision Making-এর ভিত্তি তৈরি করে।

পরবর্তী অধ্যায়: Operators, Expressions এবং Decision Making।

অধ্যায় ৩ — Operators, Expressions এবং Decision Making

শেখার লক্ষ্য

এই অধ্যায়ের শেষে পাঠক JavaScript-এর Arithmetic, Assignment, Comparison ও Logical Operators ব্যবহার করতে পারবে; `if`/`else`, `switch`, Ternary এবং Guard Clause দিয়ে সিদ্ধান্ত নিতে পারবে; Type Coercion-এর ঝুঁকি বুঝবে; এবং একটি **Shopping Discount Calculator** তৈরি করবে।

৩.১ Expression, Statement ও Operator

Expression এমন কোড যা একটি Value তৈরি করে: `5 + 3`, `price > 1000`, `user?.name`। **Statement** কোনো কাজ নির্দেশ করে: `if (...) { ... }`, `const total = ...;`। Operator এক বা একাধিক Operand নিয়ে কাজ করে।

```
const unitPrice = 850;
const quantity = 3;
const subtotal = unitPrice * quantity; // 2550
console.log(subtotal >= 2000); // true
```

`unitPrice * quantity` একটি Expression; `const subtotal = ...` একটি Declaration Statement। Expressions-এর Precedence বোঝা জরুরি; সন্দেহ হলে Parentheses ব্যবহার করো।

৩.২ Arithmetic Operators

Operator	কাজ	উদাহরণ	ফলাফল
`+`	যোগ	`7 + 3`	`10`
`-`	বিয়োগ	`7 - 3`	`4`
`*`	গুণ	`7 * 3`	`21`
`/`	ভাগ	`7 / 2`	`3.5`
`%`	Remainder	`7 % 3`	`1`
`**`	Power	`2 ** 3`	`8`

```
console.log(10 / 0);        // Infinity
console.log(0 / 0);         // NaN
console.log(-7 % 3);        // -1 (remainder; mathematical modulo নয়)
console.log(0.1 + 0.2);     // 0.30000000000000004
```

JavaScript-এর `number` হলো IEEE 754 Double-Precision Floating Point। টাকা-পয়সার হিসাবের ক্ষেত্রে সরাসরি Fractional Decimal যোগ না করে Integer Minor Units (যেমন পয়সা) বা উপযুক্ত Decimal Library ব্যবহার করা ভালো। এই Project-এ BDT-এর পূর্ণসংখ্যা Value ব্যবহার করব।

৩.৩ Assignment, Increment ও Decrement

```
let stock = 10;
stock += 5; // 15
stock -= 2; // 13
stock *= 2; // 26
stock /= 2; // 13
stock++;    // 14
--stock;    // 13
```

`x++` আগে পুরোনো Value দেয়, তারপর বাড়ায়; `++x` আগে বাড়ায়, তারপর নতুন Value দেয়। Complex Expression-এর মধ্যে এদের ব্যবহার করলে Readability কমতে পারে।

৩.৪ Comparison: `===` বনাম `==`

```
console.log(5 === '5');  // false: Type ও Value উভয়ই মিলতে হবে
console.log(5 == '5');   // true: Coercion ঘটে
console.log(5 !== '5');  // true
console.log(10 > 5);     // true
console.log(10 <= 10);   // true
```

সাধারণত `===` ও `!==` ব্যবহার করো। `NaN === NaN` false; `Number.isNaN(value)` দিয়ে যাচাই করো। Object Comparison Reference-Based:

```
console.log({ id: 1 } === { id: 1 }); // false
const a = { id: 1 };
const b = a;
console.log(a === b); // true
```

৩.৫ Logical Operators ও Short-Circuiting

`&&` (AND), `||` (OR), `!` (NOT) Boolean Logic-এ ব্যবহৃত হয়, তবে `&&` ও `||` সবসময় Boolean Return করে না—Operand-এর Value Return করে।

```
const isMember = true;
const total = 3000;
console.log(isMember && total >= 2000); // true

const displayName = '' || 'Guest'; // 'Guest'
const seats = 0 ?? 10;             // 0
const unknownSeats = null ?? 10;   // 10
```

`||` বনাম `??`: `||` যেকোনো Falsy Value (`0`, `''`, `false`) বাদ দেয়; `??` শুধু `null` ও `undefined`-এর ক্ষেত্রে Fallback দেয়। `??`-কে `||` বা `&&`-এর সঙ্গে একই Expression-এ ব্যবহার করলে Parentheses প্রয়োজন।

৩.৬ Operator Precedence এবং Parentheses

```
console.log(2 + 3 * 4);   // 14
console.log((2 + 3) * 4); // 20
const eligible = (age >= 18) && (hasId === true);
```

Precedence মুখস্থ করার চেয়ে Intent পরিষ্কার করে Parentheses ব্যবহার করা ভালো। Assignment-এর সঙ্গে Comparison মিশিয়ে জটিল One-Liner এড়িয়ে চলো।

৩.৭ `if`, `else if`, `else`

```
function shippingMessage(amount) {
  if (!Number.isFinite(amount) || amount < 0) {
    return 'Invalid amount';
  }
  if (amount >= 5000) {
    return 'Free shipping';
  } else if (amount >= 2000) {
    return 'Discounted shipping';
  }
  return 'Standard shipping';
}
```

এখানে শেষের `return` হলো **Guard/Default Flow**। Nested `if` অনেক হলে Function ভেঙে ছোট করা যায়।

৩.৮ `switch` Statement

```
function paymentLabel(method) {
  switch (method) {
    case 'card': return 'Card payment';
    case 'cash': return 'Cash on delivery';
    case 'mobile': return 'Mobile banking';
    default: return 'Unknown payment method';
  }
}
```

`switch` Strict Comparison (`===`-এর মতো) ব্যবহার করে। `break` বা `return` না দিলে পরবর্তী Case-এ Fallthrough হতে পারে। Intentional Fallthrough ছাড়া এটি এড়িয়ে চলো।

৩.৯ Ternary ও Guard Clauses

```
const badge = stock > 0 ? 'In stock' : 'Sold out';

function validateQuantity(quantity) {
  if (!Number.isInteger(quantity) || quantity <= 0) {
    return { ok: false, error: 'Quantity must be a positive integer' };
  }
  return { ok: true };
}
```

Simple Two-Way Decision-এর জন্য Ternary ভালো; জটিল Nested Ternary পড়া কঠিন।

৩.১০ Common Pitfalls

1. `=` হলো Assignment; `===` হলো Comparison।
2. `Number('')` ফল `0`; Blank Input-কে সরাসরি Number করলে ভুল Validation হতে পারে।
3. `parseInt('12abc', 10)` ফল `12`; কঠোর Numeric Validation-এর জন্য এটি যথেষ্ট নয়।
4. `0` Falsy হলেও একটি বৈধ Quantity বা Price হতে পারে।
5. Percentage Discount যোগ করার ক্রম Business Rule অনুযায়ী নির্ধারণ করতে হবে।
6. Client-Side Discount Calculator শুধু UI Preview; বাস্তব Checkout-এ Server-Side Price Recalculation জরুরি।

৩.১১ Practical Project — Shopping Discount Calculator

Business Rules (এই শিক্ষামূলক Project-এর জন্য):

- Item Price: পূর্ণসংখ্যা BDT, `0` বা বেশি।
- Quantity: ১–১০০-এর মধ্যে পূর্ণসংখ্যা।
- Coupon: `SAVE10` দিলে ১০% Discount; `SAVE20` দিলে ২০% Discount, তবে Subtotal কমপক্ষে BDT 5,000 হতে হবে।
- Member Discount: Coupon-এর পরে অবশিষ্ট Amount-এর ওপর ৫%।
- Delivery: Discounted Total BDT 3,000 বা বেশি হলে Free; অন্যথায় BDT 120।
- Percentage Discount প্রতিটি ধাপে নিকটতম পূর্ণ BDT-তে Round করা হবে।
- Unknown Coupon-এ Error দেখানো হবে; Silent Ignore নয়।

Project Structure

```
shopping-discount-calculator/
├── index.html
├── style.css
├── calculator.js
├── app.js
└── calculator.test.js
```

index.html

```
<!doctype html>
<html lang="bn">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Shopping Discount Calculator</title>
  <link rel="stylesheet" href="style.css">
  <script src="calculator.js" defer></script>
  <script src="app.js" defer></script>
</head>
<body>
  <main class="card">
    <h1>Shopping Discount Calculator</h1>
    <form id="checkout-form">
      <label for="price">Unit Price (BDT)</label>
      <input id="price" type="number" min="0" step="1" required value="1500">
      <label for="quantity">Quantity</label>
      <input id="quantity" type="number" min="1" max="100" step="1" required value="2">
      <label for="coupon">Coupon Code (optional)</label>
      <input id="coupon" type="text" maxlength="20" autocomplete="off" placeholder="SAVE10">
      <label class="check"><input id="member" type="checkbox"> Member (extra 5% off)</label>
      <button type="submit">Calculate Total</button>
    </form>
    <section aria-live="polite" id="result" role="status"></section>
  </main>
</body>
</html>
```

style.css

```
* { box-sizing: border-box; }
body { margin: 0; min-height: 100vh; padding: 24px; background: #eef4ff; color: #172a46; font:
16px/1.6 system-ui, sans-serif; display: grid; place-items: center; }
.card { width: min(100%, 520px); padding: 28px; background: white; border: 1px solid #d9e4f5;
border-radius: 16px; box-shadow: 0 10px 35px #17335c12; }
h1 { margin-top: 0; font-size: 1.5rem; }
form { display: grid; gap: 9px; }
input:not([type="checkbox"]) { width: 100%; padding: 11px; border: 1px solid #b9c8db; border-
radius: 8px; font: inherit; }
.check { display: flex; align-items: center; gap: 8px; margin: 9px 0; }
button { background: #155bd5; color: white; border: 0; padding: 12px; border-radius: 8px; font:
inherit; cursor: pointer; }
button:hover { background: #0d48af; }
#result { margin-top: 20px; }
#result.error { color: #a31730; }
.summary { margin: 0; display: grid; grid-template-columns: 1fr auto; gap: 7px 14px; }
.summary dt { font-weight: 500; }
.summary dd { margin: 0; text-align: right; }
.summary .grand { font-weight: 800; border-top: 1px solid #cbd8eb; padding-top: 8px; }
```

calculator.js — Pure Business Logic

```
'use strict';

function calculateOrder({ price, quantity, coupon = '', isMember = false }) {
  if (!Number.isSafeInteger(price) || price < 0) {
    throw new RangeError('Price must be a non-negative integer (BDT).');
  }
  if (!Number.isInteger(quantity) || quantity < 1 || quantity > 100) {
    throw new RangeError('Quantity must be an integer from 1 to 100.');
  }
  if (typeof coupon !== 'string' || typeof isMember !== 'boolean') {
    throw new TypeError('Invalid coupon or membership value.');
  }

  const subtotal = price * quantity;
  if (!Number.isSafeInteger(subtotal)) {
    throw new RangeError('Subtotal exceeds safe integer range.');
  }

  const code = coupon.trim().toUpperCase();
  let couponRate = 0;
  switch (code) {
    case '': break;
    case 'SAVE10': couponRate = 0.10; break;
    case 'SAVE20':
      if (subtotal < 5000) throw new RangeError('SAVE20 requires BDT 5,000 subtotal.');
      couponRate = 0.20;
      break;
    default: throw new RangeError('Unknown coupon code.');
  }

  const couponDiscount = Math.round(subtotal * couponRate);
  const afterCoupon = subtotal - couponDiscount;
  const memberDiscount = isMember ? Math.round(afterCoupon * 0.05) : 0;
  const discountedTotal = afterCoupon - memberDiscount;
  const deliveryFee = discountedTotal >= 3000 ? 0 : 120;
  const grandTotal = discountedTotal + deliveryFee;

  return { subtotal, couponDiscount, memberDiscount, discountedTotal, deliveryFee, grandTotal
};
}

if (typeof module !== 'undefined' && module.exports) {
  module.exports = { calculateOrder };
```

```
}
```

app.js — Form Events ও DOM Update

```
'use strict';

const form = document.querySelector('#checkout-form');
const result = document.querySelector('#result');
const money = value => `BDT ${value.toLocaleString('en-BD')}`;

form.addEventListener('submit', event => {
  event.preventDefault();
  result.replaceChildren();
  result.classList.remove('error');

  const priceText = document.querySelector('#price').value.trim();
  const quantityText = document.querySelector('#quantity').value.trim();
  const price = priceText === '' ? NaN : Number(priceText);
  const quantity = quantityText === '' ? NaN : Number(quantityText);

  try {
    const order = calculateOrder({
      price,
      quantity,
      coupon: document.querySelector('#coupon').value,
      isMember: document.querySelector('#member').checked
    });
    const rows = [
      ['Subtotal', order.subtotal],
      ['Coupon Discount', -order.couponDiscount],
      ['Member Discount', -order.memberDiscount],
      ['Delivery Fee', order.deliveryFee],
      ['Grand Total', order.grandTotal]
    ];
    const summary = document.createElement('dl');
    summary.className = 'summary';
    for (const [label, value] of rows) {
      const dt = document.createElement('dt');
      const dd = document.createElement('dd');
      dt.textContent = label;
      dd.textContent = money(value);
      if (label === 'Grand Total') {
        dt.className = 'grand';
        dd.className = 'grand';
      }
      summary.append(dt, dd);
    }
    result.append(summary);
  } catch (error) {
    result.classList.add('error');
    result.textContent = error.message;
  }
});
```

চালানোর নিয়ম: একই Folder-এ পাঁচটি File রেখে `index.html` Browser-এ Open করো। Calculator Library Pure Function হওয়ায় Browser ও Node.js—দুই জায়গায় Test করা যায়।

Manual Test Scenarios

Price	Qty	Coupon	Member	Expected Grand Total
1500	2	*(none)*	No	3000
1500	2	SAVE10	No	2820

Price	Qty	Coupon	Member	Expected Grand Total
2500	2	SAVE20	Yes	3800
1000	1	*(none)*	No	1120
1000	1	SAVE20	No	Error

Automated Tests

```
'use strict';
const assert = require('node:assert/strict');
const { calculateOrder } = require('./calculator');

assert.equal(calculateOrder({ price: 1500, quantity: 2 }).grandTotal, 3000);
assert.equal(calculateOrder({ price: 1500, quantity: 2, coupon: 'SAVE10' }).grandTotal, 2820);
assert.equal(calculateOrder({ price: 2500, quantity: 2, coupon: 'SAVE20', isMember: true
}).grandTotal, 3800);
assert.equal(calculateOrder({ price: 1000, quantity: 1 }).grandTotal, 1120);
assert.throws(() => calculateOrder({ price: 1000, quantity: 1, coupon: 'SAVE20' }),
/requires/);
assert.throws(() => calculateOrder({ price: -1, quantity: 1 }), /Price/);
assert.throws(() => calculateOrder({ price: 1, quantity: 0 }), /Quantity/);
assert.throws(() => calculateOrder({ price: 1, quantity: 1, coupon: 'UNKNOWN' }), /Unknown/);
console.log('8 tests passed');
```

Terminal-এ `node calculator.test.js` চালাও। Browser UI-এর Automated E2E Test এই অধ্যায়ে অন্তর্ভুক্ত নয়।

৩.১২ অনুশীলনী

1. `7 % 3` এবং `-7 % 3`-এর ফল ব্যাখ্যা করো।
2. `0 || 50` ও `0 ?? 50`-এর ফল কী? কেন?
3. `if/else` দিয়ে Exam Grade নির্ধারণ করো: 80+ A, 60+ B, 40+ C, নইলে F। Invalid Score Reject করো।
4. `switch` দিয়ে `pending`, `paid`, `cancelled` Order Status-এর Label Return করো।
5. Project-এ `SAVE15` Coupon যোগ করো: Subtotal কমপক্ষে BDT 2,000 হলে ১৫% Discount।
6. `calculateOrder`-এ 0 Price এবং Quantity 1-এর জন্য Test লেখো।
7. Challenge: Discount Policy আলাদা Configuration Object-এ সরিয়ে Business Logic আরও Maintainable করো।

৩.১৩ অনুশীলনীর সমাধান

১. `7 % 3` → `1`; `-7 % 3` → `-1`। JavaScript `%` Remainder Operator।

২. `0 || 50` → `50` কারণ `0` Falsy; `0 ?? 50` → `0` কারণ `0` Nullish নয়।

৩.

```
function grade(score) {
```

```
  if (!Number.isFinite(score) || score < 0 || score > 100) return 'Invalid';
  if (score >= 80) return 'A';
  if (score >= 60) return 'B';
  if (score >= 40) return 'C';
  return 'F';
}
```

৪.

```
function statusLabel(status) {
  switch (status) {
    case 'pending': return 'Awaiting payment';
    case 'paid': return 'Payment complete';
    case 'cancelled': return 'Order cancelled';
    default: return 'Unknown status';
  }
}
```

৫. `switch`-এ `case 'SAVE15': if (subtotal < 2000) throw new RangeError(...); couponRate = 0.15; break;` যোগ করতে হবে।

৬. `assert.equal(calculateOrder({price: 0, quantity: 1}).grandTotal, 120);`।

৭. `const couponPolicies = { SAVE10: {rate: 0.1, min: 0}, SAVE20: {rate: 0.2, min: 5000} };` রেখে Lookup ও Validation আলাদা Function-এ নেওয়া যায়। Object-এর Key Lookup করার আগে `Object.hasOwn()` দিয়ে Membership Check করা উচিত।

৩.১৪ MCQ Quiz

1. Strict Equality Operator কোনটি? A. `=` B. `==` C. `===` D. `=>`
2. `2 + 3 * 4` কত? A. 20 B. 14 C. 24 D. 9
3. `0 ?? 10` কত? A. 10 B. null C. 0 D. undefined
4. `5 === '5'` কী? A. true B. false C. 5 D. Error
5. `10 % 4` কত? A. 2 B. 2.5 C. 4 D. 0
6. কোনটি Logical AND? A. `||` B. `!` C. `&&` D. `??`
7. `switch`-এ Default Branch কোনটি? A. `else` B. `default` C. `otherwise` D. `fallback`
8. `Number.isNaN(NaN)` কী? A. true B. false C. 0 D. Error
9. Ternary Syntax কোনটি? A. `a ? b : c` B. `a : b ? c` C. `if ? else` D. `a ?? b : c`
10. Client-Side Price Calculation কেন Final Authority নয়? A. Slow B. User এটি পরিবর্তন করতে পারে C. JS Math পারে না D. HTML নেই

Answer Key ও ব্যাখ্যা: 1-C (Type ও Value দুটোই তুলনা), 2-B (Multiplication আগে), 3-C (0 Nullish নয়), 4-B (Type আলাদা), 5-A (Remainder 2), 6-C (AND), 7-B (Unmatched Case), 8-A (`NaN` সনাক্ত), 9-A (Condition ? True : False), 10-B (Client Input ও Code Manipulable)।

৩.১৫ Chapter Review Checklist

- ☐ Arithmetic ও Comparison Operators বুঝেছি
- ☐ `===` বনাম `==` ব্যাখ্যা করতে পারি
- ☐ `||` বনাম `??` সঠিকভাবে ব্যবহার করতে পারি
- ☐ `if/else`, `switch`, Ternary দিয়ে Decision নিতে পারি
- ☐ Shopping Discount Calculator চালিয়েছি
- ☐ Automated Tests চালিয়েছি
- ☐ Exercises ও Quiz শেষ করেছি

পরবর্তী অধ্যায়: Chapter 4 — Loops এবং Iteration।

অধ্যায় ৪ — Loops এবং Iteration: Repeated Work থেকে Cart Analyzer

শেখার লক্ষ্য

এই অধ্যায়ের শেষে পাঠক `for`, `while`, `do...while`, `for...of`, `for...in` এবং Array Iteration বুঝতে পারবে; `break`, `continue`, Nested Loop, Complexity ও Infinite Loop-এর ঝুঁকি চিনবে; এবং একটি বাস্তব **Cart Analyzer** বানাবে।

৪.১ Loop কেন দরকার?

ধরা যাক, একটি অনলাইন দোকানে ১০০টি পণ্য আছে। প্রতিটির দাম আলাদাভাবে যোগ করা অযৌক্তিক। Loop একই নিয়ম একাধিক Data Item-এর ওপর প্রয়োগ করতে সাহায্য করে।

```
const prices = [120, 250, 90];
let total = 0;
for (let i = 0; i < prices.length; i++) {
  total += prices[i];
}
console.log(total); // 460
```

এখানে `i = 0` হলো Initialization, `i < prices.length` হলো Condition এবং `i++` হলো Update। Condition মিথ্যা হলে Loop থামে। **Off-by-one Error** এড়াতে Index-এর ক্ষেত্রে `< array.length` ব্যবহার করো; `<=` নয়।

৪.২ `for` Loop: নির্দিষ্ট Range

```
for (let n = 1; n <= 5; n++) console.log(n);

// শুধু জোড় সংখ্যা
for (let n = 2; n <= 10; n += 2) console.log(n);

// Reverse countdown
for (let n = 5; n >= 1; n--) console.log(n);
```

কোনো Collection-এর Index দরকার হলে `for` উপযুক্ত। কিন্তু শুধু Value প্রয়োজন হলে `for...of` সাধারণত বেশি পাঠযোগ্য।

৪.৩ `while` এবং `do...while`

`while` আগে Condition পরীক্ষা করে। `do...while` অন্তত একবার Body চালায়।

```
let attempts = 0;
while (attempts < 3) {
  attempts++;
  console.log(`Attempt ${attempts}`);
}

let answer;
```

```
do {
  answer = "yes"; // বাস্তবে Input থেকে আসতে পারে
  console.log("Prompt shown once");
} while (answer !== "yes");
```

সতর্কতা: Condition পরিবর্তিত না হলে Infinite Loop হতে পারে। Browser Main Thread-এ এমন Loop UI আটকে দিতে পারে।

৪.৪ `break` ও `continue`

`break` পুরো Loop বন্ধ করে; `continue` বর্তমান Iteration বাদ দিয়ে পরের Iteration-এ যায়।

```
const values = [10, -1, 0, 25, 40];
for (const value of values) {
  if (value < 0) continue; // Negative উপেক্ষা
  if (value === 0) break;  // Sentinel পেলে থামো
  console.log(value);      // শুধু 10
}
```

এগুলো বেশি ব্যবহার করলে Control Flow জটিল হতে পারে। প্রয়োজনে আলাদা Function বানাও।

৪.৫ `for...of` বনাম `for...in`

`for...of` Iterable-এর **Value** দেয়। `for...in` Object-এর Enumerable **Property Keys** দেয়; Array Iteration-এর জন্য সাধারণত ব্যবহার করা উচিত নয়।

```
const destinations = ["Dhaka", "Sylhet", "Cox's Bazar"];
for (const destination of destinations) console.log(destination);

const product = { name: "Notebook", price: 180 };
for (const key in product) {
  if (Object.hasOwn(product, key)) console.log(key, product[key]);
}
```

`Object.keys()`, `Object.values()`, `Object.entries()` ব্যবহার করলে Object-এর Own Properties নিয়ে কাজ আরও পরিষ্কার হয়।

```
for (const [key, value] of Object.entries(product)) {
  console.log(`${key}: ${value}`);
}
```

৪.৬ Nested Loops ও Complexity

একটি Loop-এর ভিতরে আরেকটি Loop হলে মোট কাজ বাড়ে।

```
for (let row = 1; row <= 3; row++) {
  let line = "";
  for (let col = 1; col <= 4; col++) line += "* ";
  console.log(line);
}
```

উপরের Code মোট ১২ বার Inner Body চালায়। Input Size `n` হলে একটি সাধারণ Linear Loop **O(n)**; দুটি Independent Full Nested Loop প্রায় **$O(n^2)$**। Complexity একটি Growth Model, নির্দিষ্ট Runtime নয়।

Pattern	আনুমানিক Time Complexity	উদাহরণ
একটি স্থির কাজ	O(1)	Array Length পড়া
একবার সব Item দেখা	O(n)	Cart Total
দুটি পূর্ণ Nested Loop	$O(n^2)$	সব Pair তুলনা
Input প্রতি ধাপে অর্ধেক	O(log n)	Binary Search (Sorted Data)

৪.৭ Array Methods-ও Iteration করে

```
const numbers = [2, 4, 6];
console.log(numbers.map(n => n * 2));       // [4, 8, 12]
console.log(numbers.filter(n => n > 3));    // [4, 6]
console.log(numbers.reduce((s, n) => s + n, 0)); // 12
```

`map` নতুন Array তৈরি করে; `filter` বাছাই করে; `reduce` Accumulator তৈরি করে। কোনটি ব্যবহার করবে তা কাজের উদ্দেশ্য ও Readability-এর ওপর নির্ভর করে। `forEach` থেকে `break` করা যায় না; Early Exit দরকার হলে `for...of` বা `some`/`find` বিবেচনা করো।

৪.৮ Practical Project — Cart Analyzer

সমস্যা: একটি Cart-এ Product Name, Unit Price এবং Quantity আছে। আমাদের দরকার Line Subtotal, Total BDT, মোট Unit, সবচেয়ে বড় Line Amount এবং একটি Threshold-এর বেশি Subtotal-যুক্ত পণ্যের তালিকা।

Project Files:

```
source-code/chapter-04/
├── index.html
├── style.css
├── cart.js
├── app.js
└── cart.test.js
```

Data Contract: `[{ name: "Notebook", price: 120, quantity: 3 }]`। এখানে Price পূর্ণসংখ্যা BDT। Production Finance Application-এ Minor Units বা Decimal Strategy আলাদাভাবে নির্ধারণ করতে হবে।

Core Algorithm (`cart.js`)

```
function analyzeCart(items, threshold = 5000) {
  validateCart(items);
  let total = 0;
  let units = 0;
  let highest = null;
  const rows = [];
  for (let i = 0; i < items.length; i++) {
    const item = items[i];
    const subtotal = item.price * item.quantity;
    if (!Number.isSafeInteger(subtotal) || !Number.isSafeInteger(total + subtotal)) {
      throw new RangeError("Amount exceeds safe integer range");
    }
    total += subtotal;
    units += item.quantity;
    const row = { name: item.name.trim(), price: item.price, quantity: item.quantity, subtotal
};
```

```
    rows.push(row);
    if (highest === null || subtotal > highest.subtotal) highest = row;
  }
  const qualifying = [];
  for (const row of rows) if (row.subtotal >= threshold) qualifying.push(row.name);
  return { rows, total, units, distinctItems: rows.length, highest, qualifying };
}
```

উপরের Snippet-এ `validateCart` একটি আলাদা Function। **সম্পূর্ণ Runnable Implementation** `source-code/chapter-04/cart.js`-এ আছে। এটি Type Validation, Quantity Range এবং Integer Overflow পরীক্ষা করে। `app.js` DOM Events নিয়ন্ত্রণ করে, `index.html` UI তৈরি করে, আর `style.css` Layout দেয়।

Browser-এ চালানোর নিয়ম: একই Folder-এর `index.html` খুলে Product যোগ করো। নতুন Item যোগ হলে Summary ও List Update হবে। Input-এর Error Message UI-তে দেখাবে। `textContent` ব্যবহার করা হয়েছে, যাতে User-Provided Name-কে HTML হিসেবে Execute করা না হয়।

Node.js Test:

```
cd source-code/chapter-04
node --test cart.test.js
```

Dry Run

Product	Price	Quantity	Subtotal
Pen	50	2	100
Book	300	3	900
Total		**5 units**	**1000 BDT**

`highest.name` হবে `"Book"`। Threshold `500` হলে `qualifying` হবে `["Book"]`। একটি Pass-এ Main Summary তৈরির Time Complexity O(n); Extra Rows রাখার Space Complexity O(n)।

৪.৯ Common Mistakes এবং Debugging

1. `i <= array.length` ব্যবহার করলে শেষ Iteration-এ `undefined` আসতে পারে।
2. `while`-এ Counter Update না করলে Infinite Loop হয়।
3. `for...in` দিয়ে Array ঘুরলে Key String হয় এবং Inherited Enumerable Property-ও আসতে পারে।
4. `forEach(async () => ...)` ব্যবহার করে বাইরের Code Automatically Await করে না। Sequential Async কাজের জন্য `for...of` + `await` ব্যবহার করা যায়।
5. User Input-এর Number Conversion ছাড়া `+` ব্যবহার করলে String Concatenation হতে পারে।
6. Nested Loop-এ একই Counter Variable Reuse করলে Logic নষ্ট হতে পারে।

৪.১০ Exercises

Exercise 1: `for` ব্যবহার করে ১–১০-এর যোগফল বের করো।

Exercise 2: `while` দিয়ে ৫ থেকে ১ পর্যন্ত Countdown লেখো।

Exercise 3: `[3, 8, 11, 16, 19]` থেকে শুধু জোড় সংখ্যা Print করো।

Exercise 4: একটি Array-তে প্রথম `0` পাওয়ার আগে পর্যন্ত Positive Values যোগ করো।

Exercise 5: `[{name:"A",price:10,quantity:2}, {name:"B",price:15,quantity:3}]`-এর Total বের করো।

Exercise 6: Nested Loop দিয়ে ৩×৩ Multiplication Table তৈরি করো।

Exercise 7 (Challenge): Cart Analyzer-এ ১০% Discount যোগ করো, কিন্তু ১০০০ BDT-এর কম Total হলে Discount নয়। Integer Amount নিয়ে Rounding Rule লিখে দাও।

Exercise Solutions

```
// 1
let sum = 0;
for (let i = 1; i <= 10; i++) sum += i;
console.log(sum); // 55

// 2
let n = 5;
while (n >= 1) console.log(n--);

// 3
for (const value of [3, 8, 11, 16, 19]) {
  if (value % 2 === 0) console.log(value); // 8, 16
}

// 4
let result = 0;
for (const value of [5, 8, 0, 20]) {
  if (value === 0) break;
  if (value > 0) result += value;
}
console.log(result); // 13

// 5
const cart = [{ name: "A", price: 10, quantity: 2 }, { name: "B", price: 15, quantity: 3 }];
let cartTotal = 0;
for (const item of cart) cartTotal += item.price * item.quantity;
console.log(cartTotal); // 65

// 6
for (let r = 1; r <= 3; r++) {
  let line = "";
  for (let c = 1; c <= 3; c++) line += `${r * c} `;
  console.log(line.trim());
}

// 7 — Whole BDT, discount rounded to nearest BDT
function discountedTotal(total) {
  if (!Number.isSafeInteger(total) || total < 0) throw new TypeError("Invalid total");
  if (total < 1000) return total;
  return total - Math.round(total * 0.1);
}
console.log(discountedTotal(1250)); // 1125
```

৪.১১ Knowledge Check — ১০টি MCQ

1. `for` Loop-এর Update অংশ সাধারণত কখন চলে? (A) Body-এর আগে (B) Body-এর পরে (C) একবারই (D) কখনো না
2. `do...while` সর্বনিম্ন কতবার Body চালায়? (A) 0 (B) 1 (C) 2 (D) অসীম
3. `break` কী করে? (A) Loop বন্ধ করে (B) শুধু একটি Iteration Skip (C) Function বানায় (D) Error দেয়
4. `continue` কী করে? (A) Loop বন্ধ (B) বর্তমান Iteration Skip (C) Array Sort (D) Return
5. Array Value পেতে সাধারণত কোনটি? (A) `for...in` (B) `for...of` (C) `switch` (D) `typeof`
6. `i < arr.length` কেন গুরুত্বপূর্ণ? (A) Memory Delete (B) Valid Index Range (C) Sort (D) Encrypt
7. একটি Linear Scan-এর Time Complexity সাধারণত কী? (A) O(1) (B) O(log n) (C) O(n) (D) $O(n^3)$
8. দুটি পূর্ণ Nested Loop সাধারণত কী Complexity? (A) $O(n^2)$ (B) O(1) (C) O(log n) (D) O(n!)
9. `map()` সাধারণত কী Return করে? (A) নতুন Array (B) Boolean (C) HTML (D) Promise সবসময়
10. `forEach`-এর Callback থেকে `break` ব্যবহার করা যায়? (A) হ্যাঁ (B) না (C) শুধু Browser-এ (D) শুধু Node-এ

Answer Key ও Explanation: 1-B (Body শেষে Update), 2-B (Condition পরে পরীক্ষা), 3-A (Loop Exit), 4-B (পরের Iteration), 5-B (Iterable Values), 6-B (শেষ Index হলো length−1), 7-C (প্রতি Item একবার), 8-A (n×n), 9-A (Mapped Array), 10-B (`forEach` Loop Statement নয়)।

৪.১২ অধ্যায়ের সারসংক্ষেপ

Loop হলো Repeated Data Processing-এর ভিত্তি। সঠিক Loop বেছে নেওয়া, Termination Condition ঠিক রাখা, Input Validation এবং Complexity বোঝা—সবই Production-Ready JavaScript-এর গুরুত্বপূর্ণ দক্ষতা। পরের অধ্যায় **Functions**-এ আমরা এই Repeated Logic-কে Reusable Units-এ ভাগ করব।

অধ্যায় ৫ — Functions: Reusable Code, Parameters, Return Values এবং Scope

শেখার লক্ষ্য

এই অধ্যায় শেষে তুমি (১) Function Declaration, Expression ও Arrow Function লিখতে, (২) Parameter এবং Argument-এর পার্থক্য বুঝতে, (৩) Return Value দিয়ে ছোট ছোট Function যুক্ত করতে, (৪) Scope, Closure, Hoisting ও Recursion ব্যাখ্যা করতে, (৫) Pure Function ও Side Effect আলাদা করতে, (৬) একটি বাস্তব **Invoice Calculator** তৈরি ও Test করতে পারবে।

৫.১ Function কেন প্রয়োজন?

ধরো, একটি অনলাইন দোকানে তিন জায়গায় মোট দাম হিসাব করতে হচ্ছে। একই Calculation বারবার লিখলে একটি Bug ঠিক করার জন্য তিনটি জায়গায় পরিবর্তন করতে হবে। Function একটি নির্দিষ্ট কাজকে নাম দিয়ে পুনরায় ব্যবহারযোগ্য করে।

```
function calculateLineTotal(price, quantity) {
  return price * quantity;
}
console.log(calculateLineTotal(120, 3)); // 360
console.log(calculateLineTotal(75, 2));  // 150
```

এখানে `calculateLineTotal` হলো Function Name; `price` ও `quantity` হলো Parameters; `120` ও `3` হলো Arguments; `return` Function-এর Result ফেরত দেয়।

মনে রাখবে: `console.log()` Output দেখায়, কিন্তু `return` Caller-কে Value দেয়। এগুলো একই কাজ করে না।

```
function showTotal(a, b) {
  console.log(a + b);
}
const result = showTotal(4, 6); // Console: 10
console.log(result);            // undefined
```

৫.২ Function Declaration, Expression ও Arrow Function

```
// Declaration
function add(a, b) { return a + b; }

// Expression
const subtract = function (a, b) { return a - b; };

// Arrow
const multiply = (a, b) => a * b;

console.log(add(2, 3));      // 5
console.log(subtract(8, 3)); // 5
console.log(multiply(4, 5)); // 20
```

Function Declaration সাধারণত Declaration-এর আগেও Call করা যায়, কারণ এটি Hoisted হয়। `const`-এ রাখা Function Expression বা Arrow Function তার Initialization-এর আগে ব্যবহার করা যায় না।

```
sayHello(); // works
function sayHello() { console.log("Hello!"); }

// greet(); // ReferenceError: initialization-এর আগে access
const greet = () => console.log("Hi!");
```

Arrow Function-এর `this`: Arrow Function নিজের `this` Binding তৈরি করে না; চারপাশের Lexical Scope-এর `this` ব্যবহার করে। Object Method-এ `this` প্রয়োজন হলে সাধারণ Method Syntax অধিক উপযুক্ত।

```
const account = {
  owner: "Rafi",
  describe() { return `Account: ${this.owner}`; }
};
console.log(account.describe());
```

৫.৩ Parameters, Default Values এবং Rest Parameters

```
function greet(name = "Guest", prefix = "Welcome") {
  return `${prefix}, ${name}!`;
}
console.log(greet());
console.log(greet("Nila", "Hello"));
```

Default Parameter তখন কার্যকর হয় যখন Argument দেওয়া হয়নি অথবা Argument `undefined`। `null` পাঠালে Default বসে না।

```
function sumAll(...numbers) {
  return numbers.reduce((sum, value) => sum + value, 0);
}
console.log(sumAll(10, 20, 30)); // 60
```

`...numbers` Rest Parameter: একাধিক Argument-কে একটি Array-তে সংগ্রহ করে। বিপরীতে Function Call-এর সময় `...array` ব্যবহার করলে সেটি Spread Syntax।

```
const values = [5, 8, 12];
console.log(sumAll(...values)); // 25
```

৫.৪ Return, Early Return এবং Guard Clause

```
function getShippingFee(total) {
  if (!Number.isFinite(total) || total < 0) {
    throw new TypeError("Invalid total");
  }
  if (total >= 2000) return 0;
  return 120;
}
```

Guard Clause অগ্রহণযোগ্য Input শুরুতেই Reject করে; Nested `if` কমিয়ে Code পড়া সহজ করে। `return` Execute হলে Function-এর পরের Statement আর চলে না।

৫.৫ Scope: Global, Function ও Block

```
const shopName = "JS Store"; // module/global-level binding
function createReceipt() {
  const receiptId = "R-101"; // function scope
  if (true) {
    const message = `${shopName}: ${receiptId}`; // block scope
    console.log(message);
  }
  // console.log(message); // ReferenceError
}
createReceipt();
```

`let` এবং `const` Block Scoped; `var` Function Scoped। Browser Script-এর Top-Level `var`-এর Global Object-এর সঙ্গে সম্পর্ক এবং Module-এর Scope ভিন্ন হতে পারে। নতুন Code-এ সাধারণত `const` ও প্রয়োজনমতো `let` ব্যবহার করবে।

Shadowing

```
const rate = 5;
function printRate() {
  const rate = 10;
  console.log(rate); // 10
}
printRate();
console.log(rate); // 5
```

Inner Binding একই নামের Outer Binding-কে ওই Scope-এ Shadow করে। অতিরিক্ত Shadowing বিভ্রান্তি সৃষ্টি করতে পারে।

৫.৬ Closure: Function কীভাবে State মনে রাখে?

Closure মানে Function তার তৈরি হওয়ার সময়কার Lexical Environment-এর Binding-এ Access ধরে রাখতে পারে—Outer Function Return করার পরও।

```
function createCounter(start = 0) {
  let count = start;
  return function increment() {
    count += 1;
    return count;
  };
}
const counterA = createCounter();
console.log(counterA()); // 1
console.log(counterA()); // 2
const counterB = createCounter(10);
console.log(counterB()); // 11
```

প্রতিটি `createCounter()` Call আলাদা State তৈরি করে। Closure ব্যবহার হয় Event Handler, Configuration, Factory Function, Memoization এবং Encapsulation-এ। তবে বড় Data অপ্রয়োজনে Capture করলে Memory Retention হতে পারে।

৫.৭ Callback ও Higher-Order Function

Function JavaScript-এ First-Class Value; Variable-এ রাখা, Argument হিসেবে পাঠানো এবং Return করা যায়।

```
function applyOperation(a, b, operation) {
  return operation(a, b);
}
const result = applyOperation(10, 5, (x, y) => x - y);
console.log(result); // 5
```

যে Function অন্য Function নেয় বা Return করে তাকে Higher-Order Function বলে। Array-এর `map`, `filter`, `reduce`-এ এটি প্রচুর ব্যবহৃত হয়।

```
const prices = [100, 250, 500];
const discounted = prices.map(price => price * 0.9);
console.log(discounted); // [90, 225, 450]
```

৫.৮ Recursion: Function নিজেকেই Call করে

```
function factorial(n) {
  if (!Number.isSafeInteger(n) || n < 0 || n > 170) {
    throw new RangeError("n must be an integer between 0 and 170");
  }
  if (n <= 1) return 1; // base case
  return n * factorial(n - 1); // recursive case
}
console.log(factorial(5)); // 120
```

Recursion-এ Base Case না থাকলে Stack Overflow হতে পারে। JavaScript Engine-এ Tail-Call Optimization সর্বত্র নির্ভরযোগ্য নয়। বড় Input-এর ক্ষেত্রে Loop কখনো অধিক নিরাপদ।

৫.৯ Pure Function বনাম Side Effect

```
function addTax(amount, taxRate) {
  return amount + amount * taxRate;
}
```

একই Input-এ একই Output এবং বাইরে কোনো Observable State পরিবর্তন না হলে Function-কে Pure বলা হয়। DOM Update, Network Request, `console.log`, File Write, বাইরের Variable পরিবর্তন ইত্যাদি Side Effect। Side Effect নিষিদ্ধ নয়; বরং Calculation ও UI/IO Logic আলাদা রাখলে Testing সহজ হয়।

```
const cart = [{ name: "Pen", price: 20 }];
function copyWithItem(items, newItem) {
  return [...items, newItem];
}
const updated = copyWithItem(cart, { name: "Book", price: 100 });
console.log(cart.length);    // 1
console.log(updated.length); // 2
```

৫.১০ Practical Project — Invoice Calculator

Project Goal

একটি ছোট দোকানের Invoice App বানাব, যেখানে Product Name, Unit Price এবং Quantity দিয়ে Item যোগ করা যাবে। Discount এবং উদাহরণমূলক Tax Rate বদলালে Grand Total আপডেট হবে। এখানে **Discount আগে, Tax পরে** প্রয়োগ করছি—এটি এই Project-এর নির্দিষ্ট Calculation Rule, কোনো বাস্তব Tax Law-এর দাবি নয়।

Project File Structure

```
source-code/chapter-05/
├── index.html
├── style.css
├── invoice.js
├── app.js
└── invoice.test.js
```

Design Decision: টাকা Floating-Point BDT হিসেবে না রেখে **Integer Paisa**-তে হিসাব করা হচ্ছে। উদাহরণ: `BDT 12.50` → `1250` Paisa। Rate রাখা হচ্ছে Basis Points-এ: `5% = 500 bps`, `10% = 1000 bps`। `Math.round` ব্যবহার করে Positive Amount-এর Half-Up Rounding দেখানো হয়েছে; Production Financial Software-এ প্রতিষ্ঠানের Rounding Policy অনুযায়ী সিদ্ধান্ত নিতে হবে।

Core Engine (`invoice.js`)

```
function percentOf(amount, rateBasisPoints) {
  if (!Number.isSafeInteger(amount) || amount < 0) throw new TypeError("Invalid amount");
  if (!Number.isSafeInteger(rateBasisPoints) || rateBasisPoints < 0 || rateBasisPoints > 10000)
{
    throw new RangeError("Invalid rate");
  }
  const product = amount * rateBasisPoints;
  if (!Number.isSafeInteger(product)) throw new RangeError("Amount too large");
  return Math.round(product / 10000);
}

function calculateInvoice(lines, { discountBps = 0, taxBps = 0 } = {}) {
  let subtotal = 0;
  for (const line of lines) {
    subtotal += line.unitPricePaisa * line.quantity;
  }
  const discount = percentOf(subtotal, discountBps);
  const taxable = subtotal - discount;
  const tax = percentOf(taxable, taxBps);
  return { subtotal, discount, tax, total: taxable + tax };
}
```

উপরের Snippet মূল Algorithm বোঝানোর জন্য সরল করা। **Runnable `invoice.js`-এ পূর্ণ Input Validation, Safe Integer Check এবং Line Details আছে।** `app.js` DOM Update করে; `invoice.js` Pure Calculation Logic ধরে রাখে। এতে একই Logic Browser ও Node.js Test—দুই জায়গায় ব্যবহার করা যায়।

হিসাবের উদাহরণ

ধরা যাক একটি Item-এর দাম BDT 100.00, Quantity 1, Discount 10%, Example Tax 10%।

Step	Amount
Subtotal	BDT 100.00
Discount	BDT 10.00
Taxable amount	BDT 90.00
Example Tax	BDT 9.00
Grand Total	**BDT 99.00**

App চালানোর নিয়ম

1. `source-code/chapter-05/index.html` Browser-এ খোলো।
2. Product Name, Unit Price ও Quantity লিখে **Add Item** চাপো।
3. Discount ও Example Tax Rate পরিবর্তন করো।
4. Subtotal, Discount, Taxable Amount, Tax ও Grand Total দেখো।
5. **Clear** চাপলে Items মুছে যাবে।
6. Node.js থাকলে `node --test invoice.test.js` দিয়ে Automated Tests চালাও।

Code Architecture

```
User Input
   ↓
app.js → parseBdtToPaisa() → calculateInvoice()
   ↓                                  ↓
DOM Rendering ← Invoice Result Object
```

Security Note: User-provided Product Name `textContent` দিয়ে DOM-এ বসানো হয়েছে, `innerHTML` দিয়ে নয়। এটি সাধারণ HTML Injection এড়াতে সাহায্য করে।

৫.১১ Common Mistakes এবং Debugging

Mistake 1 — `return` ভুলে যাওয়া

```
const wrong = (a, b) => { a + b; }; // undefined
const correct = (a, b) => { return a + b; };
```

Mistake 2 — Parameter-কে Type Validation ছাড়া ব্যবহার

```
function add(a, b) { return a + b; }
console.log(add("5", 2)); // "52", 7 নয়
```

Mistake 3 — Default Parameter-এ `null` ধরে নেওয়া

```
function display(name = "Guest") { return name; }
console.log(display(undefined)); // Guest
console.log(display(null));      // null
```

Mistake 4 — Closure-এর Shared State বুঝতে না পারা: একই Counter Function-এ একাধিক Call করলে State বাড়ে; নতুন Factory Call করলে আলাদা State তৈরি হয়।

Mistake 5 — Recursive Base Case ভুল: Recursion-এর আগে Exit Condition লিখে পরীক্ষা করো।

৫.১২ Practice Exercises

1. `square(number)` Function লিখো, যেটি Number-এর Square Return করে।
2. `isEven(n)` Function লিখো; Input Safe Integer না হলে Error দেবে।
3. `getFullName(first, last)` লিখো; `last` না থাকলে শুধু `first` Return করবে।
4. Rest Parameter দিয়ে `average(...numbers)` লিখো; Empty Input Reject করবে।
5. `createMultiplier(factor)` Closure লিখো, যা নতুন Function Return করবে।
6. `sumTo(n)` Recursion দিয়ে লিখো; `n` হবে 0–1000-এর Integer।
7. Invoice App-এ একটি নতুন `serviceFeePaisa` Option যুক্ত করো, যা Tax-এর পরে যোগ হবে।
8. Invoice App-এ Remove Item Button যোগ করো এবং Item Index Validation করো।

৫.১৩ Exercise Solutions

```
// 1
const square = number => number * number;

// 2
function isEven(n) {
  if (!Number.isSafeInteger(n)) throw new TypeError("Integer required");
  return n % 2 === 0;
}

// 3
function getFullName(first, last = "") {
  return `${first} ${last}`.trim();
}

// 4
function average(...numbers) {
  if (numbers.length === 0 || !numbers.every(Number.isFinite)) {
    throw new TypeError("At least one finite number required");
  }
  return numbers.reduce((sum, n) => sum + n, 0) / numbers.length;
}

// 5
function createMultiplier(factor) {
  return value => value * factor;
}
const double = createMultiplier(2);
console.log(double(7)); // 14

// 6
function sumTo(n) {
  if (!Number.isSafeInteger(n) || n < 0 || n > 1000) throw new RangeError("Invalid n");
  if (n === 0) return 0;
  return n + sumTo(n - 1);
}
```

Solution 7 (Implementation Guide): `calculateInvoice()`-এর Option-এ `serviceFeePaisa = 0` যোগ করে Safe Integer Validation করবে। এরপর `total = taxable + tax + serviceFeePaisa` হিসাব করবে এবং Overflow Check করবে। Test-এ `serviceFeePaisa: 250` দিলে Grand Total 250 Paisa বাড়ছে কি না দেখবে।

Solution 8 (Implementation Guide): প্রতিটি List Item-এর সঙ্গে `Remove` Button তৈরি করে তার `data-index` বা Closure-bound Index ব্যবহার করো। Click Handler-এ Index `0 <= index < lines.length` যাচাই করে `splice()` দিয়ে Remove করো, তারপর `render()`। Button-এর Label ও Focus Accessibility বজায় রাখবে।

৫.১৪ MCQ — নিজেকে যাচাই করো

1. Function-এর Output Caller-কে কোন Keyword দেয়? (A) print (B) return (C) break (D) let
2. Function Declaration সাধারণত কোন বৈশিষ্ট্য রাখে? (A) Hoisting (B) Private this (C) No scope (D) Always async
3. Rest Parameter কোন Syntax ব্যবহার করে? (A) `??` (B) `...` (C) `=>` (D) `?.`
4. Arrow Function-এর `this` সাধারণত কোথা থেকে আসে? (A) New binding (B) Global always (C) Lexical scope (D) Argument
5. Closure কী সংরক্ষণ করতে পারে? (A) শুধু HTML (B) Lexical binding access (C) Browser History (D) CSS
6. `const f = x => { x * 2 }; f(3)` কী Return করে? (A) 6 (B) 3 (C) undefined (D) null
7. Base Case ছাড়া Recursion কী সমস্যা করতে পারে? (A) Stack overflow (B) CSS error (C) Syntax always (D) HTTP 404
8. Pure Function-এর মূল বৈশিষ্ট্য কী? (A) DOM বদলায় (B) Same input → same output, no side effects (C) Network call (D) Random output
9. `greet(undefined)`-এ Default Parameter কাজ করে? (A) হ্যাঁ (B) না (C) কেবল null-এ (D) কেবল Arrow Function-এ
10. Invoice Engine-এ Integer Paisa ব্যবহারের সুবিধা কী? (A) UI বড় হয় (B) Binary floating-point precision সমস্যা কমে (C) CSS লাগে না (D) Tax লাগে না

MCQ Answer Key ও Explanation

No.	Answer	Explanation
1	B	`return` Caller-কে Value দেয়।
2	A	Function Declaration Hoisted হয়।
3	B	`...args` Rest Parameters সংগ্রহ করে।
4	C	Arrow Function Lexical `this` ব্যবহার করে।
5	B	Closure Lexical Environment-এর Binding-এ Access ধরে রাখে।
6	C	Block Body-তে `return` নেই।

No.	Answer	Explanation
7	A	Exit Condition না থাকলে Call Stack শেষ হতে পারে।
8	B	Pure Function-এর Deterministic Output ও Side-Effect-Free Behaviour।
9	A	`undefined` Default Parameter Trigger করে।
10	B	Integer Minor Units ব্যবহার করলে Decimal Arithmetic-এর অনেক সমস্যা এড়ানো যায়।

৫.১৫ Interview Questions

Q: Parameter ও Argument-এর পার্থক্য কী? Parameter হলো Function Definition-এর Placeholder; Argument হলো Call-এর সময় পাঠানো Actual Value।

Q: Closure কি Outer Variable-এর Snapshot নেয়? সাধারণত না; Closure Lexical Binding-এ Access ধরে রাখে, ফলে Variable পরিবর্তিত হলে পরবর্তী Call-এ নতুন Value দেখা যেতে পারে।

Q: Arrow Function কি সবসময় Regular Function-এর বিকল্প? না। `this`, `arguments`, Constructor Usage এবং Generator Behaviour-এ গুরুত্বপূর্ণ পার্থক্য আছে।

Q: `return` ও `console.log` কেন আলাদা? `return` Computation Compose করতে সাহায্য করে; `console.log` মূলত Debug/Output Side Effect।

৫.১৬ Chapter Summary

- Function একটি Reusable Behaviour তৈরি করে।
- Parameter/Argument এবং Return Value Function Composition-এর ভিত্তি।
- Declaration, Expression ও Arrow Function-এর Scope/Hoisting/`this` পার্থক্য বুঝতে হবে।
- Closure, Callback ও Higher-Order Function আধুনিক JavaScript-এর গুরুত্বপূর্ণ Building Block।
- Pure Calculation Logic UI Code থেকে আলাদা রাখলে Testing সহজ হয়।
- Invoice Calculator Project-এ Validation, Rounding, Testability ও DOM Safety-এর ব্যবহার দেখা হয়েছে।

পরবর্তী অধ্যায়: Chapter 6 — Arrays: Data Collection, Array Methods এবং Practical Data Processing; Practical Project: Inventory Dashboard।

অধ্যায় ৬ — Arrays: Data Collection, Array Methods এবং Practical Data Processing

শেখার লক্ষ্য

এই অধ্যায় শেষে তুমি Array তৈরি ও পরিবর্তন, Indexing, Traversal, `map`/`filter`/`reduce`/`find`/`some`/`every`/`sort` ব্যবহার, Reference ও Mutation-এর পার্থক্য, Shallow Copy, Nested Arrays, Real-World Inventory Data Processing এবং Automated Test লিখতে পারবে।

৬.১ Array কেন দরকার?

ধরো একটি দোকানে ৫০০টি পণ্যের নাম আছে। `product1`, `product2` এভাবে ৫০০টি Variable তৈরি করা বাস্তবসম্মত নয়। Array হলো একটি **ordered collection**, যেখানে প্রতিটি Element-এর অবস্থান বা Index থাকে। JavaScript-এ Index শুরু হয় ০ থেকে।

```
const products = ['Mouse', 'Keyboard', 'Monitor'];
console.log(products[0]);          // Mouse
console.log(products[2]);          // Monitor
console.log(products.length);      // 3
console.log(products[3]);          // undefined
```

বাস্তব উদাহরণ: Shopping Cart-এর Items, API থেকে আসা Search Results, Students-এর Marks, Travel Packages-এর List, Booking Records—সব জায়গায় Arrays কাজে লাগে।

৬.২ Array তৈরি: Literal, Array.from, Array.of

```
const empty = [];
const scores = [80, 95, 76];
const mixed = ['Dhaka', 2026, true, null];
const squares = Array.from({ length: 5 }, (_, index) => index ** 2);
const oneNumber = Array.of(5);  // [5]
const fiveSlots = Array(5);     // length 5, কিন্তু পাঁচটি initialized element নয়
console.log(squares);          // [0, 1, 4, 9, 16]
```

Array-তে বিভিন্ন Type রাখা সম্ভব হলেও একই ধরনের Data Structure ব্যবহার করলে Maintenance সহজ হয়। `Array(5)`-এর Sparse Slots এবং `[undefined, ...]` এক জিনিস নয়।

৬.৩ Index, Length এবং Boundary

```
const cities = ['Dhaka', 'Chattogram', 'Sylhet'];
console.log(cities.at(-1));                 // Sylhet
console.log(cities[cities.length - 1]);   // Sylhet
cities[1] = 'Rajshahi';
console.log(cities); // ['Dhaka', 'Rajshahi', 'Sylhet']
```

`const` দিয়ে Array Declare করলেও তার Element পরিবর্তন করা যায়; `const` শুধু Variable-টিকে অন্য Array দিয়ে Reassign করতে দেয় না। `cities[100] = 'X'` করলে মাঝখানে অনেক Empty Slot তৈরি হয়—সাধারণ Data Model-এ এটি এড়িয়ে চলা ভালো।

৬.৪ Mutating বনাম Non-Mutating Methods

Method	কাজ	Original Array বদলায়?
`push`, `pop`	শেষে যোগ/সরানো	হ্যাঁ
`unshift`, `shift`	শুরুতে যোগ/সরানো	হ্যাঁ
`splice`	মাঝখানে যোগ/মুছা	হ্যাঁ
`sort`, `reverse`	সাজানো/উল্টানো	হ্যাঁ
`slice`	অংশ Copy	না
`map`, `filter`	নতুন Array তৈরি	না
`concat`	Array যুক্ত	না
`toSorted`, `toReversed`	Copy সাজানো/উল্টানো	না (আধুনিক Runtime)

```
const cart = ['Pen', 'Book'];
cart.push('Bag');                 // original cart changes
const firstTwo = cart.slice(0, 2); // independent outer array
const updated = [...cart, 'Pencil']; // original untouched
console.log(cart, firstTwo, updated);
```

সতর্কতা: `splice(1, 1)`-এ দ্বিতীয় Argument হলো কতগুলো Element Delete হবে; শেষ Index নয়।

৬.৫ Array Traversal: for, for...of, forEach

```
const prices = [100, 250, 75];
for (let index = 0; index < prices.length; index++) {
  console.log(index, prices[index]);
}
for (const price of prices) console.log(price);
prices.forEach((price, index) => console.log(index, price));
```

`forEach` নতুন Array Return করে না। `forEach`-এর Callback-এর মধ্যে `return` দিলে শুধু সেই Callback শেষ হয়, পুরো Iteration থামে না। Loop মাঝপথে থামাতে চাইলে `for...of` + `break` ব্যবহার করো।

৬.৬ map: প্রতিটি Element Transform করা

```
const prices = [100, 200, 300];
const withVat = prices.map(price => price * 1.15);
console.log(withVat); // [114.99999999999999, 229.99999999999997, 345]
```

JavaScript Floating-Point Representation-এর কারণে Decimal Money Calculation-এ অদ্ভুত Fraction দেখা যেতে পারে। Monetary Calculation-এর জন্য Minor Units (যেমন পয়সা) ব্যবহার করা বেশি নিরাপদ।

```
const priceInPaisa = [10000, 20000, 30000];
const afterTenPercent = priceInPaisa.map(paisa => Math.round(paisa * 1.10));
```

```
console.log(afterTenPercent); // [11000, 22000, 33000]
```

৬.৭ filter: শর্ত অনুযায়ী Data নির্বাচন

```
const stock = [
  { name: 'Mouse', quantity: 12 },
  { name: 'Keyboard', quantity: 3 },
  { name: 'Monitor', quantity: 0 }
];
const lowStock = stock.filter(product => product.quantity <= 5);
console.log(lowStock.map(item => item.name)); // ['Keyboard', 'Monitor']
```

`filter` সব Matching Items দিয়ে নতুন Array দেয়। `find` প্রথম Matching Item দেয়, না পেলে `undefined`।

৬.৮ find, findIndex, some, every, includes

```
const numbers = [4, 7, 10, 12];
console.log(numbers.find(n => n > 8));       // 10
console.log(numbers.findIndex(n => n > 8));  // 2
console.log(numbers.some(n => n % 2 !== 0)); // true
console.log(numbers.every(n => n > 0));      // true
console.log(numbers.includes(7));            // true
```

`some` অন্তত একটি Item শর্ত পূরণ করে কি না জানায়। `every` সব Item শর্ত পূরণ করে কি না জানায়। খালি Array-তে `every` true এবং `some` false Return করে—এটি Logic Design-এ মনে রাখতে হবে।

৬.৯ reduce: একটি Summary Value তৈরি

```
const orders = [
  { price: 250, quantity: 2 },
  { price: 100, quantity: 3 }
];
const total = orders.reduce((sum, item) => sum + item.price * item.quantity, 0);
console.log(total); // 800
```

এখানে `0` হলো Initial Accumulator। Empty Array-তেও নিরাপদভাবে কাজ করতে Initial Value দেওয়া উচিত।

```
const marks = [80, 95, 75, 90];
const sum = marks.reduce((total, mark) => total + mark, 0);
const average = marks.length ? sum / marks.length : 0;
console.log(average); // 85
```

`reduce` দিয়ে সব কিছু লেখা যায়, কিন্তু Readability কমে গেলে `map`/`filter` বা সাধারণ Loop বেশি উপযুক্ত।

৬.১০ Sorting: সবচেয়ে সাধারণ ভুল

```
const prices = [9, 100, 20];
console.log([...prices].sort());             // [100, 20, 9] — lexicographic
console.log([...prices].sort((a, b) => a - b)); // [9, 20, 100]
console.log([...prices].sort((a, b) => b - a)); // [100, 20, 9]
```

`sort()` Original Array পরিবর্তন করে। Copy প্রয়োজন হলে `[...array].sort(...)` বা Supported Runtime-এ `toSorted(...)` ব্যবহার করো। Object Sorting-এ Numeric ও Text Field আলাদাভাবে Compare করতে হয়।

```
const items = [
  { name: 'Pen', price: 25 },
  { name: 'Bag', price: 700 }
];
const byPrice = [...items].sort((a, b) => a.price - b.price);
const byName = [...items].sort((a, b) => a.name.localeCompare(b.name));
```

৬.১১ slice, splice এবং Immutable Updates

```
const list = ['A', 'B', 'C', 'D'];
console.log(list.slice(1, 3)); // ['B', 'C']; original intact
const changed = [...list];
changed.splice(1, 2, 'X');
console.log(changed); // ['A', 'X', 'D']
console.log(list);    // ['A', 'B', 'C', 'D']
```

Immutable Update বলতে সাধারণত Existing Collection না বদলে নতুন Collection Return করা বোঝায়। UI State Management, Testing ও Debugging-এ এটি উপকারী।

৬.১২ Spread, Destructuring এবং Shallow Copy

```
const [first, second, ...remaining] = ['A', 'B', 'C', 'D'];
console.log(first, second, remaining); // A B ['C', 'D']
const original = [{ name: 'Mouse', stock: 5 }];
const copy = [...original];
copy[0].stock = 2;
console.log(original[0].stock); // 2! nested object shared
```

Spread দিয়ে Array-এর **Shallow Copy** হয়। Nested Object একই Reference ধরে রাখে। একটি Object পরিবর্তন করতে হলে `{ ...item, stock: newStock }` দিয়ে সেই Object-ও Copy করো। গভীর Clone-এর জন্য Data Type ও Browser Support অনুযায়ী `structuredClone()` ব্যবহার করা যায়, তবে Function ও কিছু বিশেষ Object Clone করা যায় না।

৬.১৩ Multi-Dimensional Arrays

```
const matrix = [
  [1, 2, 3],
  [4, 5, 6]
];
console.log(matrix[1][2]); // 6
const flattened = matrix.flat();
console.log(flattened); // [1, 2, 3, 4, 5, 6]
```

`flat()` Default-এ এক Level Flatten করে। Matrix, Timetable বা Grid Data-তে Nested Array দরকার হয়।

৬.১৪ Real-World Data Pipeline

ধরো Inventory Dashboard-এ User লিখল `mo`, Category বেছে নিল `Accessories`, তারপর Price অনুযায়ী Sort করল। পুরো Flow:

```
Raw Products Array
      ↓
Validate / Normalize
```

```
      ↓
Search + Category Filter
      ↓
Sort (without mutation)
      ↓
Render Table
      ↓
Summary Metrics (all products)
```

Search Results ও Overall Inventory Summary আলাদা রাখা জরুরি। Search-এ তিনটি Product দেখা গেলেও Overall Total Value যেন সব Product-এর হয়।

৬.১৫ Practical Project — Inventory Dashboard

Project Objective: Product Add, Stock Increase/Decrease, Delete, Search, Category Filter, Low-Stock Filter, Sort, Total Inventory Value এবং Browser Storage।

File Structure

```
source-code/chapter-06/
├── index.html
├── style.css
├── inventory.js
├── app.js
└── inventory.test.js
```

Run: `index.html` Browser-এ Open করো। External Package বা Build Step লাগে না। Automated Test-এর জন্য Node.js (18+) Install করে Project Directory-তে `node --test inventory.test.js` চালাও।

HTML-এর মূল অংশ

`index.html`-এ চারটি Summary Card, Product Form, Search/Filter Controls এবং একটি Table আছে। নিচের অংশে Product Entry Form-এর ধারণা দেখানো হলো:

```
<form id="product-form">
  <label>ID <input name="id" required></label>
  <label>Name <input name="name" required></label>
  <label>Category <input name="category" required></label>
  <label>Price <input name="price" type="number" min="0" step="0.01" required></label>
  <label>Stock <input name="stock" type="number" min="0" step="1" required></label>
  <button>Add product</button>
</form>
```

Inventory Logic: Separation of Concerns

`inventory.js`-এ Data Operations রাখা হয়েছে; এটি DOM-এর উপর নির্ভরশীল নয়। `app.js`-এ Browser UI ও `localStorage` Handle করা হয়। ফলে Pure Functions আলাদাভাবে Test করা যায়।

```
function addProduct(items, product) {
  const item = validateProduct(product);
  if (items.some(existing => existing.id === item.id)) {
    throw new Error('Duplicate product ID');
  }
  return [...items, item];
}
```

কেন নতুন Array? `push()` দিয়ে Original পরিবর্তন না করে `[...items, item]` Return করলে Function-এর Side Effect কমে।

Stock Update

```
function updateStock(items, id, change) {
  if (!Number.isSafeInteger(change)) throw new Error('Invalid change');
  if (!items.some(item => item.id === id)) throw new Error('Not found');
  return items.map(item => {
    if (item.id !== id) return item;
    const stock = item.stock + change;
    if (stock < 0) throw new Error('Insufficient stock');
    return { ...item, stock };
  });
}
```

এই Design-এ কোনো Product-এর Stock Negative করা যায় না।

Filter + Sort

```
const visible = Inventory.sortProducts(
  Inventory.filterProducts(products, {
    query: searchText,
    category: selectedCategory,
    lowStockOnly: true
  }),
  'price',
  'asc'
);
```

`filterProducts()` এবং `sortProducts()` দুটোই নতুন Array Return করে; Master Inventory পরিবর্তন হয় না।

Summary Calculation

```
function summarize(items, threshold = 5) {
  return items.reduce((result, item) => {
    result.productCount++;
    result.totalUnits += item.stock;
    result.totalValueCents += Math.round(item.price * 100) * item.stock;
    if (item.stock <= threshold) result.lowStockCount++;
    return result;
  }, { productCount: 0, totalUnits: 0, totalValueCents: 0, lowStockCount: 0 });
}
```

Currency Integer Minor Units-এ হিসাব হচ্ছে। Dashboard-এ Display করার সময় ১০০ দিয়ে ভাগ করে BDT দেখানো হয়।

Safe DOM Rendering

User Input সরাসরি `innerHTML`-এ বসানো হয়নি। `textContent` ব্যবহার করা হয়েছে, যাতে Product Name-এ HTML-like Text থাকলেও তা Executable Markup না হয়।

```
const cell = document.createElement('td');
cell.textContent = product.name;
row.append(cell);
```

Local Storage

```
localStorage.setItem('inventory', JSON.stringify(products));
const restored = JSON.parse(localStorage.getItem('inventory') ?? '[]');
```

এটি Conceptual Example। প্রকৃত Project-এ Loaded Data পুনরায় Validate করা হয়েছে এবং Invalid/Corrupted Data Handle করার ব্যবস্থা আছে। Browser Storage Production Database বা Secure Server Storage-এর বিকল্প নয়।

Complete Runnable Files

বইয়ের Companion Source Code-এর `source-code/chapter-06/` Folder-এ **সম্পূর্ণ** `index.html`, `style.css`, `inventory.js`, `app.js` **এবং** `inventory.test.js` অন্তর্ভুক্ত।

৬.১৬ Debugging ও Common Mistakes

সমস্যা	কারণ	সমাধান
`sort()`-এ 100 আগে আসে	Default String Comparison	Numeric Comparator
`map()` Result `undefined`	Callback-এ `return` নেই	Explicit Return
Original Array বদলে গেছে	`sort`, `splice`, `push` Mutate করে	Copy First
`reduce` Empty Array-তে Error	Initial Value নেই	`reduce(callback, 0)`
Spread Copy-তে Nested Data বদলায়	Shallow Copy	Nested Object Copy
Price Calculation অদ্ভুত Decimal	Floating Point	Minor Units ব্যবহার
UI-তে Untrusted HTML	`innerHTML`	`textContent`

৬.১৭ Time Complexity

Operation	সাধারণ Complexity
Array Index Access	O(1)
`push`	Amortized O(1)
`find`, `filter`, `map`	O(n)
`sort`	সাধারণভাবে O(n log n), Implementation-Dependent
`includes`	O(n)
`unshift`, `shift`	O(n)

এই Complexity Big-O-এর সাধারণ ধারণা; Engine Implementation ও Input Pattern অনুযায়ী বাস্তব Performance ভিন্ন হতে পারে।

৬.১৮ Exercises — নিজে চেষ্টা করো

1. `[5, 10, 15, 20]` থেকে শুধু ১০-এর বেশি সংখ্যাগুলো বের করো।

2. Product Prices `[100, 250, 400]`-এর ওপর ১০% Discount দিয়ে নতুন Array বানাও।
3. `['Dhaka', 'Sylhet', 'Dhaka']` থেকে Unique City বের করো।
4. Students Array থেকে সর্বোচ্চ Marks-এর Student খুঁজে বের করো। Empty Array Handle করো।
5. Stock Array-তে `quantity <= 5` হলে Low-Stock Product Count বের করো।
6. Numeric Array Descending Order-এ Sort করো, Original অপরিবর্তিত রেখে।
7. Array of Orders থেকে `price * quantity` যোগ করে Grand Total বের করো।
8. একটি Product-এর Stock Immutable পদ্ধতিতে Update করো।
9. `categorySummary()`-এর মতো Category অনুযায়ী Product Count বের করো।
10. Inventory Dashboard-এ `Out of Stock` Filter যোগ করো।

৬.১৯ Exercise Solutions

Solution 1

```
console.log([5, 10, 15, 20].filter(n => n > 10)); // [15, 20]
```

Solution 2

```
const prices = [100, 250, 400];
const discounted = prices.map(price => Math.round(price * 90) / 100);
console.log(discounted); // [90, 225, 360]
```

Solution 3

```
const cities = ['Dhaka', 'Sylhet', 'Dhaka'];
console.log([...new Set(cities)]); // ['Dhaka', 'Sylhet']
```

Solution 4

```
const students = [{ name: 'A', marks: 78 }, { name: 'B', marks: 91 }];
const topper = students.reduce((best, student) =>
  best === null || student.marks > best.marks ? student : best, null);
console.log(topper); // { name: 'B', marks: 91 }
```

Solution 5

```
const inventory = [{ quantity: 8 }, { quantity: 3 }, { quantity: 0 }];
console.log(inventory.filter(item => item.quantity <= 5).length); // 2
```

Solution 6

```
const nums = [3, 20, 1];
const descending = [...nums].sort((a, b) => b - a);
console.log(descending, nums); // [20, 3, 1] [3, 20, 1]
```

Solution 7

```
const orders = [{ price: 100, quantity: 2 }, { price: 50, quantity: 3 }];
console.log(orders.reduce((sum, item) => sum + item.price * item.quantity, 0)); // 350
```

Solution 8

```
const items = [{ id: 'A', stock: 5 }, { id: 'B', stock: 2 }];
const next = items.map(item => item.id === 'A' ? { ...item, stock: 7 } : item);
console.log(items[0].stock, next[0].stock); // 5 7
```

Solution 9

```
const products = [{ category: 'Tech' }, { category: 'Books' }, { category: 'Tech' }];
const counts = products.reduce((acc, product) => {
  acc[product.category] = (acc[product.category] ?? 0) + 1;
  return acc;
}, Object.create(null));
console.log(counts.Tech, counts.Books); // 2 1
```

Solution 10

```
const outOfStock = products => products.filter(item => item.stock === 0);
// Dashboard UI-তে নতুন Checkbox যোগ করে এই Filter ব্যবহার করো।
```

৬.২০ Knowledge Check — ১০টি MCQ

১. Array-এর প্রথম Element-এর Index কত? (A) 1 (B) 0 (C) -1 (D) 2

২. কোন Method নতুন Array Return করে? (A) push (B) pop (C) map (D) reverse

৩. `filter()` কী Return করে? (A) Boolean (B) প্রথম Match (C) Matching Elements-এর Array (D) Number

৪. `[10, 2].sort()`-এর Result কী? (A) `[2,10]` (B) `[10,2]` (C) Error (D) `undefined`

৫. `find()` Match না পেলে কী Return করে? (A) `null` (B) `[]` (C) `undefined` (D) `false`

৬. Original Array পরিবর্তন করে কোনটি? (A) slice (B) filter (C) map (D) splice

৭. `reduce()`-এর Initial Value কেন গুরুত্বপূর্ণ? (A) Styling (B) Empty Array Support (C) Sorting (D) String Conversion

৮. `[...items]` Nested Object-কে Deep Clone করে? (A) Yes (B) No

৯. `some()` কখন true? (A) সব Match (B) অন্তত একটি Match (C) কোনো Match নয় (D) Empty হলে সবসময়

১০. Monetary Total হিসাবের ভালো পদ্ধতি কোনটি? (A) String Concatenation (B) Minor Currency Units (C) Random Rounding (D) Array Join

Answer Key ও ব্যাখ্যা

প্রশ্ন	উত্তর	ব্যাখ্যা
1	B	Array Index ০ থেকে শুরু
2	C	`map()` প্রতিটি Element Transform করে নতুন Array দেয়
3	C	`filter()` সব Matching Element দেয়
4	B	Default Sort String Comparison করে

প্রশ্ন	উত্তর	ব্যাখ্যা
5	C	Match না পেলে `undefined`
6	D	`splice()` Original Array Mutate করে
7	B	Initial Value Empty Array Handle করে
8	B	Spread Shallow Copy করে
9	B	অন্তত একটি Match হলেই true
10	B	Integer Minor Units Floating-Point Error কমায়

৬.২১ Final Challenge

Inventory Dashboard-এ তিনটি Feature যোগ করো: (১) Category-wise Inventory Value, (২) JSON Export, (৩) Price Range Filter। প্রতিটি Feature-এর জন্য অন্তত একটি Automated Test লেখো। `localStorage` থেকে Restore করা Data-ও Validate করবে।

৬.২২ Chapter Summary

এই অধ্যায়ে Array Creation, Indexing, Iteration, Transformation, Filtering, Searching, Sorting, Reducing, Immutability, Nested Data এবং Complexity শিখলে। Practical Inventory Dashboard-এ এগুলো একত্রে ব্যবহার করা হয়েছে। পরের অধ্যায়ে **Objects, Destructuring ও Real-World Data Modeling** শেখা হবে, যেখানে Inventory-এর Product Object-এর গঠন আরও গভীরভাবে বোঝা যাবে।

অধ্যায় ৭ — Objects: Properties, Methods, Destructuring এবং Real-World Data Modeling

শেখার লক্ষ্য

এই অধ্যায় শেষে পাঠক JavaScript Object তৈরি, পড়া, পরিবর্তন, Method লেখা, Reference বোঝা, Destructuring ও Spread ব্যবহার, Object Validation এবং Student Management Application তৈরি করতে পারবেন। আগের অধ্যায়ে Array-তে অনেক Record রাখা শিখেছি; এখানে প্রতিটি Record-এর অর্থপূর্ণ Structure তৈরি করব।

৭.১ Object কেন প্রয়োজন?

ধরুন একটি শিক্ষার্থীর নাম, বয়স ও নম্বর আছে। তিনটি পৃথক Variable-এ রাখা যায়, কিন্তু একাধিক শিক্ষার্থী এলে কোন নামের সঙ্গে কোন নম্বর যুক্ত তা বোঝা কঠিন হয়। Object একই Entity-এর সম্পর্কিত তথ্যকে Key–Value Pair-এ সংগঠিত করে।

```
const student = {
  id: 'ST-101',
  name: 'Amina Rahman',
  age: 19,
  marks: 86,
  isActive: true
};
console.log(student.name);        // Amina Rahman
console.log(student['marks']);    // 86
```

`student` একটি Object Reference ধরে। `id`, `name`, `age`, `marks` হলো Properties। Array সাধারণত Ordered Collection, আর Object সাধারণত Named Fields-এর জন্য ব্যবহার হয়। তবে JavaScript-এ Array-ও এক ধরনের Object; প্রয়োগের উদ্দেশ্য আলাদা।

৭.২ Object Literal, Constructor এবং `Object.create()`

```
const book = { title: 'JavaScript Mastery', pages: 300 };
const settings = new Object();
settings.theme = 'light';
const dictionary = Object.create(null);
dictionary['__proto__'] = 'safe as an ordinary key';
```

অধিকাংশ Application Data-এর জন্য Object Literal সহজ। `Object.create(null)`-এর Prototype নেই, তাই Dictionary-সদৃশ কাজে বিশেষ সুবিধা দিতে পারে; তবে এতে `toString()`-এর মতো স্বাভাবিক Inherited Method থাকে না।

৭.৩ Dot Notation বনাম Bracket Notation

```
const profile = { firstName: 'Amina', 'home-city': 'Dhaka' };
```

```
console.log(profile.firstName);
console.log(profile['home-city']);
const field = 'firstName';
console.log(profile[field]);
```

Key যদি Dynamic হয়, Space/Hyphen থাকে, অথবা Runtime-এ নির্ধারিত হয়, Bracket Notation ব্যবহার করতে হবে। `profile.field` এবং `profile[field]` এক নয়: প্রথমটি আক্ষরিক `field` Property খোঁজে, দ্বিতীয়টি Variable-এর Value ব্যবহার করে।

৭.৪ Property Read, Update, Delete এবং Existence

```
const user = { name: 'Rahim', city: 'Dhaka' };
user.city = 'Chattogram';
user.email = 'rahim@example.com';
delete user.email;
console.log(Object.hasOwn(user, 'name')); // true
console.log('toString' in user);          // true: inherited
```

`Object.hasOwn()` শুধু Object-এর নিজস্ব Property পরীক্ষা করে; `in` Operator Prototype Chain-এও খোঁজে। Property অনুপস্থিত এবং Property-এর Value `undefined`—এ দুটো আলাদা ঘটনা।

৭.৫ Methods এবং `this`

```
const account = {
  owner: 'Amina',
  balance: 500,
  deposit(amount) {
    if (!Number.isFinite(amount) || amount <= 0) throw new Error('Invalid amount');
    this.balance += amount;
    return this.balance;
  }
};
console.log(account.deposit(200)); // 700
```

Method-এর `this` সাধারণত Call-site-এর ওপর নির্ভর করে। `account.deposit()`-এ `this` হলো `account`; Method আলাদা Variable-এ রেখে Call করলে `this` হারাতে পারে। Arrow Function-এর নিজস্ব `this` নেই, তাই Object Method-এ প্রয়োজন বুঝে ব্যবহার করুন।

৭.৬ Nested Objects এবং Optional Chaining

```
const traveler = {
  name: 'Nadia',
  contact: { phone: '01700000000', address: { city: 'Dhaka' } }
};
console.log(traveler.contact?.address?.city); // Dhaka
console.log(traveler.passport?.number ?? 'Not provided');
```

`?.` অনুপস্থিত Nested Data Access-এ TypeError এড়াতে সাহায্য করে; `??` শুধু `null` বা `undefined` হলে Fallback দেয়। তবে এটি Schema Validation-এর বিকল্প নয়।

৭.৭ Object Destructuring এবং Defaults

```
const student = { id: 'ST-101', name: 'Amina', marks: 86 };
const { name, marks: score, city = 'Unknown' } = student;
console.log(name, score, city);

function printStudent({ name, marks = 0 }) {
  return `${name}: ${marks}`;
}
```

`marks: score` মানে `marks` Property-এর Value নতুন `score` Variable-এ যাবে। Default Value কেবল `undefined` হলে প্রযোজ্য; `null` হলে নয়। Function Parameter-এ Destructuring API-কে স্পষ্ট করে।

৭.৮ Spread, Rest এবং Shallow Copy

```
const original = { id: 'ST-101', name: 'Amina', address: { city: 'Dhaka' } };
const updated = { ...original, name: 'Amina Rahman' };
const { id, ...publicInfo } = updated;
console.log(publicInfo.name);
console.log(updated.address === original.address); // true
```

Spread Copy **Shallow**: প্রথম স্তরের Property Copy হয়, কিন্তু Nested Object-এর Reference ভাগাভাগি হতে পারে। Nested Data বদলাতে হলে প্রয়োজনমতো Nested Copy তৈরি করুন। `structuredClone()` অনেক Built-in Data Type Deep-copy করতে পারে, কিন্তু Function, DOM Node ইত্যাদি Clone করতে পারে না।

৭.৯ Reference, Equality ও Mutation

```
const a = { marks: 80 };
const b = a;
const c = { marks: 80 };
console.log(a === b); // true
console.log(a === c); // false
b.marks = 90;
console.log(a.marks); // 90
```

Object Equality সাধারণত Reference Identity পরীক্ষা করে, Content Equality নয়। তাই State Management-এ পুরোনো Object পরিবর্তন না করে নতুন Object Return করা সুবিধাজনক।

৭.১০ Object Utility Methods

```
const scores = { math: 80, english: 75, science: 90 };
console.log(Object.keys(scores));
console.log(Object.values(scores));
console.log(Object.entries(scores));
const doubled = Object.fromEntries(
  Object.entries(scores).map(([subject, mark]) => [subject, mark * 2])
);
console.log(doubled);
```

`Object.keys()`/`values()`/`entries()` Enumerable Own String-Keyed Properties নিয়ে কাজ করে। `Object.freeze()`-ও Shallow; Nested Objects স্বয়ংক্রিয়ভাবে Deep-Freeze হয় না।

৭.১১ JSON এবং Browser Storage

```
const record = { id: 'ST-101', name: 'Amina', marks: 86 };
const serialized = JSON.stringify(record);
const restored = JSON.parse(serialized);
```

JSON Data Exchange-এর Format, JavaScript Object-এর পূর্ণ বিকল্প নয়। JSON-এ Function, `undefined`, Circular Reference এবং `BigInt`-এর মতো Value-এর সীমাবদ্ধতা আছে। Untrusted JSON Parse করার পর Schema Validation জরুরি। `localStorage` String রাখে, তাই Serialize/Parse করতে হয়। Browser Storage-এ Password, Secret Token বা Sensitive Student Data রাখা উচিত নয়।

৭.১২ Real-World Data Modeling: Student Schema

এই অধ্যায়ের Project-এ একটি Student Record-এর Schema:

Field	Type	Validation
`id`	String	২–২০ Character, Unique
`name`	String	২–৮০ Character
`age`	Integer	৫–১০০
`marks`	Number	০–১০০

একজন শিক্ষার্থীকে এভাবে Represent করা হবে:

```
const student = { id: 'ST-101', name: 'Amina Rahman', age: 19, marks: 86 };
```

Project-এর Business Logic-এ `add`, `update`, `remove`, `search`, `gradeFor`, `summary` Function থাকবে। প্রতিটি Function Data Return করবে; Browser DOM Logic আলাদা File-এ থাকবে। এতে Testing ও Maintenance সহজ হয়।

৭.১৩ Practical Project — Student Management Dashboard

Feature List

1. Student Add, Edit এবং Delete
2. Duplicate Student ID প্রতিরোধ
3. Name ও ID দিয়ে Search
4. Grade Calculation (A+, A, A−, B, C, D, F)
5. Total Students, Average Marks, Passed Students, Topper
6. `localStorage` Persistence এবং Invalid Stored Record Filtering
7. DOM-এ User Input Render করার সময় `textContent` ব্যবহার

File Structure

```
source-code/chapter-07/
├── index.html
├── style.css
├── students.js
├── app.js
└── students.test.js
```

Runnable Code: বইয়ের Companion Source Code-এর `source-code/chapter-07/` Folder-এ প্রতিটি সম্পূর্ণ File সংযুক্ত আছে।

Business Logic-এর গুরুত্বপূর্ণ অংশ

```
function add(students, input) {
  return [...students, validateStudent(input, students)];
}
function update(students, id, patch) {
  const old = students.find(s => s.id === id);
  if (!old) throw new Error('Student not found');
  const next = validateStudent({ ...old, ...patch }, students, id);
  return students.map(s => s.id === id ? next : s);
}
```

এখানে `add` এবং `update` পুরোনো Array পরিবর্তন করে না। একই সঙ্গে Duplicate ID, Invalid Marks ও Age Reject করা হয়।

Run Instructions

1. `source-code/chapter-07/index.html` Browser-এ Open করুন।
2. Student ID, Name, Age, Marks দিয়ে Save করুন।
3. Search, Edit, Delete এবং Refresh করে Persistence পরীক্ষা করুন।
4. Node.js Installed থাকলে Terminal-এ `node students.test.js` চালান।

Manual Test Checklist

- ☐ Add করার পর Table-এ Student দেখা যায়
- ☐ Duplicate ID দিলে Error দেখায়
- ☐ Edit করলে Student-এর Data Update হয়
- ☐ Delete করলে Confirmation চায়
- ☐ Search Uppercase/Lowercase নির্বিশেষে কাজ করে
- ☐ Page Refresh-এর পর Valid Data থাকে
- ☐ Keyboard দিয়ে Form ব্যবহার করা যায়

৭.১৪ Common Mistakes

Mistake 1: Object Copy মানেই Deep Copy ভাবা। Spread-এর Nested Reference একই থাকতে পারে।

Mistake 2: `for...in` দিয়ে Blind Iteration। এটি Inherited Enumerable Properties-ও দেখতে পারে। Own Data-এর জন্য `Object.keys()`/`entries()` ব্যবহার করুন।

Mistake 3: Untrusted Data সরাসরি `innerHTML`-এ বসানো। XSS Risk তৈরি হতে পারে; Plain Text-এর জন্য `textContent` ব্যবহার করুন।

Mistake 4: `Number('')` যে `0` হয় তা ভুলে যাওয়া। Form Validation-এ Empty Input আলাদাভাবে Reject করতে হবে।

Mistake 5: Frontend Validation-কে Security Boundary ভাবা। Real Server Application-এ Backend Validation ও Authorization প্রয়োজন।

৭.১৫ Exercises

1. একটি `book` Object তৈরি করুন, যার `title`, `author`, `price` আছে। Price ১০% কমিয়ে নতুন Object তৈরি করুন।
2. `getFullName({ firstName, lastName })` Function লিখুন।
3. `Object.entries()` ব্যবহার করে Subject-wise Marks-এর Total বের করুন।
4. `address?.city ?? 'Unknown'` Expression কীভাবে কাজ করে ব্যাখ্যা করুন।
5. একটি Array of Student Objects থেকে ৮০ বা বেশি Marks পাওয়া Student Filter করুন।
6. `Object.hasOwn()` এবং `in`-এর পার্থক্য Code দিয়ে দেখান।
7. Nested Object-এর Shallow Copy Problem দেখিয়ে Corrected Copy তৈরি করুন।
8. Student Dashboard-এ Sort by Marks Feature যুক্ত করুন।
9. `validateStudent()`-এ Duplicate ID Checking কেন দরকার ব্যাখ্যা করুন।
10. Student Data Export-এর জন্য `JSON.stringify()` ব্যবহার করুন এবং Import-এর সময় Validation যুক্ত করুন।

৭.১৬ Exercise Solutions

```
// 1
const book = { title: 'JS', author: 'A', price: 500 };
const discounted = { ...book, price: book.price * 0.9 };

// 2
const getFullName = ({ firstName, lastName }) => `${firstName} ${lastName}`.trim();

// 3
const marks = { math: 80, english: 70, science: 90 };
const total = Object.entries(marks).reduce((sum, [, value]) => sum + value, 0);

// 4: address null/undefined হলে city access বন্ধ হয়; result null/undefined হলে Unknown
const profile = {};
const city = profile.address?.city ?? 'Unknown';

// 5
const topStudents = [{name:'A',marks:82},{name:'B',marks:65}].filter(s => s.marks >= 80);
```

```
// 6
const obj = Object.create({ inherited: true });
obj.own = true;
console.log('inherited' in obj);          // true
console.log(Object.hasOwn(obj,'inherited')); // false

// 7
const oldRecord = { name: 'A', address: { city: 'Dhaka' } };
const newRecord = { ...oldRecord, address: { ...oldRecord.address, city: 'Khulna' } };

// 8: data sorting; UI-তে sort select-এর change handler-এ ব্যবহার করুন
const sortByMarks = students => [...students].sort((a, b) => b.marks - a.marks);

// 9: unique ID ছাড়া update/delete ভুল শিক্ষার্থীকে প্রভাবিত করতে পারে
// 10: parse-এর পর প্রত্যেক Record validate করে গ্রহণ করুন
const exportJSON = students => JSON.stringify(students, null, 2);
const importJSON = text => {
  const parsed = JSON.parse(text);
  if (!Array.isArray(parsed)) throw new Error('Expected array');
  return parsed.reduce((acc, record) => StudentStore.add(acc, record), []);
};
```

৭.১৭ Knowledge Check — ১০টি MCQ

1. Object Property Access-এর জন্য কোনটি সঠিক? A. `obj.name` B. `obj->name` C. `obj::name` D. `obj#name`
2. Dynamic Key-এর জন্য কোন Syntax? A. `obj.key` B. `obj[key]` C. `obj(key)` D. `obj{key}`
3. Spread দিয়ে Nested Object Copy করলে কী হয়? A. Deep Copy B. Shallow Copy C. JSON Copy D. Freeze
4. Own Property পরীক্ষা করে কোনটি? A. `Object.hasOwn()` B. `in` only C. `typeof` D. `Array.isArray()`
5. `const a={}; const b=a; a===b`? A. false B. true C. null D. Error
6. `Object.entries()` কী Return করে? A. String B. Key–Value Pair Array C. Set D. Number
7. `user.profile?.name`-এ `profile` অনুপস্থিত হলে Result? A. Error B. 0 C. undefined D. false
8. JSON Data Parse করার Method? A. `JSON.read` B. `JSON.parse` C. `JSON.open` D. `JSON.decodeObject`
9. User Text DOM-এ নিরাপদ Plain Text হিসেবে বসাতে কোনটি? A. `innerHTML` B. `eval` C. `textContent` D. `document.write`
10. Student Marks 0–100 নিশ্চিত করার সঠিক পদ্ধতি? A. Only HTML `min/max` B. Explicit Validation C. `parseInt` only D. Ignore

Answer Key এবং Explanation

Q	Answer	কারণ
1	A	Dot Notation সরাসরি Named Property Access করে
2	B	Bracket Notation Variable-এর Value ব্যবহার করে
3	B	Spread Nested Reference Deep Copy করে না
4	A	Own Property-ই পরীক্ষা করে
5	B	দুই Variable একই Object Reference ধরে

Q	Answer	কারণ
6	B	`[key, value]` Pair-এর Array দেয়
7	C	Optional Chaining Access বন্ধ করে `undefined` দেয়
8	B	JSON Text-কে JavaScript Value-তে রূপান্তর করে
9	C	HTML হিসেবে Interpret না করে Text বসায়
10	B	Explicit Validation দরকার; HTML Constraint Bypass হতে পারে

৭.১৮ Chapter Summary এবং পরবর্তী অধ্যায়

Objects আমাদের Named Data, Methods, Nested Records এবং Real-World Entity Modeling-এর ভিত্তি। Destructuring, Spread, Optional Chaining ও Object Utilities দিয়ে Maintainable Code লেখা যায়। Student Management Project-এ এই ধারণাগুলো একসঙ্গে ব্যবহার করা হয়েছে। পরবর্তী অধ্যায় **Chapter 8 — Strings এবং Modern Text Processing**-এ Text Transformation, Unicode, Search এবং Input Processing শিখব।

অধ্যায় ৮ — Strings এবং Modern Text Processing

শেখার লক্ষ্য

এই অধ্যায় শেষে পাঠক String Primitive, Indexing, Unicode, Normalization, Template Literals, Searching, Splitting, Regular Expression, Safe DOM Rendering এবং Content Analysis বুঝবেন। Practical Project-এ বাংলা ও English কনটেন্ট থেকে Word Count, Slug, Excerpt, Frequency এবং Search Preview তৈরি করবেন।

৮.১ String কী এবং কেন গুরুত্বপূর্ণ?

ওয়েব অ্যাপ্লিকেশনে নাম, ইমেইল, URL, Search Query, Blog Post, API Message—সবই Text Data। JavaScript-এর `string` হলো Primitive Data Type; String Value অপরিবর্তনীয় (*immutable*)। Variable-এ নতুন String Assign করা যায়, কিন্তু আগের String-এর Character সরাসরি পরিবর্তন করা যায় না।

```
const language = 'JavaScript';
console.log(typeof language); // string
console.log(language[0]);     // J
language[0] = 'j';            // Primitive string অপরিবর্তিত থাকে (strict mode-এ error হতে পারে)
console.log(language);        // JavaScript
```

String Object Wrapper ও Primitive String আলাদা। সাধারণ কাজে `new String('hello')` ব্যবহার না করাই ভালো, কারণ সেটি Object এবং Truthiness/Equality-তে বিভ্রান্তি তৈরি করে।

```
const a = 'hello';
const b = new String('hello');
console.log(a === b); // false
console.log(Boolean(new String(''))); // true: Object truthy
```

৮.২ String লেখার তিন পদ্ধতি

Single Quote, Double Quote ও Backtick—সবই বৈধ। Backtick-এর Template Literal Expression Interpolation এবং Multiline Content-এর জন্য উপযোগী।

```
const name = 'Amina';
const city = "Dhaka";
const message = `Hello, ${name}! Welcome to ${city}.`;
const invoice = `Customer: ${name}
Location: ${city}`;
console.log(message);
```

HTML-এ Template Literal দিয়ে User Input সরাসরি `innerHTML`-এ ঢোকানো ঝুঁকিপূর্ণ। Display Text-এর জন্য `textContent` ব্যবহার করুন।

৮.৩ Length, Index ও UTF-16-এর বাস্তবতা

JavaScript String-এর `length` এবং Numeric Index মূলত UTF-16 **Code Unit** গণনা করে, মানুষের চোখে দেখা Character নয়। Emoji ও Combined Character-এ এই পার্থক্য গুরুত্বপূর্ণ।

```
console.log('ABC'.length);      // 3
console.log('□'.length);       // 2 (UTF-16 code units)
console.log([...'□'].length);   // 1 (Unicode code point)
console.log('□□□□'.length);  // 1 নয়; একাধিক code point ও joiner আছে
```

একটি দৃশ্যমান Character (*grapheme cluster*) গণনার জন্য উপযুক্ত Environment-এ `Intl.Segmenter` ব্যবহার করুন। এটি বাংলা যুক্তাক্ষর, Emoji Sequence এবং Combining Mark-এর ক্ষেত্রে সাধারণ `.length`-এর চেয়ে অর্থপূর্ণ ফল দিতে পারে। তবে Segmentation Locale ও Unicode Version অনুযায়ী আচরণ পরিবর্তিত হতে পারে।

```
const segmenter = new Intl.Segmenter('bn', { granularity: 'grapheme' });
const visible = [...segmenter.segment('বাংলা □')].length;
console.log(visible);
```

৮.৪ String Access: `at()`, `charAt()`, `slice()`

```
const text = 'JavaScript';
console.log(text.at(0));      // J
console.log(text.at(-1));     // t
console.log(text.charAt(4));  // S
console.log(text.slice(0, 4)); // Java
console.log(text.slice(-6));  // Script
```

`slice(start, end)`-এর `end` অন্তর্ভুক্ত নয়। `substring()` Negative Index-কে 0 হিসেবে ধরে এবং Start/End উল্টো হলে অদলবদল করতে পারে। নতুন Code-এ `slice()` সাধারণত সহজ ও Predictable। UTF-16 Boundary মাঝখানে কাটলে Emoji ভেঙে যেতে পারে; Unicode-Aware Truncation-এর জন্য Code Point বা Grapheme ভিত্তিক পদ্ধতি দরকার।

৮.৫ Searching: `includes()`, `startsWith()`, `endsWith()`, `indexOf()`

```
const title = 'Learn Modern JavaScript';
console.log(title.includes('Modern'));      // true
console.log(title.startsWith('Learn'));     // true
console.log(title.endsWith('Script'));      // true
console.log(title.indexOf('Java'));         // 13
console.log(title.indexOf('Python'));       // -1
```

Search Case-Sensitive। Case-Insensitive Search-এর সাধারণ কৌশল হলো দুই দিক একই Case-এ রূপান্তর করা। কিন্তু Unicode Case Mapping-এ কিছু ভাষায় String Length পরিবর্তিত হতে পারে; তাই Search Highlight-এর Index Mapping জটিল হয়ে যায়। Production Search-এ Locale-Aware Rules, Tokenization এবং প্রয়োজনে Dedicated Search Engine বিবেচনা করুন।

৮.৬ Clean Text: `trim()`, `replace()`, `replaceAll()`

```
const raw = '   Learn   JavaScript   ';
console.log(raw.trim());                    // Learn   JavaScript
console.log(raw.trim().replace(/\s+/g, ' ')); // Learn JavaScript
console.log('red blue red'.replace('red', 'green')); // green blue red
console.log('red blue red'.replaceAll('red', 'green')); // green blue green
```

`trim()` কেবল String-এর শুরু ও শেষের Whitespace সরায়। সব Internal Whitespace পরিবর্তন করা Content Formatting-এর জন্য সবসময় উপযুক্ত নয়—Poetry, Code Snippet ও Address-এর Line Break হারাতে পারে।

৮.৭ Case Conversion এবং Locale

```
console.log('HELLO'.toLowerCase());
console.log('hello'.toUpperCase());
console.log('İSTANBUL'.toLocaleLowerCase('tr'));
```

Case Folding সব ভাষায় একই নয়। Bangla Script-এ সাধারণত Uppercase/Lowercase নেই। Multilingual Search-এ Locale সম্পর্কে সচেতন থাকুন।

৮.৮ Split, Join ও Tokenization

```
const csvLike = 'apple,banana,mango';
console.log(csvLike.split(','));
console.log(['Learn', 'JavaScript'].join(' '));
const tokens = 'বাংলা JavaScript 2026'.match(/[\p{L}\p{M}\p{N}]+/gu) ?? [];
console.log(tokens);
```

`split(',')` একটি Full CSV Parser নয়: Quoted Field-এর ভেতরে Comma থাকলে ভুল ফল দিতে পারে। একইভাবে Word Tokenization-এর একটি Regular Expression সব ভাষা, Apostrophe, Hyphen, Emoji ও Punctuation নিখুঁতভাবে সামলাতে পারে না। Project-এর Word Count একটি শিক্ষামূলক Heuristic।

৮.৯ Unicode Normalization: NFC বনাম NFD

একই দৃশ্যমান Text Unicode-এ একাধিক Code Point Sequence দিয়ে প্রকাশিত হতে পারে। Normalization Search, Comparison ও Deduplication-এর কাজে সহায়ক।

```
const composed = 'café';
const decomposed = 'cafe\u0301';
console.log(composed === decomposed); // false
console.log(composed.normalize('NFC') === decomposed.normalize('NFC')); // true
```

`NFC` সাধারণ Canonical Composition; `NFD` Decomposition। `NFKC`/`NFKD` Compatibility Transformation করে, যা কিছু প্রতীকের অর্থপূর্ণ পার্থক্যও বদলাতে পারে। Identifier, Legal Text বা Security-Sensitive Data-তে Normalization Policy স্পষ্ট করুন।

৮.১০ Regular Expression-এর পরিচিতি

Regular Expression Pattern Matching-এর শক্তিশালী উপায়। পরবর্তী Advanced Validation অধ্যায়ে আরও গভীরভাবে আলোচনা হবে।

```
const text = 'Order A-102, Order B-205';
console.log(text.match(/[A-Z]-\d{3}/g)); // ['A-102', 'B-205']
console.log(/^\d{4}$/.test('2026'));     // true
```

Regex Flag `g` মানে Global Search, `i` Case-Insensitive, `u` Unicode-Aware Pattern। User-Provided Regex সরাসরি চালালে Invalid Pattern বা ReDoS ঝুঁকি থাকতে পারে। আমাদের Project-এ User Search Query-কে Literal Text হিসেবে Search করা হয়েছে।

৮.১১ Content SEO: Slug, Excerpt, Keyword Frequency

Slug হলো URL-Friendly Path Segment, যেমন `learn-javascript-basics`। English-Only Slug বানাতে ASCII Transliteration প্রচলিত হলেও বাংলা Unicode Slug-ও URL Encoding-এর মাধ্যমে ব্যবহারযোগ্য। Slug Uniqueness, Reserved Words, Maximum Length ও Redirect Strategy Production CMS-এ আলাদা সমস্যা।

```
function slugify(value) {
  return String(value).normalize('NFC').trim().toLocaleLowerCase('en')
    .replace(/[^\p{L}\p{M}\p{N}]+/gu, '-')
    .replace(/^-+|-+$/g, '');
}
console.log(slugify('Learn JavaScript — Step by Step!'));
// learn-javascript-step-by-step
console.log(slugify('বাংলা কনটেন্ট')); // বাংলা-কনটেন্ট
```

Excerpt সংক্ষিপ্ত Preview; এটি HTML Meta Description-এর সঙ্গে সম্পর্কিত হতে পারে, কিন্তু নির্দিষ্ট Character Count থাকলেই SEO Ranking নিশ্চিত হয় না। **Keyword Frequency** শুধু Count; Content Quality বা Search Intent-এর বিকল্প নয়।

৮.১২ Text Security: XSS ও Safe Rendering

Search Highlight দেখানোর সময় একটি সাধারণ ভুল হলো `innerHTML` দিয়ে User Content জুড়ে দেওয়া। এতে HTML Injection/XSS-এর ঝুঁকি তৈরি হয়। নিরাপদ পদ্ধতিতে `document.createTextNode()` এবং `mark.textContent` ব্যবহার করা যায়।

```
const mark = document.createElement('mark');
mark.textContent = '<script>alert(1)</script>'; // Text হিসেবে দেখাবে, চালাবে না
preview.append(mark);
```

`textContent` HTML Interpret করে না। তবে নিরাপত্তা Context-Specific: URL, CSS, Attribute, SQL ও Shell-এর জন্য আলাদা Safe Handling প্রয়োজন।

৮.১৩ Practical Project — Search-Friendly Content Tool

Scenario: একজন Content Editor বাংলা ও English Article লিখছেন। তিনি Word Count, Unicode Character Count, Slug, Short Excerpt, Most Frequent Words এবং Search Match দেখতে চান। আমরা একটি Offline Browser Tool বানাব।

Feature Requirements

1. বাংলা/English Input ও Search Phrase।
2. Word Count, Code Point Count এবং Grapheme Count।

3. Unicode-Compatible Slug Generation।
4. Configurable Excerpt Length।
5. Top Eight Word Frequency।
6. Case-Insensitive Search Highlight—HTML Injection ছাড়া।
7. Responsive UI এবং Clipboard Copy।
8. Content Browser-এ Process হবে; কোনো Server Request নেই।

Project Folder

```
source-code/chapter-08/
├── index.html
├── style.css
├── text-tools.js
├── app.js
└── text-tools.test.js
```

সম্পূর্ণ Runnable Source বইয়ের Companion Source Code-এর `source-code/chapter-08/`-এ সংযুক্ত। নিচে মূল Algorithm এবং Integration ব্যাখ্যা করা হলো।

Step 1: Pure Functions

```
const normalize = value => String(value ?? '').normalize('NFC').trim();
const words = value =>
normalize(value).match(/[\p{L}\p{M}\p{N}]+(?:['’][\p{L}\p{M}\p{N}]+)*/gu) ?? [];
const wordCount = value => words(value).length;
const characterCount = value => Array.from(normalize(value)).length;
```

এখানে `String(value ?? '')` Nullish Value-এর জন্য Empty String দেয়। `Array.from` Unicode Code Point Count করে; Grapheme Count নয়। Full Source-এ `Intl.Segmenter` ব্যবহার করা হয়েছে।

Step 2: Frequency Analysis

```
function frequency(value) {
  const map = new Map();
  for (const word of words(value)) {
    const key = word.toLocaleLowerCase();
    map.set(key, (map.get(key) ?? 0) + 1);
  }
  return [...map].sort((a, b) => b[1] - a[1] || a[0].localeCompare(b[0]));
}
```

`Map` প্রতিটি Word-এর Count ধরে। Sort Frequency অনুযায়ী Descending, সমান হলে Alphabetic। N হলো Token Count, U হলো Unique Word Count: Counting প্রায় O(N), Sorting প্রায় O(U log U)।

Step 3: Literal Search

```
function search(value, query) {
  const source = normalize(value);
  const needle = normalize(query);
  if (!needle) return [];
  const haystack = source.toLocaleLowerCase();
  const target = needle.toLocaleLowerCase();
  const matches = [];
```

```
  let start = 0;
  while (start <= haystack.length - target.length) {
    const index = haystack.indexOf(target, start);
    if (index < 0) break;
    matches.push({ index, text: source.slice(index, index + needle.length) });
    start = index + needle.length;
  }
  return matches;
}
```

Limitation: Case Mapping যদি Text Length বদলায়, Original Index ও Lowercased Index ভিন্ন হতে পারে। এই Beginner Tool-এ সাধারণ বাংলা/English Input-এর জন্য পদ্ধতিটি ব্যবহার করা হয়েছে। Full Unicode Search Engine তৈরি করতে Offset Mapping দরকার।

Step 4: Safe Highlight

```
const preview = document.getElementById('preview');
preview.replaceChildren();
let cursor = 0;
for (const match of search(content, query)) {
  preview.append(document.createTextNode(content.slice(cursor, match.index)));
  const mark = document.createElement('mark');
  mark.textContent = match.text;
  preview.append(mark);
  cursor = match.index + match.text.length;
}
preview.append(document.createTextNode(content.slice(cursor)));
```

Full UI-তে Analysis, Copy Slug, Error Message, Responsive Styling এবং Accessibility Label রয়েছে।

Run Instructions

```
# Folder: source-code/chapter-08
# Browser-এ index.html খুলুন; network প্রয়োজন নেই।
node --check text-tools.js
node --check app.js
node text-tools.test.js
```

Node.js Test-এর জন্য Modern Node Version প্রয়োজন। Browser-এ `Intl.Segmenter` না থাকলে Code Point Count Fallback দেখাবে; সেই ফল Grapheme Count-এর সমতুল্য নয়।

৮.১৪ Debugging এবং Common Mistakes

সমস্যা	ভুল ধারণা	সঠিক সমাধান
Emoji Length 2	একটি Emoji সবসময় 1 Code Unit	Code Point/Grapheme পার্থক্য বুঝুন
Search Result -1	`indexOf()` False দেয়	`-1` দিয়ে Not Found পরীক্ষা করুন
`replace()` একটি Match বদলায়	সব Match বদলায়	`replaceAll()` বা Global Regex
`split(',')` দিয়ে CSV	সব CSV Simple	Proper CSV Parser ব্যবহার করুন
`innerHTML` দিয়ে Highlight	User Text সবসময় Safe	Text Node + `textContent`
Slug Duplicate	একই Title মানেই Unique URL	Uniqueness Strategy যোগ করুন
`.toLowerCase()` সব Locale-এ সমান	Universal Case Mapping	Locale-Aware Search Design

৮.১৫ অনুশীলনী — ১০টি Exercise

1. একটি String-এর প্রথম ও শেষ Character বের করুন।
2. `slice()` দিয়ে একটি Filename-এর Extension আলাদা করুন।
3. Case-Insensitive Search Function লিখুন।
4. Multiple Spaces এক Space-এ রূপান্তর করুন।
5. `map()` দিয়ে Array of Names-কে Trim করুন।
6. Unicode Code Point ও UTF-16 Length-এর পার্থক্য দেখান।
7. `slugify()` Function লিখুন।
8. একটি Paragraph-এর Top Three Frequent Words বের করুন।
9. ২৫ Character-এর Excerpt তৈরি করুন।
10. User Search Term Highlight করার সময় XSS এড়ানোর DOM Strategy লিখুন।

৮.১৬ Exercise Solutions

```
// 1
const text = 'JavaScript';
console.log(text.at(0), text.at(-1));
// 2
const filename = 'report.final.pdf';
console.log(filename.slice(filename.lastIndexOf('.') + 1)); // pdf
// 3
const includesIgnoreCase = (source, query) =>
  String(source).toLocaleLowerCase().includes(String(query).toLocaleLowerCase());
// 4
console.log('A   B  C'.replace(/\s+/g, ' '));
// 5
console.log(['  Amina ', ' Karim  '].map(name => name.trim()));
// 6
console.log('𝌆'.length, Array.from('𝌆').length); // 2, 1
// 7
const slug = text => String(text).trim().toLocaleLowerCase().replace(/[^\p{L}\p{M}\p{N}]+/gu,'-
').replace(/^-+|-+$/g,'');
console.log(slug('Learn JavaScript!'));
// 8
const top3 = paragraph => TextTools.frequency(paragraph).slice(0, 3);
// 9
console.log(TextTools.excerpt('JavaScript is a versatile language.', 25));
// 10: document.createTextNode() + mark.textContent; user text কখনো HTML হিসেবে parse নয়
```

Note: Solution 8 ও 9 চালাতে Project-এর `text-tools.js` Load করা থাকতে হবে।

৮.১৭ Knowledge Check — ১০টি MCQ

1. String Primitive কি Mutable? A. Yes B. No C. Only in Browser D. Only Emoji
2. `'𝌆'.length` কত? A. 1 B. 2 C. 4 D. 0
3. Negative Index Support করে কোনটি? A. `at()` B. `charAt()` C. `indexOf()` D. `startsWith()`
4. `indexOf()`-এ Not Found Result? A. false B. null C. -1 D. undefined

5. সব Literal Occurrence বদলাতে? A. `replaceAll()` B. `trim()` C. `slice()` D. `split()`
6. Unicode Canonical Composition? A. NFC B. CSV C. DOM D. JSON
7. Template Literal Delimiter? A. `'` B. `"` C. Backtick D. Slash
8. Safe Plain Text Rendering? A. `innerHTML` B. `textContent` C. `eval` D. `document.write`
9. Unicode Letter Match-এর জন্য কোন Regex Class? A. `\p{L}` with `u` B. `\d` only C. `.` only D. `\s`
10. Search-Friendly Slug-এ কোনটি প্রয়োজন হতে পারে? A. Uniqueness Check B. `eval()` C. Password D. Global Variable

Answer Key ও Explanation

Q	Answer	কারণ
1	B	String Primitive Immutable
2	B	Emoji UTF-16-এ দুটি Code Unit
3	A	`at(-1)` শেষ Element দেয়
4	C	`indexOf()` Not Found হলে -1
5	A	`replaceAll()` সব Occurrence বদলায়
6	A	NFC Canonical Composition করে
7	C	Template Literal Backtick দিয়ে লেখা হয়
8	B	`textContent` HTML Parse করে না
9	A	Unicode Property Escape ও `u` Flag
10	A	Duplicate Slug URL Collision তৈরি করতে পারে

৮.১৮ Final Project Challenge

Project-এ তিনটি Feature যোগ করুন: (১) Local Draft Save/Load, (২) Stopword Exclusion Option, (৩) Keyword Density Percentage। প্রতিটি Feature-এর জন্য অন্তত একটি Automated Test লিখুন। Stopword List ভাষাভেদে ভিন্ন হবে—বাংলা ও English-এর জন্য আলাদা Config ব্যবহার করুন।

৮.১৯ Chapter Summary ও Next Step

String Processing শুধু `slice()` বা `replace()` শেখার বিষয় নয়; Unicode, Locale, Security, Search Quality এবং Content Usability-ও গুরুত্বপূর্ণ। এই অধ্যায়ের Search-Friendly Content Tool একটি Testable, Browser-Based Example। পরবর্তী **Chapter 9 — DOM Manipulation, Events এবং Interactive Web Applications**-এ Event Handling, Dynamic UI ও Task Manager Project তৈরি করব।

অধ্যায় ৯ — DOM Manipulation, Events এবং Interactive Web Applications

শেখার লক্ষ্য

এই অধ্যায় শেষে তুমি DOM Tree পড়তে, Elements নির্বাচন ও পরিবর্তন করতে, Event Handling করতে, Event Delegation ব্যবহার করতে, Form Validation এবং `localStorage` দিয়ে Persistent Browser Application তৈরি করতে পারবে। **Practical Project: Task Manager**—Add, Edit, Delete, Complete, Search, Filter এবং Save।

৯.১ Browser-এ JavaScript কীভাবে কাজ করে?

HTML হলো Document-এর কাঠামো, CSS উপস্থাপনা, আর JavaScript সেই Document-এর আচরণ নিয়ন্ত্রণ করে। Browser HTML Parse করে একটি **Document Object Model (DOM)** Tree তৈরি করে। প্রতিটি Element, Attribute এবং Text DOM-এর Node হিসেবে উপস্থাপিত হয়। DOM হলো Browser-provided API; Node.js-এ সাধারণভাবে `document` পাওয়া যায় না।

```
<main id="app">
  <h1>আমার প্রথম DOM</h1>
  <p class="message">শুরু করছি...</p>
</main>
```

এই HTML-এ `document`-এর নিচে `html`, `body`, `main`, `h1`, `p`—Parent/Child সম্পর্ক রয়েছে। `main` হলো `h1` ও `p`-এর Parent। `querySelector()` দিয়ে CSS Selector ব্যবহার করে Element খুঁজে পাওয়া যায়।

```
const heading = document.querySelector('#app h1');
const message = document.querySelector('.message');
heading.textContent = 'JavaScript Mastery';
message.textContent = 'DOM সফলভাবে পরিবর্তিত হয়েছে';
```

`querySelector()` প্রথম Matching Element অথবা `null` দেয়; `querySelectorAll()` একটি **static NodeList** দেয়। `getElementById()` ID দিয়ে Element খুঁজে পায়। `getElementsByClassName()` একটি **live HTMLCollection** দেয়। Element না পাওয়া গেলে `null` চেক করো।

```
const button = document.querySelector('#saveButton');
if (button) button.disabled = false;
```

৯.২ `textContent`, `innerHTML` ও নিরাপদ Rendering

`textContent` Text হিসেবে Value বসায়; `innerHTML` String-কে HTML হিসেবে Parse করে। User Input সরাসরি `innerHTML`-এ বসানো বিপজ্জনক: HTML Injection ও Cross-Site Scripting (XSS) হতে পারে।

```
const userInput = '<img src=x onerror=alert(1)>';
const output = document.querySelector('#output');
output.textContent = userInput; // text হিসেবেই দেখাবে
```

`innerHTML`-এ User Data ব্যবহার না করে `createElement()`, `textContent`, `append()` বা `replaceChildren()` ব্যবহার করো। Sanitization প্রয়োজন হলে Trusted, Maintained Library ও Context-specific Security Controls ব্যবহার করতে হবে।

৯.৩ Element তৈরি, যোগ ও সরানো

```
const list = document.querySelector('#items');
const li = document.createElement('li');
li.textContent = 'DOM Practice';
li.classList.add('task');
list.append(li);
// পরে প্রয়োজন হলে: li.remove();
```

`append()` Text বা Node যোগ করে; `appendChild()` শুধু Node নেয়। `prepend()` শুরুতে যোগ করে। `replaceChildren()` পুরোনো Child Nodes সরিয়ে নতুনগুলো বসায়। `classList.add/remove/toggle` CSS Class নিয়ন্ত্রণ করে; `element.dataset.id` `data-id` Attribute-এর সঙ্গে যুক্ত।

৯.৪ Events: User Interaction-এর প্রতিক্রিয়া

Browser Click, Input, Change, Submit, Keydown-এর মতো Events তৈরি করে। `addEventListener()` একটি Function-কে Event Listener হিসেবে নিবন্ধন করে।

```
const button = document.querySelector('#hello');
button.addEventListener('click', (event) => {
  console.log('Clicked:', event.currentTarget);
});
```

`event.target` হলো Event-এর উৎপত্তিস্থল; `event.currentTarget` হলো যে Element-এর Listener এখন চলছে। `event.preventDefault()` Cancelable Event-এর Default Action বন্ধ করে (যেমন Form Submit-এর Navigation)। `event.stopPropagation()` Event-এর পরবর্তী Propagation বন্ধ করে; সাধারণ ক্ষেত্রে অযথা ব্যবহার করা উচিত নয়।

Event Propagation

Event সাধারণত **Capture** → **Target** → **Bubble** পর্যায়ে চলে। Parent Container-এ Listener রেখে Child Button-এর Event Handle করাকে **Event Delegation** বলা হয়। Dynamic List-এ প্রতিটি Button-এ Listener বসানোর বদলে Parent List-এ একটি Listener রাখা সুবিধাজনক।

```
const list = document.querySelector('#taskList');
list.addEventListener('click', (event) => {
  const button = event.target.closest('button[data-action]');
  if (!button || !list.contains(button)) return;
  const row = button.closest('li[data-id]');
  if (!row) return;
  console.log(button.dataset.action, row.dataset.id);
});
```

৯.৫ Form Handling ও Validation

Form-এর `submit` Event Enter Key দিয়েও Trigger হয়। শুধু Button-এর `click` Listener দিলে Keyboard Flow বাদ পড়তে পারে। HTML Validation (`required`, `maxlength`) ও JavaScript Validation একসঙ্গে ব্যবহার করো। Frontend Validation User Experience বাড়ায়, কিন্তু Backend থাকলে Server-side Validation অবশ্যই প্রয়োজন।

```
form.addEventListener('submit', (event) => {
  event.preventDefault();
  const title = titleInput.value.trim();
  if (!title) return;
  console.log('New task:', title);
  form.reset();
});
```

৯.৬ State ও UI আলাদা রাখা

State হলো App-এর Data; UI হলো সেই Data-এর Visual Representation। ভালো Design হলো `state → render() → DOM`। Event হলে Data Update হবে, তারপর `render()` DOM Update করবে। এতে Logic Testing সহজ হয়।

```
let tasks = [];
function render() {
  list.replaceChildren();
  for (const task of tasks) {
    const li = document.createElement('li');
    li.textContent = task.title;
    list.append(li);
  }
}
```

বড় Application-এ Full Re-render-এর পরিবর্তে Selective Update বা Framework Rendering দরকার হতে পারে; ছোট Task Manager-এর জন্য Full Re-render যথেষ্ট।

৯.৭ Browser Storage: `localStorage`

`localStorage` Key/Value Pair সংরক্ষণ করে এবং Value String হয়। `JSON.stringify()` দিয়ে Array/Object String-এ রূপান্তর এবং `JSON.parse()` দিয়ে পুনরুদ্ধার করা যায়। Storage Browser/Origin অনুযায়ী আলাদা। Private Browsing, Quota, Browser Policy বা Disabled Storage-এর কারণে Access ব্যর্থ হতে পারে—তাই `try/catch` ব্যবহার করো। Password, Token বা Sensitive Personal Data `localStorage`-এ রাখা উচিত নয়।

```
const KEY = 'demo-tasks';
try {
  localStorage.setItem(KEY, JSON.stringify([{ id: '1', title: 'Read DOM' }]));
  const tasks = JSON.parse(localStorage.getItem(KEY) ?? '[]');
  console.log(tasks);
} catch (error) {
  console.warn('Storage unavailable or invalid:', error);
}
```

Production Application-এ Parsed Data-এর Shape Validate করা জরুরি। `JSON.parse()` সফল হলেও Data প্রত্যাশিত Array বা Task Object নাও হতে পারে।

৯.৮ Practical Project — Task Manager

Requirements

1. Task Add: ফাঁকা Title Reject, Maximum 120 Characters।
2. Task Complete/Undo: Checkbox দিয়ে Status পরিবর্তন।
3. Task Edit/Delete: প্রতিটি Row থেকে Action।
4. Search ও Filter: All / Active / Completed।
5. Summary: Total, Active, Completed Count।
6. Persistence: Reload-এর পর Saved Tasks Restore।
7. Security: Task Title DOM-এ `textContent` দিয়ে Render।
8. Accessibility: Form Label, Keyboard Submit, `aria-live` Status।

Project Structure

```
source-code/chapter-09/
├── index.html
├── style.css
├── task-core.js
├── app.js
└── task-core.test.js
```

Source Code: বইয়ের Companion Source Code-এর `source-code/chapter-09/` Folder-এ সব File সম্পূর্ণ রয়েছে।

Architecture

```
User Event → DOM Listener → TaskCore (Pure Function)
                                  ↓
                          Updated Task Array
                                  ↓
                    localStorage + render()
                                  ↓
                           Updated UI
```

`task-core.js`-এ Pure Functions রাখা হয়েছে; DOM ও Storage Code `app.js`-এ। এতে Node.js-এর Built-in Test Runner দিয়ে Core Logic আলাদাভাবে পরীক্ষা করা যায়।

Run

1. `source-code/chapter-09/index.html` Browser-এ খুলো।
2. Task Add, Complete, Edit, Delete, Search ও Filter পরীক্ষা করো।
3. Page Reload করে Persistence পরীক্ষা করো।
4. Node.js 18+ থাকলে Terminal-এ `node --test task-core.test.js` চালাও।

Limitation: Browser Storage Policy ও Local File Context অনুযায়ী Behaviour ভিন্ন হতে পারে। নির্ভরযোগ্য Local Development-এর জন্য Static HTTP Server ব্যবহার করো, যেমন `python -m http.server 8000`, তারপর `http://localhost:8000`।

৯.৯ Debugging Checklist

সমস্যা	সম্ভাব্য কারণ	সমাধান
`Cannot read properties of null`	Element Selector ভুল বা Script আগে চলছে	Script Body-এর শেষে রাখো, Selector যাচাই করো
Button Click কাজ করছে না	Listener/Selector ভুল	DevTools Console, Event Listener Inspect
Form Reload হয়ে যাচ্ছে	`preventDefault()` নেই	Submit Handler-এ যোগ করো
Tasks Reload-এ হারায়	Storage ব্যর্থ বা Key বদলেছে	DevTools Application Tab দেখো
HTML Injection	`innerHTML`-এ User Data	`textContent` ব্যবহার করো
Dynamic Buttons কাজ করছে না	নতুন Elements-এ Listener নেই	Event Delegation ব্যবহার করো

৯.১০ Exercises

Exercise 1: `#title` Element-এর Text বদলে `DOM Mastery` করো।

Exercise 2: `ul#list`-এ নতুন `li` তৈরি করে যোগ করো।

Exercise 3: একটি Button Click করলে `.dark` Class Toggle করো।

Exercise 4: Form Submit করলে Input-এর Trimmed Value Log করো; Empty Input Reject করো।

Exercise 5: একটি List-এ Event Delegation দিয়ে Clicked Item-এর `data-id` পড়ো।

Exercise 6: Array of Tasks থেকে Completed Task Filter করো।

Exercise 7: `localStorage`-এ Array Save ও Restore করো; Invalid JSON Handle করো।

Exercise 8: `innerHTML`-এর বদলে `textContent` ব্যবহার করে User Text নিরাপদে Render করো।

Exercise 9: Task Manager-এ `clearCompleted()` যুক্ত করো।

Exercise 10: Search Query Case-insensitive করো।

Exercise Solutions

```
// 1
const titleNode = document.querySelector('#title');
if (titleNode) titleNode.textContent = 'DOM Mastery';

// 2
const ul = document.querySelector('#list');
if (ul) { const li = document.createElement('li'); li.textContent = 'New item'; ul.append(li);
}
```

```
// 3
const toggle = document.querySelector('#toggle');
toggle?.addEventListener('click', () => document.body.classList.toggle('dark'));

// 4
form.addEventListener('submit', e => { e.preventDefault(); const v = input.value.trim(); if (v)
console.log(v); });

// 5
list.addEventListener('click', e => { const item = e.target.closest('li[data-id]'); if (item &&
list.contains(item)) console.log(item.dataset.id); });

// 6
const completed = tasks.filter(task => task.completed);

// 7
try { localStorage.setItem('tasks', JSON.stringify(tasks)); const restored =
JSON.parse(localStorage.getItem('tasks') ?? '[]'); console.log(Array.isArray(restored) ?
restored : []); } catch { console.log([]); }

// 8
output.textContent = userProvidedText;

// 9
const activeTasks = tasks.filter(task => !task.completed);

// 10
const results = tasks.filter(task =>
task.title.toLocaleLowerCase().includes(query.trim().toLocaleLowerCase()));
```

নোট: Solution Snippets-এ `form`, `input`, `list`, `tasks`, `output` ইত্যাদি সংশ্লিষ্ট Exercise-এর পূর্বনির্ধারিত Variables। এগুলো একত্রে Copy করে চালানোর জন্য নয়।

৯.১১ MCQ Quiz

1. DOM-এর পূর্ণরূপ কী? (A) Data Object Memory (B) Document Object Model (C) Dynamic Order Method (D) Document Output Map
2. `querySelector()` Match না পেলে কী দেয়? (A) `[]` (B) `undefined` (C) `null` (D) Error
3. User Text Render-এর জন্য নিরাপদ সাধারণ পদ্ধতি? (A) `innerHTML` (B) `eval()` (C) `textContent` (D) `document.write()`
4. Form Submit-এর Default Navigation থামায় কোনটি? (A) `stop()` (B) `preventDefault()` (C) `pause()` (D) `cancel()`
5. Event Delegation-এর Listener সাধারণত কোথায় থাকে? (A) Parent (B) প্রতিটি Child-এ বাধ্যতামূলক (C) CSS (D) Server
6. `localStorage` Value-এর Type কী? (A) Array (B) Object (C) String (D) Function
7. `JSON.stringify()` কী করে? (A) JSON Text তৈরি (B) DOM তৈরি (C) CSS তৈরি (D) Network Request
8. `classList.toggle()` কী করে? (A) Remove All (B) Class Add/Remove Toggle (C) Change ID (D) Reload
9. `event.target` কী নির্দেশ করে? (A) Listener Registered Element সবসময় (B) Event-এর Originating Target (C) Browser Window সবসময় (D) CSS Class

10. Browser DOM API ছাড়া Core Logic Test সহজ করতে কী করব? (A) সব Logic Event Handler-এ (B) Pure Functions আলাদা (C) HTML-এ Inline JS (D) Tests বাদ

Answer Key ও Explanation

1. **B** — DOM হলো Document Object Model।
2. **C** — প্রথম Matching Element না থাকলে `null`।
3. **C** — `textContent` String-কে HTML Parse করে না।
4. **B** — `preventDefault()` Cancelable Default Action থামায়।
5. **A** — Parent Container Listener Child Event Handle করতে পারে।
6. **C** — Web Storage API String Values সংরক্ষণ করে।
7. **A** — JavaScript Value থেকে JSON String তৈরি হয়।
8. **B** — নির্দিষ্ট CSS Class থাকলে সরায়, না থাকলে যোগ করে।
9. **B** — Event-এর Target এবং Listener-এর Current Target আলাদা হতে পারে।
10. **B** — Pure Logic DOM ছাড়াই Unit Test করা যায়।

৯.১২ অধ্যায়ের সারাংশ ও পরবর্তী অধ্যায়

DOM Selection, Secure Rendering, Events, Delegation, Form Handling, State, Storage ও Testing একত্রে একটি কার্যকর Browser App তৈরির ভিত্তি। পরের অধ্যায়ে শিখব **Asynchronous JavaScript, Promises, async/await ও Event Loop**—যা API-driven Web Application তৈরির জন্য অপরিহার্য।

অধ্যায় ১০ — Asynchronous JavaScript: Callbacks, Promises, async/await এবং Event Loop

শেখার লক্ষ্য

এই অধ্যায় শেষে তুমি Synchronous ও Asynchronous Execution-এর পার্থক্য, Callbacks, Promise State, `async/await`, Event Loop, Microtask ও Task Queue, `fetch()`, HTTP Error, `AbortController`, Concurrent Requests এবং User-Friendly Loading/Error States ব্যাখ্যা ও প্রয়োগ করতে পারবে। Practical Project: **User Directory API Explorer**।

১০.১ JavaScript কি Single-Threaded?

সাধারণ Browser JavaScript Execution Context-এ একটি Main Thread এক সময়ে একটি JavaScript Call Stack চালায়। কিন্তু Browser-এর Network, Timer, DOM Event-এর মতো সুবিধা Asynchronous কাজ পরিচালনা করতে পারে। Node.js-এও Event Loop এবং Host APIs আছে; CPU-intensive JavaScript Main Thread আটকে দিতে পারে। **Single-threaded JavaScript মানে সব I/O কাজ Blocking নয়।**

```
console.log('A');
setTimeout(() => console.log('C'), 0);
console.log('B');
// A, B, C
```

`setTimeout(..., 0)` সঙ্গে সঙ্গে Callback চালায় না; নির্ধারিত সময়ের আগে নয়, Event Loop সুযোগ পেলে Timer Task চলবে। Real scheduling Browser/Node Host-এর ওপর নির্ভর করে।

১০.২ Call Stack, Web APIs ও Event Loop

একটি Function Call Stack-এ Push হয়; Return করলে Pop হয়। `fetch()` Network Request শুরু করে এবং সঙ্গে সঙ্গে Promise দেয়। Response Ready হলে সংশ্লিষ্ট Promise Reaction Microtask Queue-তে Schedule হয়। Stack খালি হলে Host Microtask Checkpoint-এ Microtasks চালায়। Browser-এ Rendering-এর সুযোগ সাধারণত Task-এর মাঝামাঝি আসে; দীর্ঘ JavaScript Loop UI Freeze করতে পারে।

```
JavaScript Call Stack
        |
        v
  Host APIs (timer / fetch / events)
        |
        +--> Task queue (timers, events)
        +--> Microtask queue (Promise reactions)
        |
        v
 Event Loop schedules execution
```

এই Diagram একটি Conceptual Model, Browser Specification-এর পূর্ণ Scheduler নয়।

১০.৩ Callbacks: প্রথম Asynchronous Pattern

```
function afterDelay(message, callback) {
  setTimeout(() => callback(null, message), 300);
}
afterDelay('Hello', (error, value) => {
  if (error) return console.error(error);
  console.log(value);
});
```

Callback একটি Function যা অন্য Function-কে দেওয়া হয়। সব Callback Asynchronous নয়: `array.map(callback)` Synchronous। Nested Callbacks বেশি হলে Control Flow ও Error Handling জটিল হতে পারে।

১০.৪ Promise: Future Result-এর Representation

Promise-এর তিনটি State: **pending**, **fulfilled**, **rejected**। Fulfilled বা Rejected হলে Promise Settled। `then()` Success, `catch()` Rejection এবং `finally()` Cleanup Handle করে। `then()` সবসময় নতুন Promise দেয়, তাই Chaining সম্ভব।

```
const task = new Promise((resolve) => {
  setTimeout(() => resolve(42), 200);
});
task.then(value => value * 2)
    .then(value => console.log(value)) // 84
    .catch(error => console.error(error))
    .finally(() => console.log('finished'));
```

`throw` করলে Promise Chain Rejected হয়। `catch()`-এর ভেতরে Error Return করলে Chain Recover করে; পুনরায় Throw করলে Rejection বজায় থাকে। Unhandled Rejection এড়াতে যথাযথ Error Handling করো।

১০.৫ `async` এবং `await`

`async` Function সবসময় Promise Return করে। `await` ওই Async Function-এর Execution Pause করে, পুরো JavaScript Thread নয়। Rejected Promise `await` করলে Exception-এর মতো আচরণ করে, তাই `try/catch` ব্যবহার করা যায়।

```
async function getGreeting() {
  const value = await Promise.resolve('Welcome');
  return `${value} to JavaScript Mastery`;
}
getGreeting().then(console.log);
```

`await` শুধু Async Function বা Module-এর Top Level-এ বৈধ (সমর্থিত Module Environment-এ)। `forEach(async () => ...)`-এর Return Promises `forEach` অপেক্ষা করে না। Sequential Loop চাইলে `for...of` + `await`, Concurrent কাজ চাইলে `Promise.all(items.map(async ...))`।

১০.৬ Microtask বনাম Task: Output Prediction

```
console.log('1');
```

```
setTimeout(() => console.log('4'), 0);
Promise.resolve().then(() => console.log('3'));
console.log('2');
// 1, 2, 3, 4
```

কারণ Synchronous Statements আগে চলে, তারপর Promise Reaction Microtask, তারপর Timer Task। `queueMicrotask()` দিয়েও Microtask Schedule করা যায়। অতিরিক্ত Microtasks Schedule করতে থাকলে Rendering ও অন্য Tasks Starve হতে পারে।

Interactive Thought Experiment

```
async function demo() {
  console.log('B');
  await Promise.resolve();
  console.log('D');
}
console.log('A');
demo();
console.log('C');
// A, B, C, D
```

কেন? `await`-এর পরের অংশ Promise Reaction হিসেবে পরে চলে। এটি বোঝা Debugging-এর জন্য গুরুত্বপূর্ণ।

১০.৭ `fetch()` দিয়ে API Call

`fetch()` Network Failure বা Abort-এর জন্য Reject করতে পারে; কিন্তু HTTP `404`/`500` Response এলে সাধারণত Promise Fulfill হয়। তাই `response.ok` Check করতে হবে। JSON Parsing-ও Fail করতে পারে।

```
async function loadUsers() {
  const response = await fetch('https://jsonplaceholder.typicode.com/users');
  if (!response.ok) throw new Error(`HTTP ${response.status}`);
  const data = await response.json();
  if (!Array.isArray(data)) throw new TypeError('Unexpected API response');
  return data;
}
```

Production Note: Public Demo API Availability, Rate Limits, CORS, Privacy ও Network Reliability পরিবর্তিত হতে পারে। Secrets Frontend JavaScript-এ Embed করা যাবে না।

১০.৮ Request Cancellation ও Race Condition

User Search বা Reload দ্রুত বদলালে পুরোনো Request-এর Response পরে এসে নতুন Data Overwrite করতে পারে। `AbortController` আগের Request Cancel করতে সাহায্য করে। তবে Abort হওয়ার আগেই Response Settle হলে Request-ID Check-ও দরকার হতে পারে।

```
let controller;
let latest = 0;
async function searchRemote(query) {
  controller?.abort();
  controller = new AbortController();
  const requestId = ++latest;
  try {
    const url = `/api/search?q=${encodeURIComponent(query)}`;
```

```
    const response = await fetch(url, { signal: controller.signal });
    if (!response.ok) throw new Error(`HTTP ${response.status}`);
    const result = await response.json();
    if (requestId !== latest) return;
    console.log(result);
  } catch (error) {
    if (error.name !== 'AbortError') console.error(error);
  }
}
```

এই Snippet-এ `/api/search` একটি উদাহরণ Endpoint, বাস্তব Project-এর Dependency নয়।

১০.৯ Sequential বনাম Concurrent Work

```
// Sequential: একটি শেষ হওয়ার পরে আরেকটি
const first = await fetch('/api/first');
const second = await fetch('/api/second');

// Concurrent: উভয় Request শুরু, তারপর সব শেষ হওয়ার অপেক্ষা
const [a, b] = await Promise.all([
  fetch('/api/first'), fetch('/api/second')
]);
```

দুটি Request Independent হলে Concurrent Pattern Latency কমাতে পারে। `Promise.all()` একটি Promise Reject হলে দ্রুত Reject করে, তবে অন্য Operation স্বয়ংক্রিয়ভাবে Cancel হয় না। `Promise.allSettled()` সব ফলাফল (Fulfilled/Rejected) সংগ্রহ করে। `Promise.race()` প্রথম Settled Result দেয়; `Promise.any()` প্রথম Fulfilled Result দেয়, সব Reject হলে `AggregateError` দেয়।

API	আচরণ	ব্যবহার
`Promise.all`	সব Fulfill হলে Result, প্রথম Rejection-এ Reject	সব ফলাফল আবশ্যক
`Promise.allSettled`	সব Settle হওয়া পর্যন্ত অপেক্ষা	Partial Failure গ্রহণযোগ্য
`Promise.race`	প্রথম Settled Promise-এর ফল	Timeout Pattern-এর অংশ
`Promise.any`	প্রথম Fulfilled Result	বিকল্প Service/Replica

১০.১০ Retry, Timeout ও Backoff

সব Error-এ Retry উপযুক্ত নয়। `GET`-এর মতো Idempotent Request এবং Transient `429/503` Error-এ সীমিত Retry বিবেচ্য। Payment বা Booking Create-এর মতো Non-idempotent Action অন্ধভাবে Retry করলে Duplicate Transaction হতে পারে। `AbortSignal.timeout(ms)` আধুনিক Environment-এ Timeout Signal দিতে পারে; Compatibility যাচাই করতে হবে। `Promise.race()`-এ Timeout দিলেই Underlying Network Request Cancel হয় না।

```
async function fetchWithTimeout(url, ms = 5000) {
  const controller = new AbortController();
  const timer = setTimeout(() => controller.abort(), ms);
  try {
    const response = await fetch(url, { signal: controller.signal });
    if (!response.ok) throw new Error(`HTTP ${response.status}`);
    return response;
  } finally {
```

```
    clearTimeout(timer);
  }
}
```

১০.১১ Practical Project — User Directory API Explorer

Project Goal

একটি Responsive Browser Application, যেখানে Public API থেকে Users Load হবে; Loading, Error, Reload, Search, Sorting ও Cancellation Support থাকবে। Source Code বইয়ের Companion Source Code-এর `source-code/chapter-10/` Directory-তে রয়েছে।

Project Structure

```
source-code/chapter-10/
├── index.html
├── style.css
├── app.mjs
├── directory-core.mjs
└── directory-core.test.mjs
```

Running the Project

1. `source-code/chapter-10` Directory-তে Terminal খোলো।
2. `python -m http.server 8000` চালাও (অথবা VS Code Live Server ব্যবহার করো)।
3. Browser-এ `http://localhost:8000` খোলো। ES Modules-এর জন্য Local HTTP Server ব্যবহার করা সুবিধাজনক।
4. Internet Connection থাকলে JSONPlaceholder Demo API থেকে Users আসবে।
5. Search, Sort ও Reload পরীক্ষা করো; DevTools Network Tab-এ Request Observe করো।
6. Tests: `node --test directory-core.test.mjs` (Node.js 18+ Recommended)।

HTML: `index.html`

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width,initial-scale=1">
  <title>User Directory API Explorer</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <main>
    <header>
      <h1>User Directory API Explorer</h1>
      <p>Async JavaScript • Fetch • AbortController • Search</p>
    </header>
    <form id="controls" onsubmit="return false">
      <label>Search <input id="search" type="search" placeholder="Name, email, city"></label>
      <label>Sort by
        <select id="sort"><option value="name">Name</option><option
value="email">Email</option><option value="city">City</option></select>
      </label>
      <button id="reload" type="button">Reload users</button>
```

```
    </form>
    <p id="status" role="status" aria-live="polite">Loading…</p>
    <ul id="users" aria-label="Users"></ul>
  </main>
  <script type="module" src="app.mjs"></script>
</body>
</html>
```

CSS ও JavaScript Files

`style.css`, `app.mjs`, `directory-core.mjs`-এর সম্পূর্ণ, Executable Source Code Project Directory-তে দেওয়া আছে। এখানে গুরুত্বপূর্ণ অংশগুলো ব্যাখ্যা করছি।

```
// directory-core.mjs
export async function fetchUsers({ fetchImpl = fetch, signal,
  url = 'https://jsonplaceholder.typicode.com/users' } = {}) {
  const response = await fetchImpl(url, {
    signal,
    headers: { Accept: 'application/json' }
  });
  if (!response.ok) throw new Error(`HTTP ${response.status}: users unavailable`);
  const data = await response.json();
  if (!Array.isArray(data)) throw new TypeError('Expected an array of users');
  return data.map(normalizeUser);
}
```

`fetchImpl` Inject করার ফলে Real Network ছাড়াই Mock Function দিয়ে Unit Test করা যায়। `normalizeUser` API Response-এর Field Shape Verify করে। `filterUsers` Query Match করে এবং `sortUsers` Original Array Mutate না করে Copy Sort করে।

```
// app.mjs — important fragment
async function loadUsers() {
  controller?.abort();
  controller = new AbortController();
  const currentRequest = ++requestNumber;
  reload.disabled = true;
  status.textContent = 'Loading users…';
  try {
    users = await fetchUsers({ signal: controller.signal });
    if (currentRequest === requestNumber) render();
  } catch (error) {
    if (currentRequest !== requestNumber || error.name === 'AbortError') return;
    users = [];
    list.replaceChildren();
    status.textContent = 'Could not load users. Check connection and retry.';
    console.error(error);
  } finally {
    if (currentRequest === requestNumber) reload.disabled = false;
  }
}
```

Security, Accessibility ও UX

- `textContent` দিয়ে User Data Render করা হয়েছে; Raw `innerHTML` ব্যবহার করা হয়নি।
- `role="status"` ও `aria-live="polite"` Loading/Result Announcements-এ সাহায্য করে।
- `AbortController` দিয়ে পুরোনো Request Cancel করা হয়; Request Counter Stale Result ঠেকায়।
- Network Error হলে Clear Message দেখায়; Reload Button আবার চেষ্টা করতে দেয়।

- Browser Fetch ও JSONPlaceholder API-এর জন্য Network Access দরকার; Offline হলে Failure Message স্বাভাবিক।

১০.১২ Common Mistakes ও Debugging

ভুল	কারণ	সমাধান
`await` ছাড়া Promise Log	Result এখনো Pending	`await` বা `.then()`
HTTP 404-কে `catch` ধরবে ভাবা	`fetch` সাধারণত HTTP Error-এ Reject করে না	`response.ok` Check
`forEach(async ...)` Await করা	`forEach` Promise Aggregate করে না	`for...of` বা `Promise.all`
Loading State না দেখানো	User মনে করে App আটকে গেছে	Status Text/Spinner
পুরোনো Response নতুন Result Overwrite	Request Race	Abort + Request-ID
Unbounded Concurrent Requests	API/Browser Overload	Limit Concurrency
`innerHTML`-এ API Data	XSS Risk	`textContent`/DOM Nodes
Timer Delay-কে Exact Timing ধরা	Event Loop Scheduling	Delay-কে Minimum Treat

১০.১৩ Exercises

1. `setTimeout(...,0)` কেন Synchronous Statement-এর পরে চলে—ব্যাখ্যা করো।
2. একটি Callback-based `readLater(value, callback)` Function লেখো।
3. `Promise.resolve(5)`-কে Chain করে 20 Output করো।
4. `async/await` দিয়ে দুটি Sequential Promise Resolve করো।
5. Independent দুটি Request Concurrent করার Code লেখো।
6. `Promise.allSettled()` দিয়ে Partial Failure Handle করো।
7. `fetch()`-এর HTTP 404 Error কীভাবে Handle করবে?
8. `AbortController` দিয়ে User Search-এর পুরোনো Request Cancel করো।
9. `filterUsers()`-এ Search Case-Insensitive করো।
10. Project-এ Empty Result-এর জন্য আলাদা Message যোগ করো।

১০.১৪ Exercise Solutions

১. Timer Callback একটি পরবর্তী Task হিসেবে Queue-তে যায়; বর্তমান Call Stack শেষ হওয়ার আগে চলতে পারে না।

২.

```
function readLater(value, callback) {
  setTimeout(() => callback(null, value), 100);
}
readLater('done', (error, value) => console.log(error ?? value));
```

৩.

```
Promise.resolve(5).then(n => n * 4).then(console.log); // 20
```

৪.

```
async function sequential() {
  const a = await Promise.resolve(3);
  const b = await Promise.resolve(4);
  return a + b;
}
sequential().then(console.log); // 7
```

৫.

```
const [a, b] = await Promise.all([fetch('/api/a'), fetch('/api/b')]);
```

৬.

```
const results = await Promise.allSettled([Promise.resolve(1), Promise.reject('offline')]);
for (const result of results) console.log(result.status, result.status === 'fulfilled' ?
result.value : result.reason);
```

৭.

```
const response = await fetch('/missing');
if (!response.ok) throw new Error(`HTTP ${response.status}`);
```

৮. নতুন Request শুরু করার আগে `controller?.abort()` এবং নতুন `AbortController` তৈরি করো; `fetch(url, { signal: controller.signal })` ব্যবহার করো। Project-এর `app.mjs`-এ পূর্ণ Example রয়েছে।

৯.

```
const match = user.name.toLocaleLowerCase().includes(query.toLocaleLowerCase());
```

১০. `render()`-এ `filtered.length === 0` হলে `status.textContent = 'No matching users found'` দেখাও; List Empty রাখো।

১০.১৫ Knowledge Check — ১০টি MCQ

Q1. `async` Function কী Return করে? (A) Number (B) Promise (C) Timer (D) Event — **উত্তর: B**। Return Value Promise দিয়ে Wrap হয়।

Q2. `fetch()` HTTP 404 পেলে সাধারণত কী হয়? (A) Reject (B) Fulfill with Response (C) Crash (D) Retry — **উত্তর: B**। `response.ok` Check করতে হয়।

Q3. Promise Reaction সাধারণত কোথায় Schedule হয়? (A) Call Stack immediately (B) Microtask Queue (C) CSS Queue (D) Storage — **উত্তর: B**। Current Stack শেষ হওয়ার পরে Microtask চলে।

Q4. `Promise.all()`-এ একটি Promise Reject হলে কী হয়? (A) সব Cancel (B) Aggregate Reject (C) সব Ignore (D) Browser Close — **উত্তর: B**। অন্য Operations স্বয়ংক্রিয় Cancel হয় না।

Q5. `await` কী Pause করে? (A) Entire Browser (B) Current Async Function Continuation (C) CSS (D) Server — **উত্তর: B**। Thread পুরোপুরি Block হয় না।

Q6. Abort Signal কোন API-তে দেওয়া যায়? (A) `fetch` (B) `JSON.stringify` (C) `Math.random` (D) `parseInt` — **উত্তর: A**। Request Cancellation-এ সাহায্য করে।

Q7. সব Promise Settled হওয়ার Result পেতে কোনটি? (A) `allSettled` (B) `race` (C) `any` (D) `resolve` — **উত্তর: A**। Success ও Failure দুটোই Report করে।

Q8. API Data DOM-এ Text হিসেবে নিরাপদে দেখাতে কোনটি? (A) `innerHTML` (B) `eval` (C) `textContent` (D) `document.write` — **উত্তর: C**। Text HTML হিসেবে Parse হয় না।

Q9. Independent Requests দ্রুত শুরু করতে কোনটি? (A) Sequential `await` (B) `Promise.all` (C) `while(true)` (D) `alert` — **উত্তর: B**। Concurrent Operations-এর জন্য।

Q10. Stale Response Problem-এর সমাধান কী? (A) Ignore errors (B) Abort + Request ID (C) More timers (D) `var` — **উত্তর: B**। পুরোনো Request নতুন UI Overwrite করতে পারে না।

১০.১৬ Chapter Summary ও Next Steps

এই অধ্যায়ে Async Execution-এর Mental Model, Promise Chaining, `async/await`, Microtask Scheduling, HTTP Response Handling, Cancellation, Concurrency এবং User-Friendly Async UI তৈরি করা শিখেছ। Project Source ও Unit Tests আলাদা Files-এ রয়েছে। **পরবর্তী অধ্যায়: Error Handling, Debugging এবং Defensive Programming।**

Further Reading

- MDN — JavaScript Guide: Using Promises: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Using_promises
- MDN — `async function`: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Statements/async_function
- MDN — Fetch API: https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API
- MDN — AbortController: https://developer.mozilla.org/en-US/docs/Web/API/AbortController
- MDN — Event Loop: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Execution_model

অধ্যায় ১১ — Error Handling, Debugging এবং Defensive Programming

শেখার লক্ষ্য

এই অধ্যায় শেষে তুমি JavaScript-এর Error Type চেনা, `try/catch/finally`, `throw`, Custom Error, Asynchronous Rejection, Browser DevTools, Breakpoints, Defensive Validation এবং User-Friendly Error Reporting ব্যবহার করতে পারবে। **Practical Project: OrderGuard — Order Validation Dashboard**।

১১.১ Error কেন গুরুত্বপূর্ণ?

Production Application-এ সব Input সঠিক হবে না, Network সবসময় সচল থাকবে না, এবং External Data নির্ভরযোগ্য নাও হতে পারে। একটি ভুলকে পুরো App Crash-এ পরিণত না করে **সুনির্দিষ্টভাবে শনাক্ত, ব্যাখ্যা ও নিয়ন্ত্রণ** করাই Error Handling। Error Handling কখনো Validation-এর বিকল্প নয়; বরং Validation-এর ব্যর্থতাকে নিরাপদভাবে উপস্থাপন করার উপায়।

তিনটি গুরুত্বপূর্ণ শ্রেণি:

1. **Syntax Error:** JavaScript Parse করার সময় ভুল; Code চালুর আগেই থেমে যেতে পারে।
2. **Runtime Error:** Code চলার সময় সমস্যা, যেমন `null.name` পড়া।
3. **Logical Error:** Code চলে, কিন্তু ফল ভুল; যেমন Discount ভুলভাবে হিসাব করা। Unit Tests এ ধরনের সমস্যা ধরতে সাহায্য করে।

```
// Syntax error (শুধু উদাহরণ, চালাবে না): const = 42;
const user = null;
// console.log(user.name); // TypeError
const total = 100 - 10; // Discount 10% হলে এটি ভুল গণনা; সঠিক ফল 90, কিন্তু 1000 হলে একই formula ভুল
```

১১.২ Built-in Error Objects

`Error`, `TypeError`, `ReferenceError`, `RangeError`, `SyntaxError` প্রভৃতি Standard Error Object। `error.message` ব্যবহারকারীর উপযোগী Text নাও হতে পারে; `error.stack` মূলত Developer Diagnostics-এর জন্য।

```
function readPrice(item) {
  if (!item || typeof item !== 'object') {
    throw new TypeError('Item must be an object');
  }
  if (typeof item.price !== 'number') {
    throw new TypeError('Item price must be a number');
  }
  return item.price;
}
```

`throw` দিয়ে যে কোনো JavaScript Value Throw করা যায়, কিন্তু **Error Instance** Throw করা উত্তম: Stack ও Error Type সহজে পাওয়া যায়।

১১.৩ `try`, `catch` এবং `finally`

```
function safeParseJSON(text) {
  try {
    return { ok: true, value: JSON.parse(text) };
  } catch (error) {
    if (error instanceof SyntaxError) {
      return { ok: false, message: 'Invalid JSON format' };
    }
    throw error; // অপরিচিত Error চাপা দিচ্ছি না
  } finally {
    // প্রয়োজন হলে Cleanup বা Diagnostic Logging
  }
}
console.log(safeParseJSON('{"name":"Amina"}'));
console.log(safeParseJSON('{broken'));
```

`finally` সাধারণত `try/catch`-এর পরে চলে, এমনকি Return বা Throw থাকলেও। তবে `finally`-তে `return` দিলে আগের Return/Throw Override হতে পারে—এটি এড়িয়ে চলো।

১১.৪ Expected Failure বনাম Unexpected Failure

একটি Invalid Form Input **Expected Failure**: User-কে কোন Field সংশোধন করতে হবে বলো। কিন্তু `Cannot read properties of undefined` **Unexpected Failure**: Developer Log-এ Detail রাখো, User-কে সাধারণ বার্তা দেখাও। Sensitive Data, Access Token বা Password Console কিংবা Error Message-এ প্রকাশ করবে না।

```
class ValidationError extends Error {
  constructor(message, issues = []) {
    super(message);
    this.name = 'ValidationError';
    this.issues = issues;
  }
}

function assertName(name) {
  if (typeof name !== 'string' || name.trim().length < 2) {
    throw new ValidationError('Invalid customer', ['Name needs at least 2 characters']);
  }
  return name.trim();
}
```

১১.৫ Defensive Programming: Trust Boundaries

Browser Input, URL Parameters, `localStorage`, Server Responses—সব External Data যাচাই করা প্রয়োজন। `typeof` যথেষ্ট নয়: `typeof null === 'object'`, `NaN`-এর Type-ও `number`। Numeric Input-এ `Number.isFinite()` বা `Number.isSafeInteger()` প্রয়োগ করো।

```
function parsePositiveInteger(input) {
  const text = String(input).trim();
  if (!/^[1-9]\d*$/.test(text)) return null;
  const value = Number(text);
  return Number.isSafeInteger(value) ? value : null;
```

```
}
console.log(parsePositiveInteger('12'));   // 12
console.log(parsePositiveInteger('12px')); // null
console.log(parsePositiveInteger('1.5'));  // null
```

`parseInt('12px', 10)` কিন্তু `12` ফেরত দেয়। Strict Form Validation-এ এটি প্রায়ই অনাকাঙ্ক্ষিত। আবার অতিরিক্ত Validation-ও ক্ষতিকর হতে পারে; Business Rules অনুযায়ী পরিষ্কার Constraint নির্ধারণ করো।

১১.৬ Async Error Handling

```
async function getUser(id, signal) {
  const response = await fetch(`/api/users/${encodeURIComponent(id)}`, { signal });
  if (!response.ok) throw new Error(`HTTP ${response.status}`);
  return response.json();
}
async function loadUser(id) {
  const controller = new AbortController();
  try {
    return await getUser(id, controller.signal);
  } catch (error) {
    if (error?.name === 'AbortError') return null;
    console.error('User request failed', error);
    throw error;
  }
}
```

মনে রাখবে: `fetch()` সাধারণত HTTP 404/500-এ Promise Reject করে না; `response.ok` পরীক্ষা করতে হয়। কিন্তু Network Failure-এ Reject হতে পারে। `await` ছাড়া Promise Return করলে Outer `try/catch` সেটির Rejection ধরতে নাও পারে; Promise `await` করো বা Caller-কে Handling-এর দায়িত্ব দাও।

১১.৭ Debugging with Browser DevTools

1. **Console:** `console.log`, `console.table`, `console.error` দিয়ে State পরীক্ষা করো। Production-এ Sensitive Data Log করবে না।
2. **Sources → Breakpoint:** নির্দিষ্ট Line-এ Execution Pause করে Variables ও Call Stack দেখো।
3. **Step Over / Into / Out:** Function Flow অনুসরণ করো।
4. **Network:** Request URL, Status, Timing, Payload ও Response দেখো।
5. **Preserve Log:** Navigation বা Reload-এর পরও Debug History রাখতে সাহায্য করে।
6. **Reproduce → Isolate → Fix → Test:** Bug Report-এর নির্ভরযোগ্য Workflow।

```
function discountedTotal(price, discountRate) {
  debugger; // DevTools খোলা থাকলে এখানে Pause করতে পারে
  return price * (1 - discountRate);
}
// discountedTotal(1000, 0.1) => 900
```

১১.৮ Error Handling Anti-Patterns

```
// ✗ Error গিলে ফেলা
try { JSON.parse('{broken'); } catch (error) {}
```

```
// ✗ কোনো Context ছাড়াই generic error
// throw new Error('Oops');

// ✗ User Input HTML হিসেবে Inject করা
// element.innerHTML = userInput;

// ✓ নিরাপদভাবে Text Render করা
// element.textContent = userInput;
```

কোনো Error Retry করা উচিত কি না, সেটিও গুরুত্বপূর্ণ: Validation Error Retry করলে লাভ নেই; সাময়িক Network Error-এ সীমিত Backoff সহ Retry করা যায়। Payment বা Booking-এর মতো Side Effecting Request Retry করতে হলে Server-side Idempotency প্রয়োজন।

১১.৯ Practical Project — OrderGuard: Order Validation Dashboard

Goal: Customer Name, Email, Product ও Quantity যাচাই করে Order Total দেখানো; Invalid Input-এ স্পষ্ট Error, Unexpected Error-এ Safe Fallback। এটি **Learning Demo**, কোনো Payment/Backend নেই।

Project Structure

```
source-code/chapter-11/
├── index.html
├── style.css
├── app.mjs
├── order-core.mjs
└── order-core.test.mjs
```

Step 1 — Domain Logic

`order-core.mjs`-এ `ValidationError`, `parseQuantity`, `validateOrder` এবং `formatBDT` রয়েছে। Quantity 1–100, Product Catalog-এ উপস্থিত এবং Total Safe Integer হতে হবে। `Object.hasOwn()` ব্যবহার করা হয়েছে, যাতে Prototype Property ভুল করে Product হিসেবে গৃহীত না হয়।

Step 2 — User Interface

`index.html` Form, Feedback Region (`role="status"`) এবং Accepted Order List তৈরি করে। `app.mjs`-এ Submit Handler `try/catch/finally` ব্যবহার করে; `finally` Button পুনরায় Enable করে। DOM-এ `textContent` ব্যবহারে User Text HTML হিসেবে Execute হয় না।

Step 3 — Run and Test

```
# project folder-এর ভেতর থেকে
python3 -m http.server 8000
# browser: http://localhost:8000/

# Node.js 20+ recommended
node --test order-core.test.mjs
```

Expected Behavior: `Amina`, `amina@example.com`, `Keyboard`, Quantity `2` দিলে Total BDT 2,500। Quantity `0`, Unknown Product, Invalid Email দিলে Validation Message। Order History শুধুমাত্র চলমান Page Session-এর Memory-তে থাকবে; Refresh করলে মুছে যাবে।

Step 4 — Manual QA Checklist

- ☐ Valid Order Submit করলে Success Message এবং History Row দেখা যায়।
- ☐ Invalid Name, Email, Quantity আলাদাভাবে Reject হয়।
- ☐ Double Submit এড়াতে Processing চলাকালে Button Disabled থাকে।
- ☐ Error-এর পর Button পুনরায় Enabled হয়।
- ☐ HTML-like Customer Name Text হিসেবেই Render হয়।
- ☐ Mobile Width-এ Form ব্যবহারযোগ্য।

১১.১০ Testing ও Error Diagnosis

Automated Tests-এ Happy Path-এর পাশাপাশি Boundary ও Failure Cases পরীক্ষা করো। যেমন: Quantity `0`, `101`, `1.5`, `1e2`, `null`; Unknown Product; Empty Name; Malformed Email; Unsafe Total। UI Tests আলাদা, কারণ Node Unit Test DOM Render যাচাই করে না।

```
import assert from 'node:assert/strict';
import { validateOrder, ValidationError } from './order-core.mjs';
const catalog = { pen: { name: 'Pen', price: 20 } };
assert.throws(() => validateOrder({
  customer: 'Amina', email: 'amina@example.com',
  productId: 'pen', quantity: 0
}, catalog), ValidationError);
```

১১.১১ Exercises

1. `safeDivide(a,b)` লেখো: Invalid Input ও Division by Zero-তে Error Throw করবে।
2. `safeJSONParse(text)` Result Object (`ok`, `value`/`message`) Return করবে।
3. Custom `ValidationError` তৈরি করে Invalid Username Reject করো।
4. `fetch()` Response-এ `response.ok` যাচাই করার Function লেখো।
5. `parseQuantity()`-তে `0`, `1.5`, `101`, `1e2` Reject হচ্ছে কি না Test করো।
6. `finally` ব্যবহার করে Async Submit-এর Loading State Reset করো।
7. DevTools-এ Breakpoint বসিয়ে `validateOrder()`-এর `issues` Array পরীক্ষা করো।
8. DOM Rendering-এ `innerHTML` বদলে `textContent` ব্যবহারের Security Benefit ব্যাখ্যা করো।
9. `Object.hasOwn()` ছাড়া Catalog Lookup-এ কী সমস্যা হতে পারে দেখাও।
10. OrderGuard-এর জন্য তিনটি নতুন Unit Test এবং দুটি Manual UI Test Case লিখো।

১১.১২ Exercise Solutions

১.

```
function safeDivide(a, b) {
  if (!Number.isFinite(a) || !Number.isFinite(b)) throw new TypeError('Numbers required');
  if (b === 0) throw new RangeError('Cannot divide by zero');
```

```
  return a / b;
}
```

২.

```
function safeJSONParse(text) {
  try { return {ok: true, value: JSON.parse(text)}; }
  catch (error) { if (error instanceof SyntaxError) return {ok: false, message: 'Invalid
JSON'}; throw error; }
}
```

৩.

```
class ValidationError extends Error {
  constructor(message) { super(message); this.name = 'ValidationError'; }
}
function validateUsername(name) {
  if (typeof name !== 'string' || name.trim().length < 3) throw new ValidationError('Minimum 3
characters');
  return name.trim();
}
```

৪.

```
async function fetchJSON(url) {
  const response = await fetch(url);
  if (!response.ok) throw new Error(`HTTP ${response.status}`);
  return response.json();
}
```

৫. `parseQuantity()`-এ Strict Digit Regex, `Number.isSafeInteger()` এবং `<= 100` Constraint ব্যবহার করো; Project-এর Unit Tests-এ সব Cases আছে।

৬.

```
async function submitOrder(button, task) {
  button.disabled = true;
  try { return await task(); }
  finally { button.disabled = false; }
}
```

৭. DevTools Sources Tab-এ `validateOrder()`-এর `if (issues.length)` Line-এ Breakpoint দাও; Invalid Email Submit করে `issues` Inspect করো।

৮. `textContent` Input-কে Plain Text হিসেবে দেখায়; `innerHTML` Markup Parse করে এবং Untrusted Input-এ XSS ঝুঁকি তৈরি করতে পারে।

৯. Prototype Chain-এর `toString` বা `__proto__`-এর মতো Property Catalog Product নয়; `Object.hasOwn(catalog, id)` দিয়ে নিজস্ব Key নিশ্চিত করো।

১০. Unit Test: Empty Name, Unknown Product, Unsafe Total। Manual UI: Submit Feedback, Responsive Layout।

১১.১৩ MCQ Quiz — ১০টি প্রশ্ন

1. `try/catch` মূলত কোন Error Handle করে? (A) Syntax-only (B) Runtime thrown errors (C) CSS errors (D) HTTP automatically
2. `finally` কখন চলে? (A) শুধু Success (B) শুধু Failure (C) সাধারণত উভয় ক্ষেত্রেই (D) কখনো নয়
3. `typeof null` কী? (A) null (B) undefined (C) object (D) boolean
4. `fetch()` HTTP 404 পেলে সাধারণত কী হয়? (A) Promise Reject (B) Promise Resolve, `ok=false` (C) Page Reload (D) Browser Crash
5. `Number.isSafeInteger(1.5)` কী? (A) true (B) false (C) null (D) Error
6. `throw new TypeError()` কী করে? (A) Log Only (B) Exception Throw (C) Promise Resolve (D) DOM Update
7. Untrusted User Text Render-এর জন্য কোনটি নিরাপদ? (A) `innerHTML` (B) `eval` (C) `textContent` (D) `document.write`
8. `catch`-এ Unknown Error এলে কী করা ভালো? (A) Silent Ignore (B) Context অনুযায়ী Log ও Rethrow/Handle (C) Password Print (D) Infinite Retry
9. Debugger Breakpoint-এর উদ্দেশ্য? (A) CSS Style (B) Execution Pause ও State Inspect (C) Network Speed (D) Build Bundle
10. `Object.hasOwn(catalog,id)` কেন? (A) Async Support (B) Own Property Check (C) Price Format (D) Browser Cache

Answer Key ও ব্যাখ্যা

1. **B** — Throw হওয়া Runtime Error Catch করা যায়; Parse-time Syntax Error সাধারণ Script-এ আগে ঘটে।
2. **C** — `finally` Cleanup-এর জন্য, Success/Failure দুই পথেই সাধারণত চলে।
3. **C** — JavaScript-এর ঐতিহাসিক আচরণ।
4. **B** — HTTP Status পরীক্ষা করতে `response.ok` প্রয়োজন।
5. **B** — Safe Integer হতে পূর্ণসংখ্যা হতে হবে।
6. **B** — Execution-এর Normal Flow ভেঙে Exception তৈরি করে।
7. **C** — Plain Text Rendering, Markup Interpretation নয়।
8. **B** — Unknown Error গোপন করলে Debugging কঠিন হয়।
9. **B** — Call Stack, Variables ও Flow Inspect করা যায়।
10. **B** — Inherited Property-কে Catalog Entry হিসেবে গ্রহণ ঠেকায়।

১১.১৪ Chapter Recap এবং পরবর্তী অধ্যায়

Error Handling মানে শুধু `try/catch` নয়: Input Boundary যাচাই, User-Friendly Feedback, Safe Logging, Tests এবং Debugging Process একসঙ্গে Reliability তৈরি করে। এই অধ্যায়ের OrderGuard Project-এ Pure Validation Logic ও DOM Interaction আলাদা রাখা হয়েছে—পরবর্তী অধ্যায়ে **Modern JavaScript, ES Modules এবং Clean Code** শেখার জন্য এটি ভালো ভিত্তি।

পরবর্তী অধ্যায়: ১২ — Modern JavaScript (ES6+), Modules এবং Clean Code Practices।

অধ্যায় ১২ — Modern JavaScript (ES6+), Modules এবং Clean Code Practices

শেখার লক্ষ্য

এই অধ্যায় শেষে তুমি (১) ES2015+ syntax চিনতে, (২) named/default export ও import ব্যবহার করতে, (৩) UI, business logic ও storage আলাদা রাখতে, (৪) immutable update লিখতে, (৫) clean-code ও refactoring সিদ্ধান্ত নিতে এবং (৬) একটি Modular Shopping Cart চালাতে পারবে।

১২.১ কেন Modern JavaScript?

JavaScript-এর ভাষাগত পরিবর্তনগুলো শুধু ছোট করে Code লেখার জন্য নয়। `const`, `let`, template literal, destructuring, modules, optional chaining, nullish coalescing ইত্যাদি সঠিকভাবে ব্যবহার করলে code intent বোঝা, refactor করা এবং test করা সহজ হয়। তবে **নতুন syntax মানেই ভালো architecture নয়**: অপ্রয়োজনীয় abstraction বা একটি function-এ সব কাজ রাখলে নতুন syntax দিয়েও code জটিল থাকবে।

`const` এবং `let`

```
const taxRate = 0.05; // binding reassignment নয়
let quantity = 2;     // পরে পরিবর্তন হতে পারে
quantity += 1;
```

`const` object-কে deep immutable করে না। `const profile = {name:'A'}` হলে `profile.name = 'B'` বৈধ। তাই object mutation নিয়ন্ত্রণে immutable update pattern প্রয়োজন। `var`-এর function scope ও hoisting behaviour নতুন application code-এ সাধারণত এড়িয়ে চলাই সহজ।

১২.২ Template Literal ও Destructuring

```
const booking = { passenger: 'Nadia', fare: 4200, route: 'DAC-CXB' };
const { passenger, fare, route } = booking;
const summary = `${passenger}: ${route} — BDT ${fare}`;
console.log(summary);

const settings = { theme: 'light' };
const { theme, language = 'bn' } = settings;
console.log(theme, language);
```

Destructuring-এর default কেবল `undefined` হলে প্রয়োগ হয়; `null` হলে নয়। Nested destructuring করার সময় অনুপস্থিত parent object বিবেচনায় রাখতে হবে।

```
const response = { user: { name: 'Rafi' } };
const name = response.user?.name ?? 'Guest';
```

১২.৩ Spread, Rest ও Immutable Update

```
const prices = [100, 200];
const updatedPrices = [...prices, 300];
const student = { name: 'Mira', score: 72 };
const updatedStudent = { ...student, score: 85 };

function total(...values) {
  return values.reduce((sum, value) => sum + value, 0);
}
console.log(total(10, 20, 30)); // 60
```

Spread একটি **shallow copy** তৈরি করে। Nested object একই reference ধরে রাখতে পারে:

```
const original = { preferences: { theme: 'light' } };
const copy = { ...original };
copy.preferences.theme = 'dark';
console.log(original.preferences.theme); // dark

const safeUpdate = {
  ...original,
  preferences: { ...original.preferences, theme: 'light' }
};
```

`structuredClone()` অনেক structured value deep copy করতে পারে, কিন্তু function বা সব browser-specific object copy করতে পারে না। প্রয়োজন অনুযায়ী data model ও copy strategy বেছে নিতে হবে।

১২.৪ Optional Chaining ও Nullish Coalescing

```
const customer = { address: null, loyaltyPoints: 0 };
const city = customer.address?.city ?? 'Unknown';
const points = customer.loyaltyPoints ?? 100; // 0, কারণ 0 nullish নয়
const wrong = customer.loyaltyPoints || 100;  // 100, কারণ 0 falsy
```

`?.` null/undefined-এ short-circuit করে; এটি সব invalid input থেকে সুরক্ষা দেয় না। `??`-এর ব্যবহার `||` থেকে আলাদা, বিশেষ করে `0`, `false` এবং empty string-এর ক্ষেত্রে।

১২.৫ ES Modules: এক ফাইলে সব নয়

Module একটি ফাইলকে আলাদা scope দেয় এবং স্পষ্ট public API তৈরি করে। Browser-এ `<script type="module">` দিয়ে ES Module চালানো যায়।

```
// math.mjs
export const VAT = 0.05;
export function withVat(amount) {
  return amount * (1 + VAT);
}

// main.mjs
import { withVat } from './math.mjs';
console.log(withVat(100)); // 105
```

Named export একাধিক symbol প্রকাশ করে। **Default export** একটি primary value প্রকাশ করে। বড় project-এ named export অনেক সময় refactoring ও auto-import সহজ করে। Import path, filename casing

ও file extension পরিবেশভেদে গুরুত্বপূর্ণ। Browser-এ bare import (`import x from 'package'`) সাধারণত bundler বা import map ছাড়া কাজ করবে না।

Browser Module বনাম Node.js

Browser-এ `file://` থেকে Module Import অনেক সময় CORS restriction-এ ব্যর্থ হয়। Project চালাতে local HTTP server ব্যবহার করো:

```
cd source-code/chapter-12
python -m http.server 8000
```

তারপর `http://localhost:8000` খুলবে। Node.js-এ `.mjs` ফাইল native ESM হিসেবে চলে; `.js`-এর ক্ষেত্রে package configuration গুরুত্বপূর্ণ।

১২.৬ Clean Code: পাঠযোগ্যতা আগে

একটি function-এ data validation, DOM manipulation, calculation এবং storage একসঙ্গে থাকলে bug শনাক্ত করা কঠিন। তাই responsibility ভাগ করো:

Layer	দায়িত্ব	উদাহরণ
Domain	Pure cart calculation	`cart.mjs`
Persistence	Local storage	`storage.mjs`
Presentation	DOM ও events	`app.mjs`
Entry	HTML document	`index.html`

নামকরণে `x`, `tmp`, `data2`-এর পরিবর্তে `quantity`, `subtotal`, `productId` ব্যবহার করো। Function যেন একটি নির্দিষ্ট কাজ করে; কিন্তু অতি ক্ষুদ্র function বানিয়ে readability নষ্ট করারও দরকার নেই।

Guard Clause ও Early Return

```
function discountPrice(price, percent) {
  if (!Number.isFinite(price) || price < 0) {
    throw new RangeError('Invalid price');
  }
  if (!Number.isFinite(percent) || percent < 0 || percent > 100) {
    throw new RangeError('Invalid discount');
  }
  return price * (1 - percent / 100);
}
```

Guard clause invalid state শুরুতেই বাদ দেয়। Data validation সবসময় application boundary-তে প্রয়োজন; UI validation একাই যথেষ্ট নয়।

১২.৭ Pure Function, Side Effect ও Dependency Direction

`calculateTotal(cart)` pure হলে একই input-এ একই output দেয় এবং বাইরের state পরিবর্তন করে না। `localStorage.setItem()` বা `document.querySelector()` side effect। Pure core আলাদা রাখলে Node.js-এ browser ছাড়াই test করা যায়।

```
export function calculateSubtotal(items) {
  return items.reduce((sum, item) => sum + item.price * item.quantity, 0);
}
```

Dependency direction: UI domain function-কে call করবে; domain function UI বা storage সম্পর্কে জানবে না। এতে একটি module পরিবর্তন করলে অন্য module কম প্রভাবিত হয়।

১২.৮ Practical Project — Modular Shopping Cart

Problem Statement

একটি ছোট shop-এর জন্য Product Catalog, Cart Add/Remove, Quantity Update, Total Calculation এবং `localStorage` Persistence দরকার। এখানে উদ্দেশ্য শুধু UI বানানো নয়; **modular architecture** শেখা।

Project Structure

```
source-code/chapter-12/
├── index.html
├── style.css
├── cart.mjs          # pure cart logic
├── storage.mjs       # localStorage boundary
├── app.mjs           # browser UI
└── cart.test.mjs     # Node.js unit tests
```

Module 1 — Domain Logic

`cart.mjs`-এ `CATALOG`, `addItem`, `updateQuantity`, `removeItem`, `summarizeCart`, `restoreCart` export করা হয়েছে। সব cart operations নতুন array/object return করে; caller-এর cart সরাসরি mutate করে না।

```
import { addItem, summarizeCart } from './cart.mjs';
let cart = [];
cart = addItem(cart, 'book', 2);
cart = addItem(cart, 'mouse');
console.log(summarizeCart(cart));
// subtotal: 2900, count: 3
```

এখানে `restoreCart()` parsed storage input যাচাই করে। Unknown product ও invalid quantity বাদ দেয়। এটি browser storage-কে trusted ধরে নেয় না। Production application-এ server-side cart price পুনরায় যাচাই করতে হবে।

Module 2 — Persistence

`storage.mjs`-এ `loadCart()` ও `saveCart()` আছে। Storage disabled/full হলে exception ধরে fallback দেয়। `localStorage` শুধুই convenience; এটি secure database নয় এবং অন্য device-এর সঙ্গে synchronize হয় না।

Module 3 — Presentation

`app.mjs`-এ `render()` DOM-কে বর্তমান cart state থেকে পুনর্গঠন করে। User-generated string HTML হিসেবে inject না করে `textContent` ব্যবহার করা হয়েছে। Buttons ও Quantity Inputs-এ event listener যোগ করা হয়েছে।

Running the Project

```
cd source-code/chapter-12
python -m http.server 8000
# Browser: http://localhost:8000
node --test cart.test.mjs
```

Manual Acceptance Checklist

- ☐ Product যোগ করলে cart line তৈরি হয়
- ☐ একই product আবার যোগ করলে quantity বাড়ে
- ☐ Quantity 0 দিলে product মুছে যায়
- ☐ Reload করলে cart থাকে, যদি storage available হয়
- ☐ Invalid quantity দিলে application crash করে না
- ☐ Keyboard দিয়ে button/input ব্যবহার করা যায়
- ☐ Mobile width-এ layout পাঠযোগ্য থাকে

১২.৯ Common Mistakes ও Refactoring

Mistake 1 — Shared reference mutate করা: `const next = [...cart]` করলেও nested item একই object হতে পারে। `map(item => ({...item}))` প্রয়োজন অনুযায়ী ব্যবহার করো।

Mistake 2 — Module path ভুল: `./cart.mjs` এবং `cart.mjs` একই নয়। Browser native module path resolve করে; bare specifier সরাসরি resolve নাও হতে পারে।

Mistake 3 — Circular imports: A যদি B-কে import করে এবং B আবার A-কে import করে, initialization order জটিল হতে পারে। Domain dependencies একমুখী রাখো।

Mistake 4 — UI-এর price-কে trusted ধরা: Client-side price পরিবর্তন করা যায়। বাস্তব checkout-এ server catalog থেকে total পুনর্গণনা করবে।

Mistake 5 — অকারণে abstraction: একবার ব্যবহার হওয়া তিন লাইনের logic-এর জন্য পাঁচটি class তৈরি করা Clean Code নয়।

১২.১০ Exercises

1. `formatUser({name,city})` function-এ destructuring ও template literal ব্যবহার করো।
2. `||` ও `??`-এর পার্থক্য `0`, `false`, `''`, `null` দিয়ে দেখাও।

3. `updateProfile(profile, name)` লিখো যাতে original object mutate না হয়।
4. `filterExpensive(products, minPrice)` নামে pure function লেখো।
5. `calculateSubtotal`-কে আলাদা `.mjs` file থেকে import করো।
6. Cart-এ `clearCart()` function যোগ করো।
7. `summarizeCart()`-এর জন্য invalid discount unit test লেখো।
8. `localStorage` unavailable হলে graceful fallback পরীক্ষা করো।
9. `sortProductsByPrice()` লিখো যাতে input array-এর order অপরিবর্তিত থাকে।
10. Project-এ coupon code `SAVE10` যোগ করো; discount UI-তে নয়, domain layer-এ হিসাব করো।

১২.১১ Exercise Solutions

```
// 1
const formatUser = ({name, city}) => `${name} lives in ${city}`;
// 2
console.log(0 || 5, 0 ?? 5);           // 5 0
console.log(false || true, false ?? true); // true false
console.log('' || 'N/A', '' ?? 'N/A');  // N/A ''
console.log(null || 'N/A', null ?? 'N/A'); // N/A N/A
// 3
const updateProfile = (profile, name) => ({...profile, name});
// 4
const filterExpensive = (products, minPrice) =>
  products.filter(({price}) => price >= minPrice);
// 5 — math.mjs
export const calculateSubtotal = items =>
  items.reduce((sum, item) => sum + item.price * item.quantity, 0);
// 6
const clearCart = () => [];
// 7
// assert.throws(() => summarizeCart([], undefined, 101), RangeError);
// 8 — manually block site storage and verify no uncaught exception
// 9
const sortProductsByPrice = products => [...products].sort((a, b) => a.price - b.price);
// 10
const couponPercent = code => code === 'SAVE10' ? 10 : 0;
// Pass couponPercent(code) into summarizeCart(cart, CATALOG, percent).
```

Exercise 5-এর নোট: `export` declaration module file-এ লিখতে হবে; একই code block-এ illustrative solutions একত্র করা হয়েছে, এগুলো এক ফাইল হিসেবে paste করে চালানোর জন্য নয়।

১২.১২ Knowledge Check — MCQ

1. `const` কী আটকায়? A. Nested mutation B. Binding reassignment C. Function call D. Array access
2. `0 ?? 9` কত? A. 0 B. 9 C. null D. Error
3. `0 || 9` কত? A. 0 B. 9 C. null D. Error
4. Spread দিয়ে object copy সাধারণত কেমন? A. Deep B. Shallow C. Serialized D. Frozen
5. Named import-এর syntax কোনটি? A. `import x` B. `import {x} from './x.mjs'` C. `require(x)` D. `export from x`

6. Pure function কোনটি? A. DOM update B. `localStorage.setItem` C. Input থেকে total গণনা D. `alert`
7. Browser module load করতে কোন attribute? A. `defer="module"` B. `type="module"` C. `async="esm"` D. `lang="module"`
8. `user?.name`-এ `user = null` হলে? A. Throw B. `undefined` C. `false` D. `''`
9. `Array.sort()` default-এ কী করে? A. Immutable copy B. Original array mutate C. JSON serialize D. Remove duplicates
10. Checkout-এর final price কোথায় verify করা উচিত? A. CSS B. Browser only C. Server D. URL fragment

Answer Key ও Explanation

Q	Answer	কারণ
1	B	`const` binding reassignment বন্ধ করে; nested mutation নয়
2	A	`0` nullish নয়
3	B	`0` falsy
4	B	Spread shallow copy করে
5	B	Braces দিয়ে named import
6	C	Pure calculation বাইরের state পরিবর্তন করে না
7	B	Native ES Module script
8	B	Optional chaining nullish receiver-এ undefined দেয়
9	B	`sort()` array mutate করে
10	C	Client data trusted নয়

১২.১৩ Mini Capstone ও Review

নিজে একটি `coupon.mjs` যোগ করে coupon validation আলাদা করো, তারপর `cart.test.mjs`-এ edge-case test লেখো। Code review-এ প্রশ্ন করো: Domain module কি DOM import করছে? Storage unavailable হলে app কি চলছে? User data কি `innerHTML`-এ যাচ্ছে? Invalid quantity কি প্রত্যাখ্যাত হচ্ছে?

অধ্যায় সারসংক্ষেপ

Modern JavaScript-এর মূল শক্তি syntax নয়, বরং পরিষ্কার দায়িত্ব-বিভাজন ও maintainable code। Modules দিয়ে public API নির্ধারণ করো, pure functions দিয়ে domain logic test করো, browser/storage side effects boundary-তে রাখো। পরবর্তী অধ্যায়ে Classes, Prototypes ও OOP-এর মাধ্যমে আরও একটি architecture approach শিখবে।

Next: অধ্যায় ১৩ — Object-Oriented Programming (OOP), Classes, Prototypes, Inheritance এবং Encapsulation।

অধ্যায় ১৩ — Object-Oriented Programming (OOP): Classes, Prototypes, Inheritance এবং Encapsulation

শেখার লক্ষ্য

এই অধ্যায় শেষে তুমি (১) Object ও Class-এর সম্পর্ক, (২) JavaScript-এর Prototype Chain, (৩) Constructor ও `this`, (৪) Inheritance ও Method Overriding, (৫) `#private` Fields দিয়ে Encapsulation, (৬) Composition বনাম Inheritance এবং (৭) বাস্তব Booking Management Application-এর OOP Architecture বুঝতে ও প্রয়োগ করতে পারবে।

১৩.১ OOP কেন শিখব?

একটি Travel Booking System-এ Booking ID, Customer, Price, Status এবং Cancel করার নিয়ম আছে। Flight Booking-এ Baggage Fee, Hotel Booking-এ Number of Nights যোগ হয়। একই বৈশিষ্ট্য ও আচরণ বিভিন্ন ধরনের Booking-এ বারবার লিখলে Duplicate Logic তৈরি হয়। Object-Oriented Programming ডেটা ও সেই ডেটার ওপর কাজ করা Methods-কে একটি নির্দিষ্ট Interface-এর মধ্যে রাখার কৌশল দেয়। তবে OOP সব সমস্যার একমাত্র সমাধান নয়—ছোট Data Transformation-এ সাধারণ Function বেশি সহজ হতে পারে।

চারটি মূল ধারণা: Encapsulation (অভ্যন্তরীণ State নিয়ন্ত্রণ), Abstraction (ব্যবহারকারীর কাছে প্রয়োজনীয় Interface প্রকাশ), Inheritance (একটি Class-এর আচরণ আরেকটি Class-এ গ্রহণ) এবং Polymorphism (একই Interface দিয়ে ভিন্ন আচরণ)। JavaScript-এ এগুলো Prototype-based Object Model-এর ওপর কাজ করে।

১৩.২ Object Literal থেকে Class

```
const booking = {
  id: 'BK-101',
  customer: 'Nadia',
  price: 4500,
  status: 'confirmed',
  cancel() { this.status = 'cancelled'; }
};
booking.cancel();
console.log(booking.status); // cancelled
```

এখানে `this` সাধারণ method call-এর receiver object-কে নির্দেশ করে। কিন্তু Method আলাদা করে Function Variable-এ রাখলে Receiver হারিয়ে যেতে পারে।

```
const person = {
  name: 'Rafi',
  sayName() { return this.name; }
};
console.log(person.sayName()); // Rafi
const detached = person.sayName;
```

```
// detached() strict mode-এ this undefined হওয়ায় TypeError হতে পারে
const bound = person.sayName.bind(person);
console.log(bound()); // Rafi
```

`class` ব্যবহার করলে একই Shape-এর একাধিক Object তৈরি সহজ হয়:

```
class Booking {
  constructor(id, price) {
    this.id = id;
    this.price = price;
  }
  summary() { return `${this.id}: BDT ${this.price}`; }
}
const first = new Booking('BK-101', 4500);
const second = new Booking('BK-102', 7200);
console.log(first.summary());
```

`new` নতুন Object তৈরি করে, তার Prototype Link স্থাপন করে এবং Constructor চালায়। Class Constructor-কে `new` ছাড়া সাধারণ Function-এর মতো Call করা যায় না। Class Declaration-ও Declaration-এর আগে ব্যবহার করা নিরাপদ নয়; Temporal Dead Zone-এর নিয়ম প্রযোজ্য।

১৩.৩ Prototype Chain — Class-এর পেছনের মেকানিজম

JavaScript-এর Class Syntax আসলে Prototype-based Inheritance-এর সুবিধাজনক Syntax। Instance-এর Own Property ও Prototype Property আলাদা:

```
class Traveler {
  constructor(name) { this.name = name; }
  greet() { return `Hello, ${this.name}`; }
}
const traveler = new Traveler('Mira');
console.log(Object.hasOwn(traveler, 'name'));  // true
console.log(Object.hasOwn(traveler, 'greet')); // false
console.log(Object.getPrototypeOf(traveler) === Traveler.prototype); // true
```

`greet()` Method প্রতিটি Instance-এর Own Property হিসেবে তৈরি না হয়ে Prototype-এ থাকে। Property Lookup-এর সময় Engine আগে Object, তারপর Prototype Chain পরীক্ষা করে। Chain শেষ হলে `undefined` পাওয়া যায়।

```
traveler instance
  ├─ own: name
  └─ [[Prototype]] → Traveler.prototype
                       ├─ greet()
                       └─ [[Prototype]] → Object.prototype → null
```

সতর্কতা: `__proto__` পরিবর্তনের বদলে প্রয়োজন হলে `Object.create()` বা `Object.getPrototypeOf()` ব্যবহার করো। Runtime-এ Prototype পরিবর্তন Performance ও Maintainability ক্ষতিগ্রস্ত করতে পারে।

১৩.৪ Constructor, Instance Methods, Static Methods

```
class Currency {
  static formatBDT(amount) {
    if (!Number.isFinite(amount)) throw new TypeError('Invalid amount');
    return `BDT ${amount.toFixed(2)}`;
  }
```

```
}
console.log(Currency.formatBDT(1500)); // BDT 1500.00
```

`static` Method Class-এর ওপর Call করা হয়, Instance-এর ওপর নয়। Instance State-নির্ভর Method সাধারণ Method হওয়া উচিত। Static Factory Method দিয়ে Named Constructor Pattern তৈরি করা যায়, তবে সব Class-এ Factory প্রয়োজন হয় না।

১৩.৫ Encapsulation ও Private Fields

সাধারণ Public Property বাইরে থেকে পরিবর্তন করা যায়। Booking Price বা Status-এর মতো গুরুত্বপূর্ণ State নিয়ন্ত্রণ করতে Private Fields উপকারী:

```
class Wallet {
  #balance = 0;
  deposit(amount) {
    if (!Number.isFinite(amount) || amount <= 0) throw new RangeError('Invalid deposit');
    this.#balance += amount;
  }
  get balance() { return this.#balance; }
}
const wallet = new Wallet();
wallet.deposit(500);
console.log(wallet.balance); // 500
// wallet.#balance — SyntaxError: class-এর বাইরে private field access করা যায় না
```

Getter একটি Readable Interface; Setter থাকলে Validation প্রয়োজন। `#private` Fields JavaScript Language-level Privacy দেয়, তবে এটি Authentication, Authorization বা Encryption-এর বিকল্প নয়। Browser-এ থাকা Code ও Data ব্যবহারকারী নিয়ন্ত্রণ করতে পারে।

১৩.৬ Inheritance, `extends`, `super()` ও Overriding

```
class Booking {
  constructor(price) { this.price = price; }
  get total() { return this.price; }
}
class FlightBooking extends Booking {
  constructor(price, baggageFee) {
    super(price); // derived constructor-এ this ব্যবহারের আগে
    this.baggageFee = baggageFee;
  }
  get total() { return super.total + this.baggageFee; }
}
console.log(new FlightBooking(5000, 800).total); // 5800
```

`extends` দিয়ে Parent Class-এর আচরণ পাওয়া যায়। `super()` Parent Constructor চালায়। `super.total` Parent Getter পড়ছে। Overriding-এর মাধ্যমে একই Property/Method Name ভিন্ন Implementation দিতে পারে। Parent-এর Private Field Child সরাসরি Access করতে পারে না; প্রয়োজনীয় Getter/Method দিয়ে Access দিতে হয়।

১৩.৭ Polymorphism ও Open/Closed Design

```
function printTotals(bookings) {
```

```
  for (const booking of bookings) {
    console.log(booking.total);
  }
}
```

`printTotals`-কে Flight বা Hotel-এর Internal Calculation জানতে হয় না; `total` Interface-ই যথেষ্ট। এটিই Polymorphism-এর ব্যবহারিক সুবিধা। নতুন Booking Type যোগ করতে Existing Reporting Function পরিবর্তন না-ও লাগতে পারে।

১৩.৮ Composition বনাম Inheritance

সব সম্পর্ক `extends` নয়। "FlightBooking **is a** Booking" যুক্তিযুক্ত; "Booking **has a** NotificationService" হলে Composition উপযুক্ত।

```
class BookingNotifier {
  constructor(sendMessage) { this.sendMessage = sendMessage; }
  notify(booking) { this.sendMessage(`Booking ${booking.id} updated`); }
}
const notifier = new BookingNotifier(console.log);
notifier.notify({ id: 'BK-105' });
```

এভাবে Notification Strategy সহজে Mock করা যায় এবং Booking Domain-কে UI/Email Service থেকে আলাদা রাখা যায়। Inheritance-এর গভীর Chain সাধারণত Avoid করা ভালো।

১৩.৯ Practical Project — Booking Management Lab

আমরা একটি Browser-based Application বানাব, যেখানে Flight ও Hotel Booking একই Manager-এ থাকবে। Source Files:

```
source-code/chapter-13/
├── index.html
├── style.css
├── booking.mjs
├── app.mjs
└── booking.test.mjs
```

Domain Model: `Booking` Base Class, `FlightBooking` ও `HotelBooking` Derived Classes, `BookingManager` Collection ও Operations পরিচালনা করে। UI (`app.mjs`) শুধু Form, DOM ও User Feedback সামলায়। এই Separation-এ Business Logic Browser ছাড়া Test করা যায়।

Feature Rules:

1. Booking ID ও Customer খালি হতে পারবে না; Price অবশ্যই Positive Finite Number।
2. Flight Total = Base Price + Baggage Fee; Fee Negative নয়।
3. Hotel Total = Nightly Price × Number of Nights; Nights Positive Integer।
4. Duplicate Booking ID Reject হবে।
5. Cancelled Booking পুনরায় Cancel করা যাবে না; Confirmed Revenue-তে Cancelled Booking ধরা হবে না।

6. UI-তে User Input `textContent` দিয়ে Render হবে, `innerHTML` দিয়ে নয়।

গুরুত্বপূর্ণ Domain Code

`booking.mjs`-এর প্রধান অংশ:

```
export class Booking {
  #status = 'confirmed';
  constructor({ id, customer, price }) {
    if (!id?.trim() || !customer?.trim() || !Number.isFinite(price) || price <= 0) {
      throw new Error('Invalid booking');
    }
    this.id = id.trim();
    this.customer = customer.trim();
    this.price = price;
  }
  get status() { return this.#status; }
  get total() { return this.price; }
  cancel() { this.#status = 'cancelled'; }
}
```

এটি **শিক্ষামূলক সংক্ষিপ্ত Illustration**; Companion Source Code-এর `source-code/chapter-13/booking.mjs`-এ আরও কঠোর Validation, Private Fields, Duplicate Handling, Cancellation Guard এবং Serialization আছে।

Run Instructions

Node.js (ES Modules ও Private Fields সমর্থিত সংস্করণ) থাকলে:

```
cd source-code/chapter-13
node --test booking.test.mjs
```

Browser-এ ES Modules চালাতে Local HTTP Server ব্যবহার করো:

```
python -m http.server 8000
```

এরপর `http://localhost:8000/` খোলো। Browser UI Manual QA এখনো বাকি। এটি Frontend Learning Demo; Server, Authentication বা Durable Database নেই। Refresh করলে Booking Data Reset হয়।

১৩.১০ Common Mistakes ও Debugging

ভুল	সমস্যা	সমাধান
`new` ছাড়া Class Call	TypeError	`new ClassName()`
Child Constructor-এ `super()`-এর আগে `this`	ReferenceError	আগে `super()`
`this`-সহ Method Detached	Receiver হারায়	`.bind()` বা Arrow Wrapper
`#private` Field বাইরে Access	SyntaxError	Getter/Method ব্যবহার
সব কাজ Inheritance-এ রাখা	Tight Coupling	Composition ও Small Services
Input থেকে Price String রেখেই যোগ	String Concatenation	Explicit Number Conversion + Validation
UI-তে `innerHTML` দিয়ে Customer Name	XSS Risk	`textContent`

১৩.১১ Exercises

1. `Book` Class লিখো, যার `title`, `author` ও `summary()` থাকবে।
2. `Rectangle` Class-এ Validated Width/Height এবং `area` Getter তৈরি করো।
3. `BankAccount`-এ `#balance`, `deposit()` ও `withdraw()` যোগ করো।
4. `Employee` ও `Manager` Inheritance দিয়ে Bonus Calculation করো।
5. `Vehicle` Base Class এবং `Car`, `Bike` Derived Class-এ `describe()` Override করো।
6. `BookingManager`-এ `findByCustomer(name)` যোগ করো (Case-insensitive)।
7. `BookingManager`-এ `confirmedCount` Getter তৈরি করো।
8. Hotel Booking-এ `nights=0` কেন Reject হওয়া উচিত ব্যাখ্যা করো।
9. Composition দিয়ে Booking Confirmation Logger তৈরি করো।
10. `Object.getPrototypeOf()` দিয়ে একটি Class Instance-এর Prototype Chain পরীক্ষা করো।

১৩.১২ Exercise Solutions

```
// 1
class Book {
  constructor(title, author) { this.title = title; this.author = author; }
  summary() { return `${this.title} — ${this.author}`; }
}
// 2
class Rectangle {
  constructor(width, height) {
    if (![width, height].every(n => Number.isFinite(n) && n > 0)) throw new RangeError('Invalid
size');
    this.width = width; this.height = height;
  }
  get area() { return this.width * this.height; }
}
// 3
class BankAccount {
  #balance = 0;
  get balance() { return this.#balance; }
  deposit(n) { if (!Number.isFinite(n) || n <= 0) throw new RangeError('Invalid');
this.#balance += n; }
  withdraw(n) { if (!Number.isFinite(n) || n <= 0 || n > this.#balance) throw new
RangeError('Invalid'); this.#balance -= n; }
}
// 4
class Employee { constructor(salary) { this.salary = salary; } get bonus() { return this.salary
* 0.05; } }
class Manager extends Employee { get bonus() { return this.salary * 0.10; } }
// 5
class Vehicle { describe() { return 'Generic vehicle'; } }
class Car extends Vehicle { describe() { return 'Car'; } }
class Bike extends Vehicle { describe() { return 'Bike'; } }
// 6: BookingManager class-এর ভেতরে যোগ করো:
// findByCustomer(name) { return this.list().filter(b => b.customer.toLocaleLowerCase() ===
name.toLocaleLowerCase()); }
// 7: BookingManager class-এর ভেতরে যোগ করো:
// get confirmedCount() { return this.list({ status: 'confirmed' }).length; }
// 9
class ConfirmationLogger { constructor(log) { this.log = log; } confirmed(b) {
this.log(`Confirmed: ${b.id}`); } }
// 10
```

```
const b = new Book('JS', 'Author');
console.log(Object.getPrototypeOf(b) === Book.prototype); // true
```

৮ নম্বরের ব্যাখ্যা: `nights=0` হলে বাস্তব Hotel Stay নেই এবং `total=0` হয়ে যেতে পারে; Domain Validation-এ Positive Integer বাধ্যতামূলক। **৬–৭ নম্বর:** এগুলো Class Body-তে যুক্ত করার Method Snippets, Standalone Script নয়।

১৩.১৩ Self-Assessment Quiz (১০টি MCQ)

1. Class-এর Instance তৈরি করতে সাধারণত কোন Keyword? (A) `create` (B) `new` (C) `instance` (D) `make`
2. Class Method সাধারণত কোথায় থাকে? (A) Global Object (B) Prototype (C) JSON (D) DOM
3. Private Field Declaration কোনটি? (A) `private x` (B) `_x` (C) `#x` (D) `@x`
4. Derived Constructor-এ Parent Constructor চালাতে কী লাগে? (A) `parent()` (B) `base()` (C) `super()` (D) `this()`
5. `static` Method কীভাবে Call করা হয়? (A) Class-এর ওপর (B) সবসময় Instance-এর ওপর (C) `window` দিয়ে (D) `new` দিয়ে
6. একই `total` Interface-এ ভিন্ন Booking Calculation কোন ধারণা? (A) Hoisting (B) Polymorphism (C) Closure (D) Event Loop
7. `Object.hasOwn(obj, 'x')` কী পরীক্ষা করে? (A) Prototype-এ x আছে কি না (B) Own Property আছে কি না (C) x Function কি না (D) x Private কি না
8. একটি Booking-এর মধ্যে Logger Inject করা কোন Design? (A) Composition (B) Inheritance (C) Recursion (D) Mutation
9. Cancelled Booking Revenue থেকে বাদ দেওয়ার Logic কোথায় থাকা ভালো? (A) CSS (B) Domain/Manager (C) HTML (D) Browser URL
10. User Input DOM-এ Text হিসেবে Render করতে কোনটি নিরাপদ Default? (A) `innerHTML` (B) `document.write()` (C) `textContent` (D) `eval()`

Quiz Answers ও Explanations

Q	Answer	Explanation
1	B	`new` Class Constructor দিয়ে Instance তৈরি করে।
2	B	সাধারণ Instance Methods Prototype-এ থাকে।
3	C	`#` Language-level Private Field Syntax।
4	C	`super()` Parent Constructor চালায়।
5	A	Static Method Class Constructor-এর Property।
6	B	একই Interface ভিন্ন Object Type-এ ভিন্নভাবে কাজ করে।
7	B	Own Property পরীক্ষা করে, Inherited নয়।
8	A	Logger Dependency Inject করা Composition।

Q	Answer	Explanation
9	B	Domain Rules UI থেকে স্বাধীন রাখা ভালো।
10	C	`textContent` Text হিসেবে Render করে; HTML Parse করে না।

১৩.১৪ আরও অনুশীলন ও Production Checklist

- Booking-এ `createdAt` এবং `updatedAt` যোগ করো।
- Booking ID Random Generation-এর Collision Risk ব্যাখ্যা করো।
- `BookingManager`-এর Data Storage Adapter আলাদা করে Unit Test করো।
- Money Calculation-এ Floating-Point সমস্যা এড়াতে Minor Unit (Paisa) বা Decimal Library-এর প্রয়োজন বিবেচনা করো।
- Real Application-এ Server-side Validation, Authentication, Authorization, Database Transaction এবং Audit Log প্রয়োজন।

Chapter Summary: JavaScript-এর OOP হলো Prototype-based System-এর ওপর গড়ে ওঠা Class, Encapsulation, Inheritance ও Polymorphism-এর ব্যবহারিক প্রয়োগ। Domain Rules এবং UI আলাদা করলে Testing ও Maintenance সহজ হয়। পরের অধ্যায়ে Advanced Functions, Closures এবং Functional Patterns শিখব।

অধ্যায় ১৪ — Advanced Functions, Closures, Higher-Order Patterns এবং Functional Programming

শেখার লক্ষ্য

এই অধ্যায় শেষে তুমি lexical scope ও closure ব্যাখ্যা করতে, callback ও higher-order function ডিজাইন করতে, `this` binding বুঝতে, function composition ও currying প্রয়োগ করতে, এবং immutable data transformation দিয়ে একটি ছোট analytics application বানাতে পারবে।

১৪.১ JavaScript-এ Function কেন First-Class Value?

JavaScript-এ function একটি value। Variable-এ রাখা, argument হিসেবে পাঠানো এবং অন্য function থেকে return করা যায়। এই বৈশিষ্ট্য reusable logic ও event-driven architecture-এর ভিত্তি।

```
const greet = name => `স্বাগতম, ${name}!`;
const apply = (fn, value) => fn(value);
console.log(apply(greet, 'রাহিম'));

function makeMultiplier(factor) {
  return value => value * factor;
}
const double = makeMultiplier(2);
console.log(double(12)); // 24
```

`makeMultiplier` একটি নতুন function return করে। `factor`-এর মান return-এর পরেও সংরক্ষিত থাকে—এটাই closure-এর মূল ধারণা।

১৪.২ Lexical Scope ও Closure

Lexical scope অর্থ function কোথায় *লেখা হয়েছে* তার ভিত্তিতে outer variable খুঁজে পাওয়া। Closure হলো function এবং তার lexical environment-এর সংযোগ।

```
function createCounter(initial = 0) {
  let count = initial;
  return {
    increment(step = 1) { count += step; return count; },
    decrement(step = 1) { count -= step; return count; },
    getValue() { return count; }
  };
}
const a = createCounter(10);
const b = createCounter();
a.increment(); // 11
b.increment(); // 1 — state আলাদা
```

Closure private state-এর জন্য উপযোগী, কিন্তু এটি নিরাপত্তার পূর্ণ বিকল্প নয়। কোনো sensitive secret frontend source code-এ রাখা উচিত নয়।

Loop-এর closure trap

```
for (var i = 0; i < 3; i++) {
  setTimeout(() => console.log(i), 0); // 3, 3, 3
}
for (let j = 0; j < 3; j++) {
  setTimeout(() => console.log(j), 0); // 0, 1, 2
}
```

`var`-এর একই binding পুনঃব্যবহার হয়; `let` প্রতিটি iteration-এ নতুন binding তৈরি করে।

১৪.৩ Higher-Order Functions

যে function অন্য function গ্রহণ করে অথবা return করে তাকে higher-order function বলা হয়। `map`, `filter`, `reduce`, `some`, `every` এর পরিচিত উদাহরণ।

```
const prices = [100, 250, 400];
const taxed = prices.map(price => price * 1.05);
const expensive = taxed.filter(price => price >= 200);
const total = expensive.reduce((sum, price) => sum + price, 0);
```

কখন কোনটি? `map` প্রতিটি item বদলে নতুন array, `filter` শর্তসাপেক্ষ subset, `reduce` একটি accumulated result তৈরি করে। `forEach` side effect-এর জন্য ব্যবহার করো; এটি নতুন transformed array return করে না।

১৪.৪ Callback, `this` ও Arrow Function

```
const account = {
  owner: 'Mina',
  balance: 500,
  print() { return `${this.owner}: ${this.balance}`; }
};
console.log(account.print());
const detached = account.print;
// detached() strict-mode context-এ this undefined হতে পারে
const bound = account.print.bind(account);
console.log(bound());
```

Arrow function নিজস্ব `this` তৈরি করে না। Object method-এ সাধারণ method syntax ভালো default; nested callback-এ arrow function প্রাসঙ্গিক হতে পারে।

১৪.৫ Function Composition এবং Pipeline

একাধিক ছোট function একত্র করে বড় transformation তৈরি করা যায়।

```
const pipe = (...fns) => input => fns.reduce((value, fn) => fn(value), input);
const trim = value => value.trim();
const lower = value => value.toLowerCase();
const slugify = value => value.replace(/\s+/g, '-');
const toSlug = pipe(trim, lower, slugify);
console.log(toSlug('  Hello JavaScript  ')); // hello-javascript
```

`pipe(f, g)(x)` মানে `g(f(x))`। বিপরীতে `compose(f, g)(x)` মানে `f(g(x))`। Production-এ প্রতিটি transformation-এর input/output contract স্পষ্ট রাখো।

১৪.৬ Currying এবং Partial Application

```
const add = a => b => a + b;
const addVat = rate => amount => amount + amount * rate;
const withVat15 = addVat(0.15);
console.log(withVat15(100)); // 115
```

Currying একাধিক argument-এর function-কে এক-argument function-এর ধারায় প্রকাশ করে। Partial application কিছু argument আগেই নির্ধারণ করে। দুটোই reusable configuration তৈরিতে কাজে লাগে, তবে অকারণে nested function বাড়িয়ে readability কমানো ঠিক নয়।

১৪.৭ Pure Function, Immutability ও Side Effects

Pure function একই input-এ একই output দেয় এবং বাইরের state বদলায় না।

```
const addItem = (items, item) => [...items, item];
const updatePrice = (items, id, price) =>
  items.map(item => item.id === id ? { ...item, price } : item);
```

`sort()` মূল array বদলে দেয়; `toSorted()` নতুন array return করে (পুরোনো runtime-এ নাও থাকতে পারে)। Compatible বিকল্প `[...items].sort(...)`। Spread একটি **shallow copy**; nested object স্বয়ংক্রিয়ভাবে deep-clone হয় না।

১৪.৮ Memoization: লাভ এবং সতর্কতা

```
function memoizeUnary(fn) {
  const cache = new Map();
  return value => {
    if (cache.has(value)) return cache.get(value);
    const result = fn(value);
    cache.set(value, result);
    return result;
  };
}
const square = memoizeUnary(n => n * n);
```

Memoization pure এবং costly computation-এ সহায়ক। কিন্তু unbounded `Map` memory বাড়াতে পারে। Object keys-এর ক্ষেত্রে identity-ভিত্তিক caching, mutation এবং stale result-এর সমস্যা বিবেচনা করো।

১৪.৯ Debounce ও Throttle

Search input-এ প্রতি keystroke-এ network request না পাঠিয়ে debounce ব্যবহার করা হয়। Scroll বা resize-এর frequent handler-এ throttle কার্যকর হতে পারে।

```
function debounce(fn, delay = 250) {
  let timer;
  return (...args) => {
    clearTimeout(timer);
```

```
    timer = setTimeout(() => fn(...args), delay);
  };
}
```

এই সাধারণ implementation-এ `this` preserve হয় না এবং `cancel()` নেই; production utility-তে প্রয়োজন অনুযায়ী দুটোই যোগ করো।

১৪.১০ Practical Project — Sales Function Lab

এখানে functional patterns দিয়ে sales records filter, normalize, aggregate এবং rank করা হবে। Source files: `source-code/chapter-14/`।

Data Contract

প্রতিটি sale: `{ id, category, quantity, unitPrice }`। `quantity` positive integer এবং `unitPrice` non-negative finite number হতে হবে। Total = quantity × unitPrice।

Functional Pipeline

```
const categorySummary = pipe(
  records => records.filter(row => row.category === 'Books'),
  records => records.map(row => ({ ...row, total: row.quantity * row.unitPrice })),
  records => records.reduce((sum, row) => sum + row.total, 0)
);
```

Actual project-এ shared validation, category aggregation, ranking ও HTML table rendering রয়েছে। User input DOM-এ `textContent` দিয়ে render করা হয়; raw HTML injection নয়।

Run Instructions

1. `source-code/chapter-14/` folder খোলো।
2. `index.html` browser-এ open করো। Sample sales load, category filter এবং summary পরীক্ষা করো।
3. Terminal-এ `node --test source-code/chapter-14/analytics.test.mjs` চালাও (Node.js-এর সাম্প্রতিক version প্রয়োজন)।

১৪.১১ সাধারণ ভুল ও Debugging Checklist

- `map`-এ return ভুলে গেলে result-এ `undefined` আসতে পারে।
- `reduce`-এর initial value না দিলে empty array-এ error হতে পারে।
- `sort` mutate করে; original data রাখতে copy করো।
- Closure-এর private state-কে shared global state ভেবে ভুল করো না।
- Floating-point অর্থের হিসাবের ক্ষেত্রে currency minor units (যেমন পয়সা) অথবা decimal library বিবেচনা করো।
- Function-এর মধ্যে DOM, storage, network এবং business calculation গুলিয়ে ফেলো না।

১৪.১২ অনুশীলন

1. `createCounter(5)` দিয়ে দুটি independent counter তৈরি করো।
2. `pipe` লিখে trim → lowercase → whitespace replace করো।
3. `filter` + `reduce` দিয়ে 1000-এর বেশি মোট বিক্রয় বের করো।
4. `makeDiscount(rate)` নামের curried function তৈরি করো।
5. Mutation ছাড়া একটি object-এর nested `address.city` update করো।
6. `memoizeUnary`-তে `cache.has` কেন `cache.get(...) || ...` থেকে নিরাপদ?
7. `debounce` ও `throttle`-এর ব্যবহারিক পার্থক্য লিখো।
8. Sales Function Lab-এ category অনুযায়ী highest revenue highlight করো।
9. Invalid quantity ও negative price-এর জন্য unit test লেখো।
10. `map`, `filter`, `reduce`-এর প্রত্যেকটির জন্য একটি real-world উদাহরণ লেখো।

১৪.১৩ অনুশীলনের নমুনা সমাধান

```
// 1
const first = createCounter(5), second = createCounter(5);
first.increment(); // 6; second.getValue() এখনও 5

// 2
const slug = pipe(s => s.trim(), s => s.toLowerCase(), s => s.replace(/\s+/g, '-'));

// 3
const bigSales = sales => sales.filter(s => s.quantity * s.unitPrice > 1000)
  .reduce((sum, s) => sum + s.quantity * s.unitPrice, 0);

// 4
const makeDiscount = rate => amount => amount * (1 - rate);

// 5
const changeCity = (person, city) => ({
  ...person, address: { ...person.address, city }
});

// 6: Cached result 0, false, '' বা undefined হলেও has() key presence যাচাই করে।
// 7: debounce শেষ event-এর পরে অপেক্ষা করে; throttle সময়সীমায় call rate সীমিত করে।
// 8: Object.entries(summary).sort((a,b) => b[1]-a[1])[0] থেকে top category।
// 9: Test file-এ invalid input-এর assert.throws() আছে।
// 10: map = price transform, filter = in-stock products, reduce = cart total।
```

১৪.১৪ Knowledge Check — MCQ

1. Closure কোনটি ধরে রাখে? A. HTML B. Lexical environment C. CSS D. Browser tab
2. `map()` কী return করে? A. নতুন array B. Boolean C. DOM D. কিছুই না
3. `filter()`-এর কাজ কী? A. sort B. subset C. mutation বাধ্যতামূলক D. JSON parse
4. `reduce()`-এ initial value কেন ভালো? A. CSS B. empty input safety C. network D. hoisting
5. Arrow function-এর নিজস্ব কোনটি নেই? A. lexical `this` binding B. body C. return D. parameter

6. Pure function কী করে না? A. calculation B. return C. external state mutation D. parameters গ্রহণ
7. `pipe(f,g)(x)` কী? A. f(g(x)) B. g(f(x)) C. x D. undefined
8. `sort()`-এর আচরণ? A. original array mutate B. always immutable C. Promise D. closure
9. Memoization-এর ঝুঁকি? A. memory growth B. HTML parsing C. CSS conflict D. syntax always fails
10. Search typing-এ request কমাতে কোনটি সাধারণত উপযোগী? A. debounce B. recursion C. prototype D. `var`

Answer Key: 1-B (lexical variables), 2-A (mapped array), 3-B (matching subset), 4-B (empty array-এ safe accumulation), 5-A (arrow lexical `this` ব্যবহার করে), 6-C (side-effect এড়িয়ে চলে), 7-B (left-to-right pipeline), 8-A (in-place sort), 9-A (unbounded cache), 10-A (typing থামার পরে action)।

১৪.১৫ Chapter Summary ও Next Steps

Function-কে value হিসেবে ব্যবহার, closure দিয়ে private state, higher-order transformation, composition, immutability এবং memoization-এর সীমাবদ্ধতা—এগুলো বড় JavaScript application-এ modular design-এর ভিত্তি। পরের অধ্যায়ে Browser APIs, Storage, Fetch এবং বাস্তব Web Application তৈরি করা হবে।

অধ্যায় ১৫ — JavaScript Browser APIs, Web Storage, Fetch, URL এবং Real-World Browser Applications

শেখার লক্ষ্য

এই অধ্যায় শেষে তুমি Browser API কীভাবে কাজ করে, `window` ও `document`-এর দায়িত্ব, URL API, Web Storage, Fetch, AbortController, File/Blob API এবং browser security boundary বুঝতে পারবে। শেষে **Bookmark Vault** নামে একটি বাস্তব Browser Application তৈরি করবে, যেখানে Search, Tag Filter, Favorites, Import/Export এবং Persistent Storage থাকবে।

১৫.১ Browser API কী এবং JavaScript Language থেকে পার্থক্য

JavaScript ভাষা `Array`, `Object`, `Promise`, `Map` ইত্যাদি দেয়। Browser সেই ভাষার বাইরে `document`, `localStorage`, `fetch`, `URL`, `navigator`, `history`, `location`, `Blob` ইত্যাদি Host APIs দেয়। Node.js-এ একই Browser DOM API সাধারণত পাওয়া যায় না।

```
console.log(typeof Promise);       // function
console.log(window.location.href); // বর্তমান পেজের URL
console.log(document.title);       // বর্তমান document title
```

Browser APIs-এর কিছু অংশ secure context (HTTPS বা localhost) দাবি করে। API availability যাচাই করা ভালো:

```
if ('clipboard' in navigator) {
  // Clipboard API সম্ভবত উপলব্ধ; permission লাগতে পারে
}
```

`window` হলো browser window-এর global object; `document` হলো DOM tree-এর entry point। `location` current URL নির্দেশ করে। Browser এবং Server-এর JavaScript পরিবেশ আলাদা হওয়ায় runtime capability ধরে না নিয়ে পরীক্ষা করা উচিত।

১৫.২ URL এবং URLSearchParams

String জোড়া লাগিয়ে URL বানালে encoding, query separator, whitespace ইত্যাদি ভুল হতে পারে। `URL` ও `URLSearchParams` এই সমস্যা কমায়।

```
const url = new URL('https://example.com/search');
url.searchParams.set('q', 'বাংলা JavaScript');
url.searchParams.set('page', '2');
console.log(url.href);
console.log(url.searchParams.get('q'));
```

`URL`-এর `protocol`, `hostname`, `pathname`, `searchParams`, `hash` ইত্যাদি property রয়েছে। `new URL('/help', 'https://example.com/docs/')` দিয়ে relative path resolve করা যায়। তবে **সঠিকভাবে parse হওয়া মানেই URL নিরাপদ নয়**: `javascript:` বা `data:` scheme অনিচ্ছাকৃতভাবে গ্রহণ করা বিপজ্জনক। Bookmark-এর জন্য আমরা কেবল `http:` এবং `https:` গ্রহণ করব।

```
function safeWebUrl(value) {
  const url = new URL(value);
  if (!['http:', 'https:'].includes(url.protocol)) {
    throw new Error('Unsupported URL scheme');
  }
  return url.href;
}
```

১৫.৩ Web Storage: localStorage ও sessionStorage

`localStorage`-এর data একই origin-এর browser storage-এ সাধারণত session পেরিয়েও থাকে; `sessionStorage` tab-এর session-এ সীমাবদ্ধ। উভয় API key-value string store করে।

```
localStorage.setItem('theme', 'dark');
console.log(localStorage.getItem('theme')); // dark
localStorage.removeItem('theme');

const preferences = {language: 'bn', compact: true};
localStorage.setItem('prefs', JSON.stringify(preferences));
const saved = JSON.parse(localStorage.getItem('prefs') ?? '{}');
```

সীমাবদ্ধতা: Web Storage synchronous; বড় data লিখলে UI block হতে পারে। Browser quota, privacy settings, private mode অথবা sandbox policy-র কারণে operation ব্যর্থ হতে পারে। তাই `try/catch` ব্যবহার করো। `file://` থেকে Storage-এর আচরণ browserভেদে ভিন্ন হতে পারে; local server দিয়ে চালানো ভালো।

```
function persist(key, value) {
  try {
    localStorage.setItem(key, JSON.stringify(value));
    return true;
  } catch (error) {
    console.warn('Save failed:', error);
    return false;
  }
}
```

Security: Password, API Secret, Access Token-এর মতো Sensitive Data localStorage-এ রাখাকে নিরাপদ default ধরা যাবে না। XSS হলে page-এর JavaScript Storage পড়তে পারে। এই Chapter-এর Bookmark Data ব্যক্তিগত হলেও high-sensitivity secret নয়; ব্যবহারকারীকে Export File নিরাপদে রাখতে হবে।

১৫.৪ JSON Serialization এবং Data Validation

`JSON.stringify()` Object-কে String করে; `JSON.parse()` String থেকে Value তৈরি করে। কিন্তু Parsed Data কখনোই স্বয়ংক্রিয়ভাবে trustworthy নয়।

```
const raw = localStorage.getItem('bookmarks');
let bookmarks = [];
try {
  const parsed = raw ? JSON.parse(raw) : [];
```

```
  if (Array.isArray(parsed)) bookmarks = parsed;
} catch {
  bookmarks = [];
}
```

`JSON.stringify()` Function ও `undefined`-এর মতো কিছু value বাদ দিতে পারে, `BigInt` সরাসরি serialize করা যায় না, এবং circular reference-এ Error হয়। Stored Schema বদলালে Migration Plan দরকার—যেমন `version: 1`।

১৫.৫ Fetch API: HTTP Request ও Response

`fetch()` Promise return করে। Network Error হলে Promise reject হতে পারে, কিন্তু HTTP 404 বা 500 হলে সাধারণত `fetch` reject করে না; `response.ok` পরীক্ষা করতে হয়।

```
async function loadPosts() {
  const response = await fetch('https://jsonplaceholder.typicode.com/posts?_limit=3');
  if (!response.ok) throw new Error(`HTTP ${response.status}`);
  return await response.json();
}

loadPosts().then(console.log).catch(console.error);
```

`response.json()`-ও ব্যর্থ হতে পারে যদি response JSON না হয়। API-র response data ব্যবহার করার আগে structure validate করো। `POST` request-এ `method`, `headers` ও `body` প্রয়োজন হতে পারে:

```
const payload = {title: 'New note'};
const response = await fetch('/api/notes', {
  method: 'POST',
  headers: {'Content-Type': 'application/json'},
  body: JSON.stringify(payload)
});
if (!response.ok) throw new Error('Request failed');
```

CORS: অন্য origin-এ Browser Request করলে Server-এর CORS Policy প্রযোজ্য হয়। Frontend থেকে ইচ্ছেমতো Header যোগ করে Server-এর CORS restriction bypass করা যায় না। Authentication ও authorization Server-এ enforce করতে হয়।

১৫.৬ AbortController ও Request Cancellation

Search Box-এ দ্রুত টাইপ করলে পুরোনো request পরে শেষ হয়ে নতুন result overwrite করতে পারে। `AbortController` দিয়ে আগের request cancel করা যায়।

```
let controller;
async function searchRemote(query) {
  controller?.abort();
  controller = new AbortController();
  try {
    const url = new URL('/api/search', location.origin);
    url.searchParams.set('q', query);
    const response = await fetch(url, {signal: controller.signal});
    if (!response.ok) throw new Error('Search failed');
    return await response.json();
  } catch (error) {
    if (error.name === 'AbortError') return null;
    throw error;
```

```
  }
}
```

Request cancel হলেও Server-এ ইতিমধ্যে শুরু হওয়া কাজ সম্পূর্ণ বন্ধ হয়েছে—এমন নিশ্চয়তা নেই। Race Condition-এর জন্য Request ID/Latest-Response Check-ও ব্যবহার করা যায়।

১৫.৭ Browser Events এবং Cross-Tab Storage

`addEventListener()` দিয়ে Event Handler register করা হয়। `storage` event সাধারণত একই origin-এর **অন্য** document-এ Storage বদলালে পাওয়া যায়; যে document নিজে পরিবর্তন করেছে সেখানে এই event trigger হয় না।

```
window.addEventListener('storage', event => {
  if (event.key === 'jsmastery.bookmarks.v1') {
    console.log('অন্য tab-এ Bookmark data বদলেছে');
  }
});
```

Production App-এ অন্য Tab-এর পরিবর্তন এলে UI reload বা state merge করার ব্যবস্থা করা যায়। আমাদের ছোট Project-এ cross-tab live synchronization রাখা হয়নি—এটি Extension Exercise।

১৫.৮ File, Blob এবং Download API

Client-side JSON Export করতে Server প্রয়োজন নেই। `Blob` দিয়ে File Content তৈরি করে Object URL-এর মাধ্যমে Download করা যায়।

```
const blob = new Blob([JSON.stringify({hello: 'world'}, null, 2)],
  {type: 'application/json'});
const objectUrl = URL.createObjectURL(blob);
const a = document.createElement('a');
a.href = objectUrl;
a.download = 'example.json';
a.click();
URL.revokeObjectURL(objectUrl);
```

File Input থেকে `file.text()` দিয়ে User-selected File পড়া যায়। Import-এর আগে File Size, Format ও Data Schema validate করতে হবে। JSON-এর মধ্যে HTML বা URL থাকলে তা `innerHTML`-এ বসানো যাবে না।

১৫.৯ Practical Project — Bookmark Vault

সমস্যা ও Requirements

অনেক Developer Documentation, Tutorials ও Tools-এর URL আলাদা জায়গায় রেখে পরে খুঁজে পান না। **Bookmark Vault**-এ আমরা URL সংরক্ষণ, Tagging, Search, Favorites, Delete, JSON Export/Import এবং Local Persistence যুক্ত করব। Backend বা Account ছাড়া একটি Local-first Learning App হবে।

Project Folder:

```
source-code/chapter-15/
├── index.html
```

```
├── style.css
├── bookmarks.mjs
├── app.mjs
└── bookmarks.test.mjs
```

`bookmarks.mjs`-এ pure validation, filtering ও import/export function; `app.mjs`-এ DOM এবং Storage orchestration। এভাবে UI Logic ও Business Logic আলাদা থাকায় Unit Test সহজ হয়।

Step 1 — Data Model

```
{
  id: 'generated-unique-id',
  title: 'MDN JavaScript',
  url: 'https://developer.mozilla.org/',
  tags: ['javascript', 'reference'],
  createdAt: '2026-10-11T00:00:00.000Z',
  favorite: false
}
```

`id` হলো Identity, `url` হলো Validated Address, `tags` হলো Search/Filter Metadata। `createdAt` ISO Timestamp; এটি Client Clock-এর উপর নির্ভরশীল।

Step 2 — Validation এবং Normalization

`validateBookmark()` Title Trim করে, URL Protocol Check করে, Tags Normalize করে। `createBookmark()` Unique ID ও Timestamp যুক্ত করে। Full implementation `bookmarks.mjs`-এ রয়েছে।

```
const clean = validateBookmark({
  title: '  MDN  ',
  url: 'https://developer.mozilla.org/#docs',
  tags: 'JavaScript, Reference, javascript'
});
// title: 'MDN'; hash বাদ; tags duplicate বাদ
```

Step 3 — UI Event এবং Rendering

`app.mjs` Form Submit intercept করে, Bookmark Add করে, তারপর `render()` চালায়। `render()` DOM Element তৈরি করে `textContent` ব্যবহার করে—User Input HTML হিসেবে execute হয় না। Link-এর `target="_blank"`-এর সঙ্গে `rel="noopener noreferrer"` রাখা হয়েছে।

Step 4 — Storage

```
const items = safeParseBookmarks(localStorage.getItem(STORAGE_KEY));
localStorage.setItem(STORAGE_KEY, JSON.stringify(items));
```

Import-এ Duplicate URL বাদ দেওয়া হয়। Export JSON-এ `version` Field থাকে, যাতে ভবিষ্যতে Schema Upgrade সহজ হয়। এই Demo-তে Storage Write Fail করলে Message দেখানো হয়; Production-এ Retry/Backup Strategy লাগতে পারে।

Step 5 — App চালানো

```
cd source-code/chapter-15
python -m http.server 8000
```

তারপর Browser-এ `http://localhost:8000` খুলো। Project `type="module"` ব্যবহার করে, তাই `file://`-এর পরিবর্তে Local Server সুপারিশ করা হচ্ছে। Node.js থাকলে Test চালাও:

```
node --test bookmarks.test.mjs
```

Manual Acceptance Checklist

- ☐ Valid HTTPS URL Add করলে List-এ দেখা যায়
- ☐ `javascript:` URL Reject হয়
- ☐ Duplicate URL Add করা যায় না
- ☐ Search ও Tag Filter কাজ করে
- ☐ Favorite Toggle এবং Delete কাজ করে
- ☐ Refresh-এর পর Saved Data থাকে
- ☐ Export → Import করলে Bookmark ফিরে আসে
- ☐ Corrupt JSON Import করলে Error Message দেখায়
- ☐ Mobile Width-এ Layout ব্যবহারযোগ্য থাকে

১৫.১০ নিরাপত্তা, Privacy ও Architecture Review

1. **XSS:** User Input Render করার সময় `textContent` ব্যবহার করো, `innerHTML` নয়।
2. **URL Validation:** `http:`/`https:` ছাড়া Protocol Reject করো। এটি সমস্ত Phishing/Unsafe Destination আটকায় না।
3. **LocalStorage:** Same-Origin Scripts Data পড়তে পারে। Confidential Credential রাখবে না।
4. **Import Trust:** File Extension `.json` হলেও Content যাচাই প্রয়োজন।
5. **Network:** `fetch`-এর Error ও HTTP Status আলাদা Handle করো।
6. **Accessibility:** Form Label, Focus State, Status Announcement (`aria-live`) গুরুত্বপূর্ণ।
7. **Data Integrity:** Import-এর আগে Backup Export রাখা ভালো; Real-world App-এ Schema Migration প্রয়োজন।
8. **Scalability:** হাজার হাজার Bookmark হলে IndexedDB, Pagination, Debounced Search বিবেচনা করো।

১৫.১১ Common Mistakes এবং Debugging

ভুল	সমস্যা	সমাধান
`JSON.parse(null)` ধরে নেওয়া	Unexpected State	Missing Value ও Schema Check
`fetch()` 404-এ Reject করবে ভাবা	Error UI কাজ করে না	`response.ok` Check
`innerHTML = title`	XSS Risk	`textContent`
`new URL()` সব Scheme Allow করা	Unsafe Links	Protocol Allowlist

ভুল	সমস্যা	সমাধান
`localStorage` Write কখনো Fail করবে না ভাবা	Data Loss	`try/catch` ও Feedback
Module `file://` থেকে চালানো	CORS/Module Error	Local HTTP Server
Storage Event একই Tab-এ আশা করা	UI Sync Bug	Same-tab State Update

১৫.১২ Exercises

1. `URLSearchParams` দিয়ে `category=js` এবং `page=3` Query String তৈরি করো।
2. `localStorage`-এ `theme` সংরক্ষণ এবং পড়ার Function লেখো।
3. HTTP Status 404 হলে Custom Error Throw করে এমন Fetch Wrapper লেখো।
4. URL Validator-এ `ftp:` Scheme Reject করো।
5. Bookmark Tags থেকে Unique Tags-এর Sorted List তৈরি করো।
6. `filterBookmarks()`-এ Case-insensitive Search পরীক্ষা করো।
7. Import-এর আগে 1MB File Size Limit যুক্ত করো।
8. Export JSON-এ `version` Field যোগ করো।
9. অন্য Tab-এ Storage বদলালে Current Tab-এর List Refresh করো।
10. Favorite Bookmarks আগে দেখানোর Sorting যুক্ত করো।

১৫.১৩ Sample Solutions

১. Query:

```
const params = new URLSearchParams({category: 'js', page: '3'});
console.log(params.toString()); // category=js&page=3
```

২. Theme Storage:

```
function saveTheme(theme) { localStorage.setItem('theme', theme); }
function getTheme() { return localStorage.getItem('theme') ?? 'light'; }
```

৩. Fetch Wrapper:

```
async function getJSON(url) {
  const res = await fetch(url);
  if (!res.ok) throw new Error(`HTTP ${res.status}`);
  return res.json();
}
```

৪. Protocol: `['http:', 'https:'].includes(new URL(input).protocol)` পরীক্ষা করো; False হলে Error Throw।

৫. Unique Tags:

```
const tags = [...new Set(items.flatMap(x => x.tags))].sort();
```

৬. Search: `filterBookmarks(items, 'mdn')` ও `filterBookmarks(items, 'MDN')` একই ফল দেয় কি না Test করো।

৭. File Size: `if (file.size > 1024 * 1024) throw new Error('Too large')`।

৮. Versioned Export: `JSON.stringify({version: 1, bookmarks: items})`।

৯. Cross-tab:

```
window.addEventListener('storage', event => {
  if (event.key === STORAGE_KEY) {
    items = safeParseBookmarks(event.newValue);
    render();
  }
});
```

১০. Favorite Sorting:

```
const sorted = [...items].sort((a, b) => Number(b.favorite) - Number(a.favorite));
```

১৫.১৪ MCQ Quiz — ১০টি প্রশ্ন

১. `localStorage` কোন ধরনের Value Store করে? A) Object B) String C) Function D) DOM Node

২. `fetch()`-এ HTTP 404 এলে কী পরীক্ষা করা উচিত? A) `response.ok` B) `window.ok` C) `fetch.status` D) `JSON.ok`

৩. URL Query Parameter Encode করতে উপযোগী কোনটি? A) `URLSearchParams` B) `eval` C) `innerHTML` D) `setTimeout`

৪. Request Cancel করতে কোন API? A) `AbortController` B) `MutationObserver` C) `FileReader` D) `History`

৫. User-provided Title Render করতে কোনটি নিরাপদ Default? A) `innerHTML` B) `document.write` C) `textContent` D) `eval`

৬. JSON Text থেকে Value পেতে? A) `JSON.stringify` B) `JSON.parse` C) `JSON.load` D) `URL.parse`

৭. File Download-এর জন্য In-memory Content তৈরি করতে? A) `Blob` B) `Map` C) `WeakSet` D) `Proxy`

৮. `sessionStorage` সাধারণত কোন Scope-এ? A) সকল Browser B) Tab Session C) সব Device D) Server Database

৯. `storage` Event সাধারণত কোন Document-এ Fire হয়? A) যে Document পরিবর্তন করেছে B) অন্য Same-origin Document C) Server D) Service Worker মাত্র

১০. `javascript:` URL Allow করা উচিত? A) সবসময় B) Bookmark-এর জন্য না C) HTTPS হলে D) Only Chrome-এ

Answer Key ও ব্যাখ্যা

1. **B** — Storage Value String; Object হলে JSON Serialize করতে হয়।
2. **A** — 404/500-এ Promise সাধারণত Resolve হতে পারে; `ok` পরীক্ষা প্রয়োজন।
3. **A** — `URLSearchParams` Query Encode করে।
4. **A** — `AbortController`-এর `signal` Request-এ দেওয়া যায়।

5. **C** — `textContent` Input-কে Text হিসেবে Render করে।
6. **B** — `JSON.parse()` JSON Text Parse করে।
7. **A** — `Blob` File-like Data Represent করে।
8. **B** — `sessionStorage` Tab Session-ভিত্তিক।
9. **B** — সাধারণত অন্য Same-origin Browsing Context-এ।
10. **B** — Untrusted Bookmark URL-এ Unsafe Scheme Reject করতে হয়।

১৫.১৫ Chapter Summary ও Next Step

এই অধ্যায়ে Browser Environment, URL Processing, Storage, Fetch, Cancellation, File Import/Export, Safe Rendering এবং Validation শিখেছ। Bookmark Vault-এর Source Code ও Tests Chapter Folder-এ রয়েছে। পরের অধ্যায়ে আমরা **Error Handling, Debugging, Testing এবং Software Reliability**-তে এগোব, যেখানে Error Scenario-কে Testable Behavior-এ রূপান্তর করব।

অধ্যায় ১৬ — JavaScript Error Handling, Debugging, Testing এবং Software Reliability

শেখার লক্ষ্য

এই অধ্যায় শেষে তুমি (১) Error-এর উৎস শনাক্ত, (২) প্রত্যাশিত Validation Failure এবং অপ্রত্যাশিত System Failure আলাদা, (৩) `try/catch/finally` সঠিকভাবে ব্যবহার, (৪) Browser DevTools-এ Debug, (৫) Unit ও Integration Test লিখতে এবং (৬) Reliability-কেন্দ্রিক BookingGuard Application তৈরি করতে পারবে।

১৬.১ Software Reliability কেন গুরুত্বপূর্ণ?

কোনো Web Application শুধু *happy path*-এ কাজ করলেই যথেষ্ট নয়। ব্যবহারকারী ভুল Input দিতে পারে, Browser Storage বন্ধ থাকতে পারে, Network Request Timeout হতে পারে কিংবা একই Booking দুবার Submit হতে পারে। Reliability মানে সব Error অদৃশ্য করা নয়; বরং Error হলে Predictable, Recoverable ও Testable Behavior নিশ্চিত করা।

পরিস্থিতি	দুর্বল আচরণ	নির্ভরযোগ্য আচরণ
Invalid Email	ভুল Data সংরক্ষণ	Field-specific Validation Message
Duplicate Booking	Duplicate Record	Conflict Error
Storage Blocked	App Crash	User-facing Warning
Unexpected Exception	Blank Screen	Safe Fallback + Diagnostic Log
API Timeout	Infinite Spinner	Timeout, Retry Option ও Cancellation

মূল নীতি: User-friendly Message এবং Developer Diagnostic আলাদা। Browser Console-এ Debug Details দেখানো যায়, কিন্তু User-কে Sensitive Stack Trace দেখানো উচিত নয়।

১৬.২ Error-এর ধরন

```
// SyntaxError: কোড Parse করা যায় না
// const name = ;

// ReferenceError: অঘোষিত Identifier
// console.log(unknownVariable);

// TypeError: ভুল ধরনের Operation
// null.toString();

// RangeError: অগ্রহণযোগ্য Range
// new Array(-1);

// JSON Parsing-এর SyntaxError
try {
  JSON.parse('{broken json}');
} catch (error) {
  console.log(error.name); // SyntaxError
```

```
}
```

সব Error একইভাবে Handle করা উচিত নয়। ভুল User Input হলে Validation Error; Data Conflict হলে Conflict Error; Programming Bug হলে তা গোপন না করে Log ও Test দিয়ে Fix করতে হবে।

১৬.৩ `try`, `catch`, `finally` ও `throw`

```
function parseSettings(raw) {
  try {
    return JSON.parse(raw);
  } catch (error) {
    console.error('Invalid settings JSON', error);
    return {};
  } finally {
    console.log('Parse attempt finished');
  }
}
```

`finally` সাধারণত Resource Cleanup বা Loading State Reset-এর জন্য উপযোগী। `catch` Block-এ Error নীরবে Ignore করা বিপজ্জনক; Recovery Strategy না থাকলে Error পুনরায় `throw` করা যুক্তিযুক্ত।

```
function assertPositivePrice(price) {
  if (!Number.isFinite(price) || price <= 0) {
    throw new RangeError('Price must be a positive finite number');
  }
  return price;
}
```

১৬.৪ Custom Error Classes

Domain-specific Error ব্যবহার করলে UI Layer সিদ্ধান্ত নিতে পারে কী Message দেখাবে।

```
class ValidationError extends Error {
  constructor(field, message) {
    super(message);
    this.name = 'ValidationError';
    this.field = field;
  }
}

function validateSeats(seats) {
  if (!Number.isSafeInteger(seats) || seats < 1 || seats > 9) {
    throw new ValidationError('seats', 'Seats must be 1–9');
  }
  return seats;
}
```

`instanceof ValidationError` দিয়ে পরিচিত Validation Failure আলাদা করা যায়। কিন্তু Third-party Error বা Cross-realm Error-এ শুধু `instanceof`-এর ওপর নির্ভর করা সবসময় যথেষ্ট নয়।

১৬.৫ Defensive Programming এবং Input Boundaries

Boundary Validation: Browser Form-এর `required` বা `min` Attribute ভালো User Experience দেয়, কিন্তু এগুলো Security Boundary নয়। Backend-এ একই নিয়ম স্বাধীনভাবে Validate করতে হবে।

```
function parseQuantity(value) {
  const quantity = Number(value);
  if (!Number.isSafeInteger(quantity) || quantity < 1 || quantity > 9) {
    throw new Error('Quantity must be an integer between 1 and 9');
  }
  return quantity;
}
```

`Number('')` শূন্য হয়—তাই খালি Input আলাদা করে Reject করতে চাইলে `String(value).trim()` আগে পরীক্ষা করো। `Number.isFinite()` দিয়ে `NaN` ও `Infinity` Reject করো। Financial Calculation-এ Floating-point Precision গুরুত্বপূর্ণ; Production Payment System-এ Minor Currency Unit (যেমন পয়সা) Integer হিসেবে বা Decimal Library ব্যবহার করো।

১৬.৬ Async Error Handling

```
async function loadBookings(signal) {
  const response = await fetch('/api/bookings', { signal });
  if (!response.ok) {
    throw new Error(`HTTP ${response.status}`);
  }
  return response.json();
}

async function refreshBookings() {
  const controller = new AbortController();
  try {
    const bookings = await loadBookings(controller.signal);
    console.log(bookings);
  } catch (error) {
    if (error.name === 'AbortError') return;
    console.error('Could not refresh bookings', error);
  }
}
```

`fetch()` সাধারণত HTTP 404/500-এ Promise Reject করে না; `response.ok` পরীক্ষা করতে হয়। Cancellation ইচ্ছাকৃত হলে তাকে সাধারণ Network Failure-এর মতো দেখানো উচিত নয়। `Promise.all()`-এ একটি Promise Reject করলে Combined Promise Reject হয়; সব Result জানতে `Promise.allSettled()` ব্যবহার করা যায়।

১৬.৭ Debugging: একটি পুনরাবৃত্তিযোগ্য পদ্ধতি

1. **Reproduce:** ঠিক কোন Input-এ Bug হচ্ছে তা লিখে রাখো।
2. **Observe:** Console, Network Tab, Sources এবং Stack Trace দেখো।
3. **Hypothesis:** কোন State বা Function ভুল করছে অনুমান করো।
4. **Isolate:** সবচেয়ে ছোট Reproduction তৈরি করো।
5. **Fix:** Root Cause সংশোধন করো, শুধু Symptom নয়।
6. **Regression Test:** একই Bug যেন ফিরে না আসে, তার Automated Test লেখো।

```
function calculateTotal(items) {
  debugger; // DevTools খোলা থাকলে এখানে Pause করবে
  return items.reduce((sum, item) => sum + item.price * item.quantity, 0);
}
```

`console.table(items)` Array/Object Data তুলনা করতে সহায়ক। `console.assert(condition, message)` Development Diagnostic-এর জন্য; Business Validation-এর বিকল্প নয়। Production Code-এ অপ্রয়োজনীয় `debugger` Statement সরিয়ে ফেলো।

১৬.৮ Testing Pyramid এবং Test Types

Test Type	লক্ষ্য	উদাহরণ
Unit Test	একক Function-এর আচরণ	`validateBooking()`
Integration Test	একাধিক Component-এর সংযোগ	Form → Domain → Storage
End-to-End Test	বাস্তব User Journey	Browser-এ Booking Confirm
Regression Test	পুরোনো Bug পুনরায় না হওয়া	Duplicate ID Reject
Boundary Test	Minimum/Maximum Limit	0, 1, 9, 10 Seats

Unit Test দ্রুত, কিন্তু Browser UI কাজ করছে কি না তার নিশ্চয়তা নয়। E2E Test User Journey যাচাই করে, তবে তুলনামূলক ধীর ও Maintenance-heavy। Balanced Testing Strategy দরকার।

১৬.৯ Node.js Built-in Test Runner

Node.js-এর `node:test` ও `node:assert/strict` দিয়ে Dependency ছাড়াই Test লেখা যায়।

```
import test from 'node:test';
import assert from 'node:assert/strict';
import { validateBooking } from './booking-core.mjs';

test('valid booking calculates total', () => {
  const booking = validateBooking({
    customer: 'Amina', email: 'amina@example.com', seats: 2, unitPrice: 350
  });
  assert.equal(booking.total, 700);
});

test('rejects zero seats', () => {
  assert.throws(() => validateBooking({
    customer: 'Amina', email: 'amina@example.com', seats: 0, unitPrice: 350
  }));
});
```

Run: `node --test booking-core.test.mjs`। ভালো Test শুধু Pass হওয়ার জন্য নয়; Failure হলে কেন Fail হয়েছে তা দ্রুত বোঝার জন্য।

১৬.১০ Test Doubles: Stub, Mock ও Fake

Stub নির্ধারিত Return Value দেয়। **Spy/Mock** Function Call-এর Behaviour পরীক্ষা করে। **Fake** হলো সহজ In-memory Implementation, যেমন Test Storage।

```
const fakeStorage = {
  getItem(key) {
    return key === 'bookings' ? '[{"id":"BK-1"}]' : null;
  }
};
```

```
console.log(fakeStorage.getItem('bookings'));
```

Test-এ বাস্তব Network/Payment Gateway ব্যবহার না করে Controlled Dependency Injection করো। এতে Test Deterministic হয়।

১৬.১১ Practical Project — BookingGuard

Problem: Booking Form-এ ভুল Data, Duplicate ID, Storage Failure এবং Cancellation-এর সময় Application Crash বা ভুল Revenue দেখা যায়।

Solution: Pure Domain Functions + Browser UI + Storage Adapter + Automated Tests।

Project Folder

```
source-code/chapter-16/
├── index.html
├── style.css
├── booking-core.mjs
├── app.mjs
└── booking-core.test.mjs
```

সম্পূর্ণ File-by-File Runnable Code বইয়ের Companion Source Code-এর `source-code/chapter-16/` Folder-এ দেওয়া হয়েছে। এই অধ্যায়ে Core Logic ব্যাখ্যা করছি;

Data Flow

```
flowchart TD
  A[User submits booking] --> B[Read form data]
  B --> C{Validation valid?}
  C -- No --> D[Show field error]
  C -- Yes --> E{Duplicate ID?}
  E -- Yes --> F[Show conflict error]
  E -- No --> G[Update immutable booking list]
  G --> H[Persist in localStorage]
  H --> I[Render booking list and revenue]
```

Domain Function

```
export function summarizeBookings(bookings) {
  const active = bookings.filter(item => item.status === 'confirmed');
  return {
    totalBookings: bookings.length,
    confirmed: active.length,
    revenue: Math.round(active.reduce((sum, item) => sum + item.total, 0) * 100) / 100
  };
}
```

Cancellation-এর পরে `confirmed` Count ও Revenue কমবে; Original Array Mutation না করে New Array Return করা হয়েছে। UI-তে `textContent` ব্যবহার করে User Data Render করা হয়েছে, যাতে HTML Injection-এর ঝুঁকি কমে।

Run Instructions

1. `source-code/chapter-16/` Folder-এ যাও।

2. Terminal-এ `python -m http.server 8000` চালাও।
3. Browser-এ `http://localhost:8000` খোলো।
4. Test চালাও: `node --test booking-core.test.mjs`।
5. Invalid Email, Zero Seats, Cancel এবং Browser Storage Blocked Scenario পরীক্ষা করো।

Limitations: এটি Learning Demo; Authentication, Server-side Validation, Payment Processing, Concurrent Booking Control বা Real Database নেই। `localStorage` Production Booking Database নয়।

১৬.১২ Error-handling Anti-patterns

Anti-pattern 1: Silent Catch

```
try { riskyOperation(); } catch (error) { /* ignored */ }
```

এতে Failure লুকিয়ে যায়। Recovery Strategy থাকলে তা Document করো; না থাকলে Log বা Re-throw করো।

Anti-pattern 2: Error Message থেকে HTML Injection

```
// ভুল: element.innerHTML = error.message;
// ভালো: element.textContent = error.message;
```

Anti-pattern 3: শুধু Happy-path Test — Invalid, Empty, Boundary, Duplicate ও Cancellation Case অবশ্যই Test করো।

Anti-pattern 4: Client-side Validation-কে Security মনে করা — Server-side Revalidation অপরিহার্য।

১৬.১৩ Hands-on Exercises

1. `validateBooking()`-এ Customer Name-এর Maximum Length 80 থেকে 60 করো।
2. Seats-এর Boundary Values 0, 1, 9, 10-এর জন্য Test লেখো।
3. Duplicate Booking ID-এর জন্য `ConflictError` Assert করো।
4. `cancelBooking()`-এ Missing ID-এর জন্য Error Test লেখো।
5. `summarizeBookings()`-এ Cancelled Booking Revenue বাদ যাচ্ছে কি না Test করো।
6. Storage-এর `getItem()` Exception Throw করলে কী হয় পরীক্ষা করো।
7. UI-তে Error Message-এর জন্য `aria-live` কেন দরকার ব্যাখ্যা করো।
8. `fetch()` HTTP 500 Response-এর জন্য Testable Wrapper তৈরি করো।
9. Error Handling-এ `finally` দিয়ে Loading Indicator Reset করো।
10. BookingGuard-এর জন্য Browser E2E Test Case Outline লেখো।

১৬.১৪ Exercise Solutions (Sample)

১. `customer.length > 60` Check ব্যবহার করো এবং Error Message-ও আপডেট করো।

২.

```
for (const seats of [0, 10]) {
  assert.throws(() => validateBooking({
    customer: 'Amina', email: 'amina@example.com', seats, unitPrice: 100
  }));
}
for (const seats of [1, 9]) {
  assert.equal(validateBooking({
    customer: 'Amina', email: 'amina@example.com', seats, unitPrice: 100
  }).seats, seats);
}
```

৩. `assert.throws(() => addBooking([booking], booking), ConflictError)`।

৪. `assert.throws(() => cancelBooking([], 'missing'), ValidationError)`।

৫. একটি Confirmed ও একটি Cancelled Booking দিয়ে `revenue`-তে শুধু Confirmed Amount Assert করো।

৬. Fake Storage-এর `getItem()` থেকে Error Throw করিয়ে `safeReadBookings()` Empty Array Return করছে কি না দেখো।

৭. `aria-live="polite"` Screen Reader-কে নতুন Status Message ঘোষণা করতে সাহায্য করে।

৮.

```
async function getJSON(url, fetchImpl = fetch) {
  const response = await fetchImpl(url);
  if (!response.ok) throw new Error(`HTTP ${response.status}`);
  return response.json();
}
// Test-এ fetchImpl হিসেবে Stub Function Pass করো।
```

৯.

```
async function withLoading(task, setLoading) {
  setLoading(true);
  try { return await task(); }
  finally { setLoading(false); }
}
```

১০. Browser-এ Valid Booking Submit → Confirmation Visible → Cancel Click → Revenue Zero → Reload → Cancelled State Persist—এই Journey Test করো।

১৬.১৫ MCQ — Final Knowledge Check

১. `finally` কখন Run করে? A) শুধু Success B) সাধারণত `try/catch` Completion-এর পরে C) শুধু SyntaxError D) কখনো না

২. `fetch()`-এ HTTP 404 হলে সাধারণত কী হয়? A) Promise সবসময় Reject B) Response Resolve হতে পারে C) Browser বন্ধ D) JSON Automatic Empty

৩. `assert.throws()` কী Test করে? A) Function Throw করে B) Promise Resolve C) CSS D) HTTP Status

৪. Duplicate ID কোন Error-এর উদাহরণ? A) Validation B) Conflict C) Syntax D) Range

৫. Pure Function-এর সুবিধা কী? A) Random Output B) Deterministic Testing C) Browser Required D) Global Mutation

৬. `textContent` কেন ভালো? A) HTML Execute করে B) Text Render করে C) Database Save করে D) Network Call করে

৭. `localStorage` Production Booking Database? A) হ্যাঁ B) না C) শুধু Chrome-এ D) শুধু Offline

৮. Regression Test-এর লক্ষ্য? A) UI Design B) পুরোনো Bug ফিরে আসা আটকানো C) Images Compress D) Hosting

৯. `AbortError` সাধারণত কী বোঝায়? A) Cancellation B) Syntax Failure C) Duplicate ID D) CSS Error

১০. Unit Test Pass মানে কি Browser UI নিশ্চিতভাবে ঠিক? A) হ্যাঁ B) না C) শুধু Mobile D) শুধু Server

MCQ Answer Key ও Explanation

1. **B** — `finally` সাধারণত `try/catch` শেষে চলে; Process Termination-এর মতো ব্যতিক্রম থাকতে পারে।
2. **B** — `response.ok` পরীক্ষা জরুরি।
3. **A** — Synchronous Exception Assert করে; Async Rejection-এর জন্য `assert.rejects()`।
4. **B** — Duplicate ID একটি Conflict Scenario।
5. **B** — Same Input → Same Output হলে Test সহজ হয়।
6. **B** — `textContent` HTML Markup Interpret করে না।
7. **B** — Production-এর জন্য Server-side Storage ও Validation দরকার।
8. **B** — Fixed Bug-এর জন্য Automated Guard তৈরি করে।
9. **A** — ইচ্ছাকৃত Request Cancellation হতে পারে।
10. **B** — UI আলাদাভাবে Integration/E2E Test করতে হয়।

১৬.১৬ Chapter Recap

Reliable JavaScript মানে Error-free হওয়ার দাবি নয়; Failure Predict, Detect, Handle ও Test করার দক্ষতা। এই অধ্যায়ে Custom Errors, Boundary Validation, Async Error Handling, Debugging, Unit Testing এবং BookingGuard Project-এর মাধ্যমে এই ধারণাগুলো বাস্তবে প্রয়োগ করেছ। পরবর্তী অধ্যায়ে Regular Expressions ও Secure Input Processing আরও গভীরভাবে শিখবে।

অধ্যায় ১৭ — JavaScript Regular Expressions, Advanced Validation এবং Secure Input Processing

এই অধ্যায়ে যা শিখবে

Regular Expression (Regex) ব্যবহার করে Text Pattern খোঁজা, Extract, Replace, Validation, Unicode-aware Processing, Input Normalization এবং নিরাপদ DOM Rendering। অধ্যায়ের শেষে **FormShield** নামে একটি বাস্তব Browser Application তৈরি করবে।

১৭.১ Regex কেন প্রয়োজন?

ধরো, একটি Form-এ Email, Phone Number, Full Name এবং Password নেওয়া হচ্ছে। শুধু `value.length > 0` পরীক্ষা করলে ভুল Input আটকানো যায় না। Regex একটি *pattern language*: এটি নির্দিষ্ট Character বা Character Sequence মিলিয়ে দেখতে পারে। তবে Regex কোনো Magic Security Shield নয়—এটি কেবল Validation-এর একটি স্তর।

```
const message = 'Order ID: TRV-2048';
const found = message.match(/TRV-\d{4}/);
console.log(found?.[0]); // TRV-2048
```

`/TRV-\d{4}/`-এ `TRV-` Literal Text, `\d` Digit এবং `{4}` ঠিক চারবার উপস্থিতি বোঝায়।

১৭.২ Regex Syntax: Literals, Character Classes এবং Quantifiers

Pattern	অর্থ	উদাহরণ
`abc`	সরাসরি `abc`	`abc123`
`.`	Line Terminator ছাড়া যেকোনো Character	`a.c`
`\d`	ASCII Digit	`2026`
`\w`	ASCII Word Character	`user_1`
`\s`	Whitespace	Space, Tab
`[A-Z]`	Capital English Letter	`HELLO`
`[^0-9]`	Digit ছাড়া Character	`a`
`?`	০ অথবা ১ বার	`colou?r`
`+`	১ বা একাধিক	`a+`
`*`	০ বা একাধিক	`a*`
`{2,5}`	২ থেকে ৫ বার	`\d{2,5}`

একটি গুরুত্বপূর্ণ ভুল: `.` মানে Literal Dot নয়। Email Domain-এর Dot মিলাতে `\.` লিখতে হয়।

```
console.log(/a.c/.test('abc'));   // true
console.log(/a\.c/.test('a.c'));  // true
console.log(/a\.c/.test('abc'));  // false
```

১৭.৩ Anchors, Groups এবং Alternation

`^` String-এর শুরু, `$` String-এর শেষ বোঝায় (Multiline Flag ছাড়া সাধারণ ব্যবহার)। Validation-এর ক্ষেত্রে Full Input Match করতে এগুলো প্রয়োজন।

```
const codePattern = /^BD-(?:DAC|CGP)-\d{3}$/;
console.log(codePattern.test('BD-DAC-101')); // true
console.log(codePattern.test('XBD-DAC-101')); // false
```

এখানে `(?:...)` Non-capturing Group এবং `|` Alternation। Capturing Group চাইলে `(\d{4})-(\d{2})-(\d{2})` ব্যবহার করা যায়।

```
const date = '2026-10-11';
const parts = date.match(/^(\d{4})-(\d{2})-(\d{2})$/);
console.log(parts?.slice(1)); // ['2026', '10', '11']
```

সতর্কতা: Pattern মিললেই Date বাস্তব হয় না; `2026-99-99`-ও এই Regex-এ মিলে যাবে। পরে Calendar Validation প্রয়োজন।

১৭.৪ Regex Flags ও JavaScript Methods

- `i`: Case-insensitive Matching।
- `g`: Global Matches।
- `m`: Multiline Anchors।
- `s`: Dotall Mode।
- `u`: Unicode-aware Mode।

```
const text = 'JS 101, JS 202, JS 303';
console.log([...text.matchAll(/JS (\d+)/g)].map(match => match[1]));
// ['101', '202', '303']
console.log(text.replace(/JS/gi, 'JavaScript'));
console.log('Bangladesh'.search(/desh/i));
```

`test()` Boolean দেয়; `match()` Match Data দেয়; `matchAll()` Iterator দেয়; `replace()` Replacement করে; `split()` Regex Delimiter নিতে পারে। `g` Flag-যুক্ত Regex-এর `test()` বারবার চালালে `lastIndex`-এর কারণে Result বদলাতে পারে—এটি Debugging-এর গুরুত্বপূর্ণ বিষয়।

```
const pattern = /a/g;
console.log(pattern.test('a')); // true
console.log(pattern.test('a')); // false: lastIndex পরিবর্তিত
```

১৭.৫ Unicode ও বাংলা Text Validation

বাংলা নাম বা আন্তর্জাতিক নাম Validate করতে শুধু `[A-Za-z]` ব্যবহার করা উচিত নয়। Unicode Property Escape `\p{L}` Letter এবং `\p{M}` Combining Mark বোঝায়। `u` Flag ছাড়া Property Escape প্রত্যাশামতো কাজ করে না।

```
const humanName = /^[\p{L}\p{M}][\p{L}\p{M} .'-]*$/u;
console.log(humanName.test('রহমান'));      // true
console.log(humanName.test('Amina Rahman')); // true
```

বাস্তব নামের বৈচিত্র্য অনেক বেশি। Apostrophe, Hyphen, Script, Transliteration ও Length Requirement ব্যবসায়িক প্রয়োজন অনুযায়ী নির্ধারণ করতে হবে। `String.prototype.normalize('NFC')` Unicode Normalization-এর জন্য কাজে লাগে, তবে এটি সব Linguistic সমস্যা সমাধান করে না।

১৭.৬ Normalization বনাম Validation

Input Normalize করার অর্থ হলো অপ্রয়োজনীয় পার্থক্য দূর করা, যেমন Leading/Trailing Space Trim বা Email Lowercase করা। Validation হলো গ্রহণযোগ্যতার নিয়ম পরীক্ষা। দুটিকে আলাদা রাখলে Code Test করা সহজ হয়।

```
function normalizeSpace(value) {
  return String(value ?? '').trim().replace(/\s+/gu, ' ');
}
function validateUsername(value) {
  const username = normalizeSpace(value);
  return /^[a-z][a-z0-9_]{2,19}$/i.test(username);
}
```

কিছু Input কখনো স্বয়ংক্রিয়ভাবে বদলানো উচিত নয়—যেমন Password Trim করলে User-এর দেওয়া Secret পরিবর্তিত হতে পারে।

১৭.৭ Email, Phone এবং Password Validation-এর সীমা

Email-এর জন্য একটি সাধারণ Pattern `^[^\s@]+@[^\s@]+\.[^\s@]+$` UI-তে প্রাথমিক Check দিতে পারে। কিন্তু এটি RFC-complete Email Parser নয়। Email সত্যি ব্যবহারযোগ্য কি না জানতে Verification Link বা OTP দরকার।

Phone Number আন্তর্জাতিকভাবে জটিল; Production-এ Country-aware Phone Parsing Library প্রয়োজন হতে পারে। Password-এর ক্ষেত্রে শুধু Regex Complexity Rule যথেষ্ট নয়। Server-এ Secure Password Hashing, Rate Limiting, Breached Password Check এবং MFA গুরুত্বপূর্ণ।

```
function basicEmailCheck(value) {
  const email = String(value).trim();
  return email.length <= 254 && /^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(email);
}
```

১৭.৮ Regex Security: ReDoS ও Untrusted Input

Nested Quantifier-এর মতো কিছু Pattern-এ Catastrophic Backtracking হতে পারে। উদাহরণস্বরূপ, `^(a+)+$` দীর্ঘ `aaaa...!` Input-এ অত্যন্ত ধীর হতে পারে। তাই Untrusted Text-এর ক্ষেত্রে Regex-এর Complexity, Maximum Input Length এবং বিকল্প Parser বিবেচনা করো।

আরেকটি ঝুঁকি: User Input সরাসরি `new RegExp(userInput)`-এ বসালে Special Character Pattern বদলে দিতে পারে। Literal Search চাইলে Escape করো।

```
function escapeRegex(value) {
  return String(value).replace(/[.*+?^${}()|[\]\\]/g, '\\$&');
}
const query = 'a+b';
const safePattern = new RegExp(escapeRegex(query), 'i');
console.log(safePattern.test('Search: a+b')); // true
```

১৭.৯ XSS, Injection এবং Secure DOM Rendering

Regex দিয়ে HTML Sanitization করার চেষ্টা করো না। Untrusted Text দেখাতে `textContent` ব্যবহার করো। `innerHTML` দিয়ে User-supplied Content বসানো বিপজ্জনক হতে পারে। HTML প্রয়োজন হলে নির্ভরযোগ্য Sanitization Library ও Strict Content Security Policy বিবেচনা করো।

```
const display = document.querySelector('#output');
display.textContent = '<img src=x onerror=alert(1)>';
// Text হিসেবে দেখাবে; HTML Execute করবে না।
```

SQL Injection Regex দিয়ে প্রতিরোধ করা যায় না; Database-এ Parameterized Query ব্যবহার করতে হয়। Client-side Validation সহজেই Bypass করা যায়, তাই Server-side Validation বাধ্যতামূলক।

১৭.১০ Practical Project — FormShield

Project Goal: User-এর Full Name, Email, Phone, Password এবং Consent যাচাই করে পরিষ্কার Error Feedback দেখানো। এটি শুধুই Browser-side Demonstration; কোনো Personal Data Network-এ পাঠায় না বা Storage-এ রাখে না।

Folder Structure

```
source-code/chapter-17/
├── index.html
├── style.css
├── form-core.mjs
├── app.mjs
└── form-core.test.mjs
```

`form-core.mjs`-এ Pure Validation Functions, `app.mjs`-এ Form Events এবং DOM Update, `index.html`-এ Accessible Form এবং `style.css`-এ Responsive Layout আছে। পূর্ণ Runnable Source Code বইয়ের Companion Source Code-এর `source-code/chapter-17/` Folder-এ সংযুক্ত।

Core Validation Flow

```
import { validateRegistration } from './form-core.mjs';
const result = validateRegistration({
  name: 'Amina Rahman',
  email: 'amina@example.com',
  phone: '+8801712345678',
  password: 'SecurePass1234',
  consent: true
});
console.log(result.ok); // true
```

Run Instructions

1. Node.js 20+ ইনস্টল করো।
2. `node --test source-code/chapter-17/form-core.test.mjs` দিয়ে Tests চালাও।
3. Browser Module চালানোর জন্য Folder Root-এ `python -m http.server 8000` দিয়ে Local Server চালাতে পারো।
4. `http://localhost:8000/source-code/chapter-17/` খুলে Form পরীক্ষা করো।
5. Invalid Email, Bengali Name, Long Input, Missing Consent এবং HTML Injection Text দিয়ে Test করো।

Project Extension Ideas

Field-level Live Validation, Accessible Error Summary, Country-aware Phone Parser, Password Strength Feedback এবং Backend Validation Endpoint যুক্ত করতে পারো। Password কখনো Console-এ Log বা Browser Storage-এ Save করবে না।

১৭.১১ Exercises

Exercise 1: `BD-1234` Format Validate করার Regex লেখো।

Exercise 2: একটি String থেকে সব ৪-অঙ্কের Year বের করো।

Exercise 3: `test()`-এর Global `lastIndex` Pitfall প্রদর্শন করো।

Exercise 4: বাংলা ও ইংরেজি Letter গ্রহণ করে এমন Name Pattern তৈরি করো।

Exercise 5: Email Normalize এবং Validate করার দুটি আলাদা Function লেখো।

Exercise 6: User Search Query-কে Regex Literal হিসেবে Escape করো।

Exercise 7: Phone Number থেকে Space ও Hyphen সরিয়ে Digit Count Check করো।

Exercise 8: `innerHTML`-এর বদলে `textContent` ব্যবহার করে Safe Preview বানাও।

Exercise 9: Invalid Date Pattern এবং Invalid Calendar Date-এর পার্থক্য বোঝাও।

Exercise 10: FormShield-এ Password Confirmation Field যুক্ত করো।

১৭.১২ Sample Solutions

```
// 1
const codeOK = /^BD-\d{4}$/.test('BD-1234');
// 2
const years = [...'1999, 2026 and 2040'.matchAll(/\b\d{4}\b/g)].map(m => m[0]);
// 3
const global = /x/g; console.log(global.test('x'), global.test('x'));
// 4
const nameOK = /^[\p{L}\p{M} ]+$/u.test('রহমান Ali');
// 5
const normalizeEmail = v => String(v).trim().toLowerCase();
const emailOK = v => /^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(normalizeEmail(v));
// 6
const escapeRegex = v => String(v).replace(/[.*+?^${}()|[\]\\]/g, '\\$&');
// 7
const digits = '+880 1712-345678'.replace(/[\s-]/g, '');
const phoneOK = /^\+?\d{7,15}$/.test(digits);
// 8 (Browser)
// document.querySelector('#preview').textContent = userInput;
// 9: 2026-99-99 Regex match করতে পারে, কিন্তু Calendar Date নয়।
// 10: password === confirmation পরীক্ষা করো; Password Log/Store কোরো না।
```

১৭.১৩ MCQ Quiz

১. Regex-এ `\d` কী? A) Letter B) Digit C) Dot D) Space

২. `^` ও `$` একসঙ্গে কেন? A) Style B) Full Input Match C) API Call D) JSON

৩. বাংলা Unicode Property Escape-এর জন্য কোন Flag? A) g B) i C) u D) m

৪. `test()` কী Return করে? A) Boolean B) Array C) Promise D) Object

৫. `g` Flag-সহ Repeated `test()`-এ কোন Property বদলায়? A) length B) lastIndex C) prototype D) size

৬. Regex দিয়ে HTML Sanitization কি নিরাপদ? A) হ্যাঁ B) না C) শুধু Chrome D) শুধু Node

৭. Untrusted Text দেখানোর নিরাপদ DOM Property কোনটি? A) innerHTML B) outerHTML C) textContent D) insertAdjacentHTML

৮. ReDoS সাধারণত কোন সমস্যার সঙ্গে যুক্ত? A) CSS B) Catastrophic Backtracking C) URL Encoding D) JSON

৯. Client-side Validation কি Server-side Validation-এর বিকল্প? A) হ্যাঁ B) না C) শুধু HTTPS D) শুধু SPA

১০. Email Regex Pass করলে কি Inbox Ownership নিশ্চিত হয়? A) হ্যাঁ B) না C) শুধু Gmail D) শুধু Corporate

Answer Key ও Explanation

1. **B** — `\d` Digit Class।
2. **B** — Start ও End Anchor দিয়ে সম্পূর্ণ Input Check করা যায়।
3. **C** — `u` Unicode Mode চালু করে।
4. **A** — `test()` True বা False দেয়।

5. **B** — Global/Sticky Regex-এর `lastIndex` Stateful।
6. **B** — HTML Sanitization-এর জন্য Regex নির্ভরযোগ্য নয়।
7. **C** — `textContent` Markup Execute করে না।
8. **B** — Complex Backtracking CPU Time বাড়াতে পারে।
9. **B** — Client Validation Bypass করা যায়।
10. **B** — Ownership Verify করতে Confirmation দরকার।

১৭.১৪ Chapter Recap

Regex একটি শক্তিশালী Text Processing Tool, কিন্তু Validation, Security ও Business Logic আলাদা বিষয়। Anchors, Groups, Quantifiers, Unicode, `matchAll()`, Normalization, ReDoS Awareness এবং Safe DOM Rendering—এই দক্ষতাগুলো FormShield Project-এ একসঙ্গে প্রয়োগ করেছ। পরবর্তী অধ্যায়ে Functional Programming, Immutability এবং Advanced Data Transformation নিয়ে কাজ করব।

অধ্যায় ১৮ — Functional Programming, Immutability এবং Advanced Data Transformation

Level: Intermediate → Advanced | **Project:** SalesFlow Analytics | **Prerequisites:** Functions, Arrays, Objects, Modules।

শেখার লক্ষ্য

এই অধ্যায় শেষে তুমি Pure Function ও Side Effect আলাদা করতে, Immutable Transformation লিখতে, `map`/`filter`/`reduce`-এর মাধ্যমে Data Pipeline তৈরি করতে, Grouping ও Ranking করতে, Function Composition বুঝতে এবং Automated Test দিয়ে Data Integrity যাচাই করতে পারবে।

১৮.১ Functional Programming কী এবং কেন?

Functional Programming (FP) হলো এমন একটি Programming Style যেখানে সমস্যাকে ছোট, পুনর্ব্যবহারযোগ্য Function-এর মাধ্যমে সমাধান করা হয়। বিশেষ গুরুত্ব দেওয়া হয় **Pure Functions**, **Immutable Data** এবং **Declarative Transformation**-এ। JavaScript Multi-Paradigm Language: একই Application-এ FP, OOP এবং Imperative Code একসঙ্গে থাকতে পারে।

ধরা যাক একটি Sales Dashboard-এর জন্য Orders থেকে Paid Revenue বের করতে হবে। Imperative পদ্ধতিতে Loop, Temporary Variable এবং State Update করতে পারো। Functional পদ্ধতিতে Transformation-এর উদ্দেশ্য সরাসরি প্রকাশ করা যায়।

```
const orders = [
  { id: 1, status: 'paid', total: 800 },
  { id: 2, status: 'refunded', total: 500 },
  { id: 3, status: 'paid', total: 1200 }
];

const revenue = orders
  .filter(order => order.status === 'paid')
  .reduce((sum, order) => sum + order.total, 0);
console.log(revenue); // 2000
```

এখানে Original `orders` Array বদলায়নি। প্রতিটি Step-এর দায়িত্বও আলাদা।

১৮.২ Pure Function, Referential Transparency ও Side Effects

একটি Function-কে Pure বলা যায় যদি (১) একই Input দিলে একই Output দেয় এবং (২) External State পরিবর্তন না করে।

```
function calculateLineTotal(quantity, unitPrice) {
  return quantity * unitPrice;
}
```

এটি Pure। কিন্তু নিচের Function বাইরের `total` পরিবর্তন করে:

```
let total = 0;
function addToTotal(amount) {
  total += amount; // side effect
  return total;
}
```

Side Effect সবসময় খারাপ নয়। Browser DOM Update, Network Request, File Write এবং Logging বাস্তব Application-এর প্রয়োজনীয় কাজ। ভালো Architecture-এ এসবকে Core Calculation থেকে আলাদা রাখা হয়। এতে Test সহজ হয়।

Referential Transparency: Pure Function Call-কে একই Input-এর Result দিয়ে প্রতিস্থাপন করলেও আচরণ বদলায় না। এটি Reasoning ও Caching সহজ করে।

১৮.৩ Immutability — পরিবর্তনের বদলে নতুন Value

JavaScript-এ Object ও Array Reference-based। `const` দিয়ে Variable Reassignment বন্ধ করা যায়; Nested Data Mutation বন্ধ হয় না।

```
const student = { name: 'Amina', profile: { city: 'Dhaka' } };
student.profile.city = 'Khulna'; // বৈধ, যদিও student const

const updated = {
  ...student,
  profile: { ...student.profile, city: 'Sylhet' }
};
```

Spread Operator একটি **Shallow Copy** তৈরি করে। Deeply Nested Structure-এর ক্ষেত্রে প্রয়োজনীয় স্তরগুলো Copy করতে হয়, অথবা Supported Environment-এ `structuredClone()` ব্যবহার করা যায়। তবে Function, DOM Node, কিছু Special Object `structuredClone`-এ Unsupported হতে পারে।

Array-এর জন্য:

```
const items = ['A', 'B'];
const appended = [...items, 'C'];
const removed = items.filter(item => item !== 'A');
const replaced = items.map(item => item === 'B' ? 'New B' : item);
console.log(items); // ['A', 'B'] — unchanged
```

`sort()`, `reverse()`, `splice()` Original Array পরিবর্তন করে। Non-mutating বিকল্প হিসেবে `toSorted()`, `toReversed()`, `toSpliced()` Modern Runtimes-এ পাওয়া যায়; Compatibility পরীক্ষা করবে।

১৮.৪ `map()`, `filter()`, `reduce()` এবং `flatMap()`

`map` একেকটি Item থেকে নতুন Item তৈরি করে; `filter` শর্ত অনুযায়ী Item রাখে; `reduce` অনেক Item থেকে একটি Accumulator তৈরি করে; `flatMap` Mapping-এর পর এক স্তর Flatten করে।

```
const products = [
  { name: 'Book', price: 400, stock: 3 },
  { name: 'Course', price: 1200, stock: 0 },
  { name: 'Pen', price: 30, stock: 10 }
```

```
];
const availableNames = products
  .filter(product => product.stock > 0)
  .map(product => product.name);
// ['Book', 'Pen']
const totalStockValue = products.reduce(
  (sum, product) => sum + product.price * product.stock, 0
); // 1500
```

`reduce()`-এ Initial Value দেওয়া গুরুত্বপূর্ণ। Empty Array-এ Initial Value ছাড়া `reduce()` Error দেয়।

১৮.৫ Data Validation ও Normalization

External Data বিশ্বাস করা যায় না। API, CSV বা Form থেকে আসা Numeric Value String হতে পারে। Pipeline-এর শুরুতে Input Normalize এবং Validate করো।

```
function normalizeOrder(order) {
  const quantity = Number(order.quantity);
  const unitPrice = Number(order.unitPrice);
  if (!Number.isSafeInteger(quantity) || quantity <= 0) {
    throw new RangeError('Quantity must be a positive integer');
  }
  if (!Number.isFinite(unitPrice) || unitPrice < 0) {
    throw new RangeError('Price must be non-negative');
  }
  return { ...order, quantity, unitPrice, total: quantity * unitPrice };
}
```

Financial Precision Note: বাস্তব Payment System-এ Floating-Point Price সরাসরি ব্যবহারে Precision Issue হতে পারে। Currency Minor Units (যেমন পয়সা) বা নির্ভরযোগ্য Decimal Library ব্যবহার করা উত্তম। আমাদের Project শিক্ষামূলক।

১৮.৬ Grouping এবং Aggregation

Category অনুযায়ী Sales Group করার সময় Accumulator ব্যবহার করা যায়:

```
function revenueByCategory(orders) {
  return orders
    .filter(order => order.status === 'paid')
    .reduce((groups, order) => ({
      ...groups,
      [order.category]: (groups[order.category] ?? 0) + order.total
    }), {});
}
```

এটি Immutable, কিন্তু প্রতিটি Iteration-এ পুরো Object Spread করলে বড় Dataset-এ Cost বাড়তে পারে। Performance-sensitive Code-এ একটি Function-এর **Local Mutable Accumulator** ব্যবহার করেও বাইরে থেকে Pure আচরণ বজায় রাখা যায়:

```
function efficientRevenueByCategory(orders) {
  const groups = new Map();
  for (const order of orders) {
    if (order.status !== 'paid') continue;
    groups.set(order.category, (groups.get(order.category) ?? 0) + order.total);
  }
  return groups;
```

```
}
```

মূল কথা: FP মানেই প্রতিটি Local Variable অপরিবর্তনীয় হতে হবে, এমন নয়। External Observable Mutation এড়ানোই প্রধান উদ্দেশ্য।

১৮.৭ Function Composition ও Pipelines

একটি Function-এর Output আরেকটি Function-এর Input হলে Function Composition তৈরি হয়।

```
const pipe = (...functions) => initial =>
  functions.reduce((value, fn) => fn(value), initial);

const trim = value => value.trim();
const lower = value => value.toLowerCase();
const hyphenate = value => value.replace(/\s+/g, '-');
const slugify = pipe(trim, lower, hyphenate);
console.log(slugify('  JavaScript Mastery  '));
```

`pipe` বাম থেকে ডানে কাজ করে। `compose` সাধারণত ডান থেকে বামে কাজ করে। Function Signatures মিলতে হবে।

১৮.৮ Closures, Currying ও Partial Application

```
const greaterThan = minimum => number => number > minimum;
const premiumPrices = [100, 500, 1200].filter(greaterThan(400));
// [500, 1200]

const multiply = a => b => a * b;
const double = multiply(2);
console.log(double(7)); // 14
```

এখানে Inner Function বাইরের `minimum` বা `a` ধরে রাখে—এটি Closure। Currying একটি Multi-Argument Function-কে একাধিক Single-Argument Function-এর Chain হিসেবে প্রকাশ করে। Partial Application কিছু Argument আগেই Fix করে নতুন Function তৈরি করে।

১৮.৯ Sorting, Stability ও Non-Mutation

```
const ranked = [...products].sort((a, b) => b.price - a.price);
```

Original `products` অক্ষত থাকে। String Sort-এ `localeCompare()` উপযোগী। Multiple Sort Keys থাকলে Tie-breaker যোগ করো। Input Array বড় হলে বারবার Sorting-এর Cost সাধারণত O(n log n)।

১৮.১০ Async Pipelines এবং সীমাবদ্ধতা

`map(async ...)` একটি **Promise Array** তৈরি করে, Resolved Value Array নয়।

```
const ids = [1, 2, 3];
const results = await Promise.all(ids.map(async id => {
  return { id, valid: true };
}));
```

`Promise.all`-এ একটি Promise Reject করলে Combined Promise Reject হয়। Concurrent Request বেশি হলে Rate Limit/Memory বিবেচনা করে Bounded Concurrency দরকার হতে পারে।

১৮.১১ Practical Project — SalesFlow Analytics

Goal: Raw Sales Orders থেকে Searchable Dashboard, Paid Revenue, Unit Count ও Category Ranking তৈরি করা।

Folder Structure:

```
source-code/chapter-18/
├── index.html
├── style.css
├── sales-core.mjs
├── app.mjs
└── sales-core.test.mjs
```

Architecture

```
Raw Orders
    │
    ▼
normalize + validate
    │
    ▼
filter (search/category/status)
    │
    ├──→ Paid Revenue / Units
    ├──→ Category Aggregation + Ranking
    └──→ DOM Rendering (textContent only)
```

`sales-core.mjs`-এ `normalizeOrder`, `prepareOrders`, `filterOrders`, `sumRevenue`, `totalUnits`, `groupByCategory`, `rankCategories`, `summarize` ও `pipe` রয়েছে। এগুলো DOM-নির্ভর নয়, তাই Node.js Test Runner দিয়ে Test করা যায়। `app.mjs`-এ Event Listener ও DOM Rendering আলাদা রাখা হয়েছে। Full Runnable Files Companion Source Code-এর `source-code/chapter-18/` Folder-এ রয়েছে।

Core Usage:

```
import { sampleOrders, summarize } from './sales-core.mjs';
const report = summarize(sampleOrders, { category: 'Books', status: 'paid' });
console.log(report.revenue); // 2900
console.log(report.units);   // 3
```

চালানোর নিয়ম: Project Folder-এ Local HTTP Server চালিয়ে Browser-এ `index.html` খুলো। যেমন `python -m http.server 8000`, তারপর `http://localhost:8000`। `file://` দিয়ে ES Modules চালালে Browser Security Restrictions হতে পারে। Test: `node --test sales-core.test.mjs`।

Design Decisions: Source Array Mutate করা হয়নি; `textContent` ব্যবহার করা হয়েছে যাতে User Text HTML হিসেবে Execute না হয়; Refund/Pending Revenue-তে Count হয় না; Empty Dataset-এর জন্য Safe Default আছে।

১৮.১২ Testing Strategy

Pure Functions Test করতে Browser লাগে না। গুরুত্বপূর্ণ Cases:

1. Empty Array-তে Revenue 0।
2. Refunded Order Revenue-তে যোগ হবে না।
3. Duplicate Order ID Reject হবে।
4. Negative Price বা Zero Quantity Reject হবে।
5. Search Case-insensitive।
6. Category Ranking Descending।
7. Input Array Mutation হবে না।

Sample Test:

```
import test from 'node:test';
import assert from 'node:assert/strict';
import { summarize } from './sales-core.mjs';

test('empty report', () => {
  assert.equal(summarize([]).revenue, 0);
});
```

১৮.১৩ Common Mistakes

- `map()` ব্যবহার করে শুধু Side Effect ঘটানো; এমন ক্ষেত্রে `forEach()` বা Loop বেশি স্পষ্ট।
- `sort()` দিয়ে Original Array অজান্তে বদলে ফেলা।
- `reduce()`-এ Initial Value বাদ দেওয়া।
- `const`-কে Deep Immutability মনে করা।
- `map(async ...)` থেকে Promise Array পাওয়ার বিষয়টি ভুলে যাওয়া।
- Monetary Values-এর Floating-Point Precision উপেক্ষা করা।
- Large Dataset-এ প্রতিটি `reduce` Iteration-এ Object Spread করে অপ্রয়োজনীয় Cost বাড়ানো।

১৮.১৪ Practice Exercises

১. `[1,2,3,4]` থেকে প্রতিটি সংখ্যার Square-এর Array তৈরি করো।

২. `products` থেকে Stock-এ থাকা Product Name বের করো।

৩. Paid Orders-এর মোট Revenue বের করো।

৪. Category অনুযায়ী Order Count তৈরি করো।

৫. Input Array না বদলে Price অনুযায়ী Descending Sort করো।

৬. `pipe()` দিয়ে Trim → Lowercase → Slug Generator বানাও।

৭. Duplicate ID Reject করার Function লিখো।

৮. Empty Dataset-এর জন্য Safe Summary তৈরি করো।

৯. `async map` এবং `Promise.all` দিয়ে Fake Data Enrichment করো।

১০. `groupByCategory()`-এর Unit Test লিখো।

১৮.১৫ Sample Solutions

```
// 1
const squares = [1,2,3,4].map(n => n*n);
// 2
const inStock = products.filter(p => p.stock > 0).map(p => p.name);
// 3
const paidRevenue = orders.filter(o => o.status === 'paid')
  .reduce((sum,o) => sum + o.total, 0);
// 4
const countByCategory = orders.reduce((acc,o) => {
  acc[o.category] = (acc[o.category] ?? 0) + 1;
  return acc;
}, {});
// 5
const sorted = [...products].sort((a,b) => b.price - a.price);
// 6
const slug = pipe(s => s.trim(), s => s.toLowerCase(),
  s => s.replace(/\s+/g,'-'));
// 7
function rejectDuplicates(items) {
  const ids = items.map(item => item.id);
  if (new Set(ids).size !== ids.length) throw new Error('Duplicate IDs');
  return items;
}
// 8
const safeSummary = items => ({ count: items.length,
  total: items.reduce((sum,item) => sum + item.total,0) });
// 9
async function enrich(ids) {
  return Promise.all(ids.map(async id => ({id, valid:true})));
}
// 10: source-code/chapter-18/sales-core.test.mjs দেখো।
```

Exercise Note: Solutions 2–5-এ পূর্ববর্তী Sections-এর `products` ও `orders` Sample Variables ব্যবহার করা হয়েছে।

১৮.১৬ Knowledge Check — ১০টি MCQ

১. Pure Function-এর বৈশিষ্ট্য কোনটি? A) Global State Update B) Same Input → Same Output C) DOM Mutation D) Random Output

২. `filter()` কী Return করে? A) Boolean B) New Array C) Single Number D) Promise

৩. কোন Method Original Array Mutate করে? A) `map` B) `filter` C) `sort` D) `slice`

৪. `reduce([])`-এ Initial Value না থাকলে কী হয়? A) 0 B) null C) TypeError D) undefined

৫. `const obj = {x:1}` হলে `obj.x=2`? A) সবসময় Error B) সম্ভব C) Syntax Error D) Compile Error

৬. `map(async fn)` কী Return করে? A) Resolved Values B) Promise Array C) String D) Map Object

৭. `pipe(f,g)(x)` সাধারণত কোন Order? A) g তারপর f B) f তারপর g C) Random D) Parallel

৮. External State পরিবর্তন না করেও Local Mutation কি থাকতে পারে? A) না B) হ্যাঁ C) শুধু Browser-এ D) শুধু React-এ

৯. `structuredClone()` কি Function Clone করতে পারে? A) সবসময় B) না C) শুধু Async Function D) শুধু

Arrow Function

১০. Refund বাদ দিয়ে Revenue হিসাবের আগে কী প্রয়োজন? A) `filter` B) `alert` C) `document.write` D) `eval`

Answer Key ও Explanation

1. **B** — Deterministic Result ও Observable Side Effect না থাকা Pure Function-এর মূল বৈশিষ্ট্য।
2. **B** — `filter` নতুন Array দেয়।
3. **C** — `sort` In-place Mutate করে।
4. **C** — Empty Array-এ Initial Accumulator না থাকলে TypeError।
5. **B** — `const` Binding Reassignment আটকায়, Object Mutation নয়।
6. **B** — `async` Callback-এর Result Promise।
7. **B** — Pipe Left-to-Right।
8. **B** — Local Mutation বাইরের Observable State না বদলালে Function Pure থাকতে পারে।
9. **B** — Function `structuredClone`-এ Supported নয়।
10. **A** — Status Filter দিয়ে Eligible Orders নির্বাচন করা যায়।

১৮.১৭ Chapter Recap ও Next Step

Functional Programming একটি চিন্তার পদ্ধতি: ছোট Pure Functions, Immutable Data Flow, Explicit Validation এবং Testable Transformations। SalesFlow Analytics-এ আমরা এই ধারণাগুলোকে বাস্তব Project-এ ব্যবহার করেছি। পরবর্তী অধ্যায়ে Performance Optimization, Memory Management ও Web Application Efficiency নিয়ে কাজ করব।

অধ্যায় ১৯ — JavaScript Performance Optimization, Memory Management এবং Web Application Efficiency

Practical Project: DataPulse Explorer — ১০,০০০ রেকর্ডের Search, Sort ও Virtual Scrolling Dashboard।

Learning Outcomes

এই অধ্যায় শেষে তুমি পারবে (১) Performance Bottleneck চিহ্নিত করতে, (২) Time Complexity ও বাস্তব Benchmark-এর পার্থক্য বুঝতে, (৩) Browser Main Thread-এর কাজ কমাতে, (৪) Memory Leak প্রতিরোধ করতে, (৫) Virtualization ও Debounce প্রয়োগ করতে, এবং (৬) DevTools দিয়ে পরিবর্তনের ফল যাচাই করতে।

১৯.১ Performance বলতে কী বোঝায়?

একটি Web Application দ্রুত মনে হওয়ার জন্য শুধু JavaScript Function দ্রুত হওয়া যথেষ্ট নয়। Network, JavaScript Execution, Style Calculation, Layout, Paint এবং User Input Response—সবকিছু মিলে ব্যবহারকারীর অভিজ্ঞতা তৈরি হয়। **Latency** হলো একটি কাজ শেষ হতে সময়; **Throughput** হলো নির্দিষ্ট সময়ে কত কাজ করা যায়। ১০,০০০ রেকর্ড Filter করতে ৩ ms লাগলেও ১০,০০০ DOM Row আঁকতে ২০০ ms লাগতে পারে। তাই Optimization-এর আগে কোথায় সময় ব্যয় হচ্ছে তা জানতে হবে।

Rule: Measure → Identify Bottleneck → Change One Thing → Measure Again। অকারণে `for` বনাম `map` নিয়ে Micro-optimization করলে বড় Bottleneck অমীমাংসিত থাকতে পারে।

১৯.২ Big-O: Algorithmic Growth

Complexity	সাধারণ উদাহরণ	১০ থেকে ১০,০০০ Input হলে
O(1)	`Map.get(key)` — average-case	তুলনামূলক স্থির
O(log n)	Sorted Data-তে Binary Search	ধীরে বাড়ে
O(n)	Array Scan	প্রায় Input-এর অনুপাতে
O(n log n)	Efficient Comparison Sort	n-এর চেয়ে দ্রুত বাড়ে
O(n^2)	Nested Pairwise Comparison	বড় Data-তে অত্যন্ত ব্যয়বহুল

```
// O(n²): প্রতিটি Product-এর সঙ্গে সব Product মিলানো
for (const a of products) {
  for (const b of products) {
    if (a.id === b.id) { /* ... */ }
  }
}

// ID Lookup বারবার দরকার হলে একটি Map তৈরি করা ভালো।
const byId = new Map(products.map(item => [item.id, item]));
```

```
const product = byId.get(42); // average-case O(1)
```

`Map` তৈরিতে O(n) সময় ও অতিরিক্ত Memory লাগে। তাই একবার Lookup-এর জন্য সবসময় `Map` বানানো লাভজনক নয়। **Time–Space Tradeoff** বুঝে সিদ্ধান্ত নিতে হবে।

১৯.৩ Performance Measurement

```
const start = performance.now();
const result = expensiveCalculation();
const duration = performance.now() - start;
console.log({ duration, result });
```

`performance.now()` High-Resolution Clock দেয়। কিন্তু একটি Measurement যথেষ্ট নয়: JIT Warm-up, Garbage Collection, Background Tab Throttling ও Machine Load-এর কারণে ফল বদলায়। একই Data দিয়ে বহুবার চালিয়ে Median/Distribution দেখো। Production User Experience যাচাই করতে Browser Performance Panel ও Real-User Metrics ব্যবহার করো।

```
function benchmark(fn, iterations = 30) {
  for (let i = 0; i < 5; i++) fn(); // warm-up
  const samples = [];
  for (let i = 0; i < iterations; i++) {
    const t = performance.now();
    fn();
    samples.push(performance.now() - t);
  }
  samples.sort((a, b) => a - b);
  return { medianMs: samples[Math.floor(samples.length / 2)], samples };
}
```

সতর্কতা: Benchmark-এ Result ব্যবহার না করলে Optimizer কাজ বাদ দিতে পারে; Browser Rendering Benchmark-এ DOM Paint-এর খরচ আলাদা বিবেচনা করতে হবে।

১৯.৪ Main Thread ও Long Tasks

Browser-এর Main Thread JavaScript, Input Processing এবং Rendering-এর বহু কাজ সামলায়। দীর্ঘ Synchronous Loop User Interaction আটকে দিতে পারে।

```
// বড় কাজকে Chunk-এ ভাগ করা: Browser-কে মাঝে কাজের সুযোগ দেওয়া।
async function processInChunks(items, process, chunkSize = 500) {
  for (let i = 0; i < items.length; i += chunkSize) {
    for (const item of items.slice(i, i + chunkSize)) process(item);
    await new Promise(resolve => setTimeout(resolve, 0));
  }
}
```

`setTimeout(0)` সঙ্গে সঙ্গে চলার নিশ্চয়তা দেয় না; এটি শুধু পরবর্তী Task-এ কাজ সরায়। Heavy CPU Computation-এর জন্য **Web Worker** ব্যবহার করা যায়; Worker সরাসরি DOM পরিবর্তন করতে পারে না। Large Data Transfer-এ Structured Clone ও Transferable Objects বিবেচনা করতে হয়।

১৯.৫ DOM Rendering কেন ধীর হয়?

```
// সমস্যা: প্রতিবার ১০,০০০ Row DOM-এ রাখা
for (const item of items) {
  list.append(createRow(item));
}
```

অনেক DOM Node Memory ও Layout Cost বাড়ায়। `DocumentFragment` দিয়ে Batch Append কিছু Overhead কমাতে পারে, কিন্তু DOM-এ ১০,০০০ Row থাকলে সমস্যার মূল সমাধান হয় না। **Virtualization** শুধু দৃশ্যমান Row এবং সামান্য Overscan Render করে।

```
const rowHeight = 40;
const start = Math.max(0, Math.floor(scrollTop / rowHeight) - 5);
const end = Math.min(total, Math.ceil((scrollTop + viewportHeight) / rowHeight) + 5);
const visibleItems = items.slice(start, end);
```

এই কৌশলটি Fixed-Height Rows ধরে। Variable-Height Rows হলে Measurement Cache ও Dynamic Positioning দরকার। Accessibility, Keyboard Navigation ও Screen Reader Support-ও পরীক্ষা করতে হবে।

১৯.৬ Debounce ও Throttle

User প্রতিটি Key চাপলে ১০,০০০ Item Search চালানো অপ্রয়োজনীয় হতে পারে। **Debounce** শেষ Input-এর পর নির্দিষ্ট সময় অপেক্ষা করে; **Throttle** নির্দিষ্ট Interval-এ সর্বোচ্চ একবার কাজ চালায়।

```
function debounce(fn, delay = 200) {
  let timer;
  return (...args) => {
    clearTimeout(timer);
    timer = setTimeout(() => fn(...args), delay);
  };
}
searchInput.addEventListener('input', debounce(updateResults, 180));
```

Debounce-এর ফলে Response একটু দেরিতে আসে; Input Type ও UX অনুযায়ী Delay বেছে নিতে হবে। Scroll Rendering-এ `requestAnimationFrame` সাধারণত উপযোগী।

১৯.৭ Memory Model ও Garbage Collection

JavaScript Engine Reachability অনুসারে অপ্রয়োজনীয় Object-এর Memory Reclaim করে। `const`/`let` ব্যবহার করলেই Memory Leak বন্ধ হয় না। **Leak** হলো প্রয়োজন শেষ হওয়ার পরও কোনো Object Reference ধরে রাখা।

```
const handler = () => console.log('resize');
window.addEventListener('resize', handler);

// Component বা Feature বন্ধ করার সময়:
window.removeEventListener('resize', handler);
```

সম্ভাব্য Leak Source: Unremoved Event Listener, `setInterval`, Unbounded Cache, Detached DOM Nodes, Global Collections, Long-Lived Closure। `WeakMap`-এ Object Key-এর Weak Reference থাকে, তবে এটি সব Leak-এর সমাধান নয়। Memory Tab-এ Heap Snapshot ও Retainer Paths দেখো।

১৯.৮ Cache: সুবিধা ও ঝুঁকি

Repeated Expensive Computation-এর জন্য Memoization উপকারী, তবে Cache Memory নেয় এবং Stale Data-এর ঝুঁকি থাকে।

```
const cache = new Map();
function cachedSummary(key, items) {
  if (cache.has(key)) return cache.get(key);
  const value = items.reduce((sum, x) => sum + x.price, 0);
  cache.set(key, value);
  return value;
}
```

এখানে `key` যদি Data Version অনুযায়ী না বদলায়, ভুল Result পাওয়া যাবে। Unbounded `Map` সময়ের সঙ্গে বড় হতে পারে। TTL, LRU, Max Size অথবা Explicit Invalidation প্রয়োজন হতে পারে।

১৯.৯ Network, Bundles ও Lazy Loading

সব Performance সমস্যা JavaScript Loop-এর নয়। বড় Bundle, Blocking Script, Large Images, অতিরিক্ত API Calls এবং Third-Party Scripts Page Slow করতে পারে। `defer`, ES Modules, Dynamic `import()`, Code Splitting, Compression, Cache Headers ও Image Optimization প্রাসঙ্গিক।

```
// ব্যবহারকারী Feature খুললেই Heavy Module Load
button.addEventListener('click', async () => {
  const { renderChart } = await import('./charts.js');
  renderChart();
});
```

`import()` Promise Return করে; Loading Error Handle করতে হবে। Browser DevTools Network Panel-এ Download Size, Waterfall ও Cache Behavior যাচাই করো।

১৯.১০ Practical Project — DataPulse Explorer

Scenario: একটি Dashboard-এ ১০,০০০ Product-like Record আছে। Search, Category Filter ও Sorting থাকবে। কিন্তু Screen-এ একসঙ্গে হাজার হাজার Row না এঁকে Virtual Window ব্যবহার করব।

Project Structure

```
source-code/chapter-19/
├── index.html
├── style.css
├── data-core.mjs
├── app.mjs
└── data-core.test.mjs
```

Run: Project Directory-তে `python -m http.server 8000` চালিয়ে `http://localhost:8000` খুলো। ES Modules `file://` থেকে চালালে Browser Security Restrictions দেখা দিতে পারে। Tests: `node --test data-core.test.mjs`।

Domain Logic: `createDataset()` Deterministic Records তৈরি করে; `queryDataset()` Filter ও Sort করে, Original Array Mutate করে না; `summarize()` Count ও Average হিসাব করে; `windowRange()` Visible Slice-এর Index দেয়। Source Files-এ সম্পূর্ণ Implementation আছে।

```
const all = createDataset(10000);
const result = queryDataset(all, { category: 'Books', sort: 'price-desc' });
const { start, end } = windowRange(result.length, 1200, 40, 400, 6);
const visible = result.slice(start, end);
```

UI Rendering: `app.mjs`-এ Scroll Event-কে `requestAnimationFrame` দিয়ে Batch করা হয়েছে; DOM-এ শুধু Visible Rows রাখা হয়। `textContent` দিয়ে Safe Text Rendering করা হয়েছে। Search Input Debounced; Filter ও Sort Change সঙ্গে সঙ্গে Apply হয়। Query Duration `performance.now()` দিয়ে মাপা হয়।

Performance Experiment: DevTools Elements Panel-এ দেখো ১০,০০০ Record থাকা সত্ত্বেও Rendered Row সাধারণত কয়েক ডজন। তারপর Search ও Scroll করে FPS/Responsiveness পর্যবেক্ষণ করো। এটি Virtualization-এর সুফল দেখায়; সব Device-এ নির্দিষ্ট Millisecond Performance-এর নিশ্চয়তা নয়।

১৯.১১ Debugging & Profiling Checklist

1. Performance Panel-এ Record চালু করে Search/Scroll করো।
2. Main Thread-এ Long Task আছে কি না দেখো।
3. Scripting, Rendering ও Painting Time আলাদা করো।
4. Memory Heap Snapshot নিয়ে Repeat Interaction-এর পরে Compare করো।
5. Filter/Sort Result Correct কি না Automated Tests চালাও।
6. Low-End Device ও Slow Network Emulation-এ Test করো।
7. Keyboard, Focus ও Screen Reader Usability যাচাই করো।

১৯.১২ Common Mistakes

- **Premature Optimization:** Problem না মেপে Code জটিল করা।
- `sort()` **Mutation:** Original Array বদলে যায়; Copy করে Sort করো।
- **Unbounded Cache:** Cached Object কখনো Clear না করা।
- **Scroll Handler Overwork:** প্রতিটি Scroll Event-এ Heavy Computation চালানো।
- **Virtualization = Free Performance:** Filtering/Sorting এখনো সম্পূর্ণ Array-এ চলে।
- **Benchmark Confusion:** One-off Local Timing-কে Production Performance ধরে নেওয়া।
- **Detached DOM Leak:** Listener/Reference Cleanup না করা।

১৯.১৩ Exercises — ১০টি

1. `performance.now()` দিয়ে Array Filter-এর Duration মাপো।

2. ১০,০০০ Record থেকে `category === 'Books'` Filter করো।
3. Original Array অপরিবর্তিত রেখে Price Descending Sort করো।
4. Empty Array-এর Average শূন্য Return করার Function লেখো।
5. ৪০ px Row Height ও ৪০০ px Viewport-এর Visible Range বের করো।
6. Debounce Function লেখো এবং Rapid Calls-এর Result পরীক্ষা করো।
7. `Map`-ভিত্তিক ID Lookup তৈরি করো।
8. `setInterval`-এর Cleanup Function লেখো।
9. ৫০০টি Item করে Async Chunk Processing Implement করো।
10. DataPulse-এর Filter/Sort ও Virtualization Test Cases যোগ করো।

১৯.১৪ Sample Solutions

```
// 1
const t0 = performance.now();
const expensiveResult = numbers.filter(n => n % 2 === 0);
console.log(performance.now() - t0);

// 2
const books = items.filter(item => item.category === 'Books');

// 3
const descending = [...items].sort((a, b) => b.price - a.price);

// 4
const average = xs => xs.length ? xs.reduce((s, x) => s + x, 0) / xs.length : 0;

// 5
const start = Math.max(0, Math.floor(scrollTop / 40) - 5);
const end = Math.min(totalRows, Math.ceil((scrollTop + 400) / 40) + 5);

// 6: source-code/chapter-19/data-core.mjs → debounce()

// 7
const byId = new Map(items.map(item => [item.id, item]));
console.log(byId.get(42));

// 8
const interval = setInterval(refresh, 1000);
function cleanup() { clearInterval(interval); }

// 9
async function chunk(items, fn) {
  for (let i = 0; i < items.length; i += 500) {
    items.slice(i, i + 500).forEach(fn);
    await new Promise(resolve => setTimeout(resolve, 0));
  }
}

// 10: source-code/chapter-19/data-core.test.mjs দেখো।
```

Exercise Note: `numbers`, `items`, `scrollTop`, `totalRows`, `refresh` এগুলো নিজস্ব Example Data/Function দিয়ে Define করতে হবে।

১৯.১৫ Knowledge Check — ১০টি MCQ

১. Virtualization-এর মূল সুবিধা কী? A) API Faster B) DOM Nodes কমানো C) Database Encryption D) Type Checking

২. `Array.sort()` কী করে? A) Always New Array B) Original Mutate C) Network Request D) Promise Return

৩. Debounce কখন উপকারী? A) Rapid Search Input B) Password Hashing C) CSS Color D) Database Migration

৪. `performance.now()` কী মাপে? A) Duration Timing B) Disk Size C) IP Address D) Heap Object Count

৫. Memory Leak-এর একটি কারণ? A) Removed Listener B) Unbounded Cache C) Local Constant D) Small Function

৬. Web Worker কি DOM সরাসরি পরিবর্তন করতে পারে? A) হ্যাঁ B) না C) শুধু Chrome-এ D) শুধু CSS-এ

৭. `Map.get()` Average-case Lookup? A) $O(n^2)$ B) O(log n) C) O(1) D) O(n!)

৮. `requestAnimationFrame` সাধারণত কোথায় উপকারী? A) Visual Update B) SQL Query C) File Compression D) Password Storage

৯. `WeakMap` Key হিসেবে কী নেয়? A) Arbitrary String B) Objects এবং Non-registered Symbols C) Numbers only D) Arrays only

১০. Optimization-এর প্রথম ধাপ কী? A) Guess B) Measure C) Rewrite Everything D) Add More Libraries

Answers & Explanations

1. **B** — দৃশ্যমান Row-এর বাইরে DOM তৈরি না করলে Rendering Cost কমে।
2. **B** — `sort()` In-place Mutate করে।
3. **A** — দ্রুত Input Events Coalesce করা যায়।
4. **A** — High-Resolution Timestamp দিয়ে Elapsed Time মাপা হয়।
5. **B** — Unlimited Cache Reference ধরে রাখে।
6. **B** — Worker-এর সরাসরি DOM Access নেই।
7. **C** — সাধারণ Hash Map Implementation-এ Average-case Constant Time।
8. **A** — Browser Paint Cycle-এর সঙ্গে Visual Work Schedule করে।
9. **B** — Object এবং Non-registered Symbol Key Supported।
10. **B** — Measurement ছাড়া Bottleneck নির্ধারণ করা যায় না।

১৯.১৬ Chapter Recap ও Next Step

Performance Engineering হলো **Measured Tradeoff**। Algorithm, DOM, Main Thread, Memory ও Network একসঙ্গে দেখতে হয়। DataPulse Explorer-এ Pure Data Queries, Debounce, Virtual Window এবং Profiling-এর ভিত্তি তৈরি হলো। পরবর্তী Chapter 20-এ Web Application Security ও Secure Coding Practices শিখব।

অধ্যায় ২০ — JavaScript Security Fundamentals, Web Application Security এবং Secure Coding Practices

২০.১ Learning Objectives

এই অধ্যায় শেষে তুমি Threat Modeling, Trust Boundary, XSS, CSRF, Injection, Authentication বনাম Authorization, Browser Storage, CORS, CSP এবং Secure Coding-এর মৌলিক সিদ্ধান্ত নিতে পারবে। একই সঙ্গে SecureTrip Booking Portal-এ Input Validation, Safe DOM Rendering ও Controlled Data Flow প্রয়োগ করবে।

২০.২ Security মানে কী?

একটি Application সঠিকভাবে কাজ করলেই তা নিরাপদ হয় না। Security হলো **Confidentiality** (অননুমোদিত ব্যক্তি তথ্য না পাওয়া), **Integrity** (তথ্য অনুমতি ছাড়া বদলানো না যাওয়া), এবং **Availability** (বৈধ ব্যবহারকারীর জন্য সেবা চালু থাকা)—এই তিনটির ভারসাম্য।

উদাহরণ: একটি Booking Form-এ Client থেকে `total: 1` পাঠিয়ে যদি ৫০,০০০ টাকার Booking তৈরি করা যায়, তাহলে Server Client-এর মূল্যকে বিশ্বাস করেছে। এটি Integrity Failure। Server-কে নিজস্ব Package Catalog থেকে মূল্য গণনা করতে হবে।

Security principle: Browser-এ থাকা Data, Hidden Input, Local Storage, URL Parameter ও JavaScript State—সবই আক্রমণকারী পরিবর্তন করতে পারে। তাই Browser কখনো Security Authority নয়।

২০.৩ Threat Modeling ও Trust Boundary

একটি Feature লিখতে যাওয়ার আগে চারটি প্রশ্ন করো:

1. কোন Asset রক্ষা করতে হবে? (User Data, Session, Booking, Payment)
2. কে এই Asset ব্যবহার করতে পারে? (Guest, Customer, Staff, Admin)
3. Data কোথা থেকে আসছে? (Form, URL, API, Database, Third-party)
4. কোন Boundary পেরিয়ে Data Trust পাবে? (Browser → API → Database)

```
Untrusted Browser
   |  JSON Request (untrusted)
   v
HTTPS + Server API
   |  Validate + Authenticate + Authorize
   v
Business Rules (price, capacity, status)
   |  Parameterized Queries / Transactions
   v
```

```
Database
```

এই Diagram-এ Validation, Authentication এবং Authorization আলাদা দায়িত্ব। শুধু একটি Login Button থাকলেই User অন্যের Booking দেখতে পারবে না—এমন নিশ্চয়তা পাওয়া যায় না।

২০.৪ XSS — Cross-Site Scripting

XSS হয় যখন Untrusted Content Browser-এ Executable Markup/Script হিসেবে ব্যবহৃত হয়। নিচের Code বিপজ্জনক:

```
// বিপজ্জনক: input-এর HTML browser parse করবে
result.innerHTML = userProvidedName;
```

সাধারণ Text দেখানোর জন্য নিরাপদ পদ্ধতি:

```
const item = document.createElement('li');
item.textContent = userProvidedName;
list.append(item);
```

`textContent` HTML Interpret করে না। কিন্তু `textContent`-এর ব্যবহার সব Context-এ নিরাপত্তার বিকল্প নয়: URL, CSS, HTML Attribute এবং Script Context-এর আলাদা Rules আছে। HTML Rich Text সত্যিই প্রয়োজন হলে বিশ্বস্ত Sanitization Library এবং Context-aware Output Encoding ব্যবহার করতে হবে।

URL সতর্কতা: `a.href = untrustedUrl` করার আগে `new URL()` দিয়ে Scheme (`https:`/`http:`), Domain Allowlist এবং Business Requirement যাচাই করো। `javascript:` URL অনুমোদন করা যাবে না।

২০.৫ Input Validation বনাম Output Encoding

Validation পরীক্ষা করে Data ব্যবসার নিয়ম মানে কি না। Output Encoding নিশ্চিত করে Data যেখানে দেখানো হচ্ছে সেখানে Executable Code হয়ে না যায়। দুটি আলাদা কাজ।

```
function validateTravelers(value) {
  const n = Number(value);
  if (!Number.isSafeInteger(n) || n < 1 || n > 8) {
    throw new Error('Travelers must be 1-8');
  }
  return n;
}
```

Validation **Client ও Server উভয় জায়গায়** থাকতে পারে, কিন্তু Server-side Validation বাধ্যতামূলক। Browser Validation ব্যবহারকারীর অভিজ্ঞতা উন্নত করে; Security Boundary তৈরি করে না।

২০.৬ SQL Injection ও Parameterized Queries

Untrusted String দিয়ে SQL Query তৈরি করা বিপজ্জনক:

```
// করবেন না
const query = `SELECT * FROM bookings WHERE email = '${email}'`;
```

Parameterized Query-তে Query Structure এবং Data আলাদা থাকে:

```
// PostgreSQL node-postgres example (requires configured server and pg package)
const result = await pool.query(
  'SELECT id, status FROM bookings WHERE email = $1',
  [email]
);
```

Database Permission-ও Least Privilege নীতিতে সীমিত রাখতে হবে। Parameterization Data Query Injection ঠেকায়; এটি Authorization বা Rate Limiting-এর বিকল্প নয়।

২০.৭ Authentication বনাম Authorization

Authentication: কে Request করছে? **Authorization:** তার এই Resource-এ Access আছে কি?

ধরা যাক `GET /api/bookings/123`। শুধু Session Valid কি না দেখলে যথেষ্ট নয়। Server-কে যাচাই করতে হবে Booking `123` এই User-এর কি না অথবা তার উপযুক্ত Staff Permission আছে কি না। একে Object-level Authorization বলা হয়।

```
// Conceptual server-side example
if (!session?.userId) return reply.status(401).send();
const booking = await bookings.findById(request.params.id);
if (!booking) return reply.status(404).send();
if (booking.ownerId !== session.userId && !session.roles.includes('admin')) {
  return reply.status(403).send();
}
return reply.send(booking);
```

এই Snippet Framework-নির্ভর Pseudocode; প্রকৃত Framework অনুযায়ী Syntax ও Error Handling সমন্বয় করতে হবে।

২০.৮ Password, Sessions ও Cookies

Password কখনো Plaintext-এ Store করবে না। Server-side Password Hashing-এর জন্য Argon2id (উপযুক্ত Parameters সহ) বা সুপ্রতিষ্ঠিত Password Hashing Scheme ব্যবহার করতে হবে। General-purpose `SHA-256(password)` Password Storage-এর যথেষ্ট সমাধান নয়।

Session Cookie-এর সাধারণ Production Flags:

```
Set-Cookie: session=<opaque-session-id>; HttpOnly; Secure; SameSite=Lax; Path=/
```

`HttpOnly` JavaScript থেকে Cookie পড়া কঠিন করে, `Secure` HTTPS-only Delivery নির্দেশ করে, `SameSite` Cross-site Request-এর কিছু ঝুঁকি কমায়। এগুলো **CSRF প্রতিরোধের সম্পূর্ণ বিকল্প নয়**। State-changing Request-এ CSRF Token বা নির্ভরযোগ্য Origin Checking প্রয়োজন হতে পারে।

Sensitive Authentication Token Local Storage-এ রাখার ঝুঁকি বুঝতে হবে: XSS থাকলে JavaScript তা পড়তে পারে। Threat Model অনুযায়ী Session Architecture নির্বাচন করতে হবে।

২০.৯ CSRF, CORS এবং Same-Origin Policy

CSRF: Victim-এর Browser দিয়ে Victim-এর Authentication Context-এ অনাকাঙ্ক্ষিত Request করানো।
CORS: Browser কোন Origin-এর Script অন্য Origin-এর Response পড়তে পারবে তার Policy। **CORS কোনো Authentication বা CSRF Defense নয়।**

```
// Browser request: cookie-backed session হলে CSRF strategy লাগবে
await fetch('/api/bookings', {
  method: 'POST',
  credentials: 'same-origin',
  headers: {'Content-Type': 'application/json', 'X-CSRF-Token': csrfToken},
  body: JSON.stringify(payload)
});
```

`csrfToken` Server-issued এবং Session-bound হওয়া উচিত। Header নাম দিলেই নিরাপত্তা তৈরি হয় না—Server-কে Token যাচাই করতে হবে।

২০.১০ CSP, HTTPS এবং Security Headers

Content Security Policy (CSP) XSS-এর Impact কমাতে সাহায্য করতে পারে। উদাহরণ:

```
Content-Security-Policy: default-src 'self'; script-src 'self'; object-src 'none'; base-uri
'self'; frame-ancestors 'none'
X-Content-Type-Options: nosniff
Referrer-Policy: strict-origin-when-cross-origin
```

এটি একটি প্রাথমিক Example, সবার জন্য Copy-Paste Production Policy নয়। Script Loading, Third-party Service, Inline Script, Nonce/Hash এবং Browser Support যাচাই করতে হবে। HTTPS/TLS, Secure Cookie, HSTS Deployment ও Dependency Updates-ও গুরুত্বপূর্ণ।

২০.১১ Secrets, Dependencies এবং Logging

API Key, Database Password ও Signing Secret Frontend Bundle-এ রাখা যাবে না। `.env` ফাইল Commit না করে Secret Manager বা Server Environment-এ রাখো। Logs-এ Password, Full Card Number, Session Token, Authorization Header বা Sensitive Personal Data রাখা যাবে না।

Dependencies Update ও Vulnerability Audit করো, কিন্তু Automated Scanner-এর Output মানবিক যাচাই ছাড়া Blindly Fix করবে না। Third-party Script যত কম, Supply-chain Attack Surface তত কম।

২০.১২ Practical Project — SecureTrip Booking Portal

Goal: একটি Browser-based Booking Request Demo তৈরি করা, যাতে Form Validation, Safe Text Rendering, Fixed Price Catalog, Unique Request ID এবং Summary থাকবে। এটি Payment, Authentication, Real Reservation বা Secure Server System নয়।

File Structure

```
source-code/chapter-20/
```

```
├── index.html
├── style.css
├── booking-core.mjs
├── app.mjs
└── booking-core.test.mjs
```

Run: Folder-এ `python -m http.server 8000` চালিয়ে `http://localhost:8000` খুলতে পারো। Project Folder Root থেকে Serve করলে URL হবে `/source-code/chapter-20/`। JavaScript Module File Browser-এ `file://` দিয়ে খুললে কিছু Browser-এ CORS Restriction হতে পারে।

Tests: `node --test source-code/chapter-20/booking-core.test.mjs` (Node.js 20+ recommended)।

Core Logic

```
export function validateBooking(input) {
  // বাস্তব Implementation: booking-core.mjs
  // 1. Name/Email validate
  // 2. Catalog থেকে Package খুঁজে বের করো
  // 3. Travelers 1-8 integer কিনা দেখো
  // 4. Price = trustedCatalogPrice * travelers
  // 5. Consent যাচাই করো
}
```

এটি Architecture Outline; পূর্ণ Runnable Implementation Companion Source Code-এর `source-code/chapter-20/booking-core.mjs`-এ রয়েছে। UI-তে প্রতিটি Booking-এর Text `textContent` দিয়ে দেখানো হয়েছে, `innerHTML` দিয়ে নয়।

Security Boundary Checklist

Control	Demo Status	Production Requirement
Client validation	আছে	Server revalidation বাধ্যতামূলক
Catalog-derived price	Client demo-তে আছে	Server authoritative pricing
Safe DOM text rendering	আছে	Contextual encoding everywhere
Booking ID	Browser crypto random	Server-generated ID and uniqueness
Authentication	নেই	Secure identity/session design
Authorization	নেই	Per-resource access checks
Persistence	In-memory only	Database and transactions
CSRF / Rate limit	নেই	Threat-based server controls
Payments	নেই	Certified payment provider integration

২০.১৩ Guided Security Review

1. `booking-core.mjs`-এ `total` Field Client থেকে নেওয়া হয়েছে কি? কেন?
2. UI-তে `innerHTML` ব্যবহার করলে কোন ধরনের ঝুঁকি তৈরি হতে পারে?
3. `crypto.getRandomValues()` দিয়ে Browser ID তৈরি হলেও Server কেন Unique Constraint চাইবে?

4. Session Cookie `HttpOnly` হলেও CSRF কেন ঘটতে পারে?
5. `CORS: *` Authentication-এর বিকল্প নয় কেন?
6. Booking List যদি Server-এ থাকে, কীভাবে Object-level Authorization নিশ্চিত করবে?

২০.১৪ Exercises — ১০টি

১. Name Input-এর Length Validation ২–৮০ অক্ষর কেন, ব্যাখ্যা করো।

২. User-controlled HTML `textContent` দিয়ে Render করো।

৩. Travelers `2.5` দিলে Reject করার Function লেখো।

৪. Client-supplied `price: 1` উপেক্ষা করে Catalog Price Calculate করো।

৫. Safe Redirect-এর জন্য Relative Path Validation করো।

৬. Authorization এবং Authentication-এর মধ্যে দুটি পার্থক্য লেখো।

৭. SQL Query Parameterization-এর Example লেখো।

৮. CSRF Token ছাড়া Cookie-based POST Request-এর ঝুঁকি ব্যাখ্যা করো।

৯. Login Error Log থেকে Sensitive Fields বাদ দেওয়ার Strategy লেখো।

১০. SecureTrip-এর জন্য Server-side Deployment Checklist তৈরি করো।

Sample Solutions

```
// 1 — Length rule (trim + Unicode normalization আগে করো)
const validNameLength = name => name.length >= 2 && name.length <= 80;

// 2 — Safe DOM text
const span = document.createElement('span');
span.textContent = userText;

// 3 — Integer range
const validTravelers = n => Number.isSafeInteger(Number(n)) && Number(n) >= 1 && Number(n) <=
8;

// 4 — Authoritative price lookup (server-side-ও একই নিয়ম)
const total = catalog.find(p => p.id === packageId).price * travelers;

// 5 — Simplified safe return path (full version: booking-core.mjs)
const safePath = path => typeof path === 'string' && path.startsWith('/') &&
!path.startsWith('//') ? path : '/';
```

৬. Authentication Identity যাচাই করে, Authorization Permission যাচাই করে। ৭. `pool.query('SELECT * FROM bookings WHERE id = $1', [id])`। ৮. Cookie Automatic Attach হতে পারে; Server-side CSRF Defense দরকার। ৯. Structured Log Allowlist ব্যবহার করো; Secret Fields Redact করো। ১০. HTTPS, Server Validation, AuthN/AuthZ, CSRF, Rate Limits, Parameterized SQL, DB Constraints, Secret Management, Monitoring, Dependency Audit এবং Backup/Recovery Plan অন্তর্ভুক্ত করো।

সতর্কতা: ৫ নম্বর Simplified Snippet Production-ready URL Validation নয়; Test করা পূর্ণ Function Source File-এ দেখো।

২০.১৫ MCQ — ১০টি

১. Browser Input-কে কেন Untrusted ধরা হয়? A) Slow B) User can modify C) Unicode D) CSS

২. সাধারণ Text Output-এর জন্য কোনটি ভালো? A) `innerHTML` B) `eval()` C) `textContent` D) `document.write()`

৩. SQL Injection কমাতে কী ব্যবহার করা উচিত? A) String Concatenation B) Parameterized Queries C) Hidden Fields D) CSS

৪. Authorization কী যাচাই করে? A) Identity B) Permission C) Password Hash D) Browser Version

৫. `HttpOnly` Cookie-এর মূল কাজ? A) Encrypt Database B) Restrict JS access C) Prevent all CSRF D) Disable HTTPS

৬. CORS কি Authentication-এর বিকল্প? A) হ্যাঁ B) না C) Only Safari D) Only Node

৭. Price কে Authoritatively Calculate করবে? A) Browser B) Server C) CSS D) URL

৮. Password Storage-এর জন্য কী উপযুক্ত? A) Plaintext B) Fast SHA-256 only C) Argon2id D) Base64

৯. CSP-এর ভূমিকা কী? A) Reduce certain injection impacts B) Replace Validation C) Replace HTTPS D) Replace Login

১০. SecureTrip Demo-তে কী নেই? A) Safe DOM rendering B) Catalog pricing C) Real authentication D) Input validation

Answers & Explanations

1. **B** — Browser State User বদলাতে পারে।
2. **C** — `textContent` HTML Parse করে না।
3. **B** — Query Structure ও Data আলাদা রাখে।
4. **B** — Resource Permission যাচাই করে।
5. **B** — JavaScript থেকে Cookie পড়ার সুযোগ সীমিত করে।
6. **B** — CORS Browser Read Policy, Identity Check নয়।
7. **B** — Server authoritative pricing নিশ্চিত করে।
8. **C** — Argon2id Password Hashing-এর উপযোগী।
9. **A** — CSP Defense-in-depth Layer।
10. **C** — Demo-তে Real Authentication নেই।

২০.১৬ Chapter Recap

Security একটি Feature নয়; এটি Architecture, Coding, Deployment ও Monitoring-এর সমন্বিত প্রক্রিয়া। Client Validation, Safe DOM Rendering এবং Trusted Price Calculation-এর ধারণা SecureTrip Demo-তে প্রয়োগ করেছি। Production Security-এর জন্য Server-side Validation, Authentication, Authorization, Session Defense এবং Database Controls অপরিহার্য। পরবর্তী অধ্যায়ে Design Patterns ও Scalable Architecture আলোচনা করব।

অধ্যায় ২১ — JavaScript Design Patterns, Software Architecture এবং Scalable Application Structure

Hands-on Project: TravelFlow Booking Manager

শেখার লক্ষ্য

এই অধ্যায় শেষে তুমি (১) Design Pattern ও Architecture-এর পার্থক্য বুঝবে, (২) Factory, Strategy, Observer, Repository এবং Dependency Injection ব্যবহার করতে পারবে, (৩) Domain, Application ও Presentation Layer আলাদা করতে পারবে, (৪) State Transition এবং Error Handling সঠিকভাবে ডিজাইন করতে পারবে, (৫) Testable Booking Application তৈরি করতে পারবে।

২১.১ ছোট কোড থেকে বড় Application: সমস্যাটা কোথায়?

একটি ৫০ লাইনের JavaScript ফাইলে DOM Update, Price Calculation ও Storage একসঙ্গে লিখলে প্রথমে সহজ মনে হয়। কিন্তু Project বড় হলে একটি পরিবর্তনে একাধিক Feature ভেঙে যায়। উদাহরণস্বরূপ, Booking Price যদি Button Click Handler-এর মধ্যে হিসাব করা হয়, সেই হিসাব Unit Test, Server API বা অন্য UI-তে পুনর্ব্যবহার কঠিন।

```
// Anti-pattern: UI এবং Business Logic এক জায়গায়।
document.querySelector('#save').addEventListener('click', () => {
  const price = Number(document.querySelector('#price').value);
  localStorage.setItem('booking', JSON.stringify({ price }));
});
```

এখানে User নিজেই Price পরিবর্তন করতে পারে। Storage Failure, Validation, State Transition ও Future API Migration-এর ব্যবস্থা নেই। **Separation of Concerns** অনুসারে দায়িত্বগুলো আলাদা করা ভালো।

২১.২ Design Pattern বনাম Architecture

Design Pattern হলো পুনরাবৃত্ত Design Problem সমাধানের পরীক্ষিত কাঠামো; যেমন Observer বা Factory। **Architecture** হলো পুরো System-এর Component, Dependency ও Data Flow সাজানোর সিদ্ধান্ত; যেমন Layered বা Hexagonal Architecture। Pattern কোনো বাধ্যতামূলক Recipe নয়—প্রয়োজন ছাড়া Pattern ব্যবহার করলে অকারণ Complexity বাড়ে।

Concept	Focus	Example
Factory	Object creation	`createBookingService()`
Strategy	Swappable algorithm	Discount strategies
Observer	Event notification	Booking created event

Concept	Focus	Example
Repository	Data access abstraction	Memory/Browser storage
Dependency Injection	Dependency externalize	Service receives repository
Layered Architecture	Responsibility separation	UI → Service → Domain → Repository

২১.৩ Single Responsibility এবং Cohesion

Single Responsibility Principle (SRP) বোঝায়: একটি Module-এর পরিবর্তনের প্রধান কারণ একটি হওয়া উচিত। UI Module User Interaction সামলাবে, Domain Module Business Rules, Repository Data Persistence। Cohesion বেশি এবং Coupling কম থাকলে Refactoring ও Testing সহজ হয়।

```
// Domain: কোনো Browser API নেই।
export function calculateTotal(unitPrice, travelers) {
  if (!Number.isFinite(unitPrice) || unitPrice < 0) throw new Error('Invalid price');
  if (!Number.isSafeInteger(travelers) || travelers < 1) throw new Error('Invalid travelers');
  return unitPrice * travelers;
}
```

এটি Browser, Node.js এবং Test Runner—সব জায়গায় চলতে পারে।

২১.৪ Factory Pattern

Factory Object Creation-এর Details লুকায়। Constructor Call ছড়িয়ে না রেখে একটি Function দিয়ে Configured Instance তৈরি করা যায়।

```
function createNotifier(channel) {
  if (channel === 'console') return { send: message => console.log(message) };
  if (channel === 'silent') return { send: () => {} };
  throw new Error('Unknown channel');
}
const notifier = createNotifier('console');
notifier.send('Booking created');
```

Factory Test-এ Fake Dependency দেওয়ার পথও খুলে দেয়।

২১.৫ Strategy Pattern

একাধিক Algorithm একই Interface অনুসরণ করলে Strategy উপকারী। যেমন বিভিন্ন Discount Rule:

```
const strategies = {
  regular: total => total,
  member: total => total * 0.95,
  seasonal: total => total * 0.90
};
function applyDiscount(total, type = 'regular') {
  const strategy = strategies[type];
  if (!strategy) throw new Error('Unknown discount strategy');
  return Math.round(strategy(total));
}
console.log(applyDiscount(10000, 'member')); // 9500
```

Real Payment বা Price-এর ক্ষেত্রে Discount Server-side পুনরায় যাচাই করতে হবে।

২১.৬ Observer / Publish–Subscribe Pattern

একটি Booking তৈরি হলে UI Refresh, Analytics ও Notification—একাধিক Consumer-কে জানানোর প্রয়োজন হতে পারে। Observer Pattern-এ Producer সরাসরি Consumer-এর Implementation জানে না।

```
function createEventBus() {
  const handlers = new Map();
  return {
    on(type, handler) {
      const group = handlers.get(type) ?? new Set();
      group.add(handler);
      handlers.set(type, group);
      return () => group.delete(handler);
    },
    emit(type, payload) {
      for (const handler of handlers.get(type) ?? []) handler(payload);
    }
  };
}
```

সতর্কতা: Synchronous `emit()`-এ Listener Exception ছুড়লে পরের Listener আর নাও চলতে পারে। Production System-এ Error Isolation, Logging এবং Async Delivery Semantics আলাদাভাবে ডিজাইন করতে হয়। Unsubscribe না করলে Memory Leak হতে পারে।

২১.৭ Repository Pattern

Business Logic যেন সরাসরি `localStorage` বা Database Query না জানে—Repository সেই Boundary তৈরি করে।

```
function createMemoryRepository() {
  const records = new Map();
  return {
    async get(id) { return records.get(id) ?? null; },
    async save(record) { records.set(record.id, record); },
    async list() { return [...records.values()]; }
  };
}
```

`async` Interface ব্যবহার করায় পরবর্তীতে HTTP API বা Database Repository-তে বদলানো সহজ। তবে Production Repository-তে Concurrency, Transactions, Error Handling ও Authorization প্রয়োজন।

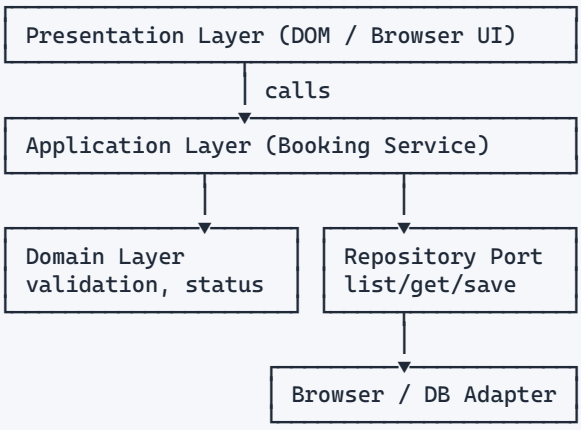
২১.৮ Dependency Injection (DI)

Service-এর ভেতরে Storage ও Event Bus তৈরি না করে বাইরে থেকে পাঠালে Testability বাড়ে।

```
function createService({ repository, events }) {
  return {
    async save(record) {
      await repository.save(record);
      events.emit('saved', record);
      return record;
    }
  };
}
```

এই পদ্ধতিকে **Constructor/Function Injection** বলা যায়। এটি Framework ছাড়াই করা যায়।

২১.৯ Layered Architecture ও Dependency Direction



Domain Layer Browser DOM-এর উপর নির্ভর করবে না। Repository-এর Storage Implementation বদলালেও Service-এর Public Contract একই থাকতে পারে।

২১.১০ State Machine ও Business Rules

Booking Status যেকোনো Value থেকে যেকোনো Value-তে পরিবর্তন করা উচিত নয়। এই Project-এ Transition Rules:

Current	Allowed Next
`pending`	`confirmed`, `cancelled`
`confirmed`	`cancelled`
`cancelled`	কোনো Transition নয়

```
const allowed = {
  pending: ['confirmed', 'cancelled'],
  confirmed: ['cancelled'],
  cancelled: []
};
function canTransition(current, next) {
  return (allowed[current] ?? []).includes(next);
}
```

State Machine Invalid Transition আটকায় এবং Workflow-কে স্পষ্ট করে।

২১.১১ Practical Project — TravelFlow Booking Manager

Project Goal: একটি Educational Booking Dashboard তৈরি করা, যেখানে Customer Package নির্বাচন করে Booking Request তৈরি করবে এবং Status Update করা যাবে। Architecture-এ Domain, Repository, Event Bus, Service ও UI আলাদা থাকবে। এটি বাস্তব Booking Engine নয়।

Folder Structure

```
source-code/chapter-21/
├── index.html
```

```
├── style.css
├── booking-core.mjs
├── repository.mjs
├── events.mjs
├── service.mjs
├── app.mjs
└── booking-core.test.mjs
```

গুরুত্বপূর্ণ File Responsibilities

`booking-core.mjs`: `validateRequest()`, `createBooking()`, `transitionBooking()`, `summarize()`। User Input থেকে Price গ্রহণ না করে Catalog থেকে Price হিসাব করা হয়।

`repository.mjs`: `createMemoryRepository()` এবং `createBrowserRepository()`। দুটিই `list/get/save` Interface অনুসরণ করে।

`events.mjs`: `createEventBus()` এবং Unsubscribe Function।

`service.mjs`: `createBookingService()`—Booking Creation ও State Change-এর Orchestration।

`app.mjs`: DOM Rendering, Form Submission, Event Listener ও UI Feedback। User-provided Text প্রদর্শনে `textContent` ব্যবহার করা হয়েছে।

সম্পূর্ণ Runnable Source Code ও Tests Companion Source Code-এর `source-code/chapter-21/` Folder-এ রয়েছে। অধ্যায়ের এই ব্যাখ্যার সঙ্গে ওই Files পড়বে।

Run Instructions

```
# Project folder থেকে:
python -m http.server 8000
# Browser: http://localhost:8000

# Node.js installed থাকলে:
node --test booking-core.test.mjs
```

ES Modules Browser-এ সাধারণত `file://` দিয়ে চালানোর বদলে Local HTTP Server ব্যবহার করবে।

Data Flow

1. User Form Submit করে।
2. UI → Service-এ Request পাঠায়।
3. Service → Domain Validation ও Booking Creation করে।
4. Service → Repository-তে Save করে।
5. Service → Event Bus-এ `booking:created` Emit করে।
6. UI Listener → নতুন Data Load করে Table Refresh করে।

Important Limitations

এই Browser Demo-র `localStorage` Trusted Database নয়। User Data পরিবর্তন করতে পারে। Multi-user Access, Authentication, Authorization, Real Booking Confirmation, Inventory Locking, Payment, Audit Trail

এবং Server-side Validation এখানে নেই। Production Implementation-এ Backend API ও Database Transaction প্রয়োজন।

২১.১২ Testing Strategy

এই Project-এ Unit ও Integration Tests দুটোই আছে। Unit Test-এ Price Calculation, Invalid Input ও State Transition যাচাই করা হয়। Integration Test-এ Service, Repository এবং Event Bus একসঙ্গে কাজ করছে কি না দেখা হয়।

```
import test from 'node:test';
import assert from 'node:assert/strict';
import { validateRequest } from './booking-core.mjs';

test('price comes from catalog', () => {
  const result = validateRequest({
    customer: 'Amina Khan', packageId: 'cox', travelers: 2, total: 1
  });
  assert.equal(result.total, 13000);
});
```

Browser UI-এর Keyboard Navigation, Accessibility, LocalStorage Error Handling এবং Manual End-to-End Testing আলাদা Review-এর অংশ।

২১.১৩ Common Architecture Mistakes

- **God Object:** সব Business Rule ও UI Logic একটি Module-এ রাখা।
- **Overengineering:** ছোট Application-এ অপ্রয়োজনীয় ১০টি Pattern যোগ করা।
- **Hidden Dependency:** Service-এর মধ্যে সরাসরি Global `localStorage` ব্যবহার করা।
- **Mutable Shared State:** Reference Copy-এর কারণে অনিচ্ছাকৃত পরিবর্তন।
- **Missing Error Boundary:** Storage Failure-এর পরে Success Message দেখানো।
- **Trusting the Client:** Browser-এর Price/Status-কে Authoritative ধরে নেওয়া।
- **Unmanaged Listeners:** Event Subscription পরিষ্কার না করা।

২১.১৪ অনুশীলন — ১০টি

১. SRP অনুযায়ী একটি Monolithic Form Handler-কে তিনটি Function-এ ভাগ করো।

২. `createNotifier('console')` ও `createNotifier('silent')` Factory তৈরি করো।

৩. Student Discount-এর জন্য নতুন Strategy যোগ করো।

৪. Event Bus-এর `on()` Function থেকে Unsubscribe Return করো।

৫. Repository-তে `remove(id)` Method যোগ করো।

৬. `confirmed → pending` Transition কেন Invalid, ব্যাখ্যা করো।

৭. Booking Service-এ `repository` Mock করে Unit Test লেখো।

৮. `localStorage`-এ Invalid JSON থাকলে App কী করবে, পরিকল্পনা করো।

৯. Domain Layer থেকে DOM Dependency সরানোর উদাহরণ দাও।

১০. TravelFlow-এ `booking:updated` Event দিয়ে Audit Logger যুক্ত করো।

Sample Solutions

১. `readForm()` Input পড়ে; `validateRequest()` Business Rule যাচাই করে; `renderResult()` UI Update করে।

২.

```
const createNotifier = type => type === 'console'
  ? { send: message => console.log(message) }
  : { send: () => {} };
```

৩.

```
strategies.student = total => total * 0.92;
```

৪.

```
const off = bus.on('saved', console.log);
off(); // Listener removed
```

৫.

```
async remove(id) { return records.delete(id); }
```

৬. Confirmed Booking-কে Pending করলে Workflow History অসংগত হতে পারে। নতুন Business Process ছাড়া Backward Transition নিষিদ্ধ রাখা সহজ ও নিরাপদ।

৭.

```
const saved = [];
const repository = { save: async x => saved.push(x) };
```

৮. JSON Parse `try/catch`-এ রেখে Safe Fallback দেখাতে হবে; Corrupt Data-কে চুপচাপ মুছে না দিয়ে Recovery/Export Strategy রাখা ভালো।

৯. `document.querySelector()`-এর বদলে Domain Function-এ Plain Object Input দাও।

১০.

```
const unsubscribe = events.on('booking:updated', booking => {
  console.info('Updated booking', booking.id, booking.status);
});
```

২১.১৫ MCQ — ১০টি

১. Repository Pattern মূলত কী Abstract করে? A) CSS B) Data Access C) Typography D) HTML

২. Observer Pattern-এর উদ্দেশ্য কী? A) Object creation B) Event notification C) Database migration D) Password hashing

৩. Dependency Injection-এর সুবিধা কী? A) Testing easier B) No errors ever C) Browser faster always D) No security needed

৪. Domain Layer-এ সাধারণত কোনটি থাকা উচিত? A) DOM Event Listener B) Business Rules C) CSS Grid D) Browser History

৫. Factory Pattern কী করে? A) Creates configured objects B) Compresses images C) Encrypts tokens D) Renders tables

৬. `cancelled → confirmed` এই Project-এ কী? A) Valid B) Invalid C) Always automatic D) CSS state

৭. `textContent` ব্যবহারের একটি কারণ কী? A) Safe plain-text rendering B) Faster payment C) Password hashing D) DB indexing

৮. Browser `localStorage` কি Trusted Server Database? A) Yes B) No C) Only Chrome D) Only HTTPS

৯. Strategy Pattern কখন উপযোগী? A) Swappable algorithms B) One fixed color C) One paragraph D) Static HTML only

১০. Service Layer সাধারণত কী করে? A) Orchestrates use cases B) Sets font family C) Manages DNS D) Paints pixels

MCQ Answers ও ব্যাখ্যা

1. **B** — Data Persistence-এর Implementation আড়াল করে।
2. **B** — Event ঘটলে Subscriber-দের জানায়।
3. **A** — Fake/Mock Dependency দিয়ে Test সহজ হয়।
4. **B** — Business Validation ও State Rule Domain-এর দায়িত্ব।
5. **A** — Object Creation-এর Logic Encapsulate করে।
6. **B** — State Transition Rules অনুমতি দেয় না।
7. **A** — Plain Text HTML হিসেবে Parse হয় না।
8. **B** — Browser Storage User পরিবর্তন করতে পারে।
9. **A** — Algorithm পরিবর্তনযোগ্য হলে Strategy উপকারী।
10. **A** — Use Case-এ বিভিন্ন Layer সমন্বয় করে।

২১.১৬ Chapter Recap ও পরবর্তী অধ্যায়

Design Patterns কোনো Magic Solution নয়; সঠিক Responsibility, Testable Boundaries এবং Explicit Business Rules-এর সঙ্গে ব্যবহার করলেই উপকার পাওয়া যায়। TravelFlow Project-এ Factory, Observer, Repository, Dependency Injection এবং Layered Architecture একসঙ্গে প্রয়োগ করেছি। পরবর্তী অধ্যায়

JavaScript Testing Mastery-তে Unit, Integration, Mocking, TDD ও End-to-End Testing আরও গভীরভাবে শিখব।

অধ্যায় ২২ — JavaScript Testing Mastery — Unit Testing, Integration Testing, Mocking, TDD এবং E2E

Hands-on Project: Booking Quality Lab — Testable Booking Workflow

শেখার লক্ষ্য

এই অধ্যায় শেষে তুমি (১) Automated Testing-এর উদ্দেশ্য ব্যাখ্যা করতে পারবে, (২) Node.js Test Runner দিয়ে Unit Test লিখবে, (৩) Mock ও Dependency Injection ব্যবহার করবে, (৪) Integration Test এবং E2E Test-এর পার্থক্য বুঝবে, (৫) TDD-এর Red–Green–Refactor Cycle অনুসরণ করবে, (৬) Test Coverage-এর সীমাবদ্ধতা ব্যাখ্যা করবে এবং (৭) Booking Quality Lab-এর Core Logic যাচাই করবে।

২২.১ কেন Testing জরুরি?

একটি Booking Application-এ Customer Name, Package, Traveler Count, Total Price এবং Status পরিবর্তনের নিয়ম আছে। একটি ছোট পরিবর্তন—যেমন `guests`-এর Limit বদলানো—অন্য Function-এ ভুল ফল দিতে পারে। **Automated Test** হলো এমন Executable Specification যা নির্দিষ্ট Input-এর বিপরীতে প্রত্যাশিত Output বা Behavior যাচাই করে।

Manual Testing প্রয়োজন, কিন্তু একই Scenario বারবার পরীক্ষা করা সময়সাপেক্ষ। Automated Test দ্রুত Regression শনাক্ত করে; তবে Test Pass মানেই Production System সম্পূর্ণ Bug-free নয়। Test কেবল যেসব Behavior পরীক্ষা করেছে, সেগুলোর বিষয়ে Evidence দেয়।

২২.২ Testing-এর স্তর

Test Type	কী যাচাই করে	উদাহরণ	সাধারণ বৈশিষ্ট্য
Unit	ছোট Function বা Module	`calculateTotal()`	দ্রুত, Isolated
Integration	একাধিক Module-এর সহযোগিতা	Service + Repository	Boundary ও Data Flow যাচাই
Component	UI Component	Form Error Message	UI Interaction
E2E	Browser থেকে সম্পূর্ণ Flow	Place → Confirm	বাস্তবের কাছাকাছি, তুলনামূলক ধীর
Regression	আগের Bug ফিরে এসেছে কি না	Cancelled Booking Confirm আটকানো	Bug Fix-এর পরে যোগ হয়
Smoke	Critical Feature কাজ করছে কি না	App Load ও Primary Form	Deploy-এর পরে উপকারী

Test Pyramid একটি সাধারণ নির্দেশনা: দ্রুত Unit Test বেশি, Integration Test প্রয়োজনমতো এবং Critical User Journey-এর E2E Test সীমিত কিন্তু অর্থবহ। এটি কঠোর অনুপাত নয়।

২২.৩ Arrange–Act–Assert Pattern

একটি Test তিনটি ধাপে পরিষ্কারভাবে লেখা যায়: **Arrange** (Input ও Dependencies প্রস্তুত), **Act** (Function চালানো), **Assert** (ফল যাচাই)।

```
import test from 'node:test';
import assert from 'node:assert/strict';
import { calculateTotal } from './booking.mjs';

test('three hill travelers cost BDT 21,600', () => {
  // Arrange
  const packageId = 'hills';
  const guests = 3;
  // Act
  const total = calculateTotal(packageId, guests);
  // Assert
  assert.equal(total, 21600);
});
```

Test Name-এ Behavior উল্লেখ করো। `works fine`-এর চেয়ে `rejects fractional guest count` অনেক বেশি অর্থবহ।

২২.৪ Node.js Built-in Test Runner

Node.js 20+ ইনস্টল থাকলে অতিরিক্ত Framework ছাড়াই `node:test` ও `node:assert/strict` ব্যবহার করা যায়।

```
node --test booking.test.mjs
node --experimental-test-coverage --test booking.test.mjs
```

`test()` একটি Test Case সংজ্ঞায়িত করে। `assert.equal()` Strict Equality পরীক্ষা করে। `assert.deepEqual()` Nested Object/Array-এর Structure তুলনা করে। `assert.throws()` Synchronous Error, আর `assert.rejects()` Rejected Promise যাচাই করে।

```
import assert from 'node:assert/strict';
assert.equal(2 + 3, 5);
assert.deepEqual({ status: 'ok' }, { status: 'ok' });
assert.throws(() => { throw new Error('bad'); }, /bad/);
await assert.rejects(Promise.reject(new Error('offline')), /offline/);
```

২২.৫ Boundary Value এবং Equivalence Partitioning

Input Validation Test-এ কেবল Valid Case যথেষ্ট নয়। ধরো Guests-এর সীমা ১ থেকে ১০। তাহলে ০, ১, ১০, ১১, Fraction, `NaN` এবং Missing Value পরীক্ষা করা দরকার।

```
import { validateBooking } from './booking.mjs';
const base = { customer: 'Amina', packageId: 'beach', guests: 2 };

for (const guests of [0, 11, 1.5, NaN]) {
  assert.throws(() => validateBooking({ ...base, guests }));
```

```
}
```

এই Code-এর `assert` Import আগের উদাহরণের মতো থাকতে হবে। Equivalence Partitioning-এ একই আচরণের Input Group থেকে Representative Case বেছে নেওয়া হয়। Boundary Testing-এ Range-এর প্রান্তগুলো গুরুত্ব পায়।

২২.৬ Test Doubles: Stub, Fake, Spy ও Mock

Stub নির্ধারিত Return Value দেয়। **Fake** সরল কিন্তু কার্যকর Alternative Implementation, যেমন In-memory Repository। **Spy** কোনো Function কতবার ও কী Arguments-এ Call হয়েছে তা রেকর্ড করে। **Mock** নির্দিষ্ট Interaction Expectation যাচাই করে। Framework অনুযায়ী Terminology কিছুটা ভিন্ন হতে পারে।

```
const calls = [];
const notifier = {
  async send(type, booking) {
    calls.push({ type, id: booking.id });
  }
};
```

এখানে `calls` Array ব্যবহার করে Notification Interaction পরীক্ষা করা যায়। External Email Service না চালিয়েও Service-এর Behavior যাচাই করা সম্ভব।

২২.৭ Dependency Injection এবং Testability

Hard-coded Dependency থাকলে Test চালানোর সময় Real Database, Email বা Network প্রয়োজন হতে পারে। Dependency Injection ব্যবহার করলে Test-এর সময় Fake Implementation দেওয়া যায়।

```
const service = createBookingService({
  repository: createMemoryRepository(),
  notifier: { async send() {} },
  idFactory: () => 'BK-TEST-1'
});
```

এখানে `createBookingService` এবং `createMemoryRepository` Project-এর `booking.mjs` থেকে Import করতে হবে। Deterministic ID Factory Test Result স্থিতিশীল রাখে।

২২.৮ Integration Testing: একাধিক Module একসঙ্গে

Unit Test Function-এর Contract যাচাই করে; Integration Test পরীক্ষা করে Module-গুলো একসঙ্গে সঠিকভাবে কাজ করছে কি না।

```
const repository = createMemoryRepository();
const events = [];
const service = createBookingService({
  repository,
  notifier: { async send(type) { events.push(type); } },
  idFactory: () => 'BK-001'
});

const booking = await service.place({
  customer: 'Amina', packageId: 'beach', guests: 2
});
```

```
await service.confirm(booking.id);
assert.deepEqual(events, ['created', 'confirmed']);
assert.equal((await repository.find(booking.id)).status, 'confirmed');
```

এখানে Service → Repository → Notifier Flow যাচাই হচ্ছে। তবে Real Database-এর Transaction, Network Retry ও Authentication এই Test-এর আওতায় নেই।

২২.৯ Async Tests, Failure Paths ও Cleanup

Async Function Test করার সময় `await` না দিলে Test আগেই শেষ হয়ে যেতে পারে। Error Expect করলে `assert.rejects()` ব্যবহার করো। Test Isolation নিশ্চিত করতে প্রতিটি Test-এ নতুন Repository বানানো ভালো। Shared Mutable State Flaky Test-এর বড় কারণ।

```
await assert.rejects(
  service.confirm('unknown-id'),
  { code: 'NOT_FOUND' }
);
```

Timeout, Fake Timers, Network Stubbing ও Cleanup প্রয়োজন হতে পারে। Fake Timer ব্যবহার করলে `setTimeout`-নির্ভর Test বাস্তব সময় অপেক্ষা না করেও চালানো যায়।

২২.১০ TDD: Red → Green → Refactor

Red: আগে এমন Test লেখো যা নতুন Requirement না থাকায় Fail করে। **Green:** সবচেয়ে সরল Implementation দিয়ে Test Pass করাও। **Refactor:** Behavior না বদলে Code পরিষ্কার করো এবং Test আবার চালাও।

ধরা যাক, Business Rule: Cancelled Booking পুনরায় Confirm করা যাবে না।

```
// Red: আগে Test
assert.throws(
  () => transition({ status: 'cancelled' }, 'confirmed'),
  { code: 'STATUS' }
);
```

এরপর `transition()` Function-এ Allowed Transition Map যোগ করা হয়। Test Pass হলে Refactor করা যায়। TDD-এর উদ্দেশ্য শুধু Test Coverage নয়; পরিষ্কার Interface ও Design Feedback পাওয়া।

২২.১১ Test Coverage: কী বোঝায়, কী বোঝায় না

Line, Branch এবং Function Coverage একটি Codebase-এর কোন অংশ Test Execution-এ এসেছে তা দেখায়। কিন্তু ১০০% Coverage থাকলেও Assertion দুর্বল হলে ভুল Behavior ধরা নাও পড়তে পারে।

```
node --experimental-test-coverage --test booking.test.mjs
```

Coverage Report দেখে Untested Branch শনাক্ত করো। **Meaningful Assertion**, Boundary Case এবং Failure Path Coverage-এর সংখ্যার চেয়ে বেশি গুরুত্বপূর্ণ।

২২.১২ Browser E2E Testing-এর ধারণা

Playwright-এর মতো Tool দিয়ে বাস্তব Browser খুলে Form Fill, Click ও Result Verify করা যায়। নিচেরটি **illustrative example**; Companion Source Code-এ Playwright Dependency Install করা হয়নি, তাই এই E2E Test চালানো হয়েছে এমন দাবি করছি না।

```
// Example only: requires @playwright/test and browser installation.
import { test, expect } from '@playwright/test';

test('visitor places a demo booking', async ({ page }) => {
  await page.goto('http://localhost:8000');
  await page.locator('#customer').fill('Amina Rahman');
  await page.locator('#guests').fill('2');
  await page.getByRole('button', { name: 'Place demo booking' }).click();
  await expect(page.locator('#bookings li')).toHaveCount(1);
});
```

E2E Test-এর জন্য Test Server চালু থাকা, Browser Installation এবং Test Data Isolation নিশ্চিত করতে হবে। Screenshot, Trace ও Video Failure Debugging-এ সাহায্য করে।

২২.১৩ CI Pipeline-এ Tests

Pull Request-এ Automatic Test চালালে Regression Merge হওয়ার আগেই ধরা পড়তে পারে। একটি Minimal GitHub Actions Example:

```
name: Node Tests
on: [push, pull_request]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: '22'
      - run: node --test source-code/chapter-22/booking.test.mjs
```

এটি Repository Root থেকে File Path সঠিক থাকলে চলবে। Production CI-তে Lockfile, Dependency Audit, Lint, Build ও E2E Steps যোগ করা উচিত।

২২.১৪ Practical Project — Booking Quality Lab

এই Project-এ তিনটি স্তর আছে: **Domain Validation**, **Application Service**, এবং **Browser Demo**। সব Source File Companion Source Code-এর `source-code/chapter-22/` Folder-এ রয়েছে।

```
source-code/chapter-22/
├── booking.mjs           # Domain + Service + Memory Repository
├── booking.test.mjs      # Automated Unit/Integration Tests
├── app.mjs               # Browser Interaction
├── index.html            # Form and Booking List
├── style.css             # UI Styling
├── package.json          # npm test Scripts
└── RUN.md                # Execution Guide
```

Project Flow

1. User Customer, Package ও Guests দেয়।
2. `validateBooking()` Input Validate করে এবং Catalog থেকে Total হিসাব করে।
3. `createBookingService()` নতুন Booking তৈরি করে Repository-তে Save করে।
4. Notifier একটি Educational Event Log করে।
5. User Confirm করলে `transition()` State Rule প্রয়োগ করে।
6. Automated Tests Validation, State Transition, Repository Isolation ও Service Integration যাচাই করে।

Run Instructions

```
cd source-code/chapter-22
node --test booking.test.mjs
node --experimental-test-coverage --test booking.test.mjs
python -m http.server 8000
```

শেষ Command-এর পর Browser-এ `http://localhost:8000` খুলবে। Browser Demo-তে In-memory Data আছে, Refresh করলে হারিয়ে যাবে। এটি ইচ্ছাকৃতভাবে ছোট Educational Application।

গুরুত্বপূর্ণ Reliability Caveat

Demo-তে Repository Save হওয়ার পরে Notifier Fail করলে `place()` Reject করে, কিন্তু Booking ইতোমধ্যে Save হয়ে থাকতে পারে। এটি **Partial Failure**-এর বাস্তব উদাহরণ। Production System-এ Transactional Outbox, Retry Policy এবং Idempotency বিবেচনা করতে হবে। এই অধ্যায়ের Test ইচ্ছাকৃতভাবে Failure Visibility যাচাই করে; Atomicity দাবি করে না।

২২.১৫ সাধারণ Testing Mistakes

- শুধু Happy Path পরীক্ষা করা; Invalid Input ও Boundary বাদ দেওয়া।
- Implementation Detail-এ অতিরিক্ত Assert করে Refactoring কঠিন করা।
- Real API/Clock/Randomness-এর ওপর নির্ভর করে Flaky Test তৈরি করা।
- Async Test-এ `await` বাদ দেওয়া।
- Test Case-গুলোর মধ্যে Shared State রেখে Order-dependent Result পাওয়া।
- Coverage Percentage-কে Quality-এর একমাত্র সূচক ভাবা।
- Mock দিয়ে সবকিছু Pass করিয়ে Real Integration Failure উপেক্ষা করা।
- Browser E2E Test না চালিয়েও UI Verified বলা।

২২.১৬ Exercises

১. `calculateTotal('city', 4)` কত হবে? Test লেখো।

২. Blank Customer Reject হওয়ার Test লেখো।

৩. Guests ০ এবং ১১ হলে Error হয় কি না যাচাই করো।

৪. `pending → cancelled` Transition-এর Test লেখো।

৫. `cancelled → confirmed` Reject হওয়ার Test লেখো।

৬. Notifier Spy দিয়ে `created` Event Count যাচাই করো।

৭. Repository থেকে পাওয়া Object পরিবর্তন করলে Stored Object বদলায় কি না পরীক্ষা করো।

৮. `confirm()`-এ Missing ID-এর Error যাচাই করো।

৯. Notifier Fail করলে `place()`-এর Result কী হয়, Test করো।

১০. E2E Test Plan লেখো: Invalid Form → Error → Valid Form → Booking Confirm।

২২.১৭ Sample Solutions

```
// 1
assert.equal(calculateTotal('city', 4), 15600);
// 2
assert.throws(() => validateBooking({customer:' ',packageId:'city',guests:2}),
{code:'CUSTOMER'});
// 3
for (const guests of [0, 11]) {
  assert.throws(() => validateBooking({customer:'Amina',packageId:'city',guests}),
{code:'GUESTS'});
}
// 4
assert.equal(transition({status:'pending'},'cancelled').status,'cancelled');
// 5
assert.throws(() => transition({status:'cancelled'},'confirmed'), {code:'STATUS'});
// 6
const events=[]; const notifier={async send(type){events.push(type)}};
// place() করার পরে assert.deepEqual(events,['created']);
// 7
// Repository-এর find() ফল পরিবর্তন করে আবার find() করলে আগের Value থাকবে।
// 8
// await assert.rejects(service.confirm('missing'), {code:'NOT_FOUND'});
// 9
// await assert.rejects(service.place(validInput), /mail offline/);
// 10
// Browser Test Plan: blank customer rejected; valid submit adds one item;
// clicking Confirm changes status from pending to confirmed.
```

এই Sample Snippet-এ `assert`, `calculateTotal`, `validateBooking`, `transition` ইত্যাদি পূর্বের উদাহরণ অনুসারে Import করতে হবে। সম্পূর্ণ Runnable Tests `booking.test.mjs`-এ দেওয়া আছে।

২২.১৮ Knowledge Check — ১০টি MCQ

১. Unit Test-এর প্রধান লক্ষ্য কী? A) Small unit behavior B) Hosting C) SEO D) DNS

২. `assert.rejects()` কোন ক্ষেত্রে? A) CSS B) Rejected Promise C) HTML Tag D) SQL Index

৩. Fake Repository-এর সুবিধা কী? A) Fast isolated tests B) Real payment C) HTTPS D) CDN

৪. TDD-এর সঠিক ক্রম কোনটি? A) Refactor–Green–Red B) Red–Green–Refactor C) Green–Red–Refactor D) Deploy–Build–Test

৫. Boundary Testing-এ কোন Input গুরুত্বপূর্ণ? A) Limit-এর প্রান্ত B) শুধু মাঝের Value C) শুধু Valid Email D) Random CSS

৬. 100% Line Coverage মানেই কি Bug-free? A) Yes B) No C) Always D) Only Node.js

৭. Integration Test কী যাচাই করে? A) Module collaboration B) Font Weight C) Icon Size D) Page Color

৮. Flaky Test-এর একটি কারণ কী? A) Shared State B) Pure Function C) Fixed Input D) Isolated Test

৯. E2E Test সাধারণত কী ব্যবহার করে? A) Real browser journey B) Database schema only C) Color palette D) Code comments

১০. Notifier Failure-এর পরে Booking Save হয়ে থাকলে কী বোঝায়? A) Partial failure B) Atomic success C) Browser bug only D) Syntax error

Answers ও ব্যাখ্যা

1. **A** — ছোট Function/Module-এর Behavior যাচাই।
2. **B** — Promise Rejection যাচাই।
3. **A** — External Database ছাড়া দ্রুত Test চালানো যায়।
4. **B** — প্রথমে Failing Test, তারপর Implementation, শেষে Refactor।
5. **A** — Boundary Value-তে Error বেশি ধরা পড়ে।
6. **B** — Coverage Execution দেখায়, Correctness নিশ্চিত করে না।
7. **A** — Module/Service-এর Interaction যাচাই।
8. **A** — Test Order ও Timing-এর ওপর নির্ভরতা তৈরি হয়।
9. **A** — Browser-এ User Journey যাচাই।
10. **A** — Save সফল হলেও পরের Step Fail করেছে; Transaction Design দরকার।

২২.১৯ Chapter Recap ও পরবর্তী অধ্যায়

Automated Testing হলো Software Quality-এর গুরুত্বপূর্ণ Feedback System। Unit Test Business Rule যাচাই করে, Integration Test Module Collaboration পরীক্ষা করে, E2E Test Critical User Journey কভার করে। Mock/Fake/Spy Dependency আলাদা করে Test চালাতে সাহায্য করে। TDD Interface Design উন্নত করতে পারে। পরবর্তী অধ্যায় **JavaScript Performance Optimization, Profiling এবং Web Application Performance**-এ বাস্তব Measurement, Memory ও Rendering Performance নিয়ে কাজ করব।

অধ্যায় ২৩ — JavaScript Performance Optimization, Memory Management, Profiling এবং Web Application Performance

Hands-on Project: TravelSearch Explorer — ১২,০০০ রেকর্ডের Search ও Virtual Scrolling

শেখার লক্ষ্য

এই অধ্যায় শেষে তুমি (১) অনুমানের বদলে Performance Measure করতে, (২) Time Complexity ও বাস্তব Benchmark-এর পার্থক্য বুঝতে, (৩) Chrome/Edge DevTools-এর Performance ও Memory Panels ব্যবহার করতে, (৪) Rendering ও Long Task-এর সমস্যা শনাক্ত করতে, (৫) Debounce, Throttle, Caching এবং Virtualization প্রয়োগ করতে, (৬) Memory Leak এড়াতে এবং (৭) Performance Regression-এর Test Plan লিখতে পারবে।

২৩.১ Performance Optimization মানে কী?

দ্রুত কোড লেখা আর দ্রুত চলা কোড এক নয়। কোনো Algorithm-এর CPU Time কম হলেও যদি ১০,০০০টি DOM Element একসঙ্গে তৈরি করে, Browser Layout ও Paint ধীর হতে পারে। আবার অত্যধিক Cache Memory বাড়িয়ে দিতে পারে। তাই Performance বিচার করতে হয় **User Experience, CPU, Memory, Network এবং Rendering**—সবকিছু মিলিয়ে।

Optimization-এর প্রথম নিয়ম: **Measure → Identify Bottleneck → Change One Thing → Measure Again**। পরিমাপ ছাড়া "এটা Faster" বলা অনুমান।

গুরুত্বপূর্ণ Metric

Metric	কী বোঝায়	কোথায় মাপবে
Interaction to Next Paint (INP)	User Interaction-এর প্রতিক্রিয়া	Field data / Web Vitals
Largest Contentful Paint (LCP)	বড় Content দৃশ্যমান হতে সময়	Lighthouse / Field data
Cumulative Layout Shift (CLS)	Layout-এর অপ্রত্যাশিত নড়াচড়া	Lighthouse / Field data
Long Task	Main Thread ৫০ ms-এর বেশি ব্যস্ত	DevTools Performance
JS Heap Size	JavaScript Object Memory	DevTools Memory
Filter/Sort Duration	Business Operation কত সময় নেয়	`performance.now()`

Lighthouse-এর Lab Result ও Real User Field Data এক নয়। একটি Fast Desktop-এর Benchmark সব Mobile Device-এর অভিজ্ঞতা বোঝায় না।

২৩.২ Complexity বনাম বাস্তব সময়

```
const items = Array.from({ length: 10000 }, (_, i) => i);
const found = items.find(x => x === 9999); // worst case O(n)
const doubled = items.map(x => x * 2); // O(n)
const pairs = [];
for (let i = 0; i < 100; i++) {
  for (let j = 0; j < 100; j++) pairs.push([i, j]); // O(n²)
}
```

Big-O Input বাড়লে Growth Rate কেমন হয়, তা বোঝায়; Millisecond নির্ধারণ করে না। একই O(n) Operation ভিন্ন Hardware, Engine ও Data Shape-এ ভিন্ন সময় নিতে পারে।

`Map` ও `Set` কখন কাজে লাগে?

```
const byId = new Map(products.map(p => [p.id, p]));
const product = byId.get(42); // average-case efficient lookup
const uniqueTags = [...new Set(tags)];
```

একবার Index তৈরির খরচ আছে। মাত্র পাঁচটি Item-এর জন্য `Map` বানানো অপ্রয়োজনীয় হতে পারে; হাজারবার Lookup হলে লাভ হতে পারে।

২৩.৩ Performance Measurement

```
const start = performance.now();
const result = expensiveOperation();
const duration = performance.now() - start;
console.log(`Operation took ${duration.toFixed(2)} ms`);
```

একবার মাপলে JIT Warm-up, Garbage Collection ও Background Task-এর প্রভাব পড়তে পারে। তাই একই Input-এ বহুবার চালিয়ে Median, p95, Worst Case বিবেচনা করবে। Development Build বনাম Production Build আলাদা করে পরীক্ষা করবে।

```
function benchmark(fn, iterations = 30) {
  for (let i = 0; i < 5; i++) fn(); // warm-up
  const samples = [];
  for (let i = 0; i < iterations; i++) {
    const start = performance.now();
    fn();
    samples.push(performance.now() - start);
  }
  samples.sort((a, b) => a - b);
  return { median: samples[Math.floor(samples.length / 2)], max: samples.at(-1) };
}
```

এটি শেখার জন্য Simple Benchmark; Professional Microbenchmark-এর বিকল্প নয়।

২৩.৪ Browser Rendering Pipeline

Browser সাধারণত JavaScript, Style Calculation, Layout, Paint ও Compositing-এর কাজ করে। বারবার DOM Read/Write মিশিয়ে করলে Forced Synchronous Layout হতে পারে।

```
// Avoid repeated read-write loops
const widths = elements.map(el => el.getBoundingClientRect().width);
```

```
elements.forEach((el, i) => { el.style.width = `${widths[i] + 10}px`; });
```

`DocumentFragment`, DOM Batch Updates, CSS `transform` এবং `requestAnimationFrame()` সঠিক জায়গায় ব্যবহার করলে Rendering Work কমতে পারে। তবে `requestAnimationFrame` CPU-heavy কাজকে স্বয়ংক্রিয়ভাবে দ্রুত করে না।

২৩.৫ Debounce, Throttle ও Input Responsiveness

Search Input-এর প্রতিটি Keystroke-এ বড় Filter চালানোর বদলে Debounce করা যায়।

```
function debounce(fn, wait = 200) {
  let timer;
  return (...args) => {
    clearTimeout(timer);
    timer = setTimeout(() => fn(...args), wait);
  };
}
const onSearch = debounce(query => console.log('Search:', query), 200);
```

Debounce: শেষ Input-এর কিছু সময় পরে Run। Throttle: নির্দিষ্ট সময়ের মধ্যে Run Frequency সীমিত। Scroll Rendering-এর জন্য `requestAnimationFrame`-এ Schedule করা উপযোগী হতে পারে।

২৩.৬ Virtual Scrolling কেন?

১২,০০০টি Result থাকলে একসঙ্গে ১২,০০০টি Row DOM-এ রাখার বদলে দৃশ্যমান অংশ এবং কিছু Extra Row Render করা যায়। Fixed Height ৫৬ px, Viewport ৪৮০ px হলে সাধারণত কয়েক ডজন Row যথেষ্ট।

```
const rowHeight = 56;
const overscan = 6;
const start = Math.max(0, Math.floor(scrollTop / rowHeight) - overscan);
const end = Math.min(total, Math.ceil((scrollTop + viewportHeight) / rowHeight) + overscan);
```

এখানে Spacer Full Scroll Height বজায় রাখে, আর Visible Rows `translateY` দিয়ে সঠিক অবস্থানে বসে। Variable Height, Focus Retention ও Screen Reader Accessibility-এর জন্য আরও উন্নত Design প্রয়োজন।

২৩.৭ Memory Management ও Garbage Collection

JavaScript Engine Reachable Object-কে ধরে রাখে। Unreachable Object Garbage Collection-এর যোগ্য হয়। Memory Leak সাধারণত এমন Reference থেকে আসে যা আর দরকার নেই, কিন্তু এখনো Reachable।

সাধারণ কারণ: Global Array-এ অনবরত Data জমা, Detached DOM Node ধরে রাখা, Cleanup ছাড়া Event Listener, `setInterval()` বন্ধ না করা, সীমাহীন Cache এবং Closure-এ বড় Data আটকে রাখা।

```
function startTicker(element) {
  const id = setInterval(() => { element.textContent = new Date().toLocaleTimeString(); },
1000);
  return () => clearInterval(id); // cleanup when view is destroyed
}
const stop = startTicker(document.querySelector('#clock'));
// Component unmount / page teardown: stop();
```

`WeakMap` নির্দিষ্ট Object-keyed Metadata-র জন্য কাজে লাগে, কিন্তু সব Memory Leak সমাধান করে না। Memory Panel-এর Heap Snapshot Compare করে Retained Object খুঁজতে হবে।

২৩.৮ Web Workers, Lazy Loading ও Network

Main Thread-এ ভারী Calculation UI Block করতে পারে। Web Worker আলাদা Thread-এ CPU-heavy Work করতে পারে, তবে Worker DOM Access করতে পারে না এবং Message Transfer-এর খরচ আছে।

```
const worker = new Worker('./worker.js', { type: 'module' });
worker.postMessage({ type: 'ANALYZE', records });
worker.addEventListener('message', event => console.log(event.data));
```

Network Optimization-এর জন্য Code Splitting (`import()`), Asset Compression, Browser Caching, Image Size ও Lazy Loading বিবেচনা করবে। অকারণে Lazy Loading করলে Critical Content দেরিতে দেখা দিতে পারে।

২৩.৯ Practical Project — TravelSearch Explorer

Project Folder: `source-code/chapter-23/`

```
chapter-23/
├── index.html
├── style.css
├── core.mjs
├── app.mjs
├── core.test.mjs
└── RUN.md
```

Features

- ১২,০০০টি Deterministic Fictional Travel Records
- Destination Search ও Filter
- Price/Rating/Title Sorting
- ১৮০ ms Debounced Search
- Fixed-height Virtualized List
- `performance.now()` দিয়ে Local Filter/Sort Timing
- Unit Tests for Filtering, Sorting, Window Range ও Memoization

Run

```
cd source-code/chapter-23
node --test core.test.mjs
python -m http.server 8000
```

তারপর Browser-এ `http://localhost:8000` খুলবে। `file://` দিয়ে ES Modules চালালে Browser Security Restriction হতে পারে। `core.mjs`-এ Business Logic, `app.mjs`-এ Browser UI আলাদা করা হয়েছে।

গুরুত্বপূর্ণ Design Trade-off

এই Demo Fixed-height Virtualization শেখায়। এটি Production-ready Accessible Data Grid নয়। Search-এর পরে Scroll Reset করা হয়; Real Application-এ Keyboard Navigation, Focus Management, Empty State, Variable Height, Loading States এবং Screen Reader Behavior আলাদাভাবে Test করতে হবে।

২৩.১০ Performance Profiling Lab

১. Browser DevTools → Performance খুলে Search Input-এ দ্রুত টাইপ করে Record করো।

২. Main Thread Flame Chart-এ Long Task আছে কি না দেখো।

৩. Search Debounce বন্ধ করে আগের Measurement-এর সঙ্গে তুলনা করো।

৪. Virtualization বাদ দিয়ে ১২,০০০ Row Render করলে DOM Node Count ও Responsiveness কীভাবে বদলায়, পরীক্ষা করো।

৫. Memory → Heap Snapshot নিয়ে Search-এর আগে-পরে Retained Object তুলনা করো।

৬. Device CPU Throttling ব্যবহার করে Low-end Device-এর কাছাকাছি পরিস্থিতি তৈরি করো।

Benchmark Report Template: Device, Browser Version, Dataset Size, Scenario, Median, p95, Memory, Observations, Change, Regression Risk। এই বই কোনো Universal Speed-up Percentage দাবি করছে না।

২৩.১১ Common Mistakes

ভুল	কেন সমস্যা	করণীয়
শুধু একবার Benchmark	Noise ও JIT প্রভাব	Multiple Samples
Sort-এ Original Array Mutate	Unexpected State Change	`[...items].sort()`
Unlimited Memoization Cache	Memory Growth	Bounded Cache / Eviction
`innerHTML`-এ User Input	XSS Risk	`textContent`
১০,০০০ DOM Row একসঙ্গে	Layout/Paint Cost	Virtualization / Pagination
Cleanup ছাড়া Listener	Memory Retention	Remove Listener / AbortSignal
শুধু Desktop-এ Test	Mobile Performance অজানা	Real Device / Throttling

২৩.১২ Exercises — ১০টি

১. ৫,০০০ Item-এর `find()` ও `Map.get()`-এর Performance Benchmark করো।

২. `normalizeQuery()`-তে Whitespace Trim ও Case-insensitive Search Test করো।

৩. `filterTrips()`-এ Price Limit যুক্ত করো।

৪. `sortTrips()` যেন Input Mutate না করে, Test লেখো।

৫. `windowRange()`-এ Scroll Position ০ হলে Correct Range নির্ণয় করো।

৬. List-এর শেষের দিকে Overscan যেন `total` অতিক্রম না করে, Test লেখো।

৭. Search Input-এ Debounce ব্যবহার করো।

৮. Event Listener Cleanup-এর একটি উদাহরণ লেখো।

৯. DevTools Performance Profile থেকে Long Task শনাক্ত করে নোট লেখো।

১০. Virtualized বনাম Full Rendering-এর তুলনামূলক Report তৈরি করো।

২৩.১৩ Sample Solutions

```
// 2
normalizeQuery('  DHAKA ') === 'dhaka';
// 3
filterTrips(makeTrips(100), { maxPrice: 4000 });
// 4
const input = makeTrips(10);
const before = input.map(x => x.id);
sortTrips(input, 'price-desc');
console.assert(JSON.stringify(before) === JSON.stringify(input.map(x => x.id)));
// 5
windowRange({ scrollTop: 0, rowHeight: 50, viewportHeight: 200, total: 100, overscan: 2 });
// 6
windowRange({ scrollTop: 4950, rowHeight: 50, viewportHeight: 200, total: 100, overscan: 2
}).end === 100;
// 8
const controller = new AbortController();
window.addEventListener('resize', () => console.log('resized'), { signal: controller.signal });
controller.abort(); // remove registered listener
```

Exercise 1, 7, 9 ও 10-এর ফল Device/Browser অনুযায়ী ভিন্ন হবে; এগুলো Hands-on Lab। `core.test.mjs`-এ Exercises 2–6-এর Runnable Verification আছে।

২৩.১৪ Knowledge Check — ১০টি MCQ

১. Optimization-এর প্রথম ধাপ কী? A) Measure B) Rewrite everything C) Add cache D) Remove tests

২. `performance.now()` কী মাপে? A) High-resolution elapsed time B) Network speed only C) Heap only D) CPU cores

৩. `sort()` ডিফল্টভাবে কী করে? A) Mutates array B) Always immutable C) Deletes values D) Caches results

৪. Debounce-এর কাজ কী? A) Wait after latest input B) Always run every ms C) Increase memory D) Block UI

৫. Virtualization কেন? A) Fewer mounted rows B) More DOM nodes C) More CSS D) Remove sorting

৬. Long Task-এর সাধারণ Threshold কত? A) 50 ms B) 1 ms C) 5 sec D) 10 sec

৭. Memory Leak-এর উদাহরণ কোনটি? A) Forgotten interval B) Pure function C) Local const D) Static text

৮. Web Worker কি সরাসরি DOM Modify করতে পারে? A) Yes B) No C) Only CSS D) Only images

৯. p95 কী নির্দেশ করে? A) 95th percentile B) Average only C) Memory limit D) Request count

১০. Lab Benchmark কেন যথেষ্ট নয়? A) Real devices differ B) JS is always same speed C) Cache never matters D) Browser never changes

Answers ও Explanations

1. **A** — Measurement ছাড়া Bottleneck বোঝা যায় না।
2. **A** — High-resolution Time Difference পাওয়া যায়।

3. **A** — `sort()` Original Array বদলায়; Copy করে Sort করা নিরাপদ।
4. **A** — শেষ Event-এর পরে Delay দিয়ে Work চালায়।
5. **A** — শুধু দৃশ্যমান Row Render করা হয়।
6. **A** — Main Thread-এর ৫০ ms-এর বেশি Task Long Task হিসেবে চিহ্নিত হয়।
7. **A** — Cleanup ছাড়া Timer Reference ধরে রাখতে পারে।
8. **B** — Worker DOM Access করতে পারে না।
9. **A** — Samples-এর ৯৫ শতাংশ ওই Value বা তার নিচে।
10. **A** — Hardware, Browser ও Network ভেদে ফল বদলায়।

২৩.১৫ Chapter Recap ও Next Chapter

Performance Optimization হলো **Measured Engineering**। Algorithm Complexity, DOM Rendering, Event Scheduling, Memory Lifetime ও Network—সবকিছু একসঙ্গে বিবেচনা করতে হয়। TravelSearch Explorer-এর মাধ্যমে ১২,০০০ Record-এ Filter/Sort, Debounced Input ও Virtual Scrolling প্রয়োগ করা হয়েছে। পরবর্তী অধ্যায় **TypeScript Integration, Static Typing এবং Professional Development Workflow**।

অধ্যায় ২৪ — JavaScript থেকে TypeScript — Static Typing, Type Safety এবং Professional Development Workflow

শেখার লক্ষ্য

এই অধ্যায় শেষে পাঠক TypeScript-এর প্রয়োজনীয়তা ব্যাখ্যা করতে, `strict` Configuration ব্যবহার করতে, `unknown` Data Validate করতে, Discriminated Union দিয়ে Success/Error Model করতে, Type-safe Service Layer লিখতে এবং Automated Tests চালাতে পারবেন। Practical Project: **TypeSafe Booking API**।

২৪.১ JavaScript-এর Dynamic Typing থেকে TypeScript

JavaScript Runtime-এ একটি Variable-এর Value পরিবর্তনের সঙ্গে তার Type-ও বদলাতে পারে। এই নমনীয়তা দ্রুত Prototyping-এ সুবিধা দেয়; কিন্তু বড় Codebase-এ ভুল Property Name, ভুল Argument Type, Nullable Data ও API Contract-এর অসামঞ্জস্য ধরতে বেশি সময় লাগে। TypeScript JavaScript-এর ওপর Static Type Checking যোগ করে। TypeScript Compile হলে Type Annotation মুছে JavaScript Output তৈরি হয়। **TypeScript নিজে Runtime Validation করে না।**

```
function total(price: number, quantity: number): number {
  return price * quantity;
}
// total('100', 2); // Compile-time error
```

`tsc` দিয়ে Type-checking করা হয়। TypeScript Type System শক্তিশালী হলেও কিছু ক্ষেত্রে Unsoundness থাকতে পারে—`any`, Type Assertion এবং External Input-এর ক্ষেত্রে সতর্কতা প্রয়োজন।

২৪.২ Project Setup

```
mkdir typesafe-booking-api
cd typesafe-booking-api
npm init -y
npm install --save-dev typescript
npx tsc --init
```

এই বইয়ের Source Folder-এ `package.json`, `tsconfig.json`, `src/domain.ts`, `src/service.ts`, `src/api.ts` এবং `domain.test.mjs` দেওয়া হয়েছে। `module` ও `moduleResolution`-এ `NodeNext` ব্যবহার করা হয়েছে। Import-এ `.js` Extension লেখা হলেও TypeScript Source `.ts` File Resolve করে; Compiled ESM-এ Extension সঠিক থাকে।

২৪.৩ Basic Types ও Inference

```
let customerName: string = 'Amina';
```

```
let travelers = 2; // inferred as number
const active: boolean = true;
const destinations: string[] = ['Sylhet', 'Cox's Bazar'];
const scores: Array<number> = [80, 90];
```

`const` Variable Reassignment আটকায়, কিন্তু Object-এর ভেতরের Property স্বয়ংক্রিয়ভাবে Immutable করে না। `readonly` Compile-time Protection দেয়; Runtime-এ Object Freeze করে না।

২৪.৪ `type`, `interface`, Union ও Literal Types

```
type BookingStatus = 'pending' | 'confirmed' | 'cancelled';
type Currency = 'BDT';
interface Package {
  readonly id: string;
  readonly title: string;
  readonly priceBDT: number;
}
```

`interface` Object Contract ও Extensibility-তে সুবিধাজনক। `type` Union, Tuple, Conditional Type ও Composite Type প্রকাশ করতে পারে। একটি Team-এর Consistent Convention সবচেয়ে গুরুত্বপূর্ণ।

২৪.৫ Optional Properties, Null এবং Strict Mode

```
interface Customer {
  id: string;
  email?: string;
}
function displayEmail(customer: Customer): string {
  return customer.email?.toLowerCase() ?? 'No email';
}
```

`strictNullChecks` চালু থাকলে `null`/`undefined` ভুলভাবে ব্যবহার করার সম্ভাবনা কমে।

`noUncheckedIndexedAccess` Array Indexing-এর ফলকে Potentially Undefined ধরে;

`exactOptionalPropertyTypes` Optional Property-এর Semantics স্পষ্ট করে।

২৪.৬ `unknown` বনাম `any`: Trust Boundary

HTTP Request, JSON File ও `localStorage` থেকে আসা Data **অবিশ্বস্ত**। `JSON.parse()`-এর Result-কে সরাসরি Domain Type হিসেবে Cast করা বিপজ্জনক। `unknown` গ্রহণ করে Runtime Check করুন।

```
function isRecord(value: unknown): value is Record<string, unknown> {
  return typeof value === 'object' && value !== null && !Array.isArray(value);
}
function getCustomerName(input: unknown): string | null {
  if (!isRecord(input)) return null;
  return typeof input.customerName === 'string' ? input.customerName : null;
}
```

Production-এ Zod বা অনুরূপ Schema Validation Library ব্যবহার করা যায়, তবে এখানে Dependency ছাড়াই Validation দেখানো হয়েছে। Type Assertion (`as`) Runtime-এ Value রূপান্তর করে না।

২৪.৭ Discriminated Union দিয়ে Error Handling

```ts
type Result<T, E> =
  | { ok: true; value: T }
  | { ok: false; error: E };

function divide(a: number, b: number): Result<number, string> {
  return b === 0 ? { ok: false, error: 'Cannot divide by zero' }
                 : { ok: true, value: a / b };
}
const answer = divide(10, 2);
if (answer.ok) console.log(answer.value);
else console.error(answer.error);
```

`ok` Field দেখে Compiler Narrowing করতে পারে। এতে Error Path বাদ পড়ার ঝুঁকি কমে।

২৪.৮ Generics এবং Reusable Abstractions

```ts
function first<T>(items: readonly T[]): T | undefined {
  return items[0];
}
const n = first([10, 20]); // number | undefined
const s = first(['a', 'b']); // string | undefined
```

Generics দিয়ে Reusable Function Type Information ধরে রাখে। অপ্রয়োজনীয় Complex Generic-এর বদলে পরিষ্কার Domain Type ব্যবহার করুন।

২৪.৯ `keyof`, `Pick`, `Omit`, `Partial`, `Record`

```ts
interface User { id: string; name: string; role: 'admin' | 'viewer' }
type PublicUser = Pick<User, 'id' | 'name'>;
type NewUser = Omit<User, 'id'>;
type UserPatch = Partial<NewUser>;
const labels: Record<User['role'], string> = {
  admin: 'Administrator', viewer: 'Read-only'
};
```

Utility Types Domain Contract রূপান্তরে সাহায্য করে; এগুলো Runtime Sanitization বা Authorization-এর বিকল্প নয়।

২৪.১০ Exhaustiveness Checking এবং State Machines

```ts
type Status = 'pending' | 'confirmed' | 'cancelled';
function assertNever(value: never): never {
  throw new Error(`Unexpected: ${String(value)}`);
}
function label(status: Status): string {
  switch (status) {
    case 'pending': return 'Pending';
    case 'confirmed': return 'Confirmed';
    case 'cancelled': return 'Cancelled';
    default: return assertNever(status);
  }
}
```

নতুন Status যোগ করলে Exhaustive Switch Compile-time Feedback দিতে পারে। কিন্তু Business Transition Rules আলাদা Validate করতে হবে—যেমন `cancelled` থেকে `confirmed`-এ ফেরানো নিষিদ্ধ।

২৪.১১ Practical Project — TypeSafe Booking API

এই Project-এর Architecture:

```
Unknown Input
   ↓ Runtime validation (domain.ts)
Typed CreateBookingInput
   ↓ BookingService (service.ts)
InMemoryBookingRepository
   ↓
Typed Booking Result
   ↓
Transport Adapter (api.ts)
```

Features: Package Catalog, Booking Validation, Price Calculation, Duplicate ID Check, Status Transition, Typed Error Codes এবং Automated Tests। Sample Packages ও Prices শুধুই শিক্ষামূলক; বাস্তব Booking বা Payment Processing নেই।

File Responsibilities

- `domain.ts`: Type Definitions, `Result`, Validation, Calculation, Exhaustive Status Label।
- `service.ts`: Dependency-injected Repository, Booking Business Rules।
- `api.ts`: HTTP-like Request Adapter; এটি Network Server নয়।
- `domain.test.ts`: Positive, Negative, Boundary ও State Transition Tests।
- `tsconfig.json`: Strict Compiler Settings।

Build ও Test

```
cd source-code/chapter-24
npm install
npm run build
npm test
```

`npm run build` TypeScript Source-কে `dist/` Folder-এ JavaScript হিসেবে Emit করবে। `npm test` Build করে Node.js Test Runner চালাবে। Dependency Installation-এর জন্য Internet প্রয়োজন হতে পারে।

Architecture Decisions

Repository Interface: Business Logic Data Storage-এর নির্দিষ্ট Implementation-এর ওপর নির্ভরশীল নয়। **Unknown Input:** API Boundary-তে Validation বাধ্যতামূলক। **Discriminated Result:** Failure Cases Explicit। **Immutable Booking Update:** Status Change-এ Spread দিয়ে নতুন Object তৈরি হয়। **Integer BDT:** Demo-তে পূর্ণ টাকায় হিসাব; Fractional Currency থাকলে Minor Unit Strategy ব্যবহার করুন।

২৪.১২ Common Mistakes

১. `any` দিয়ে Type Error চেপে দেওয়া। ২. `as Booking` লিখে JSON-কে Trusted ধরে নেওয়া। ৩. `strict` বন্ধ রাখা। ৪. `readonly`-কে Deep Runtime Immutability ভাবা। ৫. API Contract আর Database Constraint এক জিনিস মনে করা। ৬. Status Transition Validation বাদ দেওয়া। ৭. Domain Logic সরাসরি UI বা Route Handler-এ লেখা। ৮. Compile Success-কে Full Security Assurance মনে করা।

২৪.১৩ Exercises — ১০টি

১. `Trip` Interface তৈরি করুন, যেখানে `id`, `destination`, `priceBDT` থাকবে।

২. `PaymentStatus` Literal Union তৈরি করুন: `unpaid`, `paid`, `refunded`।

৩. `first<T>()` Function-এর Empty Array Case Handle করুন।

৪. `unknown` Input থেকে Valid Phone String বের করার Type Guard লিখুন।

৫. `Result<number,string>` দিয়ে Discount Calculation করুন।

৬. `Pick` দিয়ে `PublicBooking` Type তৈরি করুন।

৭. `BookingStatus`-এ নতুন `expired` Value যোগ করে Exhaustiveness Error পর্যবেক্ষণ করুন।

৮. Travelers `0`, `21`, `2.5` দিয়ে Validation Test করুন।

৯. Duplicate Booking ID-এর Negative Test লিখুন।

১০. `InMemoryBookingRepository` বদলে File-backed Repository-এর Interface Design করুন (Implementation ঐচ্ছিক)।

Sample Solutions

```
// 1
type Trip = { id: string; destination: string; priceBDT: number };
// 2
type PaymentStatus = 'unpaid' | 'paid' | 'refunded';
// 3
function first<T>(items: readonly T[]): T | undefined { return items[0]; }
// 4
function isPhone(input: unknown): input is string {
  return typeof input === 'string' && /^\+?[0-9]{7,15}$/.test(input);
}
// 5
type Result<T,E> = {ok:true;value:T}|{ok:false;error:E};
function discount(amount:number, percent:number):Result<number,string>{
  if(!Number.isFinite(amount)||amount<0||percent<0||percent>100)
    return {ok:false,error:'Invalid amount or percent'};
  return {ok:true,value:amount*(1-percent/100)};
}
// 6
type Booking = {id:string;customerName:string;totalBDT:number};
type PublicBooking = Pick<Booking,'id'|'totalBDT'>;
// 7: Add 'expired' to the union, then update the exhaustive switch.
// 8-9: See domain.test.mjs for boundary and duplicate tests.
// 10: Implement the BookingRepository interface in a new class.
```

২৪.১৪ Knowledge Check — ১০টি MCQ

১. TypeScript Type Annotation Runtime-এ থাকে? A) সবসময় B) না C) শুধু Browser-এ D) শুধু Node-এ

২. External JSON-এর জন্য নিরাপদ Starting Type? A) any B) never C) unknown D) void

৩. `strictNullChecks` কী করে? A) Null Risk Check B) Network Security C) CSS Validation D) Bundle Compression

৪. `readonly` কি Object Deep Freeze করে? A) হ্যাঁ B) না C) শুধু Array D) শুধু Class

৫. Generic-এর উদ্দেশ্য কী? A) Reusable Typed Logic B) HTML Styling C) Runtime Encryption D) Database Migration

৬. `Pick<User,'id'>` কী? A) Selected Properties Type B) Delete User C) Fetch API D) CSS Selector

৭. Discriminated Union Narrowing-এর ভিত্তি কী? A) Shared Literal Field B) File Name C) Comment D) Folder

৮. `never` কোন কাজে উপযোগী? A) Exhaustiveness Check B) Image Resize C) Cache Expiry D) HTTP Request

৯. `as Booking` কি Runtime Validation? A) হ্যাঁ B) না C) শুধু Test-এ D) শুধু Production-এ

১০. Type-safe Project-এর Production Security-এর জন্য আর কী দরকার? A) Runtime Validation ও Authorization B) শুধু Compile C) শুধু CSS D) শুধু Comments

Answer Key ও Explanations

১. **B** — Compile-এর পরে Type Annotation সাধারণত থাকে না। ২. **C** — `unknown` ব্যবহারে Narrowing বাধ্যতামূলক। ৩. **A** — Nullable Value Access Check করে। ৪. **B** — এটি Static Constraint। ৫. **A** — Type Information ধরে Reuse করা যায়। ৬. **A** — নির্দিষ্ট Properties নির্বাচন করে। ৭. **A** — `ok`-এর মতো Literal Field। ৮. **A** — Unhandled Case ধরতে। ৯. **B** — Type Assertion Runtime Check নয়। ১০. **A** — Compiler Security Boundary নয়।

২৪.১৫ Chapter Summary

TypeScript JavaScript-এর সঙ্গে Type Safety ও Maintainability যোগ করে, কিন্তু Runtime Input, Authentication, Authorization এবং Database Consistency আলাদাভাবে নিশ্চিত করতে হয়। TypeSafe Booking API-এর Typed Domain, Service, Repository ও Adapter Design পরবর্তী **Chapter 25: Node.js Backend, Express, Database ও Authentication**-এর ভিত্তি।

অধ্যায় ২৫ — JavaScript Backend Development with Node.js

REST APIs, Express, Databases, Authentication এবং Production Architecture

এই অধ্যায়ের লক্ষ্য

এই অধ্যায় শেষে পাঠক Node.js runtime, HTTP request–response, RESTful API, Express middleware, input validation, password hashing, session authentication, authorization, database persistence, deployment security ও backend testing-এর মৌলিক ধারণা ব্যাখ্যা করতে এবং একটি ছোট API তৈরি করতে পারবেন। আগের অধ্যায়ের TypeScript domain model-এর সঙ্গে এই অধ্যায়ের transport ও persistence ধারণা যুক্ত করা যাবে।

২৫.১ Browser JavaScript বনাম Node.js

Browser-এ JavaScript DOM, `window` ও browser storage পায়। Node.js-এ filesystem, network server, process এবং server-side APIs পাওয়া যায়। Backend-এর দায়িত্ব শুধু JSON পাঠানো নয়; identity, business rules, authorization, data integrity, error handling ও operational safety বজায় রাখা।

```
// hello.mjs
console.log('Node version:', process.version);
console.log('Environment:', process.env.NODE_ENV ?? 'development');
```

Node.js-এ CommonJS (`require`, `module.exports`) এবং ES Modules (`import`, `export`) উভয় ব্যবস্থা আছে। এই বইয়ের project-এ ES Modules ব্যবহার করা হয়েছে। `package.json`-এ `"type": "module"` দিলে `.js` ফাইলও ESM হতে পারে; `.mjs` নিজে থেকেই ESM।

২৫.২ HTTP এবং REST

HTTP request-এর Method, URL, Headers ও Body থাকে। Response-এর Status Code, Headers ও Body থাকে। API resource-oriented হলে endpoint বোঝা সহজ হয়।

Method	Endpoint	কাজ	সাধারণ Status
GET	`/api/packages`	Package list	200
POST	`/api/register`	Account তৈরি	201
POST	`/api/login`	Session তৈরি	200 বা 401
GET	`/api/bookings`	নিজের Booking list	200 বা 401
POST	`/api/bookings`	Booking তৈরি	201 বা 400
PATCH	`/api/bookings/:id/cancel`	Booking বাতিল	200, 404 বা 409

`400` ভুল input, `401` unauthenticated, `403` authenticated কিন্তু permission নেই, `404` resource নেই, `409` conflict, `500` unexpected server failure বোঝায়। `401` ও `403` এক নয়। API design-এ একই ধরনের error-এর জন্য একই response shape ব্যবহার করা সুবিধাজনক।

২৫.৩ HTTP Server-এর মূল ধারণা

Node.js built-in `node:http` দিয়েও API বানানো যায়। Request body streaming হওয়ায় আকার সীমিত করতে হবে; `JSON.parse()` ব্যর্থ হলে `400` দিতে হবে।

```
import http from 'node:http';
const server = http.createServer((req, res) => {
  if (req.method === 'GET' && req.url === '/health') {
    res.writeHead(200, { 'content-type': 'application/json' });
    res.end(JSON.stringify({ status: 'ok' }));
    return;
  }
  res.writeHead(404, { 'content-type': 'application/json' });
  res.end(JSON.stringify({ error: 'NOT_FOUND' }));
});
server.listen(3000);
```

এটি HTTP lifecycle শেখার উদাহরণ। Full Project-এ আলাদা Router-style Handler, Service এবং Repository রাখা হয়েছে।

২৫.৪ Express কী সুবিধা দেয়?

Express routing ও middleware chain সহজ করে। নিচের snippet চালাতে `npm install express` প্রয়োজন। এটি সম্পূর্ণ TravelCore project-এর runtime dependency নয়; project-টি পরীক্ষাযোগ্য রাখতে built-in HTTP ব্যবহার করা হয়েছে।

```
import express from 'express';
const app = express();
app.use(express.json({ limit: '16kb' }));
app.get('/health', (_req, res) => res.json({ status: 'ok' }));
app.use((err, _req, res, _next) => {
  console.error(err);
  res.status(500).json({ error: 'INTERNAL_ERROR' });
});
app.listen(3000);
```

Middleware সাধারণত logging, authentication, JSON parsing, request validation ও error handling-এর জন্য ব্যবহৃত হয়। Route-এর আগে middleware বসানোর ক্রম গুরুত্বপূর্ণ। Express error middleware-এ চারটি parameter থাকে।

২৫.৫ Layered Architecture

```
HTTP Client
    ↓
Router / Controller  → HTTP parsing, status, headers
    ↓
Service              → business rules, authorization
    ↓
```

```
Repository          → data access contract
    ↓
Database            → durable state and constraints
```

Controller-এর মধ্যে SQL, password hashing, discount policy সব একসঙ্গে রাখলে test ও maintenance কঠিন হয়। Service-এ domain rule, Repository-তে storage operation রাখা ভালো। Dependency injection করলে in-memory repository দিয়ে unit test চালানো যায়।

২৫.৬ Input Validation এবং Price Integrity

Frontend-এর input বিশ্বাস করা যাবে না। Client থেকে price নেওয়ার বদলে server catalog থেকে price নির্ধারণ করবে। Quantity integer কি না, ID সঠিক কি না, payload size সীমার মধ্যে কি না যাচাই করতে হবে।

```
export function validateTravelers(value) {
  if (!Number.isSafeInteger(value) || value < 1 || value > 20) {
    throw new Error('Travelers must be 1–20');
  }
  return value;
}
```

Validation-এর দুই স্তর: syntax/shape validation এবং business-rule validation। যেমন `travelers=2` shape-এ বৈধ, কিন্তু কোনো নির্দিষ্ট package-এ ২ জনের booking বন্ধ থাকলে business rule-এ বাতিল হতে পারে।

২৫.৭ Authentication বনাম Authorization

Authentication নির্ধারণ করে **কে** request করছেন। Authorization নির্ধারণ করে **তিনি কী করতে পারবেন**। Login থাকা মানেই অন্য ব্যবহারকারীর booking দেখা বা বাতিল করার অনুমতি নয়। Project-এ `userId` দিয়ে নিজের booking filter করা হয়।

Password plaintext রাখা যাবে না। এই educational project-এ Node.js `scryptSync` + random salt দিয়ে password hash করা হয়েছে। Production-এ Argon2id-এর মতো সুপারিশকৃত password hashing ব্যবস্থা এবং উপযুক্ত cost parameters বিবেচনা করা উচিত। Password যাচাইয়ে timing-safe comparison ব্যবহার করা হয়েছে।

Session cookie-তে `HttpOnly`, `SameSite`, `Secure` (HTTPS-এ), expiry এবং server-side session invalidation প্রয়োজন। এই demo-তে token server memory-তে থাকে; restart হলে session হারিয়ে যায়। Cookie-based state-changing endpoints-এর জন্য production-এ CSRF protection দরকার। Login rate limiting, account lockout policy এবং audit logging-ও প্রয়োজন।

২৫.৮ Database: In-memory থেকে PostgreSQL

এই অধ্যায়ের runnable example-এ `Map`-ভিত্তিক Repository ব্যবহার করা হয়েছে যাতে external database ছাড়া project পরীক্ষা করা যায়। এটি durable database নয়। Real production-এ relational schema ও constraints দরকার।

```
CREATE TABLE users (
```

```
  id UUID PRIMARY KEY,
  email TEXT NOT NULL UNIQUE,
  password_hash TEXT NOT NULL
);

CREATE TABLE bookings (
  id UUID PRIMARY KEY,
  user_id UUID NOT NULL REFERENCES users(id),
  package_id TEXT NOT NULL,
  travelers INTEGER NOT NULL CHECK (travelers BETWEEN 1 AND 20),
  total_bdt INTEGER NOT NULL CHECK (total_bdt >= 0),
  status TEXT NOT NULL CHECK (status IN ('pending','confirmed','cancelled')),
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
CREATE INDEX bookings_user_created_idx ON bookings(user_id, created_at DESC);
```

Schema migration version-controlled হতে হবে। SQL query parameterized হতে হবে, string concatenation দিয়ে untrusted data SQL-এ ঢোকানো যাবে না। Booking + inventory update একই transaction-এ করতে হতে পারে; rollback, isolation ও idempotency design জরুরি। পরের অধ্যায়ে Prisma, transactions ও query optimization বিস্তারিত থাকবে।

২৫.৯ API Security ও Production Checklist

- HTTPS, reverse proxy, secure cookie, CSRF protection, rate limiting ও request-size limit।
- CORS কেবল প্রয়োজনীয় trusted origins-এ সীমিত; CORS নিজে authentication নয়।
- Secrets source code বা Git repository-তে নয়; environment/secret manager-এ।
- Logs-এ password, session token, sensitive user data নয়।
- `npm audit`, dependency updates, least-privilege database role ও backups।
- Graceful shutdown, health checks, metrics, tracing এবং incident response plan।
- Horizontal scaling-এর ক্ষেত্রে in-memory session বা process-local data যথেষ্ট নয়।

২৫.১০ Practical Project — TravelCore Booking Backend

লক্ষ্য ও Folder Structure

```
source-code/chapter-25/
├── package.json
├── catalog.mjs
├── core.mjs
├── repository.mjs
├── server.mjs
├── core.test.mjs
├── http.test.mjs
└── README.md
```

বইয়ের Companion Source Code-এ উপরোক্ত ফাইলগুলোর **সম্পূর্ণ runnable source code** আছে। এখানে গুরুত্বপূর্ণ অংশ ব্যাখ্যা করছি; source code বাদ দেওয়া হয়নি—আলাদা project directory-তে সংরক্ষিত।

Step 1: `cd source-code/chapter-25` তারপর `npm test`। **Step 2:** `npm start`। **Step 3:** `http://localhost:3000/health` এবং `/api/packages` খুলে দেখো। **Step 4:** `POST /api/register` দিয়ে

account তৈরি, `POST /api/login` দিয়ে cookie নাও। **Step 5:** সেই cookie দিয়ে `POST /api/bookings` এবং `GET /api/bookings` চালাও।

Request Example

```
curl -i -c cookies.txt -H 'Content-Type: application/json' \
  -d '{"email":"reader@example.com","password":"a strong password 123"}' \
  http://localhost:3000/api/register

curl -i -c cookies.txt -H 'Content-Type: application/json' \
  -d '{"email":"reader@example.com","password":"a strong password 123"}' \
  http://localhost:3000/api/login

curl -i -b cookies.txt -H 'Content-Type: application/json' \
  -d '{"packageId":"cox","travelers":2}' \
  http://localhost:3000/api/bookings
```

Windows PowerShell-এ `curl` alias থাকলে `curl.exe` ব্যবহার করো। Demo-এর cookie file সংবেদনশীল; অন্যের সঙ্গে share করবে না।

Project Design Decisions

- **Catalog-based price:** client arbitrary total দিতে পারে না।
- **Password hashing:** salted scrypt; plaintext password repository-তে নেই।
- **Authorization:** booking owner ছাড়া অন্য user cancel করতে পারে না।
- **Error format:** `{ error, message }`।
- **Session:** cryptographically random token; server-side hashed token lookup।
- **Testability:** Service pure-ish dependency-injected repository-তে কাজ করে।

সীমাবদ্ধতা

এই Demo-তে real database, atomic concurrent uniqueness, persistent session, CSRF defense, rate limiting ও production-grade observability নেই। তাই এটিকে Production Booking Backend হিসেবে deploy করবে না।

২৫.১১ Common Mistakes এবং Debugging

Mistake 1: Frontend price বিশ্বাস করা। **Fix:** server catalog থেকে হিসাব।

Mistake 2: `401` ও `403` এক করা। **Fix:** login এবং permission আলাদা পরীক্ষা।

Mistake 3: password log করা। **Fix:** secrets redact করা।

Mistake 4: `await` বাদ দেওয়া। **Fix:** asynchronous repository ও errors পরীক্ষা।

Mistake 5: শুধু unit test করে database concurrency safe দাবি করা। **Fix:** real database integration ও transaction test।

Mistake 6: cookie session ব্যবহার করে CSRF threat ভুলে যাওয়া। **Fix:** origin checks, CSRF token, SameSite এবং threat model।

২৫.১২ অনুশীলনী (১০টি)

1. `GET /health`-এ uptime যোগ করো।
2. `POST /api/bookings`-এ `travelers: 0` দিলে কী status হওয়া উচিত?
3. একই email দিয়ে দুইবার registration করলে conflict কেন?
4. `PATCH /api/bookings/:id/cancel`-এ owner check কেন দরকার?
5. `GET /api/bookings`-এ pagination design করো।
6. Request body size limit না থাকলে কী ঝুঁকি?
7. Password hash-এর সঙ্গে salt কেন লাগে?
8. `401` এবং `403`-এর ব্যবহার উদাহরণ দাও।
9. In-memory Repository-কে PostgreSQL-এ বদলানোর interface লিখো।
10. CSRF protection ও rate limiting-এর integration test plan তৈরি করো।

২৫.১৩ Sample Solutions

1. `process.uptime()` seconds response-এ দাও; এটি process uptime, service availability নয়।
2. `400 INVALID_TRAVELERS`; quantity validation service-এ থাকবে।
3. Duplicate identity prevent করতে `409 EMAIL_EXISTS`; real DB-তে `UNIQUE(email)` constraint অপরিহার্য।
4. Booking ID জানা থাকলেও অন্য user-এর booking পরিবর্তনের অধিকার নেই; IDOR vulnerability ঠেকায়।
5. `?limit=20&cursor=...`; stable `(created_at,id)` order ও indexed cursor ব্যবহার করো।
6. Unbounded memory/CPU usage ও denial-of-service ঝুঁকি।
7. একই password-এর hash ভিন্ন রাখতে ও precomputed rainbow table আক্রমণ কঠিন করতে।
8. Session না থাকলে `401`; login থাকা সত্ত্বেও admin-only action করলে `403`।
9. `findUserByEmail`, `saveUser`, `saveBooking`, `findBooking`, `listBookings`—এই contract-এর async PostgreSQL implementation; transaction ও constraints যোগ করো।
10. Cross-site POST reject, valid CSRF token accept, repeated login attempts throttle, legitimate user unaffected—এই case-গুলো test করো।

২৫.১৪ MCQ — Final Check

1. Node.js-এ HTTP server তৈরির built-in module কোনটি? (A) `node:http` (B) `window` (C) `document` (D) `localStorage`
2. সফল resource creation-এর সাধারণ status? (A) 404 (B) 201 (C) 500 (D) 302
3. `401` সাধারণত কী বোঝায়? (A) No authentication (B) No storage (C) Success (D) Redirect

4. User A-এর booking User B cancel করতে না পারা কোন নীতি? (A) CSS (B) Authorization (C) Minification (D) Hoisting
5. Password plaintext না রেখে কী রাখা উচিত? (A) Base64 (B) Hash + Salt (C) URL (D) Cookie Name
6. SQL Injection কমাতে কী দরকার? (A) String concatenation (B) Parameterized query (C) `eval` (D) `innerHTML`
7. HTTP request body size limit কেন? (A) Font (B) Resource abuse prevent (C) Syntax color (D) SEO
8. CSRF মূলত কোন context-এ গুরুত্বপূর্ণ? (A) Cookie-authenticated state changes (B) Static comments (C) Array sort (D) CSS animation
9. Database transaction কেন? (A) Atomic consistency (B) Font loading (C) DOM rendering (D) Code highlighting
10. `Map`-based repository restart হলে কী হয়? (A) Data persists (B) Data disappears (C) Migrates to SQL (D) Auto backup

Answer Key ও Explanation

1. **A** — `node:http` Node.js-এর HTTP module।
2. **B** — নতুন resource তৈরিতে `201 Created`।
3. **A** — request-এ বৈধ authentication নেই।
4. **B** — resource ownership authorization rule।
5. **B** — salted, computationally expensive password hashing।
6. **B** — parameter binding data ও SQL command আলাদা রাখে।
7. **B** — অস্বাভাবিক বড় payload-এর resource exhaustion ঠেকায়।
8. **A** — browser স্বয়ংক্রিয়ভাবে cookie পাঠাতে পারে।
9. **A** — একাধিক dependent database operation একসঙ্গে commit/rollback।
10. **B** — process memory durable storage নয়।

২৫.১৫ Chapter Summary ও পরবর্তী অধ্যায়

Node.js দিয়ে backend HTTP request handle করে; Express middleware ও routing সহজ করে; Service business logic রাখে; Repository persistence abstract করে; Authentication ও Authorization আলাদা; Database constraints এবং security review ছাড়া demo production-ready নয়। পরের অধ্যায় **Advanced Database Integration, ORM, Query Optimization এবং Scalable Data Architecture**—এ PostgreSQL ও Prisma নিয়ে কাজ করা হবে।

অধ্যায় ২৬ — Advanced Database Integration, ORM, Query Optimization এবং Scalable Data Architecture

PostgreSQL, Prisma এবং TravelData Pro

শেখার লক্ষ্য

এই অধ্যায় শেষে relational schema, keys, constraints, normalization, transactions, isolation, ORM, migrations, indexes, query plans, pagination, connection pools, caching এবং scalable database design ব্যাখ্যা ও প্রয়োগ করতে পারবেন। Chapter 25-এর in-memory backend-কে স্থায়ী relational persistence-এ উন্নীত করার নকশা তৈরি করব।

২৬.১ Database কেন গুরুত্বপূর্ণ?

Chapter 25-এ server restart হলে in-memory bookings হারিয়ে যায়। Production booking system-এ account, booking, inventory ও payment-এর মধ্যে নির্ভরযোগ্য সম্পর্ক দরকার। PostgreSQL-এর মতো relational database ACID transaction, foreign keys, uniqueness এবং concurrent update-এর জন্য উপযোগী। তবে database ব্যবহার করলেই correctness নিশ্চিত হয় না: application business rules, authorization, idempotency এবং failure recovery-ও জরুরি।

উদাহরণ: ১টি আসন বাকি, একই সময়ে দুইজন বুকিং করলেন। আগে availability পড়ে পরে update করলে দুজনই সফল হতে পারেন। Database transaction এবং conditional update দিয়ে overselling আটকাতে হবে।

২৬.২ Relational Modeling ও Normalization

মূল entity: `User`, `TravelPackage`, `Booking`। একটি User-এর বহু Booking থাকতে পারে, একটি Package-এর বহু Booking থাকতে পারে। প্রতিটি Booking একটি User এবং একটি Package-এর সঙ্গে সম্পর্কিত।

Entity	গুরুত্বপূর্ণ ক্ষেত্র	Constraint
User	id, email, passwordHash	email UNIQUE, NOT NULL
TravelPackage	id, title, priceMinor, availableSeats	price ও seats >= 0
Booking	id, userId, packageId, seats, status, totalMinor	Foreign keys, seats > 0

Normalization: একই package title বা customer email প্রতিটি Booking row-তে কপি করলে update anomaly তৈরি হয়। আলাদা entity এবং foreign key ব্যবহার করলে data duplication কমে। তবে reporting-এর জন্য deliberate denormalization প্রয়োজন হতে পারে—সেটি পরিমাপ ও documentation করে করতে হবে।

Booking-এর `totalMinor`-এ purchase-time price snapshot রাখা যুক্তিযুক্ত, কারণ পরে package price বদলালেও পুরোনো booking total বদলানো উচিত নয়।

২৬.৩ PostgreSQL Schema এবং Constraints

```
CREATE TABLE app_users (
  id UUID PRIMARY KEY,
  email TEXT NOT NULL UNIQUE,
  password_hash TEXT NOT NULL,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE TABLE travel_packages (
  id UUID PRIMARY KEY,
  title TEXT NOT NULL,
  price_minor BIGINT NOT NULL CHECK (price_minor >= 0),
  available_seats INTEGER NOT NULL CHECK (available_seats >= 0)
);
CREATE TABLE bookings (
  id UUID PRIMARY KEY,
  user_id UUID NOT NULL REFERENCES app_users(id),
  package_id UUID NOT NULL REFERENCES travel_packages(id),
  seats INTEGER NOT NULL CHECK (seats > 0),
  total_minor BIGINT NOT NULL CHECK (total_minor >= 0),
  status TEXT NOT NULL CHECK (status IN ('CONFIRMED', 'CANCELLED')),
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE INDEX bookings_user_created_idx ON bookings (user_id, created_at DESC);
```

Money `float` দিয়ে গণনা না করে minor units (যেমন পয়সা) বা নির্দিষ্ট decimal type ব্যবহার করা ভালো। `BIGINT` JavaScript-এ `BigInt` হিসেবে আসতে পারে; JSON serialization-এর সময় explicit conversion strategy লাগবে। UUID collision resistance যথেষ্ট হলেও authorization-এর বিকল্প নয়।

২৬.৪ SQL Query ও Injection Prevention

```
SELECT b.id, b.status, b.total_minor, p.title
FROM bookings b
JOIN travel_packages p ON p.id = b.package_id
WHERE b.user_id = $1
ORDER BY b.created_at DESC, b.id DESC
LIMIT $2;
```

`$1`, `$2` parameter binding-এর উদাহরণ; কখনো user input string concatenate করে SQL তৈরি করবেন না। Dynamic sort column-এর ক্ষেত্রে parameter placeholder দিয়ে identifier bind করা যায় না; আগে নির্দিষ্ট allowlist ব্যবহার করুন।

২৬.৫ ORM: Prisma কীভাবে কাজ করে?

Prisma schema-তে model, relation এবং index বর্ণনা করা হয়; generated client থেকে type-aware query করা যায়। ORM সুবিধাজনক হলেও generated SQL ও database constraints বোঝা জরুরি। এই অধ্যায়ের `source-code/chapter-26/prisma/schema.prisma`-তে বাস্তব model আছে।

```
const bookings = await prisma.booking.findMany({
  where: { userId: currentUserId },
  orderBy: [{ createdAt: 'desc' }, { id: 'desc' }],
```

```
  take: 20,
  select: { id: true, status: true, totalMinor: true,
    travelPackage: { select: { title: true } } }
});
```

এই query server-side authorized user ID দিয়ে filter করতে হবে; browser পাঠানো arbitrary user ID বিশ্বাস করা যাবে না। Schema-এর enum/relations এবং generated client-এর নাম সংস্করণভেদে যাচাই করুন।

২৬.৬ Migrations এবং Data Lifecycle

Schema পরিবর্তনের জন্য version-controlled migration দরকার। Development-এ `prisma migrate dev`, deployment pipeline-এ `prisma migrate deploy` ব্যবহৃত হয়। Production-এ destructive migration দেওয়ার আগে backup, rollback/roll-forward plan এবং data compatibility পরীক্ষা করতে হবে।

Expand–Migrate–Contract: নতুন nullable column যোগ করুন → app উভয় version support করুক → data backfill করুন → constraint দিন → পুরোনো column সরান। Zero-downtime deployment-এ এই কৌশল কার্যকর।

২৬.৭ Transaction, Atomicity ও Concurrency

নিচের pattern-এ conditional inventory decrement ও booking insert একই transaction-এ করা হয়:

```
await prisma.$transaction(async (tx) => {
  const updated = await tx.travelPackage.updateMany({
    where: { id: packageId, availableSeats: { gte: seats } },
    data: { availableSeats: { decrement: seats } }
  });
  if (updated.count !== 1) throw new Error('SOLD_OUT');
  const pkg = await tx.travelPackage.findUniqueOrThrow({ where: { id: packageId } });
  await tx.booking.create({ data: {
    userId, packageId, seats,
    totalMinor: pkg.priceMinor * BigInt(seats),
    status: 'CONFIRMED'
  }});
});
```

গুরুত্বপূর্ণ: এটি illustrative code; transaction-এর বাইরে price snapshot নেওয়া, retry policy, concurrent price changes, idempotency key, cancellation refund এবং authorization production system-এ আলাদাভাবে design করতে হবে। Conditional update database-এ atomically execute হয়। Unique idempotency key দিয়ে duplicate network retry-ও আটকানো যায়।

২৬.৮ Isolation Levels এবং Deadlocks

Read Committed, Repeatable Read ও Serializable-এর পার্থক্য transaction anomaly-তে প্রকাশ পায়। Serializable শক্তিশালী guarantee দেয়, কিন্তু serialization failure হলে transaction retry দরকার। Deadlock-এর ঝুঁকি কমাতে একই resource lock order অনুসরণ করুন, transaction ছোট রাখুন এবং external HTTP call transaction-এর মধ্যে করবেন না।

২৬.৯ Index, EXPLAIN এবং Query Optimization

Index lookup দ্রুত করতে পারে, কিন্তু write/storage cost বাড়ায়। Query pattern অনুযায়ী composite index নির্বাচন করুন। PostgreSQL `EXPLAIN (ANALYZE, BUFFERS)` বাস্তব query চালিয়ে plan ও cost দেখায়—production-এ সাবধানতা প্রয়োজন। `SELECT *` এড়িয়ে প্রয়োজনীয় field নিন, N+1 query কমান এবং filter/sort-এর জন্য index design করুন।

২৬.১০ Pagination, Pooling ও Caching

`OFFSET` pagination বড় table-এ ধীর হতে পারে এবং concurrent insert-এ page drift হয়। Stable ordering ও cursor/keyset pagination বিবেচনা করুন। Connection pool সীমিত রাখুন; প্রতিটি request-এ নতুন database connection তৈরি করবেন না। Cache ব্যবহার করলে TTL, invalidation, stale-data tolerance এবং privacy নির্ধারণ করুন। Booking confirmation-এর মতো critical decision cache থেকে করবেন না।

২৬.১১ Backup, Security ও Observability

Database credential environment secret manager-এ রাখুন। Least-privilege DB user, TLS, encryption at rest, access audit, backups ও restore drills রাখুন। Query duration, error rate, lock wait, connection saturation এবং slow query পর্যবেক্ষণ করুন। শুধু backup নেওয়া যথেষ্ট নয়—restore করে পরীক্ষা করা জরুরি।

২৬.১২ Practical Project — TravelData Pro

Architecture: HTTP API → Booking Service → Repository → PostgreSQL/Prisma। `source-code/chapter-26/`-তে Prisma schema, SQL DDL, transaction-oriented example এবং **dependency-free in-memory simulation** দেওয়া আছে। Simulation-এ stock reservation, cancellation, duplicate request protection, reporting এবং cursor pagination পরীক্ষা করা যায়। এটি real PostgreSQL-এর সমতুল্য concurrency guarantee দেয় না।

Run

```
cd source-code/chapter-26
node --test
node demo.mjs
```

Real Database Setup (ঐচ্ছিক)

```
npm install
# PostgreSQL instance চালু করুন এবং DATABASE_URL সেট করুন
npx prisma migrate dev --name init
npx prisma generate
```

Migration তৈরি ও চালাতে PostgreSQL service প্রয়োজন। এই বইয়ের bundle-এ live database বা credentials নেই। `prisma/schema.prisma` ব্যবহার করার আগে নিজের environment-এর Prisma version-এর configuration নিয়ম যাচাই করুন।

২৬.১৩ Exercises

১. User–Booking relation-এর cardinality ব্যাখ্যা করুন।

২. `email`-এ UNIQUE constraint কেন দরকার?

৩. `price` float ব্যবহার করলে কী সমস্যা হতে পারে?

৪. N+1 query কীভাবে শনাক্ত করবেন?

৫. `availableSeats` 0-এর নিচে নামা কীভাবে রোধ করবেন?

৬. Offset এবং cursor pagination-এর tradeoff লিখুন।

৭. Booking cancellation-এ transaction কেন দরকার?

৮. SQL injection রোধের দুটি নিয়ম লিখুন।

৯. Index থাকলেও query slow হওয়ার তিনটি কারণ লিখুন।

১০. Expand–Migrate–Contract দিয়ে একটি field rename-এর plan লিখুন।

২৬.১৪ Sample Solutions

১. One User → Many Bookings; Each Booking → One User।

২. Duplicate account identity database স্তরে রোধ করে; app validation-এর race condition-ও আটকায়।

৩. Binary floating-point rounding error; minor units বা fixed decimal ব্যবহার করুন।

৪. Query log/trace-এ একটি parent query-এর পরে প্রতি row-এর জন্য child query দেখলে N+1 সন্দেহ। Eager loading/batched query ব্যবহার করুন।

৫. `CHECK (available_seats >= 0)` এবং conditional atomic update; দুটোই দরকার।

৬. Offset সহজ কিন্তু বড় offset ধীর; cursor দ্রুত ও stable ordering-এ ভালো, তবে implementation জটিল।

৭. Status update ও stock release একসঙ্গে commit/rollback হওয়া দরকার।

৮. Parameterized queries; dynamic identifiers-এর allowlist।

৯. Low selectivity, wrong index/order, stale statistics, expensive joins বা large result set।

১০. New column যোগ → dual read/write → backfill → switch read → constraint → old column drop।

২৬.১৫ MCQ — Self Assessment

১. ACID-এর A? (A) Atomicity (B) Availability (C) Authorization (D) Aggregation

২. Foreign key কী করে? (A) UI render (B) Referential integrity (C) Encryption (D) Caching

৩. SQL injection কমায়? (A) String concatenation (B) Parameter binding (C) eval (D) localStorage

৪. Duplicate email আটকায়? (A) INDEX only (B) UNIQUE (C) ORDER BY (D) LIMIT

৫. N+1-এর উপযুক্ত সমাধান? (A) More loops (B) Batch/eager load (C) Remove constraints (D) No index

৬. Migration deploy কোথায়? (A) Production pipeline (B) CSS (C) Browser only (D) README only

৭. Cursor pagination-এ প্রয়োজন? (A) Unstable order (B) Stable ordering (C) Random order (D) No index

৮. Critical inventory check কোথায়? (A) Browser cache (B) Database transaction (C) CSS (D) Client clock

৯. EXPLAIN ANALYZE কী করে? (A) Query execute ও profile (B) Password hash (C) UI test (D) Git merge

১০. Backup-এর নির্ভরযোগ্যতা কীভাবে জানবেন? (A) File size (B) Restore test (C) Filename (D) Commit count

Answer key: ১-A (Atomicity), ২-B (referential integrity), ৩-B (parameterization), ৪-B (unique constraint), ৫-B (batch loading), ৬-A (deployment), ৭-B (stable order), ৮-B (transaction), ৯-A (execution plan), ১০-B (restore drill)।

২৬.১৬ Chapter Summary ও পরবর্তী অধ্যায়

Database design-এ correctness প্রথম, performance দ্বিতীয়, scaling তৃতীয়—তবে তিনটি একসঙ্গে পরিকল্পনা করতে হয়। Chapter 27-এ booking updates real-time clients-এ পৌঁছাতে WebSockets, Socket.IO এবং event-driven messaging শিখব।

অধ্যায় ২৭ — Real-Time JavaScript Applications

WebSockets, Socket.IO, Event-Driven Architecture এবং Distributed Messaging

শেখার লক্ষ্য

এই অধ্যায় শেষে polling, Server-Sent Events (SSE), WebSocket, Socket.IO, event-driven design, event contracts, delivery semantics, reconnect, heartbeat, authorization, backpressure, Redis Pub/Sub ও distributed scaling-এর মূল ধারণা ব্যাখ্যা করতে পারবেন। TravelLive নামে একটি real-time booking event simulator তৈরি করবেন।

২৭.১ Real-Time Application কী?

একটি booking dashboard-এ একজন operator booking confirm করলে অন্য operator-এর screen-এ যত দ্রুত সম্ভব status update দেখা দরকার। শুধু page refresh করলে তা হয় না। Real-time application সাধারণত server থেকে client-এ নতুন event পৌঁছে দেয়; তবে network delay, connection drop ও duplicate delivery বিবেচনায় নিতে হয়। **Real-time মানে guaranteed instant নয়**—এটি low-latency update-এর একটি engineering goal।

Use cases: booking status updates, live seat availability, chat, collaborative dashboards, notifications, delivery tracking এবং support agent presence।

২৭.২ Polling, Long Polling, SSE এবং WebSocket

Technique	Communication	Suitable use	Limitations
Short polling	Repeated HTTP request	Rare updates, simple clients	Unnecessary requests, delayed updates
Long polling	Server holds request until data/timeout	Fallback	Connection churn
SSE	Server → browser, HTTP stream	Notifications, status feed	One-way, text/event format
WebSocket	Bidirectional persistent connection	Chat, live collaboration	Connection lifecycle, scaling
Socket.IO	Event-oriented library with transport abstraction	Rooms, reconnect support	Socket.IO protocol, not raw WebSocket-compatible

Socket.IO client দিয়ে arbitrary raw WebSocket server-এ সরাসরি connect করা যায় না; উভয় পাশে compatible protocol দরকার। SSE-তে `EventSource` ব্যবহার করা যায়; authenticated cross-origin requests-এর cookie/CORS আচরণ বুঝতে হবে।

২৭.৩ WebSocket Lifecycle

Browser একটি HTTP Upgrade handshake দিয়ে WebSocket সংযোগ শুরু করতে পারে। সংযোগ সফল হলে client ও server দুই দিকেই message পাঠাতে পারে। `open`, `message`, `error`, `close` lifecycle events handle করতে হয়। Browser-side উদাহরণ:

```
const socket = new WebSocket('wss://example.com/live');
socket.addEventListener('open', () => console.log('Connected'));
socket.addEventListener('message', (event) => {
  try { const update = JSON.parse(event.data); console.log(update); }
  catch { console.warn('Invalid server message'); }
});
socket.addEventListener('close', () => console.log('Disconnected'));
```

Production-এ `wss://`, authenticated session, allowed origins, per-message validation, message-size limit এবং rate limiting জরুরি। শুধু client-এ validation থাকলে নিরাপত্তা নিশ্চিত হয় না।

২৭.৪ Socket.IO Events এবং Rooms

Socket.IO server-এ booking-specific room-এ authorized clients যুক্ত করা যায়। নিচের snippet একটি **illustrative server design**, bundle-এর tested implementation নয়:

```
io.use(async (socket, next) => {
  try {
    socket.data.user = await authenticateSession(socket.request);
    next();
  } catch { next(new Error('UNAUTHORIZED')); }
});

io.on('connection', (socket) => {
  socket.on('booking:subscribe', async ({ bookingId }, ack) => {
    if (typeof bookingId !== 'string') return ack?.({ ok: false });
    if (!await canViewBooking(socket.data.user.id, bookingId)) {
      return ack?.({ ok: false });
    }
    await socket.join(`booking:${bookingId}`);
    ack?.({ ok: true });
  });
});
```

Important: Room name গোপন রাখলেই authorization হয় না। প্রতিটি subscribe ও mutation server-side অনুমতি যাচাই করতে হবে। Room membership connection-specific; reconnect হলে আবার subscribe করতে হতে পারে।

২৭.৫ Event Contract ও Versioning

অর্থপূর্ণ event envelope ব্যবহার করলে client সহজে handle করতে পারে:

```
{
  "eventId": "evt-102",
  "type": "booking.status.changed",
  "version": 1,
  "bookingId": "bk-22",
  "sequence": 8,
  "occurredAt": "2026-10-11T00:00:00.000Z",
```

```
  "payload": { "status": "CONFIRMED" }
}
```

`eventId` duplicate detection-এ, `sequence` per-booking ordering-এ, `version` schema evolution-এ কাজে লাগে। Event payload-এ password, access token, ব্যক্তিগত তথ্য বা unnecessary financial details পাঠাবেন না।

২৭.৬ Event-Driven Architecture

Command (যেমন `ConfirmBooking`) একটি action-এর অনুরোধ; Event (`BookingConfirmed`) ঘটে যাওয়া fact। Command validation ও authorization-এর পর state change হয়, তারপর event publish করা হয়। একই event দিয়ে notification, analytics এবং audit trail update করা যায়।

```
Client Command → API/Auth → Booking Service → Database Transaction
                                                    ↓
                                              Outbox Record
                                                    ↓
                                             Event Publisher
                                                    ↓
                                        Broker / Socket Gateway
                                                    ↓
                                           Authorized Clients
```

Transactional outbox pattern-এ business update ও outbox event একই database transaction-এ commit হয়; background worker পরে publish করে। এটি database commit সফল কিন্তু publish ব্যর্থ হওয়ার gap কমায়। Broker duplicate delivery করতে পারে, তাই consumers idempotent হওয়া উচিত।

২৭.৭ Delivery Semantics এবং Ordering

- **At-most-once:** Event হারাতে পারে, কিন্তু retry করে duplicate তৈরি হয় না।
- **At-least-once:** Retry-এর কারণে duplicate আসতে পারে; idempotency দরকার।
- **Exactly-once effect:** End-to-end বাস্তবায়ন কঠিন; business operation-এ deduplication, transactions ও unique constraints ব্যবহার করুন।
- **Ordering:** Global ordering ধরে নেবেন না। প্রয়োজন হলে per-aggregate sequence রাখুন এবং gap হলে server snapshot fetch করুন।

Client reconnect-এর পর সর্বশেষ `sequence` পাঠিয়ে missed events replay চাইতে পারে। Replay history না থাকলে authoritative API থেকে full snapshot আনতে হবে।

২৭.৮ Reconnect, Heartbeat এবং Backpressure

Connection drop হওয়া স্বাভাবিক। Exponential backoff + jitter দিয়ে reconnect করতে হবে; infinite immediate retry server overload করে। Heartbeat/ping-pong দিয়ে dead connection শনাক্ত করা যায়। Slow client-এ unlimited queue রাখলে memory বাড়ে; bounded buffer, coalescing অথবা disconnect policy প্রয়োজন।

```
function reconnectDelay(attempt, random = Math.random) {
```

```
  const capped = Math.min(30_000, 500 * 2 ** Math.min(attempt, 6));
  return Math.floor(capped * (0.75 + random() * 0.5));
}
```

এই উদাহরণে `random` inject করা হয়েছে, যাতে deterministic tests লেখা যায়।

২৭.৯ Redis Pub/Sub এবং Multi-Instance Scaling

একাধিক Socket.IO server থাকলে একই room-এর client বিভিন্ন instance-এ থাকতে পারে। Redis adapter ব্যবহার করে event fan-out সমন্বয় করা যায়। তবে Redis Pub/Sub **durable queue নয়**—offline subscriber-এর জন্য history ধরে রাখে না। Reliable replay দরকার হলে Redis Streams, Kafka বা persistent event store বিবেচনা করুন। Load balancer, sticky sessions, transport choice এবং shared authorization state-ও পরিকল্পনায় রাখতে হবে।

২৭.১০ Security Checklist

১. TLS (`wss://`) ও origin policy যাচাই করুন। ২. Session authentication এবং per-resource authorization দিন। ৩. Client message schema, size ও rate limit enforce করুন। ৪. Event data minimization করুন। ৫. Logout/revocation-এর পর unauthorized subscription বন্ধ করুন। ৬. Observability-তে connection count, publish lag, failed authorization, dropped events ও queue depth মাপুন।

২৭.১১ Practical Project — TravelLive

এই অধ্যায়ের `source-code/chapter-27/`-এ একটি **dependency-free, in-memory real-time event simulator** রয়েছে। এতে booking creation, status transition, room subscriptions, sequence-based replay, duplicate event suppression, unauthorized room access prevention, capped history এবং reconnect delay calculation আছে। Browser UI ও actual network socket নয়; core event architecture শেখার জন্য intentionally ছোট রাখা হয়েছে।

Project Structure

```
chapter-27/
├── package.json
├── live-core.mjs
├── live-core.test.mjs
├── demo.mjs
└── README.md
```

Run

```
cd source-code/chapter-27
node --test
node demo.mjs
```

Business rule: `PENDING → CONFIRMED/CANCELLED`; `CONFIRMED → CANCELLED`; `CANCELLED` terminal। User কেবল নিজের booking-এর event দেখতে পারবেন। Subscription authorization শুধু subscribe-time-এ নয়,

event dispatch-এর সময়ও পরীক্ষা করা হয়। এই simulator single-process; multi-process durable replay, network reconnection, DB persistence ও production security দেয় না।

২৭.১২ Debugging এবং Common Mistakes

Mistake 1: Socket connected মানেই user authorized—ভুল। **Mistake 2:** `emit` সফল মানেই client processed—ভুল; acknowledgments এবং retry দরকার হতে পারে। **Mistake 3:** Reconnect-এর পর event loss নেই—ভুল; replay/snapshot strategy লাগবে। **Mistake 4:** Event ID ছাড়াই repeated updates apply করা—duplicate side effects হতে পারে। **Mistake 5:** Redis Pub/Sub-কে persistent message queue ভাবা—ভুল।

২৭.১৩ Exercises

১. Polling ও SSE-এর একটি tradeoff লিখুন।

২. Socket.IO কেন raw WebSocket-এর সঙ্গে wire-compatible নয়?

৩. Booking-specific room-এর authorization কোথায় করবেন?

৪. `eventId` ও `sequence`-এর আলাদা কাজ কী?

৫. At-least-once delivery-তে idempotency কেন জরুরি?

৬. Client reconnect হলে gap recovery কীভাবে করবেন?

৭. Redis Pub/Sub-এর durability limitation ব্যাখ্যা করুন।

৮. Transactional outbox কী সমস্যা সমাধান করে?

৯. Slow client-এর backpressure policy ডিজাইন করুন।

১০. TravelLive-এ booking owner ছাড়া অন্য user-এর subscription reject করার test লিখুন।

২৭.১৪ Sample Solutions

১. Polling সহজ কিন্তু idle request বেশি; SSE server-initiated updates efficiently পাঠায়, তবে একমুখী।

২. Socket.IO নিজস্ব framing, handshake ও event protocol ব্যবহার করে।

৩. Server-side authenticated user ও booking ownership যাচাই করে।

৪. `eventId` deduplication, `sequence` per-booking ordering/gap detection।

৫. Retry duplicate event তৈরি করতে পারে; idempotent consumer একই action দ্বিতীয়বার করবে না।

৬. Last-seen sequence পাঠিয়ে replay; gap unrecoverable হলে full snapshot fetch।

৭. Subscriber offline থাকলে Pub/Sub message ধরে রাখে না।

৮. Database commit ও event publication-এর dual-write inconsistency কমায়।

৯. Bounded queue, coalescing, slow-client disconnect এবং metrics।

১০. `assert.throws(() => live.subscribe('other-user', bookingId, fn), /FORBIDDEN/)`।

২৭.১৫ MCQ

১. Bidirectional persistent transport কোনটি? (A) CSS (B) WebSocket (C) JSON (D) localStorage

২. SSE-এর direction? (A) Client only (B) Server → Client (C) Both (D) None

৩. Room subscription authorization কে করবে? (A) Browser (B) Server (C) CSS (D) URL alone

৪. Duplicate event চিনতে কোনটি সহায়ক? (A) eventId (B) font-size (C) margin (D) hostname

৫. Per-booking ordering-এর জন্য? (A) random sort (B) sequence (C) CSS order (D) cookie name

৬. Redis Pub/Sub offline history রাখে? (A) Always (B) No (C) 30 days (D) One year

৭. Transactional outbox কোথায় লেখা হয়? (A) Same DB transaction (B) CSS (C) Browser only (D) CDN

৮. Reconnect strategy? (A) Immediate infinite loop (B) Exponential backoff with jitter (C) No retry (D) Delete data

৯. Backpressure-এর একটি সমাধান? (A) Unlimited queue (B) Bounded buffer (C) Disable validation (D) Ignore memory

১০. At-least-once delivery-তে কী ঘটতে পারে? (A) Duplicate (B) No network (C) Guaranteed order (D) Zero latency

Answer key: ১-B, ২-B, ৩-B, ৪-A, ৫-B, ৬-B, ৭-A, ৮-B, ৯-B, ১০-A।

২৭.১৬ Chapter Summary

Real-time system তৈরি মানে শুধু socket connection নয়। Correctness-এর জন্য authorization, event schema, replay, idempotency, failure handling এবং scaling strategy দরকার। Chapter 28-এ এই ধরনের application Docker, CI/CD, Cloud ও Monitoring-এর মাধ্যমে deploy করার পদ্ধতি শিখব।

অধ্যায় ২৮ — Professional JavaScript Deployment, Docker, CI/CD, Cloud Infrastructure এবং Production Monitoring

শেখার লক্ষ্য

এই অধ্যায়ের শেষে development ও production environment-এর পার্থক্য, Twelve-Factor principles, reproducible build, container image, CI/CD, deployment strategies, health checks, observability, incident response এবং নিরাপদ rollback ব্যাখ্যা করতে পারবেন। **DeployReady Travel Operations API**-কে deployment-oriented structure-এ সাজাবেন।

২৮.১ Localhost থেকে Production: কী বদলায়?

Localhost-এ একটি process চালু হলেই কাজ হয়। Production-এ ব্যবহারকারী, network, DNS, TLS, secrets, persistent storage, uptime, monitoring, scaling এবং incident recovery বিবেচনা করতে হয়। Deployment শুধু code upload নয়; এটি একটি repeatable, reversible operational process।

Concern	Local development	Production expectation
Configuration	`.env` বা local variable	Managed configuration, secret store
Process	Terminal session	Supervised service/container
Data	Temporary/in-memory হতে পারে	Durable storage ও backup
Networking	localhost	TLS, ingress, firewall, DNS
Failures	Developer দেখে	Alerts, logs, incident process
Releases	Manual restart	CI, staged rollout, rollback

২৮.২ Twelve-Factor ধারণা ও Environment Configuration

Configuration code-এর বাইরে রাখতে হবে। `process.env.PORT` ব্যবহার করা যায়, কিন্তু value parse ও validate করতে হবে। Secret কখনো Git repository, frontend bundle বা Docker image-এ hardcode করা যাবে না। Development, staging ও production environment আলাদা হওয়া উচিত।

```
const port = Number(process.env.PORT ?? 3000);
if (!Number.isInteger(port) || port < 0 || port > 65535) {
  throw new Error('Invalid PORT');
}
```

Important: `.env` ফাইল Git-এ commit না করলেও সেটি একা Secret Management নয়; production-এ access control, rotation ও audit দরকার।

২৮.৩ Build Artifact, Dependency Lock ও Reproducibility

Source code → automated checks → immutable build artifact → deployment। Dependency lockfile থাকলে `npm ci` deterministic dependency install করতে সাহায্য করে; তবে OS, runtime, native modules ও registry provenance-ও গুরুত্বপূর্ণ। এই demo-তে কোনো external runtime dependency নেই, তাই `npm install` প্রয়োজন হয় না। বাস্তব Express app-এ lockfile ও dependency scanning যুক্ত করতে হবে।

Release identity হিসেবে Git commit SHA এবং image digest সংরক্ষণ করা ভালো। `latest` tag একা rollback-এর জন্য যথেষ্ট নয়।

২৮.৪ Docker Fundamentals

Image হলো build করা filesystem ও metadata; container হলো image থেকে চালু হওয়া isolated process। Dockerfile-এর layer cache build দ্রুত করতে পারে। Non-root user, minimal base image, `.dockerignore`, health check ও explicit runtime command ব্যবহার করা ভালো।

```
FROM node:22-alpine
ENV NODE_ENV=production
WORKDIR /app
COPY --chown=node:node package.json ./
COPY --chown=node:node src ./src
USER node
EXPOSE 3000
CMD ["node", "src/server.mjs"]
```

Security trade-off: Minimal image attack surface কমায়, কিন্তু image scanning, pinned digest, patched base image ও dependency review-এর বিকল্প নয়। Alpine-এ native dependency compatibility যাচাই করতে হয়।

২৮.৫ Docker Compose ও Local Production Simulation

`compose.yaml`-এ service, environment, ports, restart policy এবং shutdown grace period দেওয়া যায়। এই অধ্যায়ের demo-তে port শুধু `127.0.0.1`-এ bind করা হয়েছে, যাতে local machine-এর বাইরে অনিচ্ছাকৃত exposure কমে।

```
services:
  api:
    build: .
    ports:
      - "127.0.0.1:3000:3000"
    environment:
      NODE_ENV: production
      PORT: 3000
    restart: unless-stopped
```

Docker Compose development ও small deployment-এ কার্যকর; multi-node orchestration, autoscaling, high availability-এর জন্য অতিরিক্ত infrastructure দরকার।

২৮.৬ CI: প্রতিটি Change যাচাই

Continuous Integration-এ code push বা pull request হলে syntax, unit/integration tests, type checks, lint, security checks এবং image build চালানো হয়। Fail করা test থাকলে release block করতে হবে।

```
name: CI
on: [push, pull_request]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: '22'
      - run: npm test
      - run: docker build -t deployready:ci .
```

Security: Third-party actions production pipeline-এ full commit SHA দিয়ে pin করা, least-privilege permissions এবং secret exposure prevention বিবেচনা করুন। CI success মানে production readiness নয়।

২৮.৭ CD: Staging, Approval এবং Rollback

Continuous Delivery release-ready artifact তৈরি করে; Continuous Deployment সফল pipeline থেকে স্বয়ংক্রিয় release করে। একটি সাধারণ flow:

```
Pull Request → Tests → Build Image → Security Scan
     → Staging Deploy → Smoke Test → Approval
     → Production Rollout → Monitor → Rollback if needed
```

Blue/Green-এ দুটি environment থাকে; traffic switch করে দ্রুত rollback সম্ভব। Canary deployment-এ অল্প traffic নতুন version-এ পাঠানো হয়। Rolling deployment-এ instance ধাপে ধাপে বদলায়। **Database migration backward-compatible না হলে শুধু app rollback যথেষ্ট নাও হতে পারে।**

২৮.৮ Liveness, Readiness এবং Graceful Shutdown

`/health/live` process বেঁচে আছে কি না জানায়। `/health/ready` traffic গ্রহণের জন্য প্রস্তুত কি না জানায়। Readiness-এ dependency check থাকতে পারে, তবে costly query বা sensitive details প্রকাশ করা উচিত নয়। Shutdown-এর সময় প্রথমে readiness false, তারপর নতুন request বন্ধ ও চলমান কাজ শেষ করার চেষ্টা করা উচিত।

```
process.on('SIGTERM', () => {
  app.drain();          // readiness becomes 503
  server.close(() => { process.exitCode = 0; });
});
```

এই snippet illustrative; complete timeout ও error handling `source-code/chapter-28/src/server.mjs`-এ রয়েছে। Container orchestration-এর termination grace period এবং application shutdown deadline সামঞ্জস্যপূর্ণ রাখতে হবে।

২৮.৯ Logs, Metrics এবং Traces

Observability-এর তিনটি গুরুত্বপূর্ণ signal: **Logs** (ঘটনার বিবরণ), **Metrics** (পরিমাপযোগ্য trend), **Traces** (request-এর end-to-end path)। Structured JSON log এবং request ID debugging সহজ করে। চারটি Golden Signals: latency, traffic, errors, saturation।

```
console.log(JSON.stringify({ level: 'info', event: 'booking.created', requestId, bookingId }));
```

Logs-এ password, token, payment card বা personal data অপ্রয়োজনে রাখবেন না। Demo-এর `/metrics` JSON endpoint শিক্ষামূলক; এটি Prometheus exposition format নয় এবং production-এ publicly expose করা উচিত নয়।

২৮.১০ SLI, SLO এবং Alerting

SLI হলো observed indicator—যেমন successful requests ratio। SLO হলো target—যেমন rolling 30-day window-এ 99.9% successful requests। SLA হলো external contractual commitment। Alerts এমন হওয়া উচিত যাতে action নেওয়া যায়; প্রতিটি isolated error-এ paging করলে alert fatigue হয়।

উদাহরণ: ১০,০০০ request-এর মধ্যে ৯,৯৭০টি সফল হলে success rate = 99.7%। যদি SLO 99.9% হয়, target পূরণ হয়নি। Error budget ও traffic window ছাড়া শুধু একটি percentage বিভ্রান্তিকর হতে পারে।

২৮.১১ Secrets, Supply Chain ও Infrastructure Security

Production deployment-এ TLS termination, network segmentation, restricted database access, secret manager, container scanning, SBOM, dependency review এবং least-privilege CI credentials দরকার। Deployment key বা cloud credential source code-এ রাখা যাবে না। Rotatable short-lived credentials ও workload identity উত্তম।

২৮.১২ Backups, Disaster Recovery ও Incident Response

RPO (Recovery Point Objective) হলো কতটা data loss সহ্য করা যাবে; RTO (Recovery Time Objective) হলো কত সময়ের মধ্যে service ফিরিয়ে আনতে হবে। Backup আছে মানেই restore কাজ করে—এমন নয়; restore drill করতে হবে। Incident runbook-এ detection, escalation, mitigation, communication, rollback এবং postmortem থাকা উচিত।

২৮.১৩ Practical Project — DeployReady Travel Operations API

এই Project-এ Node.js built-in HTTP server দিয়ে একটি ছোট Travel Operations Jobs API বানানো হয়েছে। Production concepts শেখার জন্য intentionally dependency-free design। **এটি বাস্তব travel booking/payment backend নয়।**

Project Structure

```
source-code/chapter-28/
├── package.json
├── Dockerfile
├── .dockerignore
├── compose.yaml
├── .github/workflows/ci.yml
├── src/app.mjs
├── src/server.mjs
├── test/app.test.mjs
└── README.md
```

API Contract

Method	Route	Purpose
GET	`/health/live`	Process liveness
GET	`/health/ready`	Traffic readiness
GET	`/metrics`	Educational counters
GET	`/api/jobs`	In-memory job list
POST	`/api/jobs`	Validated job creation

Run and Verify

```
cd source-code/chapter-28
npm test
npm start
# separate terminal
curl http://localhost:3000/health/ready
curl -X POST http://localhost:3000/api/jobs \
  -H 'content-type: application/json' \
  -d '{"title":"Verify hotel booking"}'
```

Docker installed থাকলে `docker compose up --build` দিয়ে চালাতে পারবেন। CI workflow GitHub repository-তে push করলে চলবে; এই environment-এ GitHub Actions run করা হয়নি।

Design Decisions

- `app.mjs`-এ HTTP handler ও state; `server.mjs`-এ process lifecycle। এতে test-এ random port ব্যবহার করা যায়।
- `x-request-id` response header request correlation-এর পরিচিতি দেয়।
- Payload size limit ও title validation আছে; কিন্তু authentication, durable database এবং rate limiting নেই।
- `/health/ready` shutdown-এ 503 দেয়; `/health/live` process active থাকা পর্যন্ত 200 দেয়।
- In-memory data restart-এ হারিয়ে যাবে। Production deployment-এর আগে persistent database, access control, secret management ও observability দরকার।

২৮.১৪ Common Deployment Mistakes

1. `.env` বা private key Git-এ commit করা।
2. Image tag `latest` ব্যবহার করে কোন version deploy হয়েছে তা না জানা।
3. Database migration rollback compatibility পরীক্ষা না করা।
4. Readiness ও liveness একইভাবে implement করা।
5. No timeout, no graceful shutdown।
6. শুধু HTTP 200 দেখে success ধরে নেওয়া; latency ও error rate না দেখা।
7. In-memory session ব্যবহার করে multiple replicas চালানো।
8. Test ছাড়াই production release করা।
9. Alerting তৈরি করে runbook না রাখা।
10. Backup তৈরি করেও restore test না করা।

২৮.১৫ Exercises

১. Development ও production-এর পাঁচটি পার্থক্য লিখুন।
২. `PORT=abc` হলে কেন fail-fast করা উচিত ব্যাখ্যা করুন।
৩. Docker image ও container-এর পার্থক্য লিখুন।
৪. `.dockerignore`-এ `node_modules` রাখার কারণ কী?
৫. Liveness ও readiness-এর আলাদা উদ্দেশ্য উদাহরণসহ লিখুন।
৬. `POST /api/jobs`-এ empty title পাঠিয়ে status code যাচাই করুন।
৭. Request ID কীভাবে incident debugging-এ সাহায্য করে?
৮. Blue/Green ও Canary deployment তুলনা করুন।
৯. ৪,০০০ request-এর মধ্যে ৩,৯৮০ সফল হলে success rate গণনা করুন।
১০. DeployReady-তে persistent database যোগ করার migration plan লিখুন।

Sample Solutions

১. Config, storage, network, process supervision ও monitoring—প্রতিটি production-এ অধিক নিয়ন্ত্রিত।
২. Invalid port দিয়ে অনির্দেশ্য behaviour হওয়ার চেয়ে startup-এ clear error নিরাপদ।
৩. Image immutable template; container তার running instance।
৪. Host-এর dependency, unnecessary size এবং platform mismatch এড়াতে।
৫. Alive কিন্তু DB unavailable হলে live=200, ready=503 হতে পারে।
৬. `curl -i -X POST http://localhost:3000/api/jobs -d '{"title":""}'` → 422; content-type সহ JSON পাঠানো উত্তম।
৭. একই ID দিয়ে reverse proxy, application ও tracing events correlate করা যায়।
৮. Blue/Green দুই environment switch করে; Canary ছোট traffic slice দিয়ে শুরু করে।
৯. `3980 / 4000 * 100 = 99.5%`।

১০. Repository abstraction, schema migration, connection secret, readiness dependency, integration tests, backup ও rollback plan যুক্ত করুন।

২৮.১৬ MCQ — Final Check

১. Readiness failure-এ সাধারণত কী হয়? A) traffic বন্ধ B) CPU double C) image delete D) DNS reset

২. `SIGTERM` সাধারণত কী signal? A) shutdown request B) SQL error C) test pass D) port scan

৩. Container image কী? A) build artifact B) live database C) monitoring alert D) browser tab

৪. CI-এর উদ্দেশ্য কী? A) automatic checks B) শুধু CSS C) manual backup D) invoice

৫. Secret কোথায় রাখা উত্তম? A) managed secret store B) public Git C) frontend JS D) image label

৬. SLO কী? A) target objective B) request body C) CSS property D) Docker network

৭. Canary rollout-এ কী করা হয়? A) limited traffic B) delete tests C) disable logs D) expose secrets

৮. RPO কী বোঝায়? A) tolerable data loss B) uptime port C) memory speed D) request ID

৯. Request ID-এর সুবিধা? A) correlation B) encryption C) authorization D) compression

১০. Production-এ in-memory data-এর ঝুঁকি? A) restart data loss B) faster TLS C) immutable DB D) automatic backup

Answers ও ব্যাখ্যা: ১-A (unready instance-এ নতুন traffic না দেওয়া), ২-A (graceful termination trigger), ৩-A (container চালানোর template), ৪-A (repeatable validation), ৫-A (controlled access/rotation), ৬-A (service-level target), ৭-A (small cohort-এ risk সীমিত), ৮-A (maximum acceptable data loss), ৯-A (distributed debugging), ১০-A (durability নেই)।

২৮.১৭ Chapter Recap ও Next Step

Production readiness মানে শুধু server চালু থাকা নয়; reproducible builds, validated releases, security, health signals, observability, durable data এবং incident recovery একসঙ্গে কাজ করা। পরবর্তী অধ্যায়ে microservices, distributed transactions ও system design-এর trade-offs আলোচনা করব।

Further reading: Node.js HTTP/Process documentation, Docker Dockerfile best practices, GitHub Actions security hardening, OpenTelemetry concepts, Google SRE workbook। Version-specific details অফিসিয়াল documentation দিয়ে যাচাই করতে হবে।

অধ্যায় ২৯ — Advanced JavaScript System Design, Microservices, Distributed Systems এবং Enterprise Architecture

২৯.১ Learning Outcomes

এই অধ্যায় শেষে আপনি functional ও non-functional requirements আলাদা করতে পারবেন, modular monolith ও microservices-এর trade-off ব্যাখ্যা করতে পারবেন, distributed transaction-এর Saga Pattern implement করতে পারবেন, idempotency, consistency, retries, observability, event delivery এবং failure recovery সম্পর্কে সিদ্ধান্ত নিতে পারবেন।

২৯.২ System Design আসলে কী?

System design শুধু diagram আঁকা নয়। একটি business requirement-কে নির্ভরযোগ্য, নিরাপদ, maintainable, observable ও cost-aware software system-এ রূপান্তর করার প্রক্রিয়া। ধরুন একটি travel platform-এ flight seat reservation, payment এবং booking confirmation একসঙ্গে করতে হবে। একটি service fail করলে অন্য service-এর state কী হবে—এটাই design-এর গুরুত্বপূর্ণ প্রশ্ন।

Functional requirements

- Customer package search করতে পারবে।
- Booking request submit করতে পারবে।
- Inventory reserve হবে।
- Payment success হলে booking confirmed হবে।
- Failure হলে reservation release হবে।

Non-functional requirements

- Availability, latency, security, durability, scalability, auditability, disaster recovery এবং operational cost।
- উদাহরণ: 95th percentile API latency 300ms-এর নিচে রাখতে চাইলে আগে traffic pattern ও dependency latency মাপতে হবে; 300ms কোনো universal guarantee নয়।

২৯.৩ Capacity Planning: অনুমান নয়, হিসাব

ধরা যাক প্রতিদিন ৮৬,৪০০ booking-related request আসে। Average throughput ≈ ১ request/second। কিন্তু peak traffic average-এর ২০ গুণ হলে ২০ request/second handle করতে হতে পারে।

```
const dailyRequests = 86_400;
const avgRps = dailyRequests / (24 * 60 * 60);
const peakMultiplier = 20;
console.log({ avgRps, estimatedPeakRps: avgRps * peakMultiplier });
```

এটি শুধু starting estimate। Payload size, database queries, cache hit rate, concurrency, network, third-party payment latency ও retry amplification আলাদাভাবে বিবেচনা করতে হবে।

২৯.৪ Monolith, Modular Monolith ও Microservices

Monolith: একটি deployable application; সহজ deployment ও debugging, কিন্তু module boundaries দুর্বল হলে coupling বাড়ে। **Modular Monolith:** একই deployable-এর মধ্যে domain modules, interfaces এবং ownership পরিষ্কার। **Microservices:** service-গুলো independently deployable; scalability ও team autonomy বাড়তে পারে, কিন্তু network failure, distributed data, versioning এবং operational complexity বাড়ে।

Criterion	Modular Monolith	Microservices
Deployment	একটি unit	বহু unit
Local transactions	সহজতর	cross-service কঠিন
Observability	তুলনামূলক সহজ	tracing জরুরি
Team autonomy	মাঝারি	উপযুক্ত boundary হলে বেশি
Operational overhead	কম	বেশি

Recommendation: প্রমাণিত scale/team boundary না থাকলে modular monolith দিয়ে শুরু করা অনেক ক্ষেত্রে যৌক্তিক। Microservices কোনো automatic upgrade নয়।

২৯.৫ Domain-Driven Design এবং Bounded Context

Travel domain-এ `Catalog`, `Inventory`, `Booking`, `Payment`, `Notification` আলাদা bounded context হতে পারে। প্রত্যেকটির নিজস্ব language ও business rules থাকে। যেমন `BookingStatus` এবং `PaymentStatus` এক নয়। Database table অনুসারে নয়, business capability অনুসারে boundary নির্ধারণ করতে হবে।

```
Client / Web App
       |
   API Gateway
       |
 +-----+--------+---------+
 | Catalog      Booking   Identity |
 +-----+--------+---------+
                |
           Saga Orchestrator
              /       \
        Inventory    Payment
              \       /
             Event Bus
                |
            Notification
```

এটি **conceptual diagram**, deployed topology নয়। Gateway authentication, authorization, rate limits এবং observability policy-এর বিকল্প নয়।

২৯.৬ Synchronous বনাম Asynchronous Communication

REST/HTTP synchronous request-response workflow-তে সহজ। Events asynchronous decoupling ও background work-এ সাহায্য করে। তবে event broker ব্যবহারে delivery guarantees, replay, schema evolution, ordering ও dead-letter handling যোগ হয়।

```
const event = {
  eventId: 'evt-101',
  type: 'booking.confirmed.v1',
  aggregateId: 'booking-101',
  occurredAt: new Date().toISOString(),
  payload: { bookingId: 'booking-101' }
};
```

Event version পরিবর্তনের সময় পুরোনো consumers যেন অকারণে break না হয়, সে জন্য backward-compatible fields ও schema contract দরকার।

২৯.৭ Distributed Transactions এবং Saga Pattern

একটি local SQL transaction দিয়ে ভিন্ন service/database-এর সব change atomically commit করা যায় না। Saga-তে প্রতিটি local transaction-এর পর failure হলে compensating action চালানো হয়। Compensation একটি logical reversal; সব external side effect নিখুঁতভাবে undo করা যায় না।

Happy path

1. Booking request validate।
2. Inventory seat reserve।
3. Payment charge।
4. Booking confirm।
5. Notification event publish।

Failure path

Payment fail করলে Inventory release করতে হবে। Inventory release-ও fail করলে `MANUAL_REVIEW` বা reconciliation queue প্রয়োজন।

```
Booking Requested
      |
  Reserve Inventory ----failure----> Reject
      |
  Charge Payment ------failure-----> Release Inventory
      |
  Confirm Booking
      |
  Publish Event
```

Orchestration-এ একটি coordinator workflow পরিচালনা করে। **Choreography**-তে services events শুনে নিজেদের action নেয়। Orchestration traceability সহজ করতে পারে; choreography coupling কমালেও emergent workflow বোঝা কঠিন হতে পারে।

২৯.৮ Idempotency এবং Duplicate Request

Network timeout মানেই operation fail নয়। Client একই request retry করলে duplicate charge বা duplicate booking হতে পারে। Idempotency key ব্যবহার করে একই logical request-এর ফলাফল reuse করতে হয়। একই key দিয়ে ভিন্ন payload এলে conflict ফিরিয়ে দিতে হবে।

```
const idempotencyKey = 'trip-101';
// Server must store key + request fingerprint + result atomically.
```

In-memory `Map` একটি learning tool; production-এ durable unique constraint ও transaction দরকার। Concurrent requests-এ per-process lock যথেষ্ট নয় যখন একাধিক replica থাকে।

২৯.৯ Consistency, CAP এবং Data Ownership

Distributed systems-এ latency, partition, availability ও consistency-এর trade-off আছে। CAP-কে "যেকোনো দুটি বেছে নাও" বলে সরলীকরণ করা ভুল; এটি network partition চলাকালে নির্দিষ্ট consistency ও availability trade-off বোঝায়।

Inventory-এর মতো scarce resource-এ strong coordination প্রয়োজন হতে পারে। Analytics ও notification eventual consistency সহ্য করতে পারে। প্রত্যেক service-এর database ownership আলাদা হলে অন্য service সরাসরি table modify করবে না।

২৯.১০ Reliability Patterns

- **Timeout:** অনির্দিষ্ট অপেক্ষা এড়ায়।
- **Retry with exponential backoff and jitter:** transient error সামলায়; permanent validation failure retry নয়।
- **Circuit breaker:** failing dependency-তে অপ্রয়োজনীয় load কমায়।
- **Bulkhead:** একটি failure যেন সব resources consume না করে।
- **Dead-letter queue:** repeatedly failing messages investigation-এর জন্য রাখে।
- **Transactional outbox:** business state change ও event record একই database transaction-এ commit করে; পরে publisher broker-এ পাঠায়।
- **Inbox/deduplication:** at-least-once delivery-তে duplicate processing প্রতিরোধ করে।

```
function backoffMs(attempt, base = 100, cap = 5_000) {
  const exponential = Math.min(cap, base * 2 ** attempt);
  return Math.floor(Math.random() * exponential);
}
```

Retry-এর আগে operation idempotent কি না এবং request deadline শেষ হয়েছে কি না যাচাই জরুরি।

২৯.১১ Observability ও Security

Metrics দিয়ে rate/errors/latency বোঝা যায়; structured logs দিয়ে ঘটনা অনুসন্ধান করা যায়; distributed traces দিয়ে cross-service request path দেখা যায়। Trace ID লগে থাকলেও password, token, payment card বা private customer data লগ করা যাবে না। Service-to-service authorization, least privilege, TLS, secret rotation এবং audit trail প্রয়োজন।

২৯.১২ Practical Project — TravelSaga Booking Orchestrator

Goal: একটি in-memory educational Saga তৈরি করা, যেখানে inventory reservation, payment charge, booking confirmation, rollback ও idempotency দেখানো হবে।

Project Structure

```
source-code/chapter-29/
├── package.json
├── saga.mjs
├── demo.mjs
├── README.md
└── test/
    └── saga.test.mjs
```

Run

```
cd source-code/chapter-29
node --test
node demo.mjs
```

Core API

```
import {
  createInventory,
  createPaymentGateway,
  createBookingRepository,
  createTravelSaga
} from './saga.mjs';

const inventory = createInventory({ seats: 2 });
const payments = createPaymentGateway();
const bookings = createBookingRepository();
const saga = createTravelSaga({ inventory, payments, bookings });

const booking = await saga.book({
  id: 'trip-101', customer: 'Rahim', amount: 12500000
});
console.log(booking.status); // CONFIRMED
```

Code walkthrough: `createInventory()` seat reservation ও release পরিচালনা করে; `createPaymentGateway()` charge/refund simulate করে; `createBookingRepository()` confirmed booking রাখে; `createTravelSaga()` compensation stack reverse order-এ চালায়। একই id-র concurrent calls single-flight করে এবং repeat call-এ saved result দেয়।

Money: `amount` integer minor units (যেমন পয়সা) হিসেবে গণ্য; floating-point currency arithmetic এড়ানোর জন্য।

Limitations: এটি real microservices নয়; in-memory state restart-এ হারায়। Crash between charge and confirmation, external payment uncertainty, cross-instance race, durable outbox, payment reconciliation ও distributed lock এই demo-তে implement করা হয়নি। Production use-এর আগে এগুলো অপরিহার্য।

২৯.১৩ Failure Scenario Lab

১. `shouldFail: () => true` দিয়ে payment failure simulate করুন; inventory পুনরুদ্ধার হয়েছে কি না দেখুন।
২. `seats: 0` দিয়ে sold-out case পরীক্ষা করুন; কোনো charge হয়নি কি না দেখুন।
৩. একই booking ID দুইবার পাঠিয়ে seat count অপরিবর্তিত আছে কি না যাচাই করুন।
৪. একই ID কিন্তু ভিন্ন amount পাঠিয়ে `IDEMPOTENCY_CONFLICT` দেখুন।
৫. `Promise.all()` দিয়ে concurrent duplicate requests পাঠান।
৬. Compensation function-এ artificial error inject করে manual reconciliation-এর প্রয়োজন বোঝার চেষ্টা করুন।

২৯.১৪ Common Mistakes

১. Network timeout-কে নিশ্চিত failure ধরে আবার charge করা।
২. Microservices নিলেই scalability নিশ্চিত হবে মনে করা।
৩. Database ownership ভেঙে অন্য service-এর table সরাসরি update করা।
৪. At-least-once events-এ consumer idempotency না রাখা।
৫. Business operation-এর পরে event publish ব্যর্থ হলে outbox ছাড়া state mismatch তৈরি করা।
৬. Retry-তে jitter বা upper bound না রাখা।
৭. Compensation fail হলে error গোপন করা।
৮. Distributed trace context propagate না করা।
৯. PII ও credentials logs-এ লেখা।
১০. Production SLA নির্ধারণের আগে measurement না করা।

২৯.১৫ Exercises

১. Modular monolith এবং microservices-এর তিনটি trade-off লিখুন।
২. Booking, Payment ও Inventory-এর bounded context আলাদা করুন।
৩. Saga-র happy path এবং payment-failure path আঁকুন।
৪. Idempotency key ও request fingerprint-এর কাজ ব্যাখ্যা করুন।
৫. `at-least-once delivery`-তে duplicate handling কেন প্রয়োজন?
৬. Transactional outbox pattern-এর কাজ লিখুন।
৭. ৪৩২,০০০ daily requests হলে average RPS কত?
৮. Retry storm কেন বিপজ্জনক?
৯. Inventory strong coordination চাইতে পারে, কিন্তু notification eventual consistency সহ্য করে—কেন?
১০. TravelSaga-তে durable database ও reconciliation worker যোগ করার architecture proposal লিখুন।

Sample Solutions

১. Deployment autonomy বনাম operational complexity; independent scaling বনাম network overhead; local transaction বনাম distributed consistency।

২. Booking booking lifecycle, Payment charge/refund, Inventory available/reserved seats-এর মালিক।

৩. Success: reserve→charge→confirm; failure: reserve→charge fail→release।

৪. Key logical request চিহ্নিত করে; fingerprint একই key-তে payload পরিবর্তন শনাক্ত করে।

৫. Broker একই event পুনরায় deliver করতে পারে; duplicate side effects এড়াতে deduplication দরকার।

৬. Business change ও outbox row একই transaction-এ commit; publisher পরে event পাঠায়।

৭. `432000 / 86400 = 5 RPS`।

৮. Failure-এর সময় repeated requests downstream overload বাড়ায়।

৯. Overselling ঠেকাতে inventory coordination; delayed notification business invariant ভাঙে না।

১০. Unique idempotency constraint, transactional booking/outbox tables, async worker, retry policy, audit logs, manual review এবং reconciliation schedule।

২৯.১৬ MCQ — Final Check

১. Saga কী? A) distributed workflow with compensation B) CSS library C) encryption D) browser storage

২. Idempotency কী প্রতিরোধ করে? A) duplicate side effects B) all latency C) every network failure D) schema migration

৩. Outbox কেন? A) state/event consistency B) CSS minification C) token creation D) image compression

৪. Circuit breaker কী করে? A) failing dependency calls সীমিত করে B) database encrypt করে C) UI render করে D) package install করে

৫. Bounded context কী নির্দেশ করে? A) domain boundary B) TCP port C) Git branch D) memory size

৬. Eventual consistency কী? A) replicas সময় নিয়ে converge করে B) always immediate C) no data D) no failures

৭. Compensation কী? A) semantic undo action B) process kill C) memory leak D) CDN

৮. Trace ID কী কাজে লাগে? A) cross-service correlation B) password hashing C) SQL index D) CSS theme

৯. Dead-letter queue কী রাখে? A) repeatedly failed messages B) browser cache C) JWT secrets D) image thumbnails

১০. Distributed system-এ retry করার আগে কী জরুরি? A) idempotency ও deadline B) font selection C) color scheme D) no logs

Answers ও ব্যাখ্যা: ১-A (local steps + compensation), ২-A (same request repeated হলেও double charge এড়ানো), ৩-A (database change ও event publication gap কমানো), ৪-A (cascade failure সীমিত), ৫-A (business model ownership), ৬-A (replicas eventually converge), ৭-A (previous step-এর logical reversal), ৮-A (distributed request debugging), ৯-A (investigation/reprocessing), ১০-A (unsafe retries নতুন সমস্যা তৈরি করতে পারে)।

২৯.১৭ Architecture Review Checklist

- ☐ Requirements, traffic estimates ও failure assumptions documented
- ☐ Domain boundaries এবং data ownership defined
- ☐ Authentication/authorization ও secrets strategy
- ☐ Idempotency, retries, timeouts এবং compensations
- ☐ Event schema/versioning ও outbox/inbox plan
- ☐ Monitoring, alerting, tracing ও incident runbooks
- ☐ Backup, disaster recovery, reconciliation
- ☐ Load, integration, chaos এবং security testing plan

২৯.১৮ Recap ও Next Chapter

এই অধ্যায়ে scalable system design-এর বাস্তব trade-offs, microservice boundaries, distributed consistency, Saga, idempotency এবং resilience শিখলাম। পরবর্তী **Chapter 30**-এ full-stack capstone ও career roadmap-এর মাধ্যমে বইয়ের সব দক্ষতা একত্র করা হবে।

Further reading: Martin Fowler-এর microservices ও patterns writings; Microsoft Azure Architecture Center-এর Saga/Outbox patterns; AWS Builders Library-এর retries/timeouts/idempotency; OpenTelemetry documentation। Version-specific guidance অফিসিয়াল উৎস দিয়ে যাচাই করতে হবে।

অধ্যায় ৩০ — JavaScript Professional Mastery, Full-Stack Capstone এবং Career Roadmap

শেখার লক্ষ্য

এই অধ্যায় শেষে পাঠক (১) end-to-end product requirement থেকে implementation plan তৈরি করতে, (২) JavaScript frontend/backend/data/security একসঙ্গে ভাবতে, (৩) tested booking domain লিখতে, (৪) API contract ও architectural trade-off বুঝতে, (৫) deployment-readiness checklist বানাতে এবং (৬) নিজের portfolio ও interview roadmap সাজাতে পারবেন।

৩০.১ JavaScript Mastery মানে কী?

শুধু syntax মনে রাখা mastery নয়। Mastery হলো: **correctness, maintainability, security, observability, performance, accessibility, product thinking**—এগুলো মিলিয়ে বাস্তব সমস্যা সমাধান করা। একজন professional developer প্রশ্ন করেন: *ব্যবহারকারী কে? ব্যর্থ হলে কী হবে? data কে বদলাতে পারবে? duplicate request এলে কী হবে? পরে কীভাবে debug করব?*

স্তর	প্রধান দক্ষতা	প্রমাণ
Foundation	Variables, functions, objects, async	ছোট utility ও unit tests
Application	DOM, API, validation, state	ব্যবহারযোগ্য UI এবং error states
Backend	HTTP, auth, database	documented API ও integration tests
Engineering	Types, tests, security, CI	reproducible build ও code review
Architecture	scale, observability, failure modes	ADR, diagram, incident plan
Professional	collaboration, communication	portfolio, README, decisions

৩০.২ Capstone: TravelMaster Pro

ধরা যাক একটি travel booking service-এ ব্যবহারকারী package দেখবে, account তৈরি করবে, login করবে, booking দেবে, নিজের booking দেখবে এবং cancel করবে। সিস্টেমকে ভুল input, unauthorized request, duplicate submission এবং seat oversell থেকে রক্ষা করতে হবে। এই অধ্যায়ের runnable reference হলো **TravelMaster Pro**, `source-code/chapter-30/`-এ।

Functional requirements

1. Public catalog-এ package name, price (BDT) এবং available seats দেখাতে হবে।
2. Registration-এ নাম, email ও password validate করতে হবে।
3. Login-এর পরে short-lived bearer session token পাওয়া যাবে।

4. Booking-এর দাম **server-side package price** থেকে হিসাব হবে; client-sent price বিশ্বাস করা যাবে না।
5. Idempotency key দিয়ে একই booking request পুনরায় এলে দ্বিতীয়বার seat কমবে না।
6. নিজের booking দেখা ও cancel করা যাবে; অন্যের booking access করা যাবে না।
7. Cancel করলে একবারই seat ফেরত যাবে।
8. Tests-এ success, failure ও edge cases থাকবে।

Non-functional requirements

- Sensitive error-এ password বা internal stack trace ফেরত নয়।
- JSON body size সীমিত; invalid JSON-এ 400।
- Token মেয়াদ শেষ হলে 401।
- HTTP responses-এ consistent `{data}` অথবা `{error:{code,message}}` contract।
- Code-এ domain ও transport layer আলাদা।
- README-তে setup, limitations ও production gaps স্পষ্ট।

৩০.৩ Architecture — বাস্তব আর educational scope আলাদা রাখুন

```
Browser / Mobile Client
        |
        | HTTPS + JSON + bearer token
        v
   HTTP Router / Controller
        |
        v
    Domain Service --------> Audit events
        |
        +-----> Users / Sessions
        +-----> Package Inventory
        +-----> Bookings / Idempotency

Educational reference: সব state একটি process-এর Map-এ
Production target: SQL database + transactional inventory + durable sessions
```

এই diagram-এর *production target* বাস্তবে implement করা হয়নি। Project-এ Node.js built-in `http` দিয়ে transport layer এবং `TravelMaster` class দিয়ে domain layer আলাদা করা হয়েছে। Frontend UI-ও এই runnable reference-এর অংশ নয়; নিচের section-এ frontend integration-এর design দেওয়া আছে।

৩০.৪ Project structure

```
source-code/chapter-30/
├── package.json
├── README.md
├── demo.mjs
├── src/
│   ├── domain.mjs
│   └── server.mjs
└── test/
    └── capstone.test.mjs
```

Node.js 20+ দিয়ে চালান:

```
cd source-code/chapter-30
npm test
npm run check
npm run demo
npm start
```

Dependencies install করতে হবে না। `npm start`-এর default address `http://127.0.0.1:3000`। API example:

```
curl http://127.0.0.1:3000/api/packages
curl -X POST http://127.0.0.1:3000/api/register \
  -H 'Content-Type: application/json' \
  -d '{"name":"Demo User","email":"demo@example.com","password":"long-password-123"}'
```

Login response থেকে token নিয়ে protected route-এ `Authorization: Bearer TOKEN` পাঠাতে হবে। Booking request-এর body:

```
{"packageId":"BT-4D","travelers":2,"idempotencyKey":"unique-key-001"}
```

API contract

Method	Endpoint	Auth	Expected result
GET	`/health`	No	200 health status
GET	`/api/packages`	No	200 catalog
POST	`/api/register`	No	201 new user
POST	`/api/login`	No	200 bearer token
POST	`/api/logout`	Token	200 session revoked (demo implementation does not validate token first)
POST	`/api/bookings`	Yes	201 booking
GET	`/api/bookings`	Yes	200 user's bookings
PATCH	`/api/bookings/:id/cancel`	Yes	200 cancellation

৩০.৫ Core JavaScript decisions

Validate at trust boundaries

`typeof travelers === 'number'` যথেষ্ট নয়: `NaN`, `Infinity`, fraction, negative value—সবই সমস্যা। `Number.isInteger(travelers) && travelers >= 1 && travelers <= 10` প্রয়োজন। Email regex এখানে basic format check, বাস্তব email ownership verification নয়।

Password handling

Plaintext password store করা যাবে না। Project-এ Node-এর `scryptSync` ও per-user random salt ব্যবহার হয়েছে। Production-এ calibrated Argon2id/scrypt parameters, credential-stuffing defense, login rate

limiting এবং password-reset process প্রয়োজন। Authentication মানে কে login করেছে; authorization মানে সে **কোন data** access করতে পারবে।

Idempotency

একজন ব্যবহারকারী Submit button দুবার চাপলে দুইটি booking হওয়া উচিত নয়। Project-এ `(userId, idempotencyKey)` lookup করে একই payload-এ একই booking ফেরত দেওয়া হয়; ভিন্ন payload-এ 409। বাস্তব multi-instance server-এ এই guarantee দিতে database unique constraint এবং atomic transaction দরকার।

```
const first = service.createBooking(userId, {
  packageId: 'BT-4D', travelers: 2, idempotencyKey: 'booking-001'
});
const retry = service.createBooking(userId, {
  packageId: 'BT-4D', travelers: 2, idempotencyKey: 'booking-001'
});
console.log(first.id === retry.id); // true
```

Inventory consistency

`if (seats >= requested) seats -= requested` একটি process-এ synchronous path-এ কাজ করে, কিন্তু একাধিক server/database transaction-এর ক্ষেত্রে race condition তৈরি করতে পারে। Production SQL উদাহরণ:

```
BEGIN;
UPDATE packages
SET seats = seats - $1
WHERE id = $2 AND seats >= $1
RETURNING id, price_bdt;
-- zero rows => sold out, ROLLBACK
INSERT INTO bookings(user_id, package_id, travelers, total_bdt)
VALUES ($3, $2, $1, $4);
COMMIT;
```

বাস্তবে booking insert, idempotency uniqueness, payment state এবং rollback—সব একই transactional design-এ নিশ্চিত করতে হবে। `$1`-style parameters SQL injection রোধের একটি গুরুত্বপূর্ণ অংশ।

৩০.৬ Frontend integration plan

একটি responsive UI-তে Home/Catalog, Package Details, Login/Register, Booking Checkout, My Bookings এবং Error/Empty States থাকবে। `fetch`-এর success path-এর পাশাপাশি loading, network failure, 401, 409 sold-out, duplicate submit, accessibility এবং keyboard navigation ভাবতে হবে।

```
async function loadPackages() {
  const response = await fetch('/api/packages');
  if (!response.ok) throw new Error('Packages unavailable');
  const { data } = await response.json();
  return data;
}
```

Important: Exampleটি conceptual frontend integration; capstone archive-এ browser UI তৈরি বা test করা হয়নি। Token browser storage-এ রাখার সিদ্ধান্ত XSS threat model অনুযায়ী নিতে হবে। HttpOnly cookie নিলে CSRF protection-ও ডিজাইন করতে হবে।

৩০.৭ Testing pyramid এবং quality gates

- **Unit:** validation, price calculation, idempotency, seat restoration।
- **Integration:** HTTP register → login → booking → history।
- **Security-focused:** unauthenticated request 401, অন্যের booking 404, malformed JSON 400।
- **E2E (future):** real browser, mobile viewport, screen-reader behavior, production-like DB।

CI gate হিসেবে `npm run check` এবং `npm test` চালানো যায়। Tests pass করলেও full security audit বা production correctness প্রমাণ হয় না।

৩০.৮ Production architecture upgrade plan

1. **PostgreSQL:** users, packages, bookings, idempotency_requests, sessions, audit_events schema।
2. **Migrations:** version-controlled migrations; constraints, foreign keys, unique keys।
3. **Atomic transactions:** seat reservation ও booking creation একই transaction।
4. **Payments:** gateway webhook verification, replay protection, reconciliation, refunds।
5. **Session security:** durable session store, token rotation/revocation, HTTPS, rate limits।
6. **Observability:** request ID, structured logs, latency/error metrics, alerting।
7. **Delivery:** Docker, CI, staged rollout, backups, recovery drill, rollback।
8. **Privacy:** data minimization, retention policy, access control, secret management।

Never expose demo server directly to the public internet as a production travel-booking service.

৩০.৯ Architecture Decision Record (ADR)

ADR-001: In-memory domain for educational reference. Context: পাঠককে dependency ছাড়াই code run করাতে হবে। Decision: Node built-ins, small modular service, in-memory repositories। Benefit: সহজ test ও setup। Cost: restart-এ data হারায়, horizontal scaling নেই, persistent transaction নেই। Next decision: PostgreSQL + transactional repository। ADR লিখলে future developer বুঝবেন কেন একটি সিদ্ধান্ত নেওয়া হয়েছিল এবং কখন বদলাতে হবে।

৩০.১০ Final assessment rubric (100 marks)

ক্ষেত্র	Marks	মূল্যায়ন
Requirements ও user flows	10	edge cases স্পষ্ট
JavaScript design	15	modularity, validation
API & authentication	15	correct status, ownership
Data consistency	15	idempotency, inventory
Tests & debugging	15	automated coverage

ক্ষেত্র	Marks	মূল্যায়ন
Security & privacy	10	threats, mitigations
Deployment & monitoring plan	10	readiness checklist
Documentation & communication	10	README, ADR, demo

Score interpretation: 85–100 strong capstone; 70–84 good foundation; 50–69 targeted improvement; below 50 revise requirements and tests first। এটি learning rubric, official certification নয়।

৩০.১১ Professional roadmap — 12 weeks

Weeks	Focus	Deliverable
1–2	Fundamentals revision, async, Git	tested utility library
3–4	DOM, accessibility, API calls	responsive package catalog
5–6	Node.js, auth, REST	booking API
7–8	SQL, transactions, Prisma/ORM	persistent booking flow
9–10	tests, TypeScript, security	CI-tested refactor
11	Docker, deploy, monitoring	staging demo
12	portfolio, README, interviews	case study + walkthrough

Portfolio-তে শুধু screenshot নয়: **problem, architecture, trade-offs, tests, limitations, what I would improve** লিখুন। Interview-এ event loop, closures, async errors, authorization, SQL transaction, caching, idempotency এবং system design নিয়ে যুক্তি ব্যাখ্যা করার অভ্যাস করুন।

৩০.১২ দশটি অনুশীলন ও সমাধান নির্দেশনা

1. **Email normalization:** `TEST@EXAMPLE.COM` দিয়ে duplicate account কেন reject হয়? **সমাধান:** `trim().toLowerCase()`-এর পরে `usersByEmail` lookup।
2. **Price tampering:** Client `{price:1}` পাঠালে total কত? **সমাধান:** catalog-এর `price * travelers`; client price উপেক্ষিত।
3. **Retry:** একই idempotency key ও payload দুইবার পাঠান। **সমাধান:** একই booking ID, একবার seat decrement।
4. **Key collision:** একই key-তে travelers বদলান। **সমাধান:** 409 `KEY_CONFLICT`।
5. **Unauthorized access:** token ছাড়া `/api/bookings`। **সমাধান:** 401 `UNAUTHORIZED`।
6. **Ownership:** user A-এর booking user B cancel করতে পারে? **সমাধান:** না, 404।
7. **Cancellation:** একই booking দুবার cancel করলে seats কতবার বাড়বে? **সমাধান:** একবার।
8. **Expiry:** clock এক ঘণ্টা এগিয়ে দিলে token valid? **সমাধান:** `expiresAt <= now` হলে 401।

9. **Database design:** booking-এ foreign key কী হবে? **সমাধান:** `user_id → users.id`, `package_id → packages.id`; constraints ও indexes।
10. **Operational readiness:** deployment-এর আগে পাঁচটি কাজ লিখুন। **সমাধান:** persistent DB, transactions, auth hardening, rate limits, backups/monitoring।

৩০.১৩ দশটি MCQ — উত্তর ও ব্যাখ্যা

1. **Server-side price কেন?** A. UI দ্রুত B. tampering প্রতিরোধ C. CSS D. caching। **উত্তর B:** client input বিশ্বাসযোগ্য নয়।
2. **Idempotency key-এর উদ্দেশ্য?** A. SEO B. duplicate side effect রোধ C. image resize D. login। **উত্তর B।**
3. **Authentication বনাম authorization?** A. একই B. identity বনাম permission C. CSS বনাম HTML D. network বনাম disk। **উত্তর B।**
4. **Atomic booking-এর জন্য কী দরকার?** A. `setTimeout` B. DB transaction C. localStorage D. alert। **উত্তর B।**
5. **401 কখন?** A. login প্রয়োজন B. package নেই C. sold out D. server success। **উত্তর A।**
6. **409 কোন ক্ষেত্রে যুক্তিযুক্ত?** A. duplicate/conflicting request B. healthy server C. logout success D. list packages। **উত্তর A।**
7. **Unit test-এর কাজ?** A. পুরো cloud deploy B. ছোট behavior isolate করা C. DNS বদলানো D. design logo। **উত্তর B।**
8. **ADR কী document করে?** A. animation B. architecture decision/trade-off C. only CSS D. invoice। **উত্তর B।**
9. **In-memory data-এর সীমাবদ্ধতা?** A. restart-এ state হারায় B. automatically replicated C. ACID guarantee D. infinite scale। **উত্তর A।**
10. **Portfolio case study-তে কী জরুরি?** A. শুধু screenshot B. problem, decisions, evidence, limitations C. শুধু color D. শুধু title। **উত্তর B।**

৩০.১৪ Final project extension challenges

Beginner: catalog filtering ও pagination। **Intermediate:** PostgreSQL persistence, Prisma migration, secure auth session। **Advanced:** payment webhook, reservation expiry, queue-based notifications, concurrency tests, observability। **Expert:** multi-region trade-offs, disaster recovery, load test, incident simulation। প্রতিটি feature-এ requirement → threat model → test → implementation → review → documentation flow অনুসরণ করুন।

৩০.১৫ বই শেষ, শেখা নয়

এই ৩০ অধ্যায়ের উদ্দেশ্য হলো আপনাকে independent engineering decisions নিতে সাহায্য করা। প্রতিটি নতুন technology শেখার আগে নিজেকে জিজ্ঞেস করুন: *এটি কোন সমস্যার সমাধান? এর ব্যর্থতার ধরন কী? আমি কীভাবে test করব?* এই mindset-ই long-term JavaScript mastery-এর ভিত্তি।